CoreLoop

前有数据结构

从原理到实现

CoreLoop

UNKNOWN VOICE

是的，没错，这片大陆叫做《数据结构》，是历代先贤们以智慧为剑刃开辟的疆土，也是无数求知者在 0 与 1 的荒原上寻找真理的朝圣之地。寻理者皆向北而行，并了解了古老律法的真相。

This land is called Data Structures. It is a realm carved out by sages of past generations with wisdom as their blade, and it is also a sacred pilgrimage site where countless seekers of knowledge search for truth in the wilderness of 0 and 1. In venturing north, the seekers discover the truth of logic.

位不见王影，然火焰映彻苍穹。当传承火焰燃烧之时，钟声将响彻四周，进而唤醒遍布这片大陆的思渡者们。

时秩双王 — 栈和队列，分界统御者 — 树与二叉树，以及万象之都的孤独王者 — 图。不过啊，王者们一定会筑起不可逾越的高墙吧。而无火的余灰将会纷至踏来，那是无名、成不了薪，被命运牵绊的思渡者。但也正因如此，灰烬才会如此渴求余火吧。

落叶捎来消息：在雾的彼端，我们的故乡“交界地”，那伟大的艾尔登法环已经破碎，“永恒女王”玛莉卡销声匿迹，在黑刀阴谋之夜，“黄金”葛德文最先失去性命，玛莉卡之子 – 诸位半神拿到艾尔登法环的碎片。却因为那股力量堕入歧途、陷入疯狂、引发碎片战争..... 在那场无王存在的战争最后 – 无上意志放逐了他们。奥，所以啊，褪色者啊 – 依旧无法永眠的死者啊，那许久以前我们失去的赐福，正在出声呼唤。“蛮荒地王者”荷莱露。光耀金面具、“死眠少女”菲雅、受尽唾弃的食粪者、“百智爵士”基甸奥夫尼尔。..... 那失去的赐福又再一次，回到默默无名的褪色者身上。朝雾的彼端前进，抵达交界地，觐见艾尔登法环 – 成为艾尔登之王吧。

— BONFIRE DISCOVERED —

欢迎来到营火，无火的余灰大人。我正是维火女，专门维护营火以及服侍您的人。

正如您所见，穿过山隘，前方就是数据结构大陆中不显眼的一角——《前有数据结构》，这片大陆的守护神亦是开创者，大家称其为 CoreLoop。如果您为了觐见**树**、**图**等王者们而需要帮助，我很愿意为此帮忙。

首先，CoreLoop 托我带给您以下谏言，愿它们能照亮您前行的路。

1. 《前有数据结构》假设您已经有了比较好的 C 语言基础，但他也会在某些合适的地方强调某些 C 语言的知识点。
2. 在您思渡的过程中，遇到的所有“思维拓展”**都！可！以！不！必！停！留！！**我向您保证，即使跳过这些，您也决不会错过任何核心内容。您可根据适合自己的方式选择是否驻足。

如果你对本书中的内容存在疑问，欢迎联系作者。【联系方式】

许多地方添加一些内容，能够对正文有很好的补充及/或帮助大家理解的作用，但是为了保证在本书中不过分多地引入与《数据结构》本身无关的内容，作者并未给出这些“DLC”。如果想要了解，请参考我的视频。

知识绝不会少，但更重要的是思考。

很多有意思的东西都是藏在习题里面。

即使引导早已破碎，也请您成为数据结构之王。

恭喜你找到了第一个篝火。你可以在此处整理背包，回忆这段旅程的所见所闻，并且将所得卢恩转化为力量，并解锁一些新的能力。后续可以在适合的篝火处和读者说说话。

继续向前，便是 CoreLoop 所开创的大陆。在旅途中，若我的出现偶尔成了您的负累，还请您不必驻足与我的谏言。我并非大陆的探拓者，因此这不会让您错过任何数据结构大陆的美丽风景。无论如何，我都将不离不弃地伴您同行，直至旅途的终点。

进入大陆前，请您收下这份地图。

地 图

第一章 数据结构 **1**

1.1 数据结构 1

1.2 算法与算法效率的度量 3

1.2.1 时间复杂度 3

1.2.2 空间复杂度 5

1.3 **标题还没想好** 6

第二章 线性表 **9**

2.1 顺序表和链表简介 10

2.2 顺序表 13

2.2.1 从使用者视角理解“数据结构” 16

2.3 链表 18

2.4 顺序表与单链表的对比 24

2.4.1 随机访问 25

2.4.2 插入和删除 25

2.4.3 空间利用 25

2.4.4 如何选择 25

习题 1 28

2.5 单链表的常见变态 32

2.5.1 双链表 32

2.5.2 循环链表 34

2.5.3 静态链表 34

2.6 本章小结 36

第三章 栈和队列 **37**

3.1 栈 38

3.1.1 顺序栈的常见操作实现 40

3.1.2 链栈的常见操作实现 41

3.1.3 * 浏览器的前进后退功能实现 . . . 42
3.1.4 *“函数栈”、“调用栈”是啥？ . . . 44
3.2 队列 . . . 46
3.2.1 队列的链式存储 . . . 46
3.2.2 队列的顺序存储 . . . 49
3.3 栈的进一步理解 . . . 53
3.3.1 表达式的转换与求值 . . . 56
习题 2 . . . 63

第四章 树与二叉树 71
4.1 树 . . . 71
4.1.1 * 目录树的简单实现 . . . 74
4.1.2 树的一些基本概念与性质 . . . 76
4.2 二叉树的基本内容介绍 . . . 78
4.2.1 二叉树的实现 . . . 79
4.2.2 二叉树的遍历 . . . 81
习题 3 . . . 92
4.3 哈夫曼树与哈夫曼编码 . . . 97
4.3.1 哈夫曼编码 . . . 100
4.4 堆 . . . 103
4.4.1 堆及其常见操作的实现 . . . 103
4.4.2 堆的经典应用 . . . 109
习题 4 . . . 111
4.5 二叉查找树 . . . 115
4.5.1 二叉查找树的实现 . . . 116
4.6 平衡二叉树 . . . 121
4.6.1 平衡二叉树的插入操作 . . . 124
习题 5 . . . 132

第五章 图 137
5.1 图的存储 . . . 140
5.1.1 邻接矩阵 . . . 140
5.1.2 邻接表 . . . 141
5.1.3 二者获取邻接信息的对比 . . . 144
5.2 图的遍历 . . . 146
5.2.1 深度优先搜索（DFS） . . . 146
5.2.2 广度优先搜索（BFS） . . . 148

5.2.3 DFS 和 BFS 的实现 149
习题 6 155
5.3 最短路径 162
5.3.1 迪杰斯特拉算法 162
5.3.2 Bellman-Ford 算法 174
5.3.3 Floyd 算法 178
习题 7 181
5.4 最小生成树 184
5.4.1 Prim（普里姆）算法 185
5.4.2 Kruskal（克鲁斯卡尔）算法 186

C++ 重点内容整理 **189**
vector 189
引用 190
stack 191
queue 192
C++ 的 Hello, World 193

习题参考答案 **197**

第一章　数据结构

waypoint-1

欢迎来到我的大陆。请您放心，我不会在此处引入诸如“数据”的定义、什么是“数据结构三要素”等等内容。但是同时也请您放心，我会让您彻底搞懂，什么是“数据结构”，“数据结构”究竟是在研究什么。

1.1　数据结构

事实上，“数据结构”看起来抽象晦涩的定义，恰恰能够最好地理解“什么是数据结构”。接下来就请你随我一起进入这段拨云见日的旅程。

定义

数据结构是相互之间存在一种或多种特定“关系”的数据元素的集合，以及定义在这些数据元素上的“操作”的集合。

我们以一个例子开启这段旅程：假设你是一个游戏程序员，你要编写一个游戏，玩家操控具有四个“角色”组成的小队，如：邪念、银心、甘尔以及张三这四个人，那么，每个“角色”就都可以被看成是一个数据元素。

> **数据元素**，就是我们要研究和处理的一个个“东西”。这些“东西”可以是一个游戏角色、一件装备、一条聊天消息、一个整数、甚至一张地图等等。

但是，仅仅有了四个数据元素，还不能叫“数据结构”，我们还要解决一个问题：这四个人是什么关系？比如你设计的游戏玩法是将四人排成一个有前后顺序的队伍，那么四人便会有如下的线性关系：

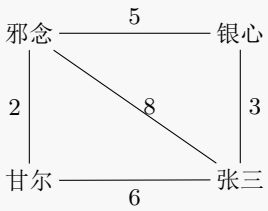
邪念 → 银心 → 甘尔 → 张三

或者你设计的玩法是一个队长三个队员，则四个人便有如下的树状关系：



waypoint-5

再假设，四个人之间互相存在好感度，则四个人可能有如下的图结构：



请注意，这里我们使用无向图描述，是因为我们假设了 A 对 B 的好感度与 B 对 A 的好感度相同，但现实中往往并非如此，要描述这种更现实的情况，就要使用后面将会介绍到的有向图。

到现在，“数据结构”这个词是不是已经变得具体了？再回看数据结构的定义，我们看到它还有后半句：“操作”。

假设现在我们的游戏队伍：

邪念 → 银心 → 甘尔 → 张三

玩家可能会在游戏中让某些人“加入队伍”（Insert），某些人“退出队伍”（Delete），如卡姐加入：

邪念 → 银心 → 甘尔 → 张三 → 卡姐

邪念退出：

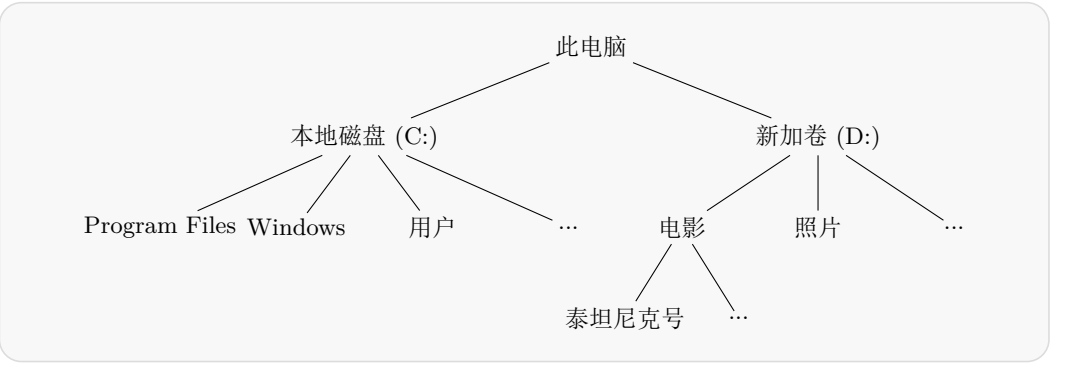
银心 → 甘尔 → 张三 → 卡姐

这里“入队”、“出队”，就都是所谓的“定义在这些数据元素上的操作”。再比如，新人入队前，你可能需要判断当前队伍是否已满，因此需要“获取队伍人数”（GetSize），这也是“定义在这些数据元素上的操作”。

有了以上的介绍，再回过头来看那个之前觉得它晦涩难懂的数据结构定义，就豁然开朗了。

任何一本《数据结构》的教材，都一定会有“线性表”、“队列”、“树”、“图”等

这些内容。为什么我们要重点研究这些数据结构呢？经过简单的思考不难发现，“线性表”、“队列”、“树”、“图”等，正是我们处理实际需求时，组织数据所需要的最有用的、最常见的结构。比如，一个有趣的事实是：你的个人计算机中目录结构的组织，其实就是一棵树。如：



请注意，以上出现的线性结构、树状结构、图结构，都只是逻辑意义上的结构，展示出数据元素之间的逻辑关系，而非数据元素物理存储（如在主存或磁盘中）的位置关系。这种逻辑上的关系称为数据的**逻辑结构**。《数据结构》不仅研究数据的逻辑结构，还要研究如何实现物理存储，这通常通过计算机语言来实现。本书中后面我们介绍到的每一种数据结构，都会给出其比较常见的、一般的实现方式。如果你感兴趣的话可以想一想，以上出现的数据结构，你如何通过你已有的编程语言知识来实现？

1.2 算法与算法效率的度量

我们已经知道，数据结构研究的是数据的组织方式以及对这些数据的操作。事实上，同样的一批数据，由于组织方式不同，可能会导致操作效率产生巨大的差异。因此，在实际程序中，我们在考虑数据应该如何组织的同时，也要考虑面对这些数据，我们应该采用什么步骤和方法去解决问题？例如：

- 如何在一组数据中快速找到目标元素？
- 如何将一组无序数据排列成有序序列？
- 如何在复杂的网络关系中寻找最短路径？

对于同一个问题，往往存在多种不同的算法。它们可能都能得到正确结果，但执行过程中的时间消耗、空间占用可能存在巨大差异。一个算法的效率正是通过其“时间消耗”与“空间占用”来衡量的，正如下面将要介绍的**时间复杂度**和**空间复杂度**。

1.2.1 时间复杂度

我们描述一个算法的时间消耗，不能简单地说出类似“算法 A 需要 3 秒，算法 B 需要 0.001 秒”这样的话。因为代码的运行时间受到 CPU 性能等诸多因素的影响。因

此，我们需要一种与机器无关的评价方法，这便是**时间复杂度**。算法的**时间复杂度**，是指算法执行时间随问题规模增长而变化的数量级关系。我们以一个最简单的，在数组中顺序查找某个元素的算法为例来说明：

```
for(int i = 0; i < n; i++)
{
    if(a[i] == x)
        return i;
}
```

这里 n 是数组的大小，也即问题规模大小。如何描述该算法的时间复杂度？一个很准确的做法是用该算法的**基本操作**的次数来描述。所谓**基本操作**，是指该算法中执行次数最多的、能代表性能瓶颈的操作。该例中，循环执行若干次 i<n, a[i]==x, 以及 i++ 的操作，这两个判断及一个变量自增就是该算法的基本操作。我们使用 $T(n)$ 来描述算法的基本操作次数，显然，该例中，最坏情况下查找到最后一个，算法才会返回，因此最坏情况下：

$$T(n) = n$$

而平均情况下：

$$T(n) = \frac{1}{n} \cdot 1 + \frac{1}{n} \cdot 2 + ... + \frac{1}{n} \cdot n = \frac{n+1}{2}$$

这表示若输入规模 n 增加一倍，则算法在最坏情况下执行次数也会增加一倍，而平均情况下，执行次数会增加 $\frac{n+1}{2}$。这便是时间复杂度的“大 T 表示法”。

而实际的学术或工程中最最常用的，也相信你一定已经见过了的，并非大 T 表示法，而是**“大 O 表示法”**：

考虑某算法的时间复杂度为

$$T(n) = 3n^2 + 5n + 10$$

当 n 很大时，该算法的时间代价几乎完全由 $3n^2$ 这一项决定，而后面两项可以忽略不计。而 3 是常数，不随问题规模 n 的变化而变化。因此，使用大 O 表示法，我们说该算法的时间复杂度是 $O(n^2)$。若某算法的时间复杂度不随问题规模 n 的变化而变化，则我们说该算法的时间复杂度为 $O(1)$。对于大 O 表示法严谨的数学定义，感兴趣的同学请自行参考相关资料。根据我们已有的数学知识，不难得出，在问题规模 n 很大的情况下：

$O(1)$ 优于 $O(logn)$ 优于 $O(n)$ 优于 $O(nlogn)$ 优于 $O(n^2)$ 优于 $O(n^3)$ 优于 $O(a^n)$

*** 思维拓展**

这里还有一个拉扯感比较强的问题：所谓“问题规模很大”，“很大”的边界在哪？比如某个问题我们确定问题规模 n 一定满足 $n <= 100$，那么，$T(n) = n^3$, 即 $O(n) = n^3$ 的算法，是不是反而优于 $T(n) = 101n^2$, 即 $O(n) = n^2$ 的算法？答案是：是的。这个问题也说明了，大 O 复杂度并不是算法实际运行效率的绝对比较。但是在实际的工程中，性能瓶颈通常是由大规模（n 很大）的数据部分产生的，这也是实际中都在使用大 O 复杂度的原因。

1.2.2 空间复杂度

某算法的**空间复杂度**是指实现该算法所需额外存储空间随问题规模增长而变化的数量级关系。同时间复杂度一样，绝大多数情况下我们关注的是算法所需额外存储空间随问题规模 n 的增长趋势，因此我们同样使用大 O 表示法来描述空间复杂度。

请注意，计算空间复杂度时我们只考虑处理问题而额外引入的空间，并不计算输入数据本身的空间。比如以下数组求和的例子，我们只引入了存储变量 i 和 result 的两个额外 int 大小的空间，该空间大小与问题的输入规模 n 无关，因此该算法的空间复杂度为 O(1)，而数组 a[] 是问题本身的数据，其存储空间并不算入空间复杂度：

```
int sum(int a[], int n)
{
    int result = 0;
    for(int i=0; i<n; i++)
    {
        result += a[i];
    }
    return result;
}
```

提示

分析空间复杂度很简单，只需要看程序额外使用了多少内存。如果你对时间复杂度没有“感觉”，笔者在这里给出一个建立感觉的提示。如前所述，时间复杂度要看程序执行次数最多、最能代表瓶颈的操作。通常来说，执行次数最多的循环就是程序的瓶颈。如：

```
void f(int n) { // 模拟问题的输入规模为 n
    int a = 10;
    int b = a + 2;
    // 该循环约执行 n 次，复杂度为 n 量级
    for(int i=0; i<n; i++) {
        // ...
    }
    // 该循环约执行 n+(n-1)+...+2+1 次，复杂度为 n^2 量级
    for(int i=0; i<n; i++) {
        for(int j=i; j<n; j++) {
            // ...
        }
    }
    // 该循环约执行 nlogn 次，复杂度为 nlogn 量级
    for(int i=0; i<n; i++) {
        for(int j=1; j<n; j*=2) {
            // ...
        }
    }
}
```

以上代码中，显然

```
// 该循环约执行 n+(n-1)+...+2+1 次，复杂度为 n^2 量级
for(int i=0; i<n; i++) {
    for(int j=i; j<n; j++) {
        // ...
    }
}
```

为程序的性能瓶颈。因此我们说算法 f 的时间复杂度是 $O(n^2)$。

1.3 标题还没想好

通过以上的介绍，我们就了解了《数据结构》或者叫《数据结构与算法》这个学科，就是研究在不同的实际需求下，数据如何存储、组织，以及高效操作的学科。

任何一本数据结构的教材都会重点介绍“线性表”、“队列”、“树”、“图”等数据结构，这是因为这些结构并不是枯燥的理论模型，而是计算机世界中最常见、最高效的组织方式。当你打开一个游戏背包，当你浏览一棵文件目录树，当你在地图中寻找一条最短路径，当你在社交网络中查看好友关系，你看到的表象虽然不同，但它们背后都隐藏着数据结构的思想。

笔者认为，《数据结构》这个学科次在“掌握知识”，而重在“培养能力”。学习《数

据结构》，真正重要的是，在学习的过程中不断思考、体会和理解这些数据结构与算法背后的思想及其代码实现原理，训练自己灵活的思维和解决问题的能力。因此本书在组织编排上，传递知识的同时更注重的是培养思维能力、创造力以及解决问题的能力。

那接下来，就我们将一起走进这段探索之旅，逐一认识这些经典结构，理解它们为什么诞生，以及它们如何帮助我们更加高效地解决实际问题。在此之前，如果你是一个喜欢思考问题的人，不妨看看下面的“思考”。说不定天才的你能够自己探索出最优解（甚至想出了比目前已知的最优解更优的解法，顺便发表一篇论文什么的）。以下的这些问题呢，也会随着本书后续的开展，而不断地展开讨论，相信你每次回看它们的时候，都会有全新的不同理解。

思考

这里给出一些思考吧，学习本书之前，可以自己瞎想一下，学完本书之后，都给出一个很优的答案，尽量做到每种结构一个例子。

代码 1.1 测试代码

```
1
2  int main() {
3      int a = 10;
4      int b = 20;
5
6      int c = a + b;
7
8      return 0;
9  }
10
```

带页码：如图 2.1（第 10 页）所示。如代码 1.1（第 7 页）所示。不带页码：如图 2.1 所示。如代码 1.1 所示。

第二章　线性表

欢迎来到第二章。

假设某游戏的玩法是玩家操控一个四人组成的小队，队伍成员存在严格有序的位置关系，类似这样：

邪念 → 银心 → 甘尔 → 张三

这，其实就是一个由四个“游戏中的可操控角色”构成的线性表，表中有四个元素，元素的类型是“角色”，可能类似这样：

代码 2.1

```
struct Role{
    int hp;             // 生命值
    int attack_power;   // 攻击力
    // ...              // 其他
};
```

有了这个例子，我们给出如下线性表的定义：

定义

线性表是具有相同数据类型的 $n(n >= 0)$ 个元素构成的有限有序序列，其中 n 为表长，当 $n = 0$ 时线性表是一个空表。

请注意，线性表所称元素“有序”，这并不是说元素的值大小有序，而是元素有严格的位置关系。例如，我们可以用 L 将线性表表示为：

$$L = (a_1, a_2, ..., a_n)$$

其中，a_1 是唯一的第一个元素，a_n 是唯一的最后一个元素。将其中任意两个元素位置互换后，得到的新的线性表与原来的线性表并非同一个线性表。

2.1 顺序表和链表简介

线性表最常见的实现方式有两种。在 C 语言中，我们可以利用数组让这些元素在内存中连续地存放。这种利用一段连续的存储空间依次保存表中元素的线性表被称为**顺序表**。顺序表的存储结构定义如代码 2.2 所示。

代码 2.2 线性表的顺序存储结构

```c
#define MaxSize 100
struct SeqList{  // SeqList, 即 Sequential List
    ElemType data[MaxSize];  // 存放表中元素, 可用数组下标为 0-99
    int current_size;        // 当前表中的元素数量
};
```

其对应的物理存储结构如图 2.1 所示。

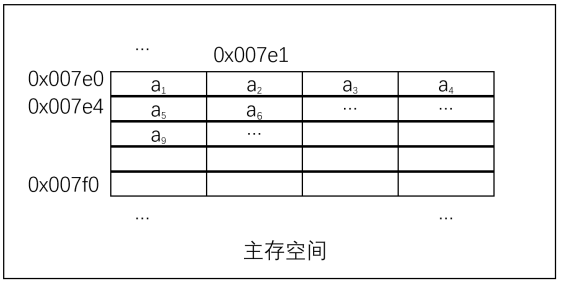


图 2.1 顺序存储的线性表

初见提示

相信你懂我的意思，但初见我还是要多啰嗦一句：图 2.1 中我们假设了某顺序表起始地址为 0x007e0，每个元素大小为 1 字节。而在后续诸多内容中，在不会引起误解的前提下，我们将省去这种用于严谨的话术，而专注于“传意”的意图。

提示

代码 2.2 中，我们用 ElemType(Element Type) 表示表中的元素类型。比如你要处理一组 int 型数据，把它们放在线性表中，ElemType 就可以是 int 类型；再比如你要处理四个游戏角色，ElemType 就可以是 Role 类型 (类似代码 2.1)。

顺序表保证了逻辑上相邻的两个元素物理上也一定相邻。因此，表中任何一个元素的地址都可以由

$$\text{Address}(a_i) = \text{Address}(a_1) + \text{sizeof(ElemType)} \cdot (i-1)$$

计算得来。因此，访问（读或写）顺序表中的任意一个元素，都可以在 O(1) 时间内实现，这种特性我们称为**顺序表支持随机访问**。

*** 思维拓展**

若线性表可以对任意元素 a_i 进行读写，而且操作时间为 $O(1)$，与 i 和数组大小 n 无关，则这种访问方式称为**随机访问（random access）**。如何理解“随机”？第一，可以理解为表可以对任一位序实现 O(1) 的访问效率，即使位序是“随机”给出的。第二，与链表做对比。我们将马上看到，链表只能顺序访问（sequential access），而无法随机访问。也就是说，要访问链表中位序为 i 的元素，我们必须依次访问 $a_1, a_2, ..., a_{i-1}, a_i$。而无法跳过前面直接访问 a_i。

不难想到，可以使用如下代码初始化一个顺序表并为其构造元素：

```
// 假设我们要将存放在数组 a 中，长度为 n 的数据插入某个顺序表中
struct SeqList L;         // 定义一个顺序表
L.current_size = 0;      // 空表初始元素数量为 0
// 插入数据
for(int i=0; i<n; i++) {
    if(L.current_size < MaxSize) {
        L.data[L.current_size] = a[i];
        L.current_size++;
    } else {
        // 表已满，拒绝插入
    }
}
```

通过 C 语言提供的指针，我们也可以让每个元素通过一个指针指向下一个元素，从而链接起整个线性表。这种通过指针，将一个个物理上未必连续的存储空间链接起来的线性表被称为**链表**。在这种实现方式下，表的每个节点除了存储数据元素本身，还要额外存储一个“指向下一个元素的指针”。因此，我们需要一个 struct 来描述表的一个“节点”：

```
struct ListNode{
    ElemType data;  // 该节点的数据元素
    struct ListNode* next_node; // 下一个节点的地址
};
```

由于有了第一个链表节点的指针，就可以顺着链条找到所有节点，因此，一个链表可以用其第一个节点的指针来表示：我们定义

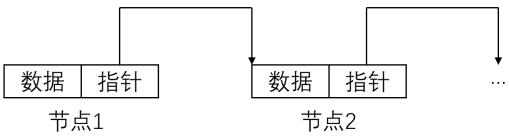


图 2.2 链表的节点

```
typedef struct ListNode* LinkList;  // LinkList 是指向 ListNode 的指针类型
```

于是，就可以用

```
LinkList L;
```

来定义一个“链表”对象 L（就像用”int a;” 来定义一个 int 型对象 a 一样）。

提示

LinkList 与 ListNode* 是同一种类型。只不过语义上强调的重点不同：ListNode* 强调“这是一个节点”，而 LinkList 强调“这是一个链表”。我们也经常使用

```
typedef struct ListNode* head;
```

来突出 head 是链表第一个节点的指针。

链表的物理存储结构如图 2.3 所示。现在假设我们要将存放在数组 a 中，长度为 n

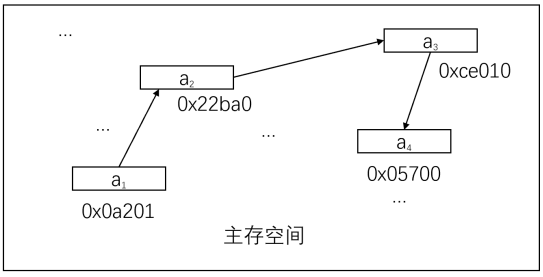


图 2.3 链式存储的线性表

的数据插入某个链表中，请你尝试写出 C 语言代码。后面我们会给出更一般化的代码。

2.2 顺序表

经过上面的介绍我们可以看出，顺序表的优点在于：①支持随机访问，访问表中任意元素可以 O(1) 时间实现；②几乎无需额外存储空间存储额外信息，不像链表那样需要额外存储 n 个指针。顺序表的缺点也很明显：插入或删除表中元素时需要移动大量元素。

接下来我们给出顺序表常见操作的代码实现，通过代码大家也可以进一步体会这一点。对于一个未初始化的顺序表 L：

```
struct SeqList L;
```

1. 顺序表的初始化操作

顺序表中通过一个整数变量 current_size 来维护当前表长（表中已有元素数量）。初始化时，将其置为 0：

```
void InitList(struct SeqList *L) {
    L->current_size = 0;
}
```

2. 顺序表的插入操作

以下函数实现了操作：在顺序表 L 中，第 index 个位置插入一个值为 value 的元素。index=1 表示在表头插入。

```
    int InsertSeqList(struct SeqList *L, int index, ElemType value) {
    if (L->current_size >= MaxSize) {
        return 0; // 表已满
    }
    if (index < 1 || index > L->current_size+1) {
        return -1; // 插入位置非法
    }
    // 移动元素，为待插元素“腾出位置”
    for (int i = L->current_size; i >= index; i--) {
        L->data[i] = L->data[i - 1];
    }
    // 在第 index 个位置插入新元素，即数组下标为 index-1 的位置
    L->data[index-1] = value;
    L->current_size++;
    return 1;
}
```

插入过程如图 2.4 所示。对于顺序表，要在 index 位置处插入新元素，就要将大

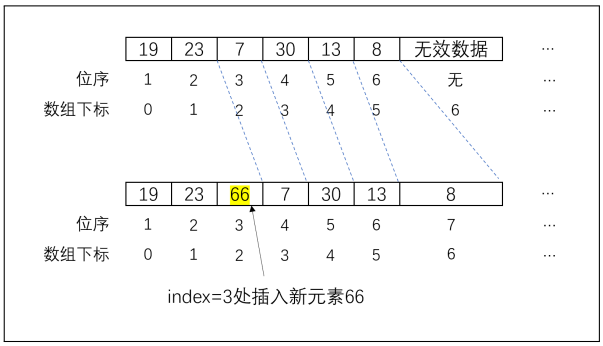


图 2.4 顺序表的插入操作

量元素依次向右推移一个位置，为新元素腾出位置。显然该算法的瓶颈在于元素移动。不难得出该算法最好情况下（在表尾插入，无需移动元素）时间复杂度为 $O(1)$。最坏情况下（在表头插入）元素需要移动 n 次，$T(n)=n$，时间复杂度为 $O(n)$。平均情况下，假设在各个位置插入概率相等，则有

$$T(n)=\frac{1}{n+1}\cdot 1+\frac{1}{n+1}\cdot 2+...+\frac{1}{n+1}\cdot n=\frac{n}{2}$$

所以平均情况下，算法时间复杂度为 $O(n)$。

3. 顺序表的删除操作

以下函数实现了操作：将顺序表 L 中第 index 个元素删除，并将其值以 value 变量返回。index=1 表示删除表头元素。

```
int DeleteSeqList(struct SeqList *L, int index, ElemType *value) {
if (index < 1 || index > L->current_size) {
    return -1; // 删除位置非法
}
// 将被删元素赋值给变量 value
if (value != NULL) {
    *value = L->data[index-1];
}
// 用第 index+1 个元素（如果有）覆盖第 index 个元素，并将后面所有元素“前移一格”
for (int i = index-1; i < L->current_size - 1; i++) {
    L->data[i] = L->data[i + 1];
}
L->current_size--;
return 1;
}
```

与插入操作的思想类似，显然删除操作的时间复杂度同样为 O(n)。

初见提示

事实上绝大多数时候我们关注的都是平均复杂度，因此，在未指出“最好”“最坏”或“平均”时，我们提到的“时间复杂度”或“空间复杂度”都是指平均。

*** 思维拓展**

相信这里已经是一个非常合适的位置来指出这几件事：

1. 顺序表插入操作的实现代码中，我们规定表满时返回 0，插入位置非法时返回-1，插入成功时返回 1。为什么要这么规定？为何元素的位序 index 从 1 开始算，而不是从 0？为何插入操作名字叫做 InsertSeqList？叫别的名字可以吗？
回答：事实上，这些并非死规定，而是由该函数的实现者来定义的。如果你来实现这个函数，你当然可以做出其他合理规定，如：“只要插入失败都返回 0，只要插入成功就返回 1”，再如你将函数名起为“SeqListInsert”，这都是可以的，只要你正确地实现了该函数，并给出清晰的说明让该函数的使用者知道这个函数具体怎么用就行。类似的事情后面将不再赘述。

2. 对于顺序表的删除操作，你也许会问，“删除操作”不就是删掉元素吗？为何还要将被删元素的值返回？这是因为在实际需求中，我们删除一个元素往往需要同时拿到它的值。比如，若银心出队去营地休息，我们将她从玩家队伍中移出，可能同时需要将她放入营地，这就需要获取“银心”这一数据元素，因此删除的同时要返回该数据。你当然可以将删除操作实现为“只删除，不返回被删元素”，只要你实现得正确。但正如 1.1 节末尾所指出，本书中每一种数据结构，我们给出的都是较常见的、一般的实现方式。这一点后面将不再赘述。

说到这里是时候指出《数据结构》的一个重要思想，请看下面的“3”。

3. 你在解决实际问题时（很可能是某个 OJ 网站上的问题），在你玩转某个数组的时候，你顺着自己的思路写出的很可能其实就是顺序表的插入、删除等操作，而若逻辑不长，你决不会把它封装成一个函数类似 InsertSeqList(...)。但是，《数据结构》的研究在于抽象和封装。实际的工程中，你也可以将某个结构及其对应的操作封装好，并和别人说：“这个结构是一棵平衡二叉树，它支持操作 123456 等、作用分别是 xxxxxx，你可以直接拿去用（而无需关注其底层是怎么实现的，因为我已经帮你实现好了）”。当然，你最好给出每个接口实现（函数实现）的详细说明，如时间、空间复杂度，各个返回值表示的含义等等。相信随着对《数据结构》的研究，你将逐步加深对该思想的体会。

4. 按位序访问/修改元素

如前所述，顺序表的存储特性决定了它支持随机访问，我们可以直接根据数组下标来访问元素。

```c
// 获取位序为 index 的元素
int GetElem(struct SeqList *L, int index, ElemType *value) {
    if (index < 1 || index > L->current_size) {
        return -1; // 访问失败
    }
    *value = L->data[index-1];
    return 1;      // 访问成功
}
// 将位序为 index 的元素修改为 new_value, 旧值通过 value 返回
int SetElem(struct SeqList *L, int index, ElemType new_value, ElemType
↪ *value) {
    // 此处略去合法性检查

    *value = L->data[index-1];
    L->data[index-1] = new_value;
    return 1;      // 访问成功
}
```

显然该操作时间复杂度为 O(1)。

5. 按值查找元素

以下函数实现了操作：返回表 L 中第一个值等于 value 的元素所在位序。若不存在，返回-1.

```c
int SeqListFind(struct SeqList *L, ElemType value) {
    for (int i = 0; i < L->current_size; i++) {
        if (L->data[i] == value) {
            return i+1;
        }
    }
    return -1;
}
```

显然该操作时间复杂度为 O(n)。在后面我们将会看到，若顺序表中元素按从小到大或从大到小有序，我们还可以使用二分查找提高查找效率。

到这里我们给出了顺序表的五种常见操作。当然你还可以为它定义更多的操作，如“获取表长”、“判断表是否为空”等操作，感兴趣的同学可以自己尝试实现。

2.2.1 从使用者视角理解“数据结构”

以上我们以“实现者”的视角提供了顺序表这一数据结构及其操作的实现。有了以上实现，我们就有了一个具有基本功能的顺序表可以使用。接下来我们进入“使用者”的视角，来看一看我们可能会如何使用一个顺序表。我们依然以游戏设计为例。假设

你设计的游戏玩法是玩家操控一个四人小队，“出征队伍”最多四人，而你可以随时让“出征队伍”中的队友去营地休息，也可以随时让营地中的队友加入“出征队伍”。

在你编写代码时，程序启动之初你可能会创建一个用来保存“出征队伍”的顺序表，以及一个用来保存“营地队员”的顺序表并初始化：

```
#define MaxSize 4
struct RoleSeqList {
    struct Role roles[MaxSize];
    int current_size;
}
RoleSeqList expedition_team; // 出征队伍列表
RoleSeqList camp;    // 营地中的队员列表
InitList(expedition_team);
InitList(camp);
```

游戏之初出征队伍可能只有主角“邪念”一个人，最初你将邪念加入表中：

```
// 假设游戏默认的主角初始数据已经保存在某 Role 类型的对象"XieNian" 中。
InsertSeqList(expedition_team, 1, XieNian);
```

而营地中暂时没有队友。随着游戏进行，由于你的好人缘，可能不断有队友“归顺于你”：

```
int result = InsertSeqList(expedition_team, expedition_team.current_size+1,
↪  YinXin);    // YinXin 入队
if(result == 0) {
    // 可以在此提示玩家：你的小队最多只能有四个人
} else if(result == -1) {
    // 错误处理，据实际情况而定
}
result = InsertSeqList(expedition_team, ...);   // 其他人入队
// ...
```

当你的出征队伍种有了一定数量的人员之后，假设某时刻玩家可能选择让位序为 index 的人出队并且去营地休息：

```
int result = DeleteSeqList(expedition_team, index, value);  // 位序为 index
↪  的人出队
if(result != 1) {
    // ...
}
```

```
int result = InsertSeqList(camp, camp.current_size, value); // 把这个人放入营
↪ 地
if(...) {
    ...
}
```

以上便是我们对“顺序表”这一数据结构可能的使用。事实上，游戏中的“出征队伍”和“营地”只是我们为了实现游戏玩法而定义的两个概念。对于顺序表的实现代码来说，它们并没有什么特殊之处：出征队伍是一个顺序表，营地也是一个顺序表。顺序表并不知道自己保存的 Role 是何含义，也不知道这些“Role”为什么需要在两个列表之间移动。它只负责提供一组通用的操作，例如插入、删除、查找和访问。

这正是数据结构的价值所在：我们是在设计一种种通用的数据组织方式，然后在不同的实际问题中重复使用它。

在这个例子中，我们甚至可以进一步把游戏中的具体概念暂时抛开。无论顺序表中存储的是游戏角色、商品，还是其他任何类型的数据，对于顺序表本身而言，它们都只是一个个数据元素。只要这些元素满足顺序表的基本要求，我们就可以使用同一套数据结构及其操作来管理它们。因此，从使用者的角度来看，数据结构实际上是一种可以被反复复用的通用工具。它将“如何组织数据”以及“如何操作这些数据”封装起来，使上层程序可以直接利用这些能力（如直接调用 InsertSeqList(camp, camp.current_size, value)），而不必重复考虑底层的实现细节。这也体现了计算机这门学科一个很重要的思想：“封装”。

至此，相信你已经对《数据结构》这门学科“到底在研究啥”已经有了一个很深刻的认知。此后，除非必要，我们将不再强调类似内容，而专注于《数据结构》本身的研究上。

2.3 链表

在 2.1 节中我们已经给出了链表存储结构的定义：

```
struct ListNode{
    ElemType data;  // 该节点的数据元素
    struct ListNode* next_node; // 下一个节点的地址
};
typedef struct ListNode* LinkList;
```

链表的节点通常是通过 malloc 或 new 动态分配的，因此逻辑上相邻的节点物理存储上未必相邻。如前所述，我们有了上述定义，就可以像定义一个 int 变量一样定义一个链表对象 my_linklist。我们首先给出链表的初始化操作，对于一个定义了但尚未

初始化的链表 my_linklist：

```
LinkList my_linklist;  // 此时 my_linklist 指向未知数据。再次提醒：这个
↪  my_linklist 语义上既可以表示“一个链表”，也可以表示“该链表的第一个节点指针”
```

1. 链表的初始化操作

初始化一个空的链表，只需要将指向第一个节点的指针置为 NULL 即可：

代码 2.3 不带头节点的链表初始化

```
int InitLinkList(LinkList* L) {
    *L = NULL;
    return 1;
}
```

调用

```
InitLinkList(&my_linklist);
```

即可对 my_linklist 这个链表进行初始化。如图 2.5 a）所示。函数将链表第一个节点的指针 my_linklist 置为 NULL，表示此时表中没有数据元素。事实上，为了操作

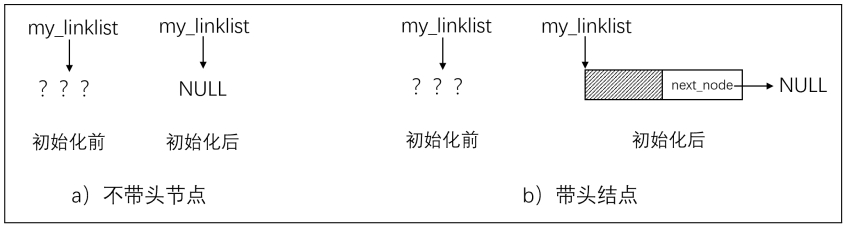


图 2.5 链表初始化前后示意

上的方便，我们在实现链表时，通常在表的第一个有效的数据节点之前附加一个节点，称为**头节点**，如图 2.6 所示。头节点同样可以用 ListNode 结构来定义，只不过这个节点不保存有效数据（data 字段可以不填值），而仅仅是链表的实现者为了代码简单易实现、不出错而额外引入的一个辅助节点。

提示

（特别是北方的朋友）请注意，对于带**头节点**的链表，头节点确实等同于“头一个节点”，即第一个节点。而对于不带**头节点**的链表，不能把第一个节点叫做“头节点”，“头节点”是一个专有名词，而不是表达“头一个节点”的语义。

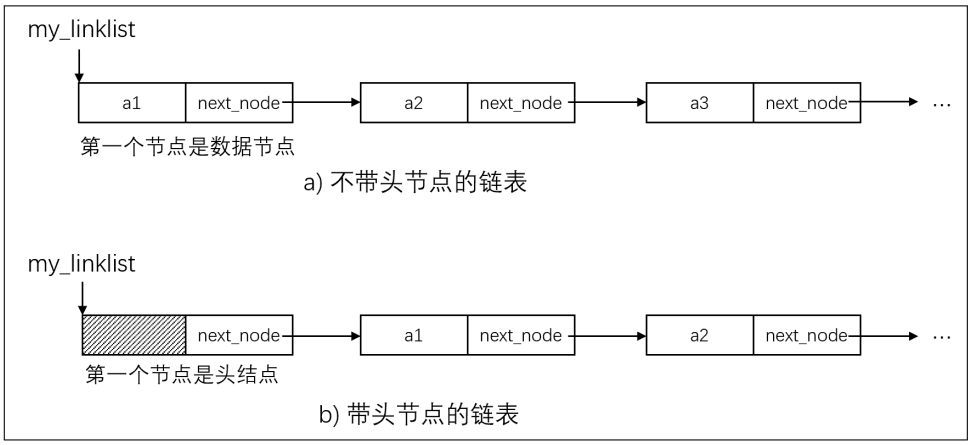


图 2.6 带头节点与不带头节点的链表

引入了头节点后，头节点即第一个节点，因此对于带头节点的链表，第一个节点的指针就是指向头节点的指针。带头节点的链表初始化操作代码如下：

代码 2.4 带头节点的链表初始化

```
int InitLinkList(LinkList* L) {
    *L = (ListNode*)malloc(sizeof(ListNode));     // 创建头节点
    if(*L == NULL) {
        return 0;
    }
    (*L)->next_node = NULL;
    return 1;
}
```

初始化过程如图 2.5 b）所示。

提示

在代码 2.3 及代码 2.4 中，LinkList 本身类型是 ListNode*，因此 LinkList* 的类型是 ListNode**, 是一个二级指针。所以 *L 类型是 ListNode*，即指向某链表节点的指针。

要在函数栈内改变栈外普通变量的值就要用其指针作为参数传递。而栈外的变量 my_linklist 本身是一个指针，因此要在函数栈内改变其值，就要用指针的指针。

如果你熟悉 C++，就可以使用 C++ 中的引用传参来解决多级指针带来的理解上的问题。后续内容中在一些地方我们将使用引用降低语言本身带来的理解负担，这样可以将更多精力专注于《数据结构》本身的思想。

由于带头结点的链表更常用，如无特殊说明，本节中均默认链表带头结点。

2. 销毁链表

销毁链表与初始化相对应。销毁链表需要释放每个节点的内存空间。

```
1  void DestroyLinkList(LinkList* L) {
2      struct ListNode* p = *L;     // p 始终指向当前待释放位置
3      struct ListNode* q;
4      while (p != NULL) {
5          q = p->next_node;        // q 是下一个待释放位置
6          free(p);
7          p = q;
8      }
9      *L = NULL;
10 }
```

这里需要注意，第 7 行不能写成“p = p->next_node”。因为 p 在第 6 行已经被释放了，再访问 p 会导致未定义行为。这也是为什么要先用 q 保存 p 的值。

3. 判断链表是否为空

由于我们采用的是带头结点的设计，因此第一个节点的 next_node 指针指向的才是实际的第一个数据元素节点。

```
1  int ListIsEmpty(LinkList L) {
2      return L->next_node == NULL;
3  }
```

4. 求链表长度

求链表长度的操作是计算链表中数据节点的个数。对于链表，我们想要求得表长就需要遍历一遍链表，每访问到一个有效的数据节点就将计数器加一，最后得到表长。

```
1  int GetLength(LinkList L) {
2      int length = 0;
3      struct ListNode* p = L->next_node;
4      while (p != NULL) {
5          length++;
6          p = p->next_node;
7      }
8      return length;
9  }
```

显然求链表长度操作的时间复杂度为 O(n)。另外请注意，链表长度是不包括头结点的，这一点很好理解。

*** 思维拓展**

事实上，我们可以对 LinkList 类型做进一步的封装，类似：

```
struct WrappedLinkList {
    LinkList L;
    int elem_num;   // 额外增加的变量
};
```

通过额外维护一个变量 elem_num 实时记录当前链表的元素数量，链表初始化时该变量赋值 0，插入时加一，删除时减一。这样可以做到 O(1) 时间求链表的长度。熟悉 C++ 的同学可以知道，C++ 标准库中的 std::list 就是采用了类似的实现方式，所以 std::list 的 size() 方法时间复杂度为 O(1)。

5. 在第 i 个位置插入元素

在链表 L 中的第 i 个位置插入一个值为 e 的新节点 s，需要断开原来的链条，将新元素“链接上”，其过程如图 2.7 所示。实现代码如下。

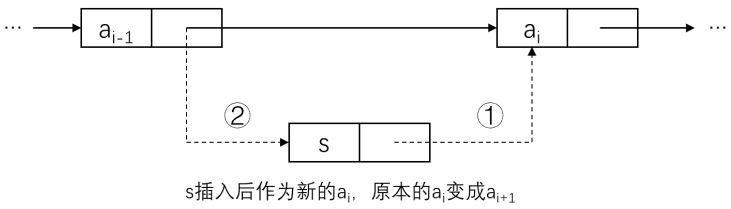


图 2.7 链表的插入过程示意

```
int ListInsert(LinkList L, int i, ElemType e) {
    struct ListNode* p = L;
    // 找到第 i-1 个节点
    for (int j = 1; j < i; j++) {
        if (p->next_node == NULL)
            return 0;   // 插入位置有误
        p = p->next_node;
    } // 该循环结束后，p 指向第 i-1 个节点 a_{i-1}
    // 创建新节点
    struct ListNode* s = (struct ListNode*)malloc(sizeof(struct ListNode));
    s->data = e;
    // 插入节点
    s->next_node = p->next_node;     // ①
    p->next_node = s;                // ②
    return 1;
}
```

可以看到在第 i 个及第 i-1 个节点之间插入，要操作链条，就要拿到这两个节点的信息。这也是该算法的主要开销：找到这两个节点的指针。不难看出该算法时间复杂度为 O(n)。通过该例大家也能出头结点所带来的好处之一——简化边界处理：即使插入位置是表首，也不需要特殊处理。

初见提示

相信你发现了，代码第十行我们分配内存后没检测是否分配成功。在之后的诸多场景，我们将略去类似代码，而专注于算法的核心思想上。

6. 删除第 i 个元素

同插入操作的思想类似，删除操作的核心同样是链条操作，如图 2.8。

```
    int ListDelete(LinkList L, int i, ElemType* e) {
    struct ListNode* p = L;
    struct ListNode* q;
    // 找到第 i-1 个节点
    for (int j = 1; j < i; j++) {
        if (p->next_node == NULL)
            return 0;
        p = p->next_node;
    }
    if (p->next_node == NULL)
        return 0;
    q = p->next_node;
    if (e != NULL)
        *e = q->data;
    // 删除节点
    p->next_node = q->next_node;
    free(q);
    return 1;
}
```

显然该算法时间复杂度同样为 O(n)。

7. 按位置获取元素

由于链表的存储结构，其无法直接通过访问位置 i 计算得到元素的地址，而只能从第一个节点顺着链条一个一个往下找。代码实现如下：

```
int GetElem(LinkList L, int i, ElemType* e) {
    struct ListNode* p = L->next_node;
    // 控制指针 p 始终指向第 j 个数据节点，j==i 成立时，循环结束，p 指向第 i 个
    ↪  节点，即要找的节点
```

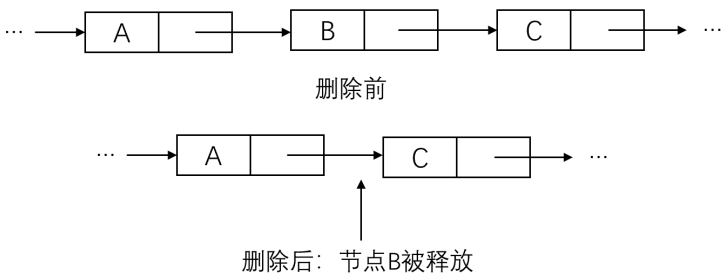


图 2.8 链表的删除过程示意

```
    for (int j = 1; j < i; j++) {
        if (p == NULL)
            return 0;
        p = p->next_node;
    }
    if (p == NULL)
        return 0;
    *e = p->data;
    return 1;
}
```

前面我们说链表只能顺序访问（sequential access），而不能随机访问（random access），这里我们也从代码的角度进一步认识了这一点。并且同时我们不难看出，要修改位序为 i 的节点，思想和该 GetElem 函数非常相似，只不过把获取元素变成了修改元素。

至此，我们已经对链表进行了较为深入的探讨。请大家注意，以上我们所谈的“链表”也可以叫做**“单链表”**，因为它只有由前指向后的一条“单链”，而没有由后指向前的链条。事实上，链表还有多种变态，用于灵活地适应不同需求，我们将在接下来的章节中展开讨论。

2.4 顺序表与单链表的对比

顺序表和单链表都是线性表的常见实现方式，它们都能够表示一组具有线性关系的数据。二者最大的区别在于数据的存储方式不同：

- 顺序表使用一段连续的存储空间保存数据，元素之间通过下标表示逻辑关系；
- 链表通过指针将多个离散存储的结点连接起来，元素之间通过指针表示逻辑关系。

这种存储方式的不同，决定了二者具有不同的特点。

2.4.1 随机访问

由于顺序表中的元素连续存储，因此可以通过下标直接计算元素地址，访问任意位置的元素只需要 $O(1)$ 时间。因此，顺序表适合需要频繁随机访问的场景。

而单链表中的结点在内存中通常不是连续存储的，无法根据位置直接计算地址。如果想访问第 i 个元素，必须从第一个结点开始依次遍历，因此时间复杂度为 $O(n)$。

2.4.2 插入和删除

顺序表在插入或删除元素时，通常需要移动若干元素。因此，顺序表的插入和删除操作平均时间复杂度为 $O(n)$ 。

而链表中的结点通过指针连接。插入元素只需要修改相关结点的指针关系，就可以完成插入或删除操作，不需要移动其他元素。在已经拿到节点指针的情况下，单链表的插入操作时间复杂度为 $O(1)$，但删除操作的时间复杂度仍然为 O(n)。即使没提供节点指针，而只提供了插入位序 i：ListInsert(LinkList L, int i, ElemType e); 此时虽然插入的时间复杂度为 O(n)，但是该算法的主要开销在于找到插入位置，主要操作是“比较”，而非“移动”，因此虽然都是 O(n)，但链表的插入或删除操作也比线性表更快。

2.4.3 空间利用

顺序表需要一段连续的存储空间，并且通常需要提前申请一定容量。如果申请空间过大，会造成浪费；如果空间不足，则可能导致一些后果。

而链表不要求连续存储空间，可以根据需要动态创建结点，因此空间利用更加灵活。但是，链表中的每个结点需要额外保存指针信息，因此会增加额外的存储开销。

2.4.4 如何选择

顺序表和链表没有绝对的优劣，它们分别适用于不同的问题场景。二者总结如下表：

	顺序表	链表
存储方式	连续存储	离散存储
随机访问	$O(1)$	$O(n)$
插入删除	$O(n)$	$O(1)$ 或 $O(n)$，但更快
空间利用	不灵活	动态申请, 较灵活
额外空间开销	较小	需要保存指针, 较大

因此：

如果程序需要频繁访问任意位置的数据，顺序表通常更加合适；

如果程序需要频繁进行插入和删除操作，链表通常更加合适。

...

通过本节内容也给了我们一个启示：数据结构的设计，甚至很多工程上的设计，都是在不同需求之间进行权衡，一种设计提高了某方面的表现，通常也会付出其他方面的代价。我们要在不同需求场景下灵活地进行取舍。

* 思维拓展

请你思考这样一个问题：遍历线性表，如以下伪代码：

```
void TravelList(List L) {
    for( every data in L ) {
        do_something();
    }
}
```

顺序表和链表哪个更快？

我们先来看二者的遍历代码。顺序表的遍历：

```
for (int i = 0; i < n; i++) {
    visit(arr[i]);
}
```

每次访问 arr[i] 前，要计算其地址：arr[i]=arr[0]+(i-1)*sizeof(ElemType). 之后访问内存；链表的遍历：

```
Node* p = head;
while (p != NULL) {
    visit(p->data);
    p = p->next;
}
```

可以看到，链表的遍历是没有乘法操作的。从硬件角度讲，乘法操作是一种很慢的操作。那是不是链表的遍历就比顺序表快呢？并不是。

我们还要考虑一点：现代计算机有一个非常重要的机制：**Cache（高速缓存）**。大家如果了解过《计算机组成原理》或《操作系统》相关内容应该已经恍然大悟了。

什么是 Cache（高速缓存）？这其实也是这门学科一个非常重要的思想，操作系统的实现、高并发服务的架构设计等许多场景下都有 Cache 的思想。它基于一个经验性规律，叫做**程序的局部性原理**：

程序在运行过程中，访问的数据和指令往往不是随机分布的，而是在一段时间内集中访问某些

局部区域。具体来说：

- 如果一个数据或指令最近被访问过，那么它近期很可能再次被访问。
- 如果一个数据被访问，那么它附近的数据近期也很可能被访问。

Cache 的设计思想就是，将部分“近期可能频繁被访问的数据”装入更高速的设备中，来提升程序的运行性能。举个例子，某大型 3D 游戏的地图非常大，有 10 个 GB，程序并非会将所有地图数据都装入主存，而是只装载玩家所在位置附近的地图，玩家移动到新区域或进行传送等操作，再进行主存和磁盘的“数据交换”，将新区域的地图装入主存。在这个场景下，我们可以称“主存”是“磁盘”的“Cache（高速缓存）”。

事实上，现代计算机都会有比内存更高速的缓存，通常称作 CPU Cache。根据局部性原理，将近期常用的指令和数据从主存装入 CPU Cache，来提升程序的运行性能。说到这里，大家应该发现了，对于顺序表，由于其物理连续的存储特性，它的“局部性”是更好的。同一个顺序表极有可能同时被装入 CPU Cache。而链表则不同，遍历链表时，会发生频繁的 CPU Cache 和主存的“数据交换”，这种“抖动”会带来极大的性能损耗。

顺序表虽然存在乘法操作，但其局部性更好，而且，现代编译器通常会做出非常棒的编译优化，对于顺序表的遍历代码，可以将乘法操作带来的影响降得很低很低。所以，对于遍历操作，顺序表和链表哪个更好，这个问题没有统一的结论，但通常我们认为，在数据量较大时，顺序表的遍历是更快的。

习题 1 （参考答案见习题 1 参考答案，第 198 页）

1. 关于线性表，下列说法正确的是（ ）。

A. (银心, 甘尔, 张三) 和 (甘尔, 银心, 张三) 是两个相同的线性表

B. 线性表是一种物理结构，线性表的所有元素物理地址一定连续

C. 同一个线性表中各个元素的数据类型可以不同

D. 不同线性表的数据元素类型可以不同

2. 对于顺序表，假设表中已有 n 个元素，下列操作中时间复杂度一定为 $O(1)$ 的是（ ）。

A. 在表中某位置插入一个元素

B. 删除表头元素

C. 获取第 i 个位置上的元素

D. 查找值为 x 的元素

3. 对于一个带头结点的单链表：

head → A → B → C → NULL

若 p 指向节点 A，现在希望在 A 后面插入节点 X，正确的核心操作是（ ）。

A. p->next_node = X; X->next_node = p->next_node;

B. X->next_node = p; p->next_node = X;

C. X->next_node = p->next_node; p->next_node = X;

D. p = X; X->next_node = p->next_node;

4. 以下单链表的操作中，时间复杂度一定是 O(1) 的是（）。

A. 求表长

B. 在表头插入元素

C. 在表尾插入元素

D. 获取第 i 个位置上的元素

5. 在代码 2.2（第 10 页）中，我们存储顺序表元素使用的数组是静态存储的，定义其最大存储数量为 100. 若存满了，则无法再扩充。事实上，我们可以使用 malloc 动态分配数据元素的存储空间，而只保存一个该空间的起始地址指针：

```
1 struct SeqList{
2     ElemType* data;          // 数组起始地址
3     int current_size;        // 当前表中的元素数量
```

```
    // ...
};
```

请你想一想，动态存储下如何初始化一个顺序表？对于插入操作，动态存储和静态存储会有什么不同？

6. 假设数组 a 中下标 0-9 处存放了 10 个 int 型数据 a_1-a_{10}，请你用数组 a 中的数据分别构建以下两个链表，填充对应代码：

1）$a_1 \rightarrow a_2 \rightarrow ... \rightarrow a_{10}$

```
LinkList f1(int a[], int n=10) {

}
```

2）$a_{10} \rightarrow a_9 \rightarrow ... \rightarrow a_1$

```
LinkList f2(int a[], int n=10) {

}
```

建议采用带头结点的实现。

7. 请你实现一个算法 DeleteValue(LinkList L, ElemType e)，删除带头结点的单链表 L 中所有值等于 e 的节点。

8. 假设线性表采用带头结点的单链表保存，结点定义如下：

```
typedef struct node
{
    int data;
    struct node *next;
}NODE;
```

请你设计一个尽可能高效的算法，对于任意输入的链表 L，都能返回其中间节点。当表长为奇数时，返回中间的那个节点；当表长为偶数时，返回中间两个节点中左边（位序更小）的那个；当表为空表时，返回空指针 NULL。

9.

1）请你设计实现将带头结点的单链表逆置的算法。即：将

$$\text{head} \to a_1 \to a_2 \to \ldots \to a_n$$

变为

$$\text{head} \to a_n \to a_{n-1} \to \ldots \to a_1$$

2）设计实现将不带头结点的单链表逆置的算法。

10. （某年 408 真题）设线性表：

$$L = (a_1, a_2, a_3, \cdots, a_{n-2}, a_{n-1}, a_n)$$

采用带头结点的单链表保存，链表中的结点定义如下：

```
typedef struct node
{
    int data;
    struct node *next;
}NODE;
```

请设计一个空间复杂度为 $O(1)$ 且时间上尽可能高效的算法，重新排列 L 中的各结点，得到：

$$L' = (a_1, a_n, a_2, a_{n-1}, a_3, a_{n-2}, \cdots)$$

要求：

1）给出算法的基本设计思想；

2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释；

3）说明所设计算法的时间复杂度。

waypoint-6

11. C++ 标准库为我们封装好了某个数据结构：vector<>。这是一个不定长数组，你可以在尖括号中定义你想要存放的元素类型，如 vector<int> 表示“元素类型为 int 的不定长数组”类型。你可以使用如下代码定义一个空的不定长数组 arr：

```
/* 类似 “int a;” 中 int 是类型，a 是变量名。
这里 vector<int> 是类型，arr 是你定义的变量名。*/
vector<int> arr;
```

已知 vector 下标从 0 开始，即首元素下标为 0；vector 对象支持如下操作：

1） int n = arr.size(); // 返回 arr 中元素的数量并赋值给 n

2） arr.push_back(e); // 在数组尾部加入元素 e

3） int tmp = arr[idx]; // 返回 arr 中下标为 idx 的元素并赋值给 tmp, 若 idx<0 或 idx>=arr.size()，则访问是非法的。

现在给定某不定长数组 arr（其中元素数量大于 0），请你将其中所有值大于 k 的元素按原位序保存在一个新的 vector 中，并返回这个新的 vector。

```
vector<int> FilterGreaterThanK(vector<int> arr, int k) {

}
```

2.5 单链表的常见变态

2.5.1 双链表

单链表的节点只有一个指向其后继的指针，如果我们要访问某个给定节点的前驱，还是要从头开始遍历。很多人会想到，如果我有这样的需求：频繁地访问某个给定节点的前驱节点（例如习题 1 第 10 题），是不是可以在链表的结构定义中增加一个指针 prior_node，指向该节点的前驱节点，类似这样：

```
1 struct ListNode{
2     ElemType data;  // 该节点的数据元素
3     struct ListNode* next_node; // 下一个节点的地址
4     struct ListNode* prior_node;    // 新增：前驱节点的地址
5 };
```

事实上，这种链表就是我们常说的**双链表**，如图 2.9 所示。第一个节点无前驱，最后一个节点无后继，相应指针指向 NULL。

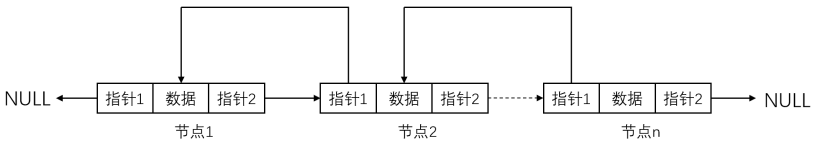


图 2.9 双链表示意图

这样，我们就能以 O(1) 时间立即找到某个给定节点的前驱节点。同时新增的节点也带来的编码上的不同，比如，在插入、删除节点时，要多维护一个指针（一个很有趣的事实是，你以为这是让编码更加复杂，但实际上是更加简单。相信在你填充了代码 2.5 和代码 2.6 之后会深有体会）。

思考

双链表可不可以带头节点？若不能，为什么？若可以，带头结点会有和单链表类似的好处吗？

双链表的以下操作时间复杂度是多少？

1. 插入操作；2. 删除操作；3. 按位序访问；4. 按值访问；5. 求表长。

正如笔者一再指出，对《数据结构》的研究和学习应该是“重在培养能力”。大家通过前面的研究，熟悉了单链表的思想之后，应该能较轻松地写出双链表相关操作的代码。因此，笔者在这里进行第一次“留白”，请大家自行实现双链表操作的代码，并分析时间空间复杂度。

1. 双链表的插入操作

时间复杂度：_____，空间复杂度：_____。

代码 2.5 双链表的插入操作

```
// 在双链表 L 中，第 index 个位置插入一个值为 e 的元素. DL: Double Linked.
    void DLListInsert(ListNode* L, int index, ElemType e) {









    }
```

2. 双链表的删除操作

时间复杂度：_____，空间复杂度：_____。

代码 2.6 双链表的删除操作

```
// 在双链表 L 中，删除第 index 位置的元素。
    void DLListDelete(ListNode* L, int index) {







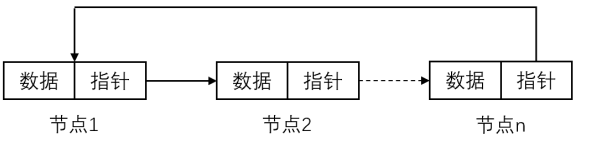


    }
```

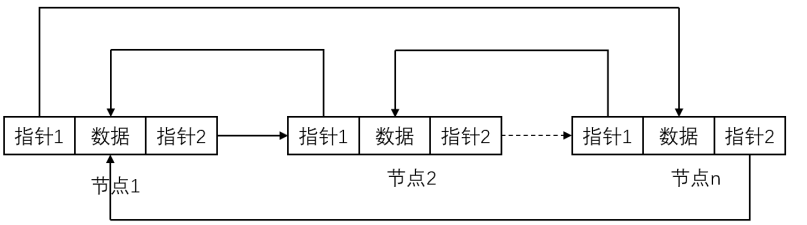
这里笔者仅提示一点：我们在 2.4.2 曾经指出，在已经拿到待删节点指针的情况下，单链表的删除操作的时间复杂度仍然为 O(n)。但是双链表却不同，在已经拿到待删节点指针的情况下，可以达到 O(1) 的时间复杂度。

2.5.2 循环链表

如果让单链表的表尾节点的后继指针指向表首节点，而不是 NULL，这就成为了一个**循环单链表**，如图 2.10 a)；类似地，将双链表的表首节点的前驱指针指向表尾节点，同时将表尾节点的后继指针指向表首节点，而不是 NULL，就成为了**循环双链表**，如图 2.10 b)。可以看出，循环链表由于其循环的特性，给出其任一位置的指针，都可



a）循环单链表



b）循环双链表

图 2.10 循环链表示意图

以顺着该指针遍历链表。在这里，我们指出两点循环链表带来的好处。

1. 与普通双链表相比，循环双链表可以在 O(1) 时间完成尾插。

2. 事实上对于循环单链表来说，保存尾指针（而不是头指针）通常是一种更好的做法，因为有了尾指针 p，即可以通过 p->next 立即找到头指针。并且保存尾指针的循环单链表在表头和表尾插入元素都可以在 O(1) 时间实现。

2.5.3 静态链表

实际需求中，有些场景需要频繁地创建、销毁对象，如某空战游戏，战机频繁地打出子弹，子弹碰撞到某物体后或飞出屏幕外之后消失。这种场景下，如果我们频繁地调用 malloc/free 就会产生较严重的内存分配开销以及内存碎片（相信如果你学习过《操作系统》就会对此有深刻理解）。这种场景：1. 普通链表非常适合频繁插入删除，但链表的动态节点分配机制，在高频创建/销毁场景下带来了额外开销；2. 顺序表又不适合频繁地插入删除。

所以这个时候，**静态链表**就可以派上用场。如图 2.11 所示，静态链表将链表中的所有节点存放在一个数组中。使用数组中的位置编号（下标）来模拟指针，从而实现节

点之间的链接关系。对于单链表，若某节点的后继指针指向 NULL 说明该节点是表尾节点。而静态链表，通常用一个不存在的下标（如-1）表示没有下一个节点了。

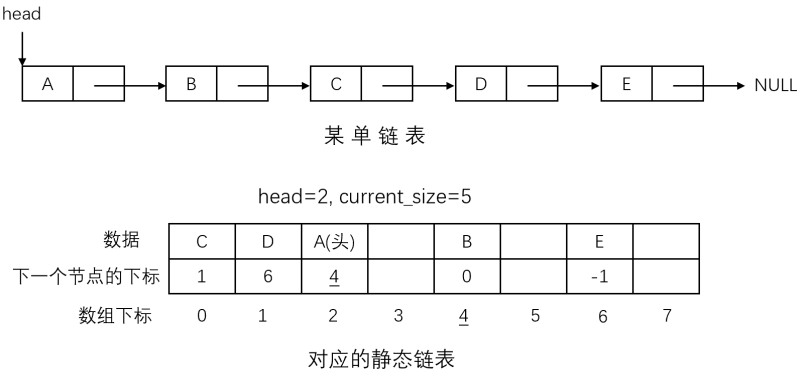


图 2.11 静态链表示意图

静态链表的一个节点的存储结构如以下代码所示：

代码 2.7

```
typedef struct {
    ElemType data;  // 对应图 2.11 中的“数据”
    int next_node;  // 对应图 2.11 中的“下一个节点的下标”
}SLinkListNode;
```

普通链表可以用其第一个节点的指针表示链表对象，对于静态链表，如何知道数组中哪个节点是第一个节点呢？一种做法是进行类似如下的封装：

```
struct SLinkList {
    SLinkListNode data[MaxSize];    // 存放链表中节点的数组
    int current_size;   // 当前链表中的节点数量
    int head;   // 第一个节点所在的下标
}
```

这样，我们便可以通过如下代码初始化一个静态链表：

```
SLinkList L;
L.current_size = 0;
L.head = -1;    // 目前是空表
```

静态链表的其他操作，感兴趣的同学可以自行尝试实现。

2.6 本章小结

“数据结构是相互之间存在一种或多种特定“关系”的数据元素的集合，以及定义在这些数据元素上的“操作”的集合。“数据结构”的物理存储、操作通常通过计算机语言来实现。”相信通过本章的探讨，大家已经能进一步理解这句话的含义。“线性表”作为一种逻辑结构，要求数据元素之间有“位置关系”，每个节点（除表尾节点）有且仅有一个直接后继，每个节点（除表首节点）有且仅有一个直接前驱。而顺序表和链表是线性表的两种不同实现方式，分别将线性表的元素以不同的物理结构存储，并通过不同手段建立元素间的逻辑关系。

笔者再次强调，研究数据结构的核心，并不是掌握某一种数据结构的代码，而是理解如何组织数据、如何分析问题、如何设计解决方案。以链表为例，我们学习的并不仅仅是：

- 链表的定义是什么；
- 链表有哪些操作；
- 创建一个结点的代码是什么；
- 插入和删除操作的代码是什么；

更重要的是理解：

- 为什么需要链表的存在，它能解决什么问题？
- 为什么链表不需要连续的存储空间；
- 为什么它能够高效地完成插入和删除；
- 为什么它无法像数组一样快速随机访问；
- 如何根据问题特点，在灵活性和效率之间进行选择。

因此，学习数据结构时，不要只关注“这个结构怎么实现”（虽然这固然重要），而更应该思考：

- 为什么要这样设计？
- 它解决了什么问题？
- 它付出了什么代价？
- 如果换一种设计，会发生什么变化？

当你能够从这些角度理解一个数据结构时，你掌握的就不再只是链表、栈、队列或者树，而是一种分析问题和设计方案的能力。这也是学习数据结构最重要的意义。

第三章　栈和队列

先说栈。在实际应用中有很多类似栈的数据调度方式。想象冬天你去澡堂洗澡。你脱衣服的顺序一定是从外到内，而穿衣服刚好相反。如图 3.1。

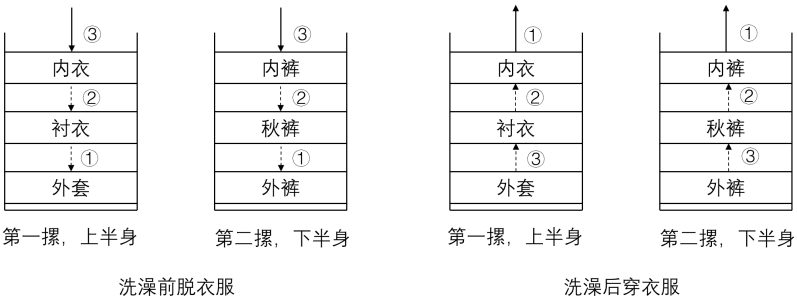


图 3.1

事实上，如果你将两摞衣物放在某个容器里，你会发现这个容器有这样的特点：先放入的在下方，后取出；后放入的在上方，先取出，称为**“后进先出”**，或者叫**“先进后出”**。这样，其实你就是在将这两个容器当成两个**栈**来使用。

而**队列**就很好理解了，正得名于生活中的“队列”。想象某门店很火爆，门前排满了人，队首的人买完东西出队，新来的人入队到队尾。

接下来我们给出完整、严谨的栈和队列的定义。

定义

栈（Stack）是只允许在一端进行插入或删除操作的线性表。

如图 3.2 所示，允许插入或删除的那一端叫做**栈顶**，不允许插入或删除的另一端叫做**栈底**，不包含任何元素的空表叫做**空栈**。栈的插入操作叫做**入栈**，删除操作叫做**出栈**，入栈和出栈通常也更生动地叫做“压栈”和“弹出”。

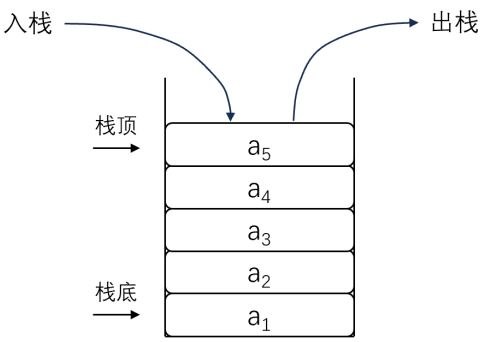


图 3.2 栈的示意图

下面给出**队列**的定义。

定义

队列（Queue）是只允许在其中一端插入、另一端删除的线性表。

如图 3.3 所示，允许出队的那一端叫做**队首**，允许入队的那一端叫做**队尾**，队列的插入操作叫做**入队**，删除操作叫做**出队**；不包含任何元素的空表叫做**空队列**。

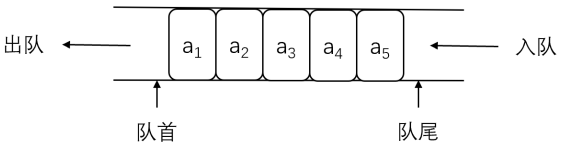


图 3.3 队列示意图

很多人喜欢将栈和队列的特征进行对比，称**队列是先进先出（First In First Out，FIFO）的，栈是后进先出（Last In First Out，LIFO）的。**

以上便是栈和队列这两种逻辑结构的介绍，接下来我们研究如何实现栈和队列。

3.1 栈

上一章中我们研究了顺序表与链表的实现。既然栈是一种特殊的线性表，是不是可以用同样的物理存储来实现栈，并且对操作加以特殊限制呢？笔者建议大家在这里暂停，思考一下这个问题：想象一下，分别用数组、链表该如何实现栈？如何设计是好的？会面临哪些问题吗？

此处应该有长时间留白.....

开始。

相信大多数人的都会有明确而正确的思路：

1. 如果使用顺序表来实现栈，由于顺序表“尾插”、“尾删”均为 O(1) 的特性，可以将“表尾”作为“栈顶”，通过操作的定义和实现来限制该表只能在表尾插入、删除，这样既有高效率，又满足栈的特点。如图 3.4 所示，你可能会想到用一个 int 型变量 top 来动态维护“栈顶”的位置。

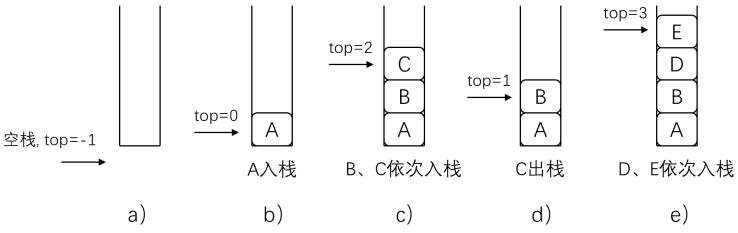


图 3.4 顺序栈实现示意图

这个思路其实就是**顺序栈**的实现思路。采用顺序存储的栈称为**顺序栈**，它利用一组地址连续的存储单元存放自栈底到栈顶的数据元素，同时附设一个指针 top，来指示当前栈顶的位置。

顺序栈的存储结构如以下代码所示：

```
#define MaxSize 100
struct SeqStack {
    ElemType data[MaxSize]; // 存储实际的数据元素
    int top;    // 栈顶指针的下标
};
```

*** 思维拓展**

你也许会发现，这特么不就是线性表么？你不就把 current_size 换成 top 了么？没错，顺序栈和顺序表的关系确实有点暧昧不清。从存储结构角度看，二者存储结构完全相同，只不过语义不同：top 表示栈顶元素的下标，空栈时为-1；而 current_size 表示顺序表中元素的数量，空表时为 0。二者之间微妙的关系在于，current_size 在数值上等于表尾元素（栈顶元素）的下标加一。而如果你将 top 的语义规定为“表尾元素（栈顶元素）的下一个位置”，那么就应将 top 初始化为 0，而不是-1，此时 top 与 current_size 语义虽然不同，但编码上是相同的。

2. 如果使用链表来实现栈，由于链表“头插”、“头删”均为 O(1) 的特性，可以将“表头”作为“栈顶”，通过操作的定义和实现来限制该表只能在表头插入、删除，这样同样既有高效率，又满足栈的特点。事实上，将链表的插入和删除操作限制在头部，即

得到了**链栈**。

接下来我们分别给出顺序栈和链栈常见操作的实现。

3.1.1 顺序栈的常见操作实现

1. 顺序栈的初始化

不难看出，初始化一个顺序栈仅需要将其栈顶指针 top 置为-1。

```
int InitStack(SeqStack* S) {
    S.top = -1;
    return 1;
}
```

对于一个未初始化的顺序栈 S:

```
SeqStack S;
```

就可以通过以下代码对 S 进行初始化：

```
InitStack(S);
```

2. 进栈操作

top 指针指向“当前栈顶元素”，因此进栈时要先移动指针到下一个“更高的”可用位置，再将元素放入该位置，此时指针仍指向栈顶。

```
int Push(SeqStack& S, ElemType e) {
    if(S.top == MaxSize - 1) {
        return 0;   // 栈已满，无法入栈
    }
    S.top += 1;
    S.data[S.top] = e;
    return 1;
}
```

如以下代码便可以为栈 S 压入一个值为 3 的元素：

```
Push(S, 3);
```

初见提示

注意这里我们首次使用了 C++ 的引用作为函数形参。”SeqStack& S” 中，符号”&” 并不是取地址的意思，而是表示形参 S 是一个 SeqStack 的**引用**类型。相信你也看出引用的作用了，类似使用指针一样，使用引用作为形参，同样可以改变函数栈外的原变量的值。关于引用的更多内容感兴趣的同学可以参考 C++ 相关资料。

3. 出栈操作

思想类似进栈，此处留白，请大家自行写出代码。

```
// 将栈 S 中当前的栈顶元素赋值给 e，并将其弹出。
    int Pop(SeqStack& S, ElemType& e) {



    }
```

4. 获取栈顶元素，但不出栈

略。

5. 判断栈是否为空

```
int Empty(SeqStack S) {
    if(S.top == -1) return 1;   // Empty 成立
    return 0;
}
```

3.1.2 链栈的常见操作实现

根据前面的介绍相信你已经看出来了，链栈的实现比链表还要简单：其他操作和链表几乎完全相同，而对于“插”和“删”，你只实现链表的“头插”和“头删”操作，而不实现任意位置的插入删除操作，你就得到了一个链栈。显然链栈也可以采用带头结点和不带头结点的实现。这里拟采用带头结点的实现。

提示

对于带头结点的实现，链表的第一个节点指针 L 指向头节点，L->next_node 指向的是第一个数据元素，即栈顶元素。

1. 链栈的入栈操作（实际上就是链表的“头插”操作）

```
int Push(ListNode* L, ElemType e) {
    ListNode* new_node = (ListNode*)malloc(sizeof(ListNode));
    new_node->data = e;
    new_node->next_node = L->next_node;
    L->next_node = new_node;
    return 1;
}
```

2. 链栈的出栈操作（实际上就是链表的“头删”操作）

```
int Pop(ListNode* L, ElemType& e) {
    if(L == NULL || L->next_node == NULL) {
        return -1;
    }
    e = L->next_node->data;
    ListNode* node_to_free  = L->next_node;
    L->next_node = L->next_node->next_node;
    free(node_to_free );
    return 1;
}
```

显然实现一个栈没有必要使用双链表，更没有必要将首尾链接起来做成循环链表。

由于顺序栈和链栈就是有特殊限制的顺序表和链表，其优劣我们不再赘述，而是作为习题放在习题 2 中。

3.1.3 * 浏览器的前进后退功能实现

这里给出一个作为思维训练、理解栈的意义的极好的例子。大家应该都用过浏览器的“后退”和“前进”功能，如图 3.5。当你按住鼠标右键从右向左滑动，或者点击右键选择“后退”时，浏览器会访问你上一个访问过的网页，向右滑或点“前进”则回到上一个回退前的网页（如果你没用过，建议先打开浏览器试一下）。假设你已经访问过了：

网页 1 -> 网页 2 -> 网页 3

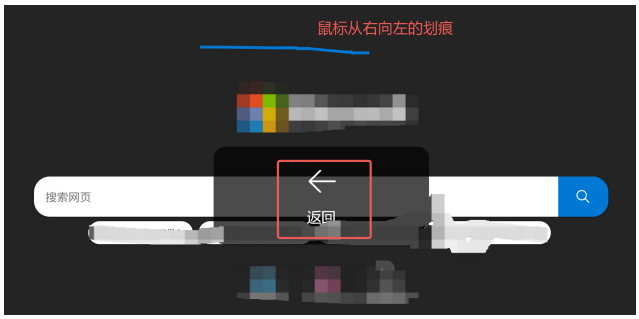


图 3.5

此时你处在网页 3，点击“后退”，会回到网页 2，再后退，回到网页 1，再前进，回到网页 2，再前进，回到网页 3。

假设：

1. 用户随时可能访问一个新的网站；
2. 用户随时可能点击“后退”（鼠标左滑）；
3. 用户随时可能点击“前进”（鼠标右滑）。

如果你是某浏览器的实现者，请你实现这个功能，要求当用户点击“前进”或“后退”时，能迅速地拿到对应网页的地址。

提示：网页地址（如 https://xxxxxx）可以用一个 char 的数组（或 string 类型）来表示。你可以用任何你喜欢的方式来模拟用户的输入，并让你的程序做出合理反应。

此处应该有留白。

具体的模拟方式各有千秋，我们只给出核心算法的实现思路。不难看出，当用户访问了若干页面之后：

```
网页 1 -> 网页 2 -> 网页 3
```

“后退”操作实际上就是依次访问与之前的顺序相反的页面，满足“后进先出”的特点，因此用户每访问一个新网页 B，我们就可以将用户所处在的当前网页 A 压入一个栈 back_stack，当用户进行后退时，取出栈顶网页访问即可。back_stack 语义上可以理解为：为用户的后退行为而预备的一个栈，栈顶元素永远是用户此时点击后退时所要访问的页面。

还有前进操作。“后退”的顺序是与用户主动访问的新网页顺序相反，不难看出，“前进”的顺序是与“后退”相反的。因此可以维护令一个栈 forward_stack，用户未进行过后退操作时该栈为空，当用户连续进行“后退”时，将网页依次压入栈 forward_stack。需要注意：当用户访问了一个新网页后无法“前进”，需要清空 forward_stack 重新期

待用户的下一次后退操作。流程如图 3.6 所清晰地展示。

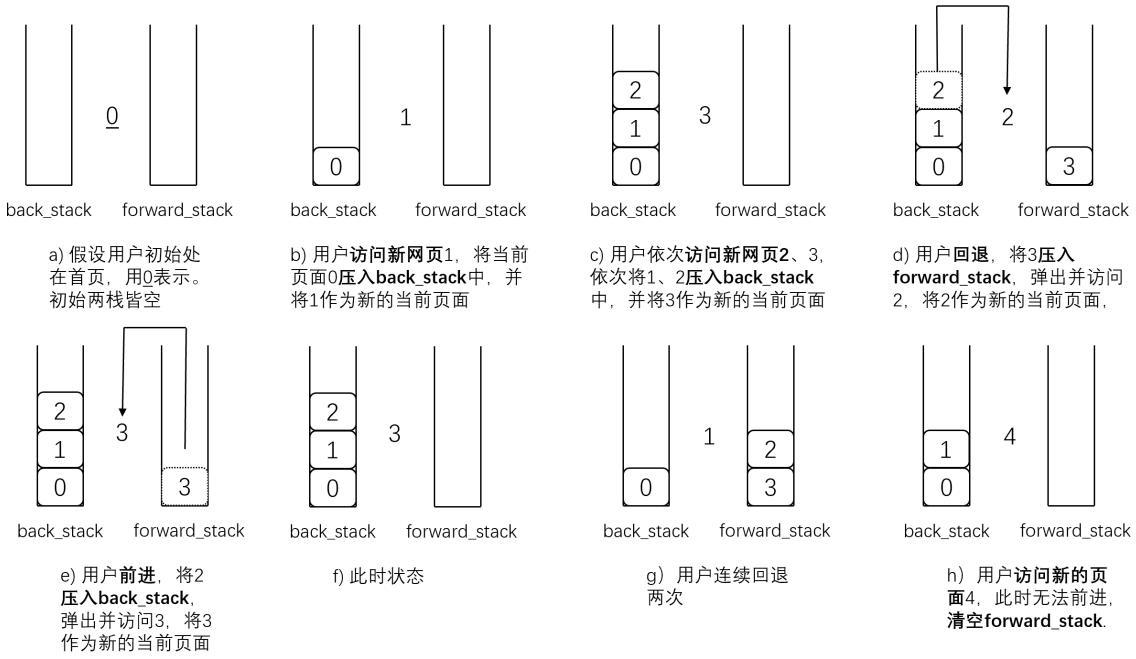


图 3.6 使用两个栈实现浏览器前进后退

3.1.4 *“函数栈”、“调用栈”是啥？

相信大家从未注意但一个很有趣的事实是，函数调用、返回的顺序恰好具有“后进先出（LIFO）”的特点。如：

```
void C() {
    return ;
}
void B() {
    C();
}
void A() {
    B();
}
int main() {
    A();
    return 0;
}
```

程序实际执行的顺序是：main -> A -> B -> C -> B -> A -> main -> 退出。函

数执行结束时，需要返回上层，因此调用一个新函数时，需要将当前函数的上下文信息压入一个记录“函数调用记录”的栈中（是不是有点像浏览器？）。从更底层的开发者角度来看，“调用栈”就是存放这些函数信息的一个栈，栈中的每一个元素是相应的函数上下文信息，如函数参数、局部变量、返回地址等。

但对于应用程序员，特别是 C/C++ 程序员来说，“调用栈”或“函数栈”通常存在其他角度的含义。对于 C/C++ 来说，非静态局部变量存放在栈区，其可访问区域仅在函数内部。如：

代码 3.1

```
int f(int t) {
    int a = 10;
    t = t + a;
}

int main() {
    int x = 2;
    f(x);
    return 0;
}
```

事实上，调用函数时，操作系统通常会为其分配一个栈区空间，其局部变量便存储在这个栈区空间中。main 函数中以 x 为参数调用 f(x)，仅仅是将 x 的值拷贝给了“栈区变量”t。因此，这种场景下我们常称 t 是“f 的函数栈（或调用栈）中的一个栈变量”，此时“函数栈”或“调用栈”的含义便是内存中的“栈区”空间。

提示

事实上，函数的形参可以理解为函数栈中的一个局部变量。如代码 3.1，以 x 为实参调用 f，可以等效理解为如下伪代码：

```
int f() {
    int t;
    t = x;  // 外界传入的 x 的值拷贝给 t
    ...

}
```

这也进一步解释了值传递无法改变函数栈外的原变量的值，而需要通过指针传递。

3.2 队列

如前所述，队列同样是一种特殊的线性表，因此队列同样可以有顺序存储和链式存储两种物理结构，并在顺序表或链表的基础上实现特殊机制。队列的链式存储稍简单些，我们先研究链式存储，再研究顺序存储。

3.2.1 队列的链式存储

队列需要实现两个操作：在链表的头部删除元素、尾部插入元素。单链表头插本身可以达到 O(1)。我们只要额外保存一个链表的尾指针，那么在尾部插入元素的操作也可以达到 O(1)。显然链式队列同样可以带头结点或不带头节点，这里拟采用带头节点的实现。

第二章中已经给出过链表节点的定义，这里为了体现出节点是队列的节点，我们把结构改个名字：

```
struct QueueNode {
    ElemType data;
    struct QueueNode* next_node;
};
```

额外保存一个表尾指针来使出队操作时间达到 O(1)：

```
typedef struct {
    struct QueueNode* front;     // 表头（队首）指针，指向头节点
    struct QueueNode* rear;     // 表尾（队尾）指针，指向最后一个节点
}LinkQueue;
```

以上代码将 LinkQueue 定义为一个“链队列”类型。有了如上定义，我们就可以用如下代码定义链队列对象：

```
LinkQueue happy_linkqueue;  // 定义一个开心的链队列
LinkQueue sad_linkqueue;    // 定义一个悲伤的链队列
```

一个链队列如图 3.7 所示。前面的 happy_linkqueue 以及 sad_linkqueue 都是定义了但未初始化的。接下来我们给出链式队列的常见操作实现（带头结点）。

1. 初始化操作

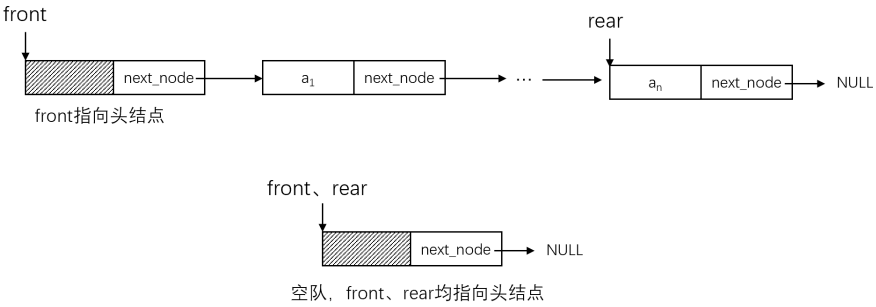


图 3.7 带头结点的链式队列示意图

```
int InitLinkQueue(LinkQueue& q) {
    q.front = (QueueNode*)malloc(sizeof(LinkQueue));      // 为头节点分配空间
    q.rear = q.front;     // 由于插入时要在尾后插入，所以 rear 初始指向头节点符合条件

    q.front->next_node = NULL;
    q.rear->next_node = NULL;
    return 1;
}
```

使用：

```
InitLinkQueue(happy_linkqueue); // 初始化开心的链队列
InitLinkQueue(sad_linkqueue);   // 初始化悲伤的链队列
```

2. 判断是否空队

```
int Empty(LinkQueue q) {
    if(q.front == q.rear) {
        return 1;
    }
    return 0;
}
```

3. 入队操作（尾插）

```
int EnQueue(LinkQueue& q, ElemType e) {
    if(q.front == NULL || q.rear == NULL) {
        return -1;  // 你给我个没初始化的队列我咋给你插入元素？
```

```
    }
    QueueNode* nwe_node = (QueueNode*)malloc(sizeof(QueueNode));
    nwe_node->data = e;
    // 操作链条
    nwe_node->next_node = NULL;
    q.rear->next_node = nwe_node;

    // 更新尾指针
    q.rear = nwe_node;
    return 1;
}
```

4. 出队操作（头删）

一般情况下出队不会改变尾指针的指向，但需要注意一个特殊情况，即队列中仅有一个节点，如图 3.8 所示。此时队首节点就是队尾节点，出队后，队列变成空队，需要维护队尾节点指针，将其指向头节点。

```
int DeQueue(LinkQueue& q, ElemType& e) {
    if(q.front == NULL || q.rear == NULL) {
        return -1;
    }
    if(q.front == q.rear) {
        return 0;   // 空队
    }   // 注意代码若能走出这个 if，说明队中至少有一个数据节点，可以直接安全地访
    ↪  问 q.front->next_node 的 data 域或 next_node 域。
    e = q.front->next_node->data;
    QueueNode* t = q.front->next_node;
    q.front->next_node = q.front->next_node->next_node;

    // 若队列仅有一个节点，更新 q.rear
    if(q.rear == t)
        q.rear = q.front;
    }
    free(t);
    return 1;
}
```

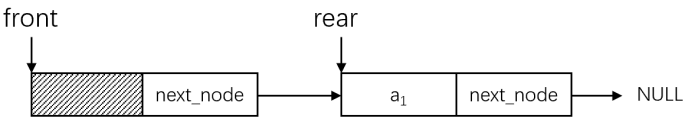


图 3.8 仅有一个数据节点的队列

3.2.2 队列的顺序存储

不难想到，要在顺序存储的线性表上实现一个队列，最简单的做法就是标记两个指针（数组下标）front 和 rear，分别指向队头和队尾，初始时将 front 和 rear 均置为 0 即可，这样我们可以规定 front 始终指向队首元素，rear 始终指向队尾元素的下一个可插入位置。

```
#define MaxSize 100
struct SeqQueue {
    ElemType data[MaxSize];
    int front;  // 队头，初始时为 0
    int rear;   // 队尾，初始时为 0
};
```

如图 3.9，入队时将元素插入到队尾并推移队尾指针，出队时推移队头指针。

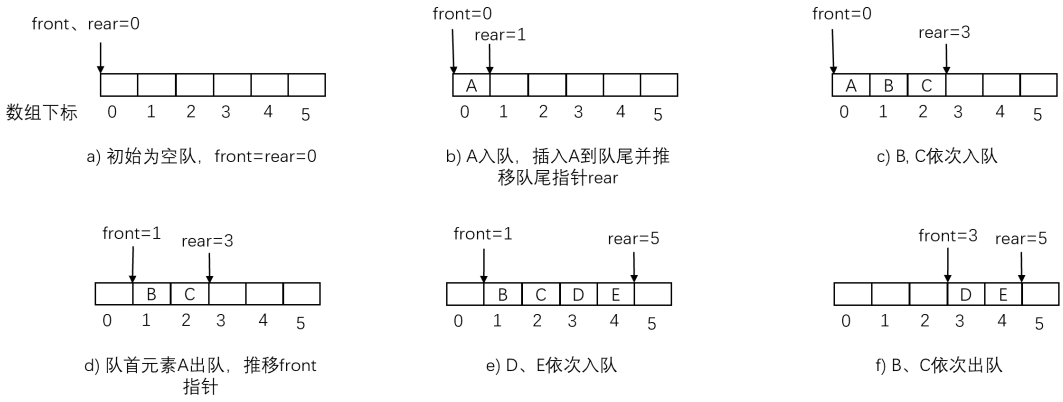


图 3.9 在顺序表上实现队列

然而，一个看似简单的问题很快就会出现：

> 当 rear 已经走到了数组的尾头，而数组的前半部分却明明还有大量空闲空间时，我们还能继续入队吗？

例如，一个容量为 10 的队列，经过几次入队、出队操作后，可能变成：

$$\underbrace{\square\ \square\ \square\ \square}_{\text{已经出队，空闲}}\quad\underbrace{A\ B\ C\ D\ E\ F}_{\text{队列中的元素}}$$

此时数组中明明还有 4 个空闲位置，但如果我们把数组看成一条只能从左向右移动的“道路”，rear 已经走到了道路的尽头。按照普通顺序表的存储方式，我们似乎只能无奈地宣布：

“队列满了。”

可是，真的满了吗？

显然不是。相信聪明的你已经想到了，如图 3.10 所示，当 rear 走到尽头时，我们可以让 rear “回到起点”，继续利用那些已经空闲了的空间，**让 rear 始终“追着 front 跑”，当 rear 真正追上 front 时，确实没有可利用的空间了，此时才是真正的“队满了”**。

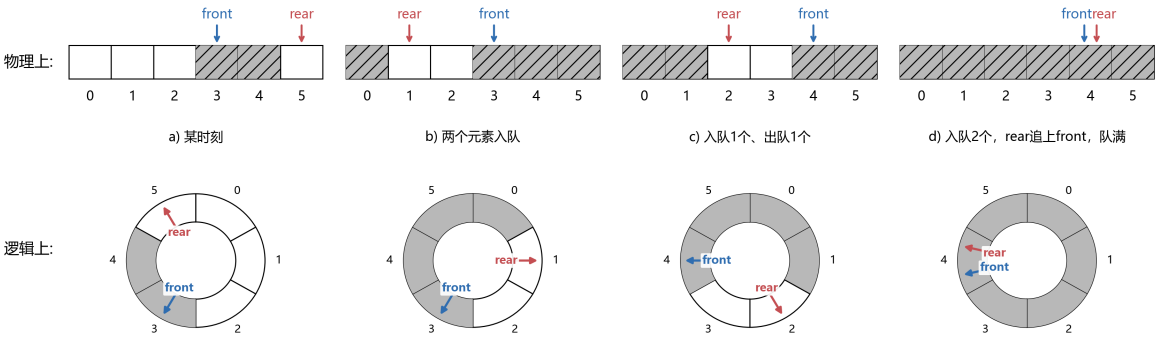


图 3.10 循环队列示意图

这正是**循环队列**所要解决的问题。

循环队列不再把数组看成一条有起点、有终点的直线，而是把数组的首尾连接起来：当指针走到数组末尾时，就“绕回”数组开头，整个数组物理上是一条直线，但逻辑上是一个环，因此叫**循环队列**或**环形队列**。

因此，队列的顺序存储通常都会实现为循环队列。

“回到起点”，代码上如何实现？举个例子某数组大小为 100，下标为 0-99，当 rear 的值已经为 99 时，如果不加以控制，下一步将走到 100，但我们希望它回到 0。不难发现，这正是“取模”运算的功能。因此入队或出队时，若指针从 99 推移到 100，我们将其对 100 取模，即可重新回到起点 0：

```
if(rear > MaxSize-1) {    // 若 MaxSize 为 100，MaxSize-1 为最大下标 99
    rear = rear % MaxSize;
}
```

*** 思维拓展**

在计算机科学中大家往往喜欢将“取模”和“取余”同等对待，但事实上存在负数参与运算时，取模（modulo）和取余（remainder）数学上的定义可能不同。“%”运算符事实上更接近数学上的取余运算。但这里还是用“取模”来描述，原因在于二者在语感上有差异：“取余”关注的是除法“除完还剩多少”；而“取模”关注的是“映射到哪个模类/固定范围”。此处“%”运算

的语义更接近“取模”。

还有第二个问题需要考虑。循环队列如何判断“队空”？可以简单地通过

$$\text{front} == \text{rear} \;\&\&\; \text{front} == 0$$

来判断吗？经过分析发现是不可以的。因为队满的时候，上面的条件也可能成立。如图 3.10 ，对于 d）所示场景，若再进行两次出队、两次入队，就可以达成 front == rear && front == 0 的条件，并且此时队列是满的。如何解决这个问题？相信大家自然地想到的做法是多维护一个字段，动态维护队列中当前元素个数：

```
#define MaxSize 100
struct SeqQueue {
    ElemType data[MaxSize];
    int front;
    int rear;
    int current_size;   // 新增字段
};
```

这当然没问题。但更多教材给出的做法是少利用一个数组空间，通过 (Q.rear+1)%MaxSize == Q.front 来作为队满的条件，如图 3.11 所示。我们这里遵从多数教材给出的做法，

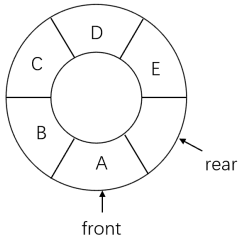


图 3.11 队满示意图

少利用一个数组空间，使用 (Q.rear+1)%MaxSize == Q.front 来作为队满的条件。同时可以看到，在此做法下，除队空时 front 和 rear 初始化为 0 之外，其余情况不可能出现 front == rear 的情况（因为我们少利用一个空间，所以 rear 永远无法追上 front，最多只能在它后面一格）。所以可以将 front == rear 直接作为判断队空的条件。

至此我们已经完成了所有“思维”上的准备，接下来给出循环队列的常见操作实现。

1. 初始化循环队列

```
int InitQueue(SeqQueue& q) {
    q.front = q.rear = 0;
    return 1;
}
```

2. 判断队空

```
int Empty(SeqQueue q) {
    if(q.front == q.rear) {
        return 1;
    }
    return 0;
}
```

3. 入队

```
int EnQueue(SeqQueue& q, ElemType e) {
    if( (q.rear+1) % MaxSize == q.front) {
        return 0;   // 队满，无法入队
    }

    // 先“放入元素”，再“推移指针”
    q.data[q.rear] = e;
    q.rear += 1;
    if(q.rear>=MaxSize) {
        q.rear = q.rear % MaxSize;
    }
    return 1;
}
```

4. 出队

```
int DeQueue(SeqQueue& q, ElemType& e) {
    if(Empty(q)) {
        return 0;   // 空队
    }

    // 先“取出元素”，再“推移指针”
    e = q.data[q.front];
    q.front += 1;
    if(q.front >= MaxSize) {
        q.front %= MaxSize;
```

```
    }
    return 1;
}
```

实际应用中，通常不会有“获取队列中的元素数量”的需求，这里笔者稍作提示：对于循环队列，可以通过下式在 O(1) 时间得到队列长度：

$$(\text{rear}-\text{front}+\text{MaxSize}) \ \% \ \text{MaxSize}$$

而对于链队列，需要求队列长度，则需要遍历一遍队列（链表），时间复杂度为 O(n)。

在解决实际问题时，有时会顺其自然地使用到两端均能入队、出队，或一端能入两端能出、两端能入一端能出的队列，它们通常被称为**双端队列**。**相关内容我们通过习题 2 作为练习。**

3.3 栈的进一步理解

提示：本节内容略有难度。

栈后进先出的特性，决定了其可以应用于一些很有趣的场景。我们这里先给出一般化的理解：许多场景下，可以用栈来保存这样的数据，即：**“那些仍然等待未来某个事件发生，并且优先处理后来者”的数据**。现在看这句话过于抽象，不过别急，本节过后你的心会怦怦直跳。

waypoint-3

我们给出一例：现在假设天气预报已经报导了未来 n 天的温度，保存在一个数组 temperatures[] 中，请你设计一种算法，对于每一天 i，找出其后方第一个温度高于第 i 天的日子。

最直接的办法就是“检查”每一天 i，向后扫描，直到找到温度高于 i 的那一天，之后回到 i+1，重复这个过程，伪代码如下所示：

```
for(int i=0; i<n; i++) {
    int found = 0;
    for(int j=i+1; j<n; j++) {
        if(temperatures[j] > temperatures[i]) {
            // 找到了.
            found = 1;
            // j 就是要找的那一天

```

```
            break;
        }
    }
    if(found == 0) {
        // 第 i 天的后面没有比第 i 天温度更高的了。
    }
}
```

显然这个算法时间复杂度为 $O(n^2)$。分析以上过程我们会发现，事实上我们做了很多查找是“不必要”的。比如，考虑 temperatures[] 输入如下：

25, 23, 19, 18, 20, 27, 26, 27

当我们“检查”25 时，就由前往后查找到了第一个 27 的位置，事实上此时我们就已经能够知道 23 的“答案天”就是 27，并且 19、18 的“答案天”就是 20。只不过，此时我们虽然已经“看到了”这些信息，却没有把它们保存下来。于是，一个很自然的想法出现了：如果我们能通过某种手段将这些**尚未找到答案的天**记录下来，那么未来的温度出现时，就可以立即检查这些等待中的天是否能够得到答案。当我们检查 25 时，将它记下来：

some_structure: {25}

检查 23，some_structure 中没有元素会得到答案，将 23 记下来：

some_structure: {25, 23}

继续检查 19、18：

some_structure: {25, 23, 19, 18}

当检查到 20 时，终于找到了 19 和 18 的答案，19 和 18 两个元素期待的事情终于发生，因此将 19 和 18 从 some_structure 中移出，将 20 记下来：

some_structure: {25, 23, 20}

你会惊奇地发现一件事情：留存在 some_structure 中的温度序列一定是单调不增的，因为低温一定会在高温来到时“找到答案”而被移出。因此当新的温度 t 来到时，它能移出哪些温度？其实只需要依次从 some_structure 的尾部依次向前比较，“清除”掉 some_structure 所有小于 t 的温度即可，这样一定不会漏。

说到这里你会发现，some_structure 正是满足“只会在一端进行插入删除”。或者换个角度想，当新温度 t 到来时，后进入 some_structure 的元素一定要先被检查，因为后进入的一定是更小的，如果都不能被 t “清理掉”，前面的就更不会被清理掉。

这不就是栈吗！

想通了这一点，这个问题就变得非常简单：我们设一个栈 s，保存所有“尚未找到答案”的天，并用一个数组 ans[] 保存已经得到的答案：ans[i]=j 表示 i 的“答案天”是 j。如图 3.12，我们遍历一遍数组，依次检查 temperatures[] 中的每一个温度 temperatures[j] = t，将其与栈顶温度 temperatures[s.top] 进行比较，若 t 大于栈顶温度，则栈顶的这一天 s.top 的“答案天”就是 j，并继续将 t 与新的栈顶进行比较，重复这个过程，直至 t 不大于栈顶温度或栈空后，将当前检查的温度 j 入栈。检查完所有温度后，栈中剩余的这些天，就是后面没有温度高于他们的。最后留在栈中的（下标为 5 和 7 的）两

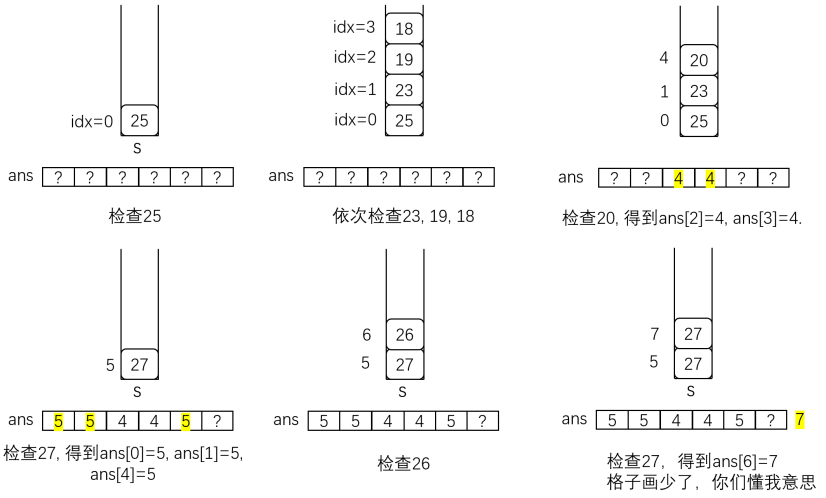


图 3.12

天，后面没有温度高于它们。伪代码如下所示：

```
for(int i=0; i<n; i++) {
    while(!Empty(s) && temperatures[i]>temperatures[s.top]) {
        ans[s.top] = i;
        Pop(s);
    }
    Push(s, i);
}
while(!Empty(s)) {
    ans[s.top] = 0; // 这里拟用 0 表示后面不存在一天，比 s.top 这一天温度更高
    Pop(s);
}
```

该算法的时间复杂度为 O(n)。至于为什么是 O(n)，相比于上一种算法，该算法的提升在哪里，这里留给读者自行思考。可以看到，栈 s 中的元素是单调不增的，很多人喜欢将这种元素具有单调性的栈称为**单调栈**，但笔者希望强调的更多是思考问题、解决问题的这个创造性的过程，次之才是：下次遇见类似的问题，可以用“单调栈”来解决。

相信通过该例大家已经建立了足够的感觉，我们来回看这句话：许多场景下，可以用栈来保存这样的数据，即：**“那些仍然等待未来某个事件发生，并且优先处理后来者”的数据**。该例并非巧合，而是存在大量类似的问题，比如大名鼎鼎的**“表达式求值”**问题。

提示

实际上队列也有类似的用途：存放“那些仍然等待某事发生，并且优先处理先来者”的数据。**我们将在第五章，图的广度优先遍历看到。**

3.3.1 表达式的转换与求值

表达式的转换与表达式的求值的过程中实际上也有类似上面的思想。我们先来介绍中缀表达式、后缀表达式的概念。

维火女

亲爱的余灰大人，如果您已经了解中缀、后缀表达式的概念，并且能够熟练地将二者相互转换，请放心地随我前往—waypoint-2—（第 57 页）处。

我们平时书写算术表达式时，通常采用这样的形式：

$$3+5$$

如果表达式稍微复杂一些，例如：

$$3+5\times 2$$

我们知道应该先计算乘法，再计算加法，这种运算符写在两个操作数之间的表达式，称为**中缀表达式（Infix Expression）**。

但是，计算机真的喜欢这样的表达式吗？

仔细观察上面的表达式：

$$3+5\times 2$$

如果我们从左到右依次读取它，会先看到 $3+5$。可是，此时却不能立即计算，因为后面的乘法具有更高的优先级。也就是说，当我们看到一个运算符时，有时可以立即计算，有时却必须**暂时把它保存起来**，等待后面的运算符被处理之后才能决定。

如果再加入括号，情况会变得更加复杂：

$$(3+5)\times 2$$

这时我们不仅需要考虑运算符的优先级，还需要处理括号所带来的运算顺序。

如果我们将

$$(3+5)\times 2$$

改写成

$$3\ 5\ 2\ \times\ +$$

此时我们再从左向右读取它，你会发现：

1. 读取 3；
2. 读取 5；
3. 读取 2；
4. 读取 ×，将 5 和 2 相乘，得到 10，表达式变成了 3 10 +；
5. 读取 +，将 3 和 10 相加，得到最终结果。

整个计算过程不需要我们再判断“乘法是不是应该优先计算”，因为**运算符出现的位置本身，就已经把运算顺序表达出来了**。

这种将运算符写在其操作数之后的表达式，称为**后缀表达式（Postfix Expression）**，也称为**逆波兰表达式（Reverse Polish Notation, RPN）**。我们再给出几例。中缀表达式

$$b\times c+(a-d)$$

的后缀形式为：

$$bc\times ad-+$$

至于如何手动将中缀转后缀，限于篇幅，此处不再给出，请大家参考 CoreLoop 本人相关 DLC。

中缀表达式

$$a+b\times c-(b-d)\div e$$

的后缀形式为：

$$abc\times +bd-e\div -$$

在后缀表达式中，每一个运算符都紧跟在它所需要的操作数之后，因此不需要再使用括号，也不需要额外规定运算符的优先级。

waypoint-2

栈与表达式求值

我们首先考虑较简单的，计算机如何求值后缀表达式。

计算机按照从左到右的顺序处理后缀表达式，当看到一个数字时，应该把它放在哪里？当看到一个运算符时，又应该从哪里取出它所需要的操作数？

例如，对于：

$$3\ 5\ 2\ \times\ +$$

当读取到 $\times$ 时，需要取出最近读取的两个操作数 5 和 2；

而计算出 $5 \times 2 = 10$ 后，又需要把结果暂时保存起来，等待后面的 $+$。

这种**“最后保存的操作数最先被取出使用”**的特点，恰好符合栈的**后进先出**原则。于是，后缀表达式求值的算法就变得很好理解：对于后缀表达式，我们从左到右依次读取每一个元素：

- 如果读取到的是操作数，则将其压入栈中；
- 如果读取到的是运算符，则从栈中弹出所需数量的操作数，完成运算后，再将运算结果压入栈中。

当整个表达式处理完毕后，栈中剩下的唯一元素就是整个表达式的计算结果。

后缀表达式最重要的特点并不是“把运算符放到了后面”，而是：

> **它把原本隐藏在括号和运算符优先级中的运算顺序，直接编码到了表达式的排列顺序中。**

因此，对于后缀表达式，计算机不再需要理解“谁的优先级更高”，也不需要处理括号的嵌套关系，只需要从左到右读取表达式，并按照遇到的运算符立即进行计算即可。求后缀表达式

$$abc \times + bd - e \div -$$

的值的过程如表 3.1 所示。

事实上，计算机可以**直接求值中缀表达式**，而无需先将中缀表达式转换为后缀表达式，再进行求值。接下来我们来看计算机如何直接求值中缀表达式。与后缀表达式不同的是，当计算机从左到右读取中缀表达式时，遇到一个运算符 op 后，不能总是立即计算。某些运算符必须暂时保存下来，等待后面的运算符出现之后，才能决定自己什么时候参与计算。有了前面的思想很容易知道，扫描过程中遇到的运算符 op 就是**“那些仍然等待未来某个事件发生，并且优先处理后来者”**的数据。

于是，一个非常自然的设计出现了：

- **操作数栈**：保存当前暂时不能参与运算的操作数以及中间计算结果；
- **运算符栈**：保存当前暂时不能执行的运算符。

表 3.1: 后缀表达式求值过程

步骤	扫描项	项类型	动作	栈中内容（栈底 → 栈顶）
1	a	操作数	a 入栈	a
2	b	操作数	b 入栈	$a,\ b$
3	c	操作数	c 入栈	$a,\ b,\ c$
4	$\times$	运算符	弹出 b、c，计算 $b \times c$，将结果 bc 入栈	$a,\ bc$
5	$+$	运算符	弹出 a、bc，计算 $a + bc$，将结果入栈	$a + bc$
6	b	操作数	b 入栈	$a + bc,\ b$
7	d	操作数	d 入栈	$a + bc,\ b,\ d$
8	$-$	运算符	弹出 b、d，计算 $b - d$，将结果入栈	$a + bc,\ b - d$
9	e	操作数	e 入栈	$a + bc,\ b - d,\ e$
10	$\div$	运算符	弹出 $b - d$、e，计算 $\dfrac{b-d}{e}$，将结果入栈	$a + bc,\ \dfrac{b-d}{e}$
11	$-$	运算符	弹出 $a + bc$、$\dfrac{b-d}{e}$，计算 $a + bc - \dfrac{b-d}{e}$，将结果入栈	$a + bc - \dfrac{b-d}{e}$

当计算机从左到右读取一个中缀表达式时，可以按照下面的规则进行处理：

1. 从左到右依次读取中缀表达式中的每一个元素。
2. 如果读取到的是操作数，则将其压入操作数栈。
3. 如果读取到的是左括号，则将其压入运算符栈。
4. 如果读取到的是右括号，则不断弹出运算符栈的栈顶运算符，从操作数栈中取出相应的操作数进行计算，并将计算结果重新压入操作数栈，直到遇到左括号为止。最后将左括号弹出并丢弃。
5. 如果读取到的是运算符，则比较当前运算符与运算符栈栈顶运算符的优先级：
   - 如果运算符栈为空，或者栈顶为左括号，或者当前运算符的优先级高于栈顶运算符，则将当前运算符压入运算符栈；
   - 否则，不断弹出栈顶运算符，从操作数栈中取出相应的操作数进行计算，并将计算结果重新压入操作数栈，直到当前运算符可以入栈为止，然后将当前运算符压入运算符栈。
6. 中缀表达式读取完毕后，不断弹出运算符栈中的运算符，从操作数栈中取出相应的操作数进行计算，并将计算结果重新压入操作数栈。
7. 当运算符栈为空时，操作数栈中剩余的唯一元素即为整个中缀表达式的计算结果。

这样，我们就不需要先将中缀表达式转换成后缀表达式，而是可以直接在读取中缀表达式的过程中完成计算。

这个过程我没有画图，并不是因为懒，而是相信通过前面的寻找温度的例子，你已经对类似场景建立了足够的直觉。如果你还没有搞清楚“栈与表达式”是怎么回事，可以参考 CoreLoop 本书相关的 DLC。

*** 栈与表达式转换**

根据前面的介绍，事实上计算机可以借助两个栈，直接计算中缀表达式的值。但出于某种原因，这里我们依然介绍一下计算机如何借助栈将中缀表达式转换为后缀表达式。有了前面的内容相信大家已经能够自行想出正确的做法了。我们这里直接给出：

1. 从左到右依次读取中缀表达式中的每一个元素。
2. 如果读取到的是操作数，则直接将其加入后缀表达式。
3. 如果读取到的是左括号，则将其压入运算符栈。
4. 如果读取到的是右括号，则不断弹出运算符栈的栈顶元素，并将其加入后缀表达式，直到遇到左括号为止。左括号弹出后丢弃，不加入后缀表达式。

5. 如果读取到的是运算符，则比较它与运算符栈栈顶运算符的优先级：
   - 如果栈为空，或者栈顶为左括号，或者当前运算符的优先级高于栈顶运算符，则将当前运算符压入栈中；
   - 否则，不断弹出栈顶运算符并加入后缀表达式，直到当前运算符可以入栈为止。
6. 中缀表达式读取完毕后，将运算符栈中剩余的运算符依次弹出，并加入后缀表达式。

整个过程的核心其实非常简单：

> **操作数直接输出，运算符暂存于栈中；当一个运算符已经可以确定其位置时，就将它从栈中弹出并输出。**

中缀表达式

$$a + b \times c - (b - d) \div e$$

转后缀表达式的过程如表 3.2 所示。

表 3.2: 中缀表达式转换为后缀表达式的过程

步骤	扫描项	项类型	动作	运算符栈（栈底 → 栈顶）	后缀表达式
1	a	操作数	a 直接输出		a
2	$+$	运算符	栈为空，$+$ 入栈	$+$	a
3	b	操作数	b 直接输出	$+$	ab
4	$\times$	运算符	$\times$ 的优先级高于栈顶的 $+$，$\times$ 入栈	$+,\ \times$	ab
5	c	操作数	c 直接输出	$+,\ \times$	abc
6	$-$	运算符	$-$ 的优先级低于栈顶的 $\times$，弹出 $\times$ 并输出；$-$ 的优先级与栈顶的 $+$ 相同，弹出 $+$ 并输出；然后 $-$ 入栈	$-$	$abc \times +$
7	$($	左括号	左括号直接入栈	$-,\ ($	$abc \times +$
8	b	操作数	b 直接输出	$-,\ ($	$abc \times +b$
9	$-$	运算符	栈顶为左括号，不进行优先级比较，$-$ 直接入栈	$-,\ (,\ -$	$abc \times +b$
10	d	操作数	d 直接输出	$-,\ (,\ -$	$abc \times +bd$
11	$)$	右括号	依次弹出栈顶运算符并输出；遇到左括号后将其弹出，但不输出	$-$	$abc \times +bd-$
12	$\div$	运算符	$\div$ 的优先级高于栈顶的 $-$，$\div$ 入栈	$-,\ \div$	$abc \times +bd-$
13	e	操作数	e 直接输出	$-,\ \div$	$abc \times +bd-e$
14	结束	—	扫描结束，将栈中剩余的运算符依次弹出并输出	—	$abc \times +bd-e \div -$

习题 2 （参考答案见习题 2 参考答案，第 205 页）

1. [多选] 下列说法正确的是 _____。

A. 栈是一种有特殊要求的线性表。

B. 队列是一种有特殊要求的线性表。

C. 最先入栈的元素一定最先出栈。

D. 最先入队的元素一定最先出队。

2. 以下哪一项不是栈可以进行的操作？_____

A. 将元素压入栈中

B. 判断栈是否为空

C. 删除栈底元素

D. 将栈置为空栈

3. 以下哪一项不是队列可以进行的操作？_____

A. 将元素插入队尾

B. 获取队首元素的值

C. 获取位序为 i 的元素的值

D. 位序为 i 的元素出队

4. 某个栈，元素的入栈序列是

$$1, 2, 3, 4$$

则以下序列中，不可能作为出栈序列的是：_____

A. 2, 1, 3, 4

B. 2, 3, 4, 1

C. 3, 2, 4, 1

D. 3, 4, 1, 2

5. 关于时间复杂度，以下说法正确的是：

A. 顺序栈的 Push 和 Pop 时间均为 $O(n)$

B. 链栈的 Push 和 Pop 均可实现为 $O(1)$

C. 使用单链表实现队列时，只要有 head 指针，入队和出队均可实现为 $O(1)$

D. 循环队列的入队操作必须移动大量元素，因此时间复杂度为 O(n)

6. 采用数组实现容量为 N 的循环队列，并规定：

front 指向队首元素；

rear 指向下一个插入位置；

front 和 rear 初始全部指向下标 $N-1$ 处，队列采用保留一个空闲位置的方式区分队满和队空。

则判断队列为空和队满的条件分别为：

A. front == 0 && rear == 0, rear + 1 == front

B. front == 0 && rear == 0, (rear + 1) % N == front

C. front == rear, rear + 1 == front

D. front == rear, (rear + 1) % N == front

7. [多选] 某同学做了如下设计：采用一个长度为 100 的数组实现两个栈 S1 和 S2。该同学规定两栈栈底分别在两个端点处，栈顶向中间生长，如图 3.13。同时规定两栈栈顶元素相邻时，存储空间满，均无法再入栈。以下说法中正确的是 _____

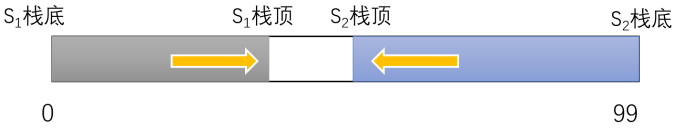


图 3.13

A. 两栈入栈时间复杂度均为 O(1)

B. 设两栈元素数量分别为 n、m，则两栈栈顶元素的出栈时间复杂度分别为 O(n)、O(m)

C. 要计算两栈栈中已有元素数量之和，时间复杂度为 O(1)

D. 若 S1.top 初始化为-1，S2.top 初始化为 100，且两个栈顶指针始终指向各自的栈顶元素，则 S1.top+1==S2.top 时，空间满，两栈均无法再入栈

E. 若 S1.top 初始化为-1，S2.top 初始化为 100，且两个栈顶指针始终指向各自的栈顶元素，则可能出现 S1.top>S2.top

F. 若 S1.top 初始化为 0，S2.top 初始化为 99，且两个栈顶指针始终指向各自栈顶元素的下一个位置，则 S1.top+1==S2.top 时，空间满，两栈均无法再入栈

G. 若 S1.top 初始化为 0，S2.top 初始化为 99，且两个栈顶指针始终指向各自栈顶元素的下一个位置，则可能出现 S1.top>S2.top

8. 一个初始为空的栈和一个初始为空的队列，都依次接收：

$$1, 2, 3, 4, 5$$

如果只允许进行正常的入栈、出栈、入队、出队操作，关于它们能够产生的输出顺序，下列说法正确的是：

A. 栈和队列都可以产生任意排列

B. 栈只能产生一种输出顺序，队列也只能产生一种输出顺序
C. 队列的输出顺序固定为 $1,2,3,4,5$，而栈的输出顺序受到栈操作过程限制
D. 栈的输出顺序固定为 $5,4,3,2,1$，队列的输出顺序受到操作过程限制

9.[多选] 假设输入序列是 1,2,3,4,5，利用两个队列进行出入队操作，不可能的输出序列有 _____
A. 5,4,3,2,1
B. 5,1,2,3,4
C. 5,2,3,1,4
D. 2,3,4,5,1

10. [多选] 与循环队列相比，链式队列 _____
A. 优点是进队和出队时间复杂度更低
B. 优点是长度可以在内存限制内动态变化
C. 缺点是节点除数据外需要额外的存储空间存放指针
D. 缺点是不能根据队首和队尾指针以 O(1) 时间算出队列长度

11. 假设某队列采用带头结点的循环双链表实现，第一个数据节点表示队头，最后一个数据节点表示队尾。若只设头指针指向头节点，则进队和出队的时间复杂度分别为：
A. O(1), O(1)
B. O(1), O(n)
C. O(n), O(1)
D. O(n), O(n)

12. （某年 408 真题）现有队列 Q 和栈 S，初始时 Q 中的元素依次是 1,2,3,4,5,6（1 在队头），S 为空。若仅允许下列的 3 种操作：①出队并输出出队元素；②出队并将出队元素入栈；③出栈并输出出栈元素。则不能得到的输出序列是 _____
A. 1,2,5,6,4,3
B. 2,3,4,5,6,1
C. 3,4,5,6,1,2
D. 6,5,4,3,2,1

13. （某年 408 真题）某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a,b,c,d,e 依次入此队列后再进行出队操作，则不可能得到的出队序列是 _____
A. b,a,c,d,e

B. d,b,a,c,e
C. d,b,c,a,e
D. e,c,b,a,d

14. （某年 408 真题）初始为空的队列 Q 的一端仅能进行入队操作，另外一端既能进行入队操作，又能进行出队操作。若 Q 的入队序列是 1,2,3,4,5，则不能得到的出队序列是 _____
A. 5,4,3,1,2
B. 5,3,1,2,4
C. 4,2,1,3,5
D. 4,1,3,2,5

15. （某年 408 真题）假设栈初始为空，将中缀表达式 a/b+(c*d-e*f)/g 转换为等价的后缀表达式的过程中，当扫描到 f 时，栈中的元素依次是 _____
A. +(*-
B. +(-*
C. /+(*-*
D. /+-*

16. （某年 408 真题）若栈 S1 中保存整数，栈 S2 中保存运算符，函数 F() 依次执行下述的各步操作：
1) 从 S1 中依次弹出两个操作数 a 和 b；
2) 从 S2 中弹出一个运算符 op；
3) 执行相应的运算 b op a；
4) 将运算结果压入 S1 中。
假定 S1 中的操作数依次是 5,8,3,2（2 在栈顶），S2 中的运算符依次是 *, -, +（+ 在栈顶）。调用 3 次 F() 后，S1 栈顶保存的值是 _____
A. -15
B. 15
C. -20
D. 20

waypoint-7

17. C++ 标准库为我们封装好了某个数据结构：stack<>。这是一个栈，你可以在尖括号中定义你想要存放的元素类型，如 stack<int> 表示“元素类型为 int 的栈”。你可以使用如下代码定义一个空的栈 s：

```
/* 类似 “int a;”中 int 是类型，a 是变量名。
   这里 stack<int> 是类型，s 是你定义的变量名。*/
   stack<int> s;
```

stack 对象支持如下操作：

1） s.push(e); // 将值为 e 的元素入栈

2） int t = s.top(); // 返回 s 的栈顶元素的引用（但不弹出该元素），将其值拷贝给 t

3） s.pop(); // 弹出栈顶元素

4） s.empty(); // 若栈 s 为空，此处返回值为 true，否则返回 false。

5） s.size(); // 返回栈 s 中元素的数量

现对于空栈 s，给定如下操作序列，说出每次调用 s.top()、s.size() 时的返回值。

1） s.push(3);

2） s.size(); //

3） s.push(5);

4） s.top(); //

5） s.top(); //

6） s.size(); //

7） s.push(6);

8） s.pop();

9） s.pop();

10） s.size(); //

11） s.top(); //

18. C++ 标准库为我们封装好了某个数据结构：queue<>。这是一个队列，你可以在尖括号中定义你想要存放的元素类型，如 queue<int> 表示“元素类型为 int 的队列”类型。你可以使用如下代码定义一个空的队列 q：

```
/* 类似 “int a;” 中 int 是类型，a 是变量名。
   这里 queue<int> 是类型，q 是你定义的变量名。*/
   queue<int> q;
```

queue 对象支持如下操作：

1） q.push(e); // 将值为 e 的元素入队

2） int t = q.front(); // 返回 q 的队首元素的引用（但不将该元素出队），将其值拷贝给 t

3） q.pop(); // 将队首元素出队

4） q.empty(); // 若队列 q 为空，此处返回值为 true，否则返回 false。

5） q.size(); // 返回队列 q 中元素的数量

现有一数组：

```
int arr[5] = {1,2,3,4,5};
```

请你将数组中的五个元素依次入栈，然后从栈顶将所有元素依次出栈，打印每个出栈元素的同时将出栈元素入队，之后从队首依次出队，打印每个出队元素，直至队列为空。请利用上题中介绍的 stack 以及本题中介绍的 queue 实现这个过程。

*19. 某游戏服务器持续记录某位名为 CoreLoop 的玩家的网络延迟情况。现假设服务器连续记录了 n 个时刻的网络延迟，第 i 个时刻的延迟为 delay[i]，这些数值均为正整数。为防止数值闪动，该服务器将时刻 i 的近 k 个时刻延迟的最大值作为该玩家在时刻 i 的**显示延迟**。也就是说，时刻 i 的显示延迟为：

$$max(delay[max(0, i-k+1)], delay[max(0, i-k+2)], ..., delay[i])$$

如，对于：

$$delay[] = 40, 33, 38, 37, 42, 35$$

k=3 时，显示延迟为

$$ans[] = 40, 40, 40, 38, 42, 42$$

k=4 时，显示延迟为

$$ans[] = 40, 40, 40, 40, 42, 42$$

请你设计一个尽可能高效的算法，求出该玩家这 n 个时刻的**显示延迟**。

```
// 将时刻 0 到 n-1 的显示延迟分别保存在 ans[] 数组下标 0 到 n-1 处。
    void MaxDelay(int delay[], int n, int k, int ans[]) {

    }
```

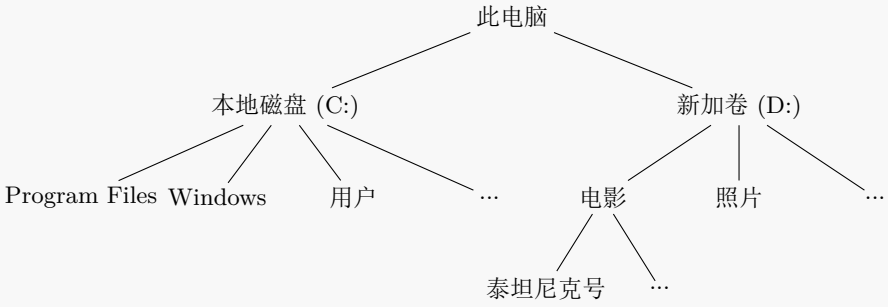
第四章　树与二叉树

4.1　树

在 1.1 节中，我们已经对“什么是树”有了一个感性的认知。与现实中的大树类似，数据结构中的树也是一种由根开始分叉生长的结构。比如游戏中一个队长三个队员的四人小队可以看成一棵树：



你的个人计算机中目录结构的组织是一棵树：



下面给出树的定义。

定义

树是 n（n>=0）个数据元素组成的有限集合，当 n=0 时，称为**空树**。任何一棵非空树都满足以下条件：

1. 有且仅有一个数据元素无前驱，称为**根节点**；
2. 除根节点外，其余每个数据元素都有且仅有一个前驱节点，称为**父节点**；

3. 每个数据元素可以有零个或多个后继节点，称为**子节点**。

由以上定义可知，对于一棵非空树中的任意一个节点，以该节点为根节点，连同该节点的所有后代节点在内所构成的仍是一棵树，称为原树的一棵**子树**。通常认为一棵树本身是自己的子树。很少存在一些场景会关心树中节点的各个子树从左至右是否有次序（即互换后还能不能认为是同一棵树），但通常将节点有次序的树称为**有序树**，节点没有次序的树称为**无序树**。

“树”这个逻辑结构该如何实现？如果采用链表类似的思想，每个节点除保存数据外，再保存所有指向其孩子节点的指针，像这样：

```
struct TreeNode {
    ElemType data;
    TreeNode* child1;
    TreeNode* child2;
    TreeNode* child3;
    ...
}
```

显然，我们无法预知每个节点有多少孩子节点，因此无法为 TreeNode 结构定义固定数量的孩子节点指针。熟悉 C++ 的同学可能会想到，可以使用一个不定长数组，将节点的所有孩子指针保存到该数组中：

```
struct TreeNode {
    ElemType data;
    std::vector<TreeNode*> childs;  // childs 是一个不定长数组，childs[0], ..., childs[n] 分别表示该节点的各个孩子节点指针
}
```

如图 4.1 所示。这是完全可行，也非常自然的一种树的表示方式。但本书不要求大家熟悉 C++，因此我们给出树的另一种表示方法，也是更高效的**孩子-兄弟表示法（First Child - Next Sibling Representation）**。该方法存储每个节点的第一个孩子的指针，以及下一个兄弟的指针：

```
struct TreeNode {
    ElemType data;
    TreeNode* first_child;
    TreeNode* next_sibling;
};
```

这种表示法如图 4.2 所示。

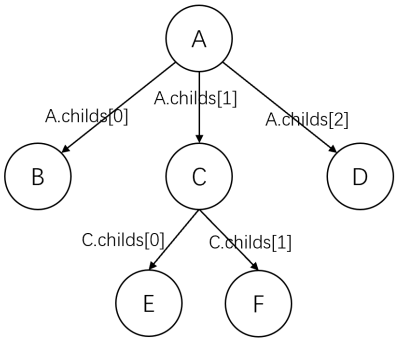


图 4.1 用不定长数组表示树的孩子

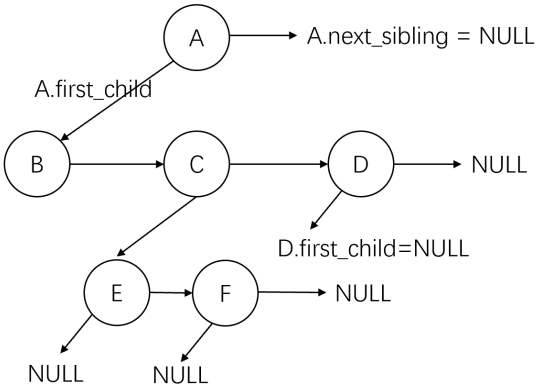


图 4.2 树的孩子-兄弟表示法

显然该表示法下，只要拿到根节点指针，就能找到树中任何一个节点。因此，可以用根节点指针表示一棵树。如：

```
TreeNode* root;
```

root 这个对象既可以理解为树的根节点的指针，又可以理解为以 root 节点为根的一棵树。

提示

图 4.1 与图 4.2 逻辑上是同一棵树，只不过物理指针的含义不同。

有了以上的内容介绍，我们来做一点有意思的事情（当然你大可以放心地跳过）：用孩子-兄弟表示法尝试实现操作系统的目录树。

4.1.1 * 目录树的简单实现

我们已经知道，操作系统对目录的管理其实就是一棵树。用户随时可能在任何目录新建目录、删除某个目录，我们暂且不考虑文件。并假设目录节点的“数据”只有一个字符串类型的成员，表示目录的名字，如“电影”等。以下代码实现了简单的新建目录、删除目录的操作，具体过程请结合注释。

```
// 在目录 Dad 中，新建一个名为 name 的目录
void DirInsert(TreeNode* Dad, char* name) {
    // 函数实现：创建一个新节点，将其加入 Dad 的孩子链表末尾
    TreeNode* new_node = (TreeNode*)malloc(sizeof(TreeNode));
    strcpy(new_node->name, name);
    new_node->first_child = NULL;
    new_node->next_sibling = NULL;

    // Dad 原本是空目录，则需要更改 first_child 指针。
    if(Dad->first_child == NULL) {
        Dad->first_child = new_node;
        return;
    }

    // 原本不是空目录，修改 next_sibling 指针。
    TreeNode* last_child = Dad->first_child;
    while(last_child->next_sibling != NULL) {
        last_child = last_child->next_sibling;
    }
    last_child->next_sibling = new_node;
```

```
    return;
}

// 销毁一个节点及其所有后代
void DestroyTree(TreeNode* root) {
    if(root == NULL) return;
    // 先递归销毁其子树
    DestroyTree(root->first_child);

    // 再释放根节点
    free(root);
}

// 在目录 Dad 中，删除目录 to_del
void DirDelete(TreeNode* Dad, TreeNode* to_del) {
    // 函数实现：先把 to_del 从父节点的孩子链表中摘掉，再销毁以 to_del 为根的整棵树
    if(Dad == NULL) {
        return;
    }
    // 要删除的就是第一个孩子节点，需要更新 first_child 指针
    if(to_del == Dad->first_child) {
        Dad->first_child = to_del->next_sibling;;
        // 注意这里不能简单地 free(to_del)，因为该目录中可能还有若干目录，目录中可能还有目录，因此要递归销毁。
        DestroyTree(to_del);
        return;
    }

    // 找到要删除的节点的前一个兄弟节点
    TreeNode* pre = Dad->first_child;
    while(pre->next_sibling != to_del) {
        pre = pre->next_sibling;
    }

    // 修改链条，类似单链表的删除操作
    pre->next_sibling = to_del->next_sibling;
    // 注意同样要递归销毁
    DestroyTree(to_del);
    return;
}
```

提示

以上例子也给了我们一个重要提示：树中每个节点的子节点很多时候不能单纯的看成一个节点，而是要看成一棵子树。因此与树相关的操作，很多都会涉及到递归，这一点非常重要。

4.1.2 树的一些基本概念与性质

我们回到逻辑上的数据结构：树。树中有一些大家常用的专业术语与性质，我们在这里列出。

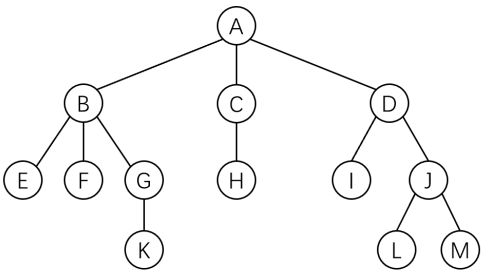


图 4.3

a）树中，一个节点拥有的子树数量称为**节点的度**。一棵树中所有节点的度的最大值称为这棵**树的度**。

b）树中度为 0 的节点称为**叶子节点**，简称**叶节点**。显然叶节点没有孩子节点。不是叶子节点的节点称为**非叶节点**。

图 4.3 中，节点 A、B 的度均为 3，整棵树的度为 3；节点 E、F、K、H、I、L、M 的度为 0，是叶子节点。

c）可以看出树是一种具有层次结构的数据结构。根节点处于最高层，通常定义根为第 1 层，根的孩子为第 2 层。若某节点处在第 i 层，其子树的根节点就处在第 i+1 层。

d）**节点的深度**通常定义为节点的所在层次；而**节点的高度**通常是指从最底层叶子节点（高度为 1）开始逐层累加至该节点时的高度值。所有节点的深度的最大值称为**树的深度**或**树的高度**。即

$$\text{树的高度} = \max\{\text{所有节点的深度值}\} = \max\{\text{所有节点的高度值}\}$$

图 4.3 中，节点 A 的深度为 1，高度为 4；节点 B 的深度为 2，高度为 3；节点 K 的深度为 4，高度为 1。整棵树的深度，或称整棵树的高度，为 4。

提示

1. 有些人喜欢将根节点的高度定义为 0，本书若无特别说明，默认按照根节点高度为 1 来定义。
2. 注意区分节点与树的深度和高度：节点的高度和节点的深度通常不是一回事；而树的高度和树的深度一定是相等的。

e）类似现实中的说法，数据结构中多棵树同样可以构成**森林**。

树的重要性质

我们首先给出一个重要性质：对于任何一棵非空树，其节点数 n 与边数 e 一定有

$$n = e + 1$$

成立。显然，当 $n = 1$ 时，树只有一个根节点，上式显然成立。此后，如图 4.4 所示，从任何一个节点引出一条新的边，都会使得边数 e 和节点数 n 同时加一，因此等式仍然成立。另外，不难发现，每从节点引出一条新的边，会使边数加一的同时，会使该节

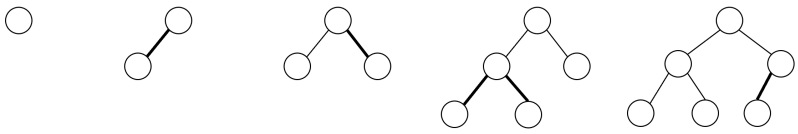


图 4.4

点的度数加一，所以不难得知，所以对任何一棵非空树，所有节点的度数之和即等于该树的总边数 e。

整理以上两个结论，有：对任何一棵非空树，都有以下结论成立：

树的边数 e = 树中所有节点的度数之和 = n - 1（n 为树的节点数）

由

以上的思维过程也可以反推出一件事：树中度为 n 的节点，会为树“贡献”n 条边。比如：一棵度为 3 的树，若已知度为 3、2、1 的节点分别有 7、5、8 个，则该树的边数一定是

$$7 * 3 + 5 * 2 + 8 * 1 = 39$$

于是可知该树的节点数为 $39+1 = 40$，于是叶节点的数量就是 $40-7-5-8 = 20$ 个。

4.2 二叉树的基本内容介绍

简单地说，“最多两个分叉的树”就是二叉树。一棵普通的二叉树本身通常难以在实际问题中应用上，但是它却是后面一些非常有趣的特殊二叉树（如**二叉查找树**、**哈夫曼树**、**堆**等）的理论基础。所以我们有必要首先进行一些理论铺垫。

定义

二叉树要么是一棵空树，要么是一棵：

由根节点以及它的两棵子树构成的树，这两棵子树有左右位置之分，分别称为**左子树**和**右子树**，并且左子树和右子树依然是一棵二叉树。

可以看到，二叉树是**递归定义**的，即用二叉树自身来定义二叉树。如图 4.5 所示的几棵树都是二叉树，根据图中文字介绍，大家也能进一步体会二叉树的“递归定义”。请注意，由图 4.5 我们可以看出，二叉树的递归定义中，递归边界就是二叉树为一棵空

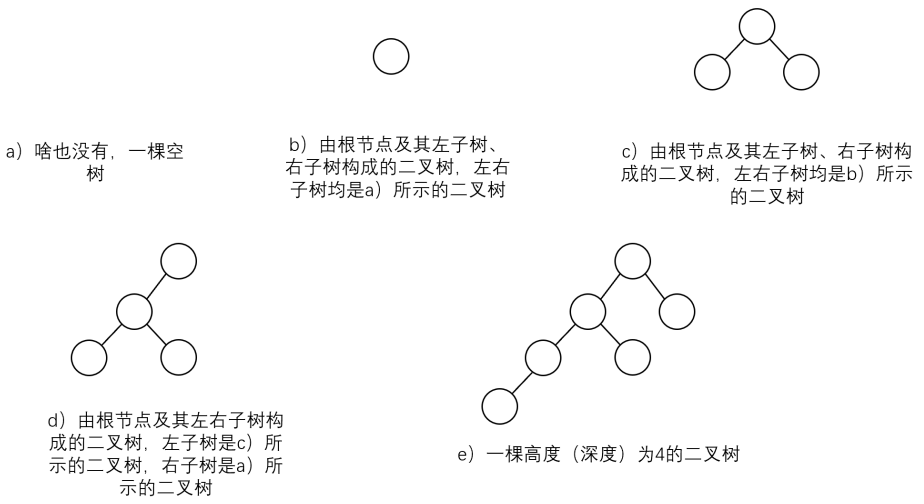


图 4.5 几棵不同的二叉树

树，这一点对我们二叉树中的诸多编程思想尤其重要，大家在后面将会慢慢体会到。显然二叉树的度只可能是 0、1 或 2，二叉树中每一个节点的度也只可能是 0、1 或 2。

提示

“二叉树”与“度为 2 的有序树”有如下区别：

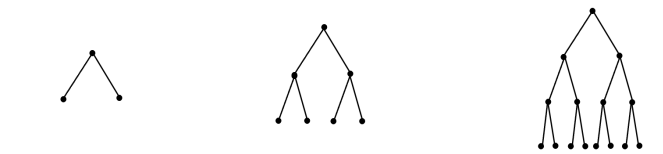
1. 二叉树的度未必是 2，还可能是 0 或 1；

2. 二叉树即使只有一个孩子节点，该节点也要区分左右。而度为 2 的有序树，其“有序”是兄弟之间的位置有序，若某个节点是“独生子女”，则其位置是唯一确定的，不分左右。二叉树明确区分孩子的左右位置，这是因为在更多的场景下这是有用的，在后面的内容中我们将有所体会。

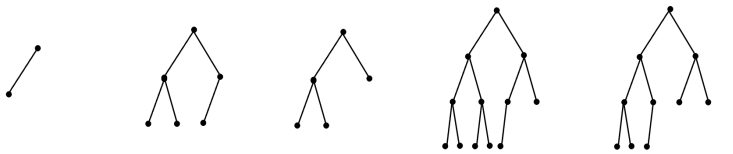
有两种比较丰满的二叉树需要特别关注：满二叉树与完全二叉树。

如果二叉树中的任意一层节点数都达到了当层能达到的最大节点数，这种二叉树称为**满二叉树**；如果除最下面一层外，其余层都达到了当层最大节点数，并且最后一层的节点都一个挨一个地挤在左边，则二叉树称为**完全二叉树**。显然满二叉树一定都是完全二叉树。

图 4.6 展示了一些满二叉树与完全二叉树。



a）一些满二叉树



b）一些完全二叉树

图 4.6 一些满二叉树与完全二叉树

4.2.1 二叉树的实现

将二叉树存储为二叉链表

有了链表的思维，相信大家非常自然地就能想到二叉树的实现方式：树的节点设两个指针分别指向其左孩子和右孩子如下所示：

```
struct BTreeNode {
    ElemType data;
    BTreeNode* lchild;  // 左孩子的指针
    BTreeNode* rchild;  // 右孩子的指针
};
```

这正是二叉树最为经典的实现方式，这种物理结构也称为**二叉链表**，它与普通的单链表相比“多分出了一个叉”。

正如本节之初所指出，二叉树的实际应用在于一些有趣的特殊二叉树。人们对队列、栈等数据结构有“入队、出队、入栈”等常见操作的定义，是因为它们切实有用，有重要的实际意义，而普通二叉树几乎不存在实际应用，因此通常不会为它定义一些常见操作，以上介绍的理论，是为了后面的特殊二叉树服务。此处如果你感兴趣，可以尝试完成下面的“思考”。

思考

建立一棵你中意的二叉树，并遍历这棵树，打印所有节点的数据内容。

二叉树的顺序存储

基于二叉树有上下的层次关系以及左右的次序关系，如果我们从上到之下、从左到右地为二叉树的每一个节点依次编号，就可以将这些节点依次存储在数组中的对应下标之处，如图 4.7 所示，我们这里遵循多数人的习惯，根节点从 1 开始编号，即从数组下标 1 处开始存储，而非从 0。根据规律不难得出，**在这种存储方式下，任一节点 i，其父节点的编号一定是 $\lfloor \frac{i}{2} \rfloor$（前提是 i>1，即该节点有父节点）；并且其左孩子如果存在的话，编号一定是 2i，右孩子如果存在的话，编号一定是 2i+1。**

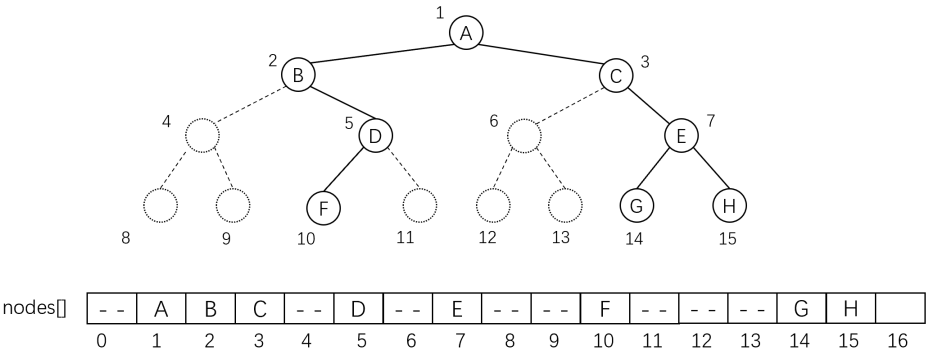


图 4.7 二叉树的顺序存储

这种表示法可以由以下代码实现：

```
struct BTreeNode{
    ElemType data;
    int valid;  // 该节点是否存在
};
BTreeNode nodes[MaxSize];   // 该数组中保存的就是二叉树的所有节点
```

提示

一般情况下每个节点还需要一个字段 valid 标识是否有效。但如果你存放的数据满足某种特点，比如一定都是正整数，那么你也可以不要 valid 字段，而是通过将数据的值置为，如-1，来表示该位置处节点不存在。

二叉树的这种存储方式的问题在于，即使某个节点不存在，也要浪费一个编号位，最坏情况下，当二叉树形状趋于深度为 h 的一条直线时，我们用约 2^h 个空间仅存储了约 h 个数据。

经过观察不难发现，**该表示法对完全二叉树（以及满二叉树）非常友好**。因为完全二叉树的节点从上到下、从左到右是连续的，不存在空缺的编号。在介绍后面的内容时，我们也能看到该表示法在完全二叉树中的妙用。

在进入本章真正有趣的部分之前，我们还要进行一场最后的理论准备：二叉树的遍历。

4.2.2 二叉树的遍历

遍历一个顺序表有多种方式，可以从前向后，也可以从后向前等。类似地，遍历二叉树也有多种方式。常见的、有意义的遍历方式通常有如下 4 种：**层序遍历**、**先序遍历**、**中序遍历**、**后序遍历**。相信大家最容易想到的遍历方式就是从上到下、从左到右地一个一个节点地访问，这就是**层序遍历**。对于图 4.8 中所示的二叉树，层序遍历得到的访问序列是 A,B,C,D,E,F,G,H,I,J。

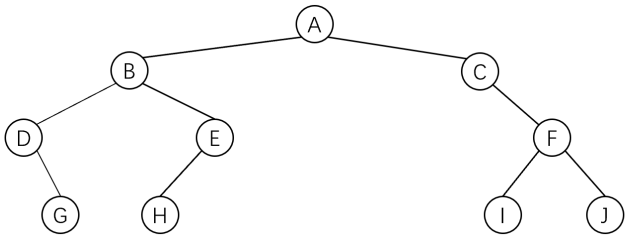


图 4.8

提示

我们与多数教材保持一致，以二叉链表存储的二叉树为例，给出二叉树的遍历方式，这也是应用得更多的场景。

二叉树的层序遍历

对于链式存储的二叉树：

```
struct BTreeNode {
    ElemType data;
    BTreeNode* lchild;  // 左孩子的指针
    BTreeNode* rchild;  // 右孩子的指针
};
```

如何从上到下，从左到右地依次访问每个节点？显然我们仅通过二叉树已有的物理结构是不行的，以图 4.8 为例，当我们从 A 走到 B 时，我们知道下一步要访问 C，但却没有一种有效的手段优雅地实现接下来的操作。所以我们需要一种手段，每走到一个节点之时，先将其两个孩子按正确的顺序保存起来，它们就是接下来即将被访问的节点。

承先贤之智，将下法传与思渡者：可使用一个队列，将更上层的节点优先入队，并且确保同层级的节点中，更左边的优先入队。这样，队中节点顺序即是层序遍历的顺序，依次出队访问即可。具体来说，实现二叉树层序遍历的过程如下：

1） 将根节点 root 加入队列 q；

2） 出队访问队首节点 front_node；

3） 如果 front_node 有左孩子，先将左孩子入队；

4） 如果 front_node 有右孩子，再将右孩子入队；

5） 若队列 q 不空，回到第 2）步；否则说明遍历完成，算法结束。

显然第一步将根节点加入队列是毋庸置疑的。而第 3）、4）两步，先加入左孩子再加入右孩子，则保证了从左到右的访问顺序。

*** 思维拓展**

如果你希望从上到下、从右到左地遍历二叉树，将上面的 3）、4）两步交换位置即可。

层序遍历的伪代码如下所示。

```
// 对根为 root 的二叉树进行层序遍历。对每个节点执行 visit() 函数。
    void LayerOrderTravel(BTreeNode* root) {
        if(root == NULL) return ;
        Queue q;
        InitQueue(q);
        EnQueue(q, root);    // 根节点入队
        while(!Empty(q)) {
            BTreeNode* front_node = NULL;
            DeQueue(q, front_node);      // 队首节点出队，赋值给变量 front_node
```

```
            visit(front_node);          // 访问 front_node
            if(front_node->lchild != NULL) {
                EnQueue(q, front_node->lchild);     // 入队左孩子
            }
            if(front_node->rchild != NULL) {
                EnQueue(q, front_node->rchild);     // 入队右孩子
            }
        }
    }
```

对于图 4.8 中所示的二叉树，其详细的层序遍历过程如下表 4.1 所示。可以看出，正是队列的先进先出特性，保证了节点按照从上到下、同层从左到右的顺序被访问。

表 4.1: 二叉树层序遍历的执行过程

步骤	出队节点	访问节点	新入队节点	操作完成后的队列（队首 → 队尾）
初始	—	—	A	A
1	A	A	B，C	B，C
2	B	B	D，E	C，D，E
3	C	C	F	D，E，F
4	D	D	G	E，F，G
5	E	E	H	F，G，H
6	F	F	I，J	G，H，I，J
7	G	G	—	H，I，J
8	H	H	—	I，J
9	I	I	—	J
10	J	J	—	空

除层序遍历外，对一棵二叉树的根节点、左子树和右子树也可以有不同的访问顺序。访问一棵二叉树时：

1. 若先访问根节点，再访问左子树，最后访问右子树，这种“根-左-右”的遍历方式称为**先序遍历**，也称**先根遍历**。

2. 若先访问左子树，再访问根节点，最后访问右子树，这种“左-根-右”的遍历方式称为**中序遍历**，也称**中根遍历**。

3. 若先访问左子树，再访问右子树，最后访问根节点，这种“左-右-根”的遍历方式

称为**后序遍历**，也称**后根遍历**。

提示

请注意，无论何种访问方式，左子树一定是先于右子树访问。所谓的“先、中、后”指的是“根节点”被“先”访问，还是“中”或“后”访问。

*** 思维拓展**

注意：本内容可能带来一些认知和思维上的启发，但对思渡者来说，可能带来知识记忆上的混乱。

我们为什么不将“根右左”、“右根左”、“右左根”分别定义为先根、中根、后根遍历？这是因为，二叉树的左右子树具有天然的对称性。将左与右交换，树的结构和**本质**并没有发生变化，它描述的还依然是那个“东西”，只是我们人类需要换一个方向去观察和描述它。
比如你将左边视为头时，[3,5] 的**本质**可以理解为“3 在前，5 在后的两个数”；而事实上 [5,3] 同样可以描述“3 在前，5 在后的两个数”，只需要你把右边视为头，或者把“前”定义为后。二者本质相同，只是我们人类观察和理解它的视角不同。
更进一步地讲，“前”与“后”、“左”与“右”的方向本身就是人为定义的，左子树之所以叫左子树，只是因为我们把“左”定义为了左而已。“左”与“右”本质上并无区别。

我们约定先左，只是因为人类的习惯就是左在前，右在后，这更符合绝大多数人的思维方式。

如果你没有阅读上述“思维拓展”，请直接跳过。如果你阅读了，请放声朗读以下内容：

先序遍历根左右 根左右 根左右； 先序所以根在先；

中序遍历左根右 左根右 左根右； 中序所以根在中；

后序遍历左右根 左右根 左右根； 后序所以根在后。

左一定要先于右； 左一定要先于右； 左一定要先于右；

先序遍历根左右 根左右 根左右； 先序所以根在先；

中序遍历左根右 左根右 左根右； 中序所以根在中；

后序遍历左右根 左右根 左右根； 后序所以根在后。

左一定要先于右； 左一定要先于右； 左一定要先于右；

二叉树的先序遍历

参照图 4.9，先序遍历下，对任何一棵二叉树 BT，总是先访问其根节点，再访问其左子树 LBT 和右子树 RBT。递归地，对于 LBT 和 RBT 又有同样的规则。因此递

归遍历便是很自然的想法，而递归边界就是某棵子树为空树。先序遍历的递归代码如代码 4.1 所示：

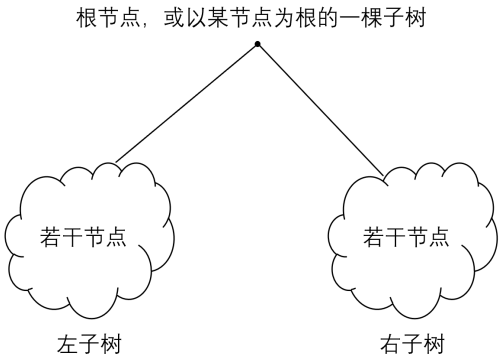


图 4.9

代码 4.1

```
1 void PreOrderTravel(BTreeNode* root) {
2     if(root == NULL) return ;   // 递归边界
3     visit(root);    /* 先序遍历，先访问根节点。请注意：这里 root 应理解为递归过
      ↪  程中访问的每棵子树的根节点，不能仅简单地理解为整棵树的根节点。*/
4     PreOrderTravel(root->lchild);   /* 递归遍历左子树。请注意：一定不能写成
      ↪  visit(root->lchild)。此处场景要把 root->lchild 看成一棵左子树，而不能仅
      ↪  仅看成一个节点。*/
5     PreOrderTravel(root->rchild);   // 递归遍历右子树
6 }
```

维火女

亲爱的余灰大人，如果您思渡至此，已经通晓了先序、中序及后序的遍历过程，请握紧我的手，我将带您传送至—waypoint-4—（第 86 页）处。

我们以图 4.8 模拟一遍整个过程：

暂且拟用 Tree(T) 表示以 T 为根节点的子树。则先序遍历过程如下：

走到 Tree(A) -> 先根遍历，因此直接访问 A；然后访问 A 的左子树，所以

走到 Tree(B) -> 同样地，直接访问 B；然后访问 B 的左子树，

走到 Tree(D) -> 访问 D；然后先访问 D 的左子树，再访问 D 的右子树。先

走到 Tree(NULL，即 D 的左孩子)，为空树，递归边界，返回到 D；该访问 D 的右子

树了：

走到 Tree(G，即 D 的右孩子)，访问 G；

走到 Tree(NULL, G 的左孩子)；返回到 G；

走到 Tree(NULL, G 的右孩子)；返回到 G；

返回到 D；

返回到 B；B 的左子树由上面的过程已经访问完成，该访问 B 的右子树了。

走到 Tree(E)；访问 E；

走到 Tree(H)；访问 H；我们不再赘述空子树的访问过程。

返回到 E-> B -> A；此时 A 的左子树已经全部访问完成，根据先序规定的访问顺序：①A ②A 的左子树 ③A 的右子树，A 及 A 的左子树已访问完成，接下来访问 A 的右子树；

走到 Tree(C)；访问 C；

走到 Tree(F)；访问 F；

走到 Tree(I)；访问 I；

走到 Tree(J)；访问 J；

一路回退到 A；代码结束。先序遍历得到的序列为 A,B,D,G,E,H,C,F,I,J。

waypoint-4

可以看出，先序遍历的思想，简单说就是：

1. 访问根节点（先根）；
2. 递归地遍历左子树；
3. 递归地遍历右子树。

后面的中序、后序遍历，实际上也是这三步，只不过访问根节点的顺序有所不同。

请注意，根据先序遍历一定最先访问根节点的特性，对一棵二叉树的先序遍历序列，序列的第一个节点一定是根节点。如上例中，整棵树的遍历序列是

$$A, B, D, G, E, H, C, F, I, J$$

因此 A 是整棵树的根节点。而对于 A 的左子树中的五个节点，遍历序列是

$$B, D, G, E, H$$

因此 B 是该子树的根节点。再如以 F 为根节点的子树，遍历序列是

$$F, I, J$$

因此 F 是根节点。

二叉树的中序遍历

参照图 4.9，中序遍历总是先访问左子树，再访问根节点，最后访问右子树。中序遍历的思想与先序遍历类似，只不过在访问根节点之前，要先递归遍历左子树：

1. 递归遍历左子树；
2. 访问根节点（中根）；
3. 递归遍历右子树。

可以看到，先序、中序、后序在代码上的区别其实就是把“访问根节点”这一操作放在哪一步的区别。中序遍历的代码如下所示。

```
void InOrderTravel(BTreeNode* root) {
    if(root == NULL) return ;   // 递归边界
    InOrderTravel(root->lchild);
    visit(root);         // 中根
    InOrderTravel(root->rchild);
}
```

我们以图 4.8 模拟一遍整个过程。拟用 Tree(T) 表示以 T 为根节点的子树。则先序遍历过程如下：

走到 Tree(A) -> 中根遍历，因此先访问 A 的左子树，再访问 A，最后访问 A 的右子树。所以暂且不访问 A，而是先

走到 Tree(B) -> 同样地，先访问 B 的左子树：

走到 Tree(D) -> 同样地，先访问 D 的左子树，因此

走到 Tree(NULL，即 D 的左孩子) -> 为空树，递归边界，返回到 D；该访问 D 这个节点了：

访问 D，此时 D 及 D 的左子树均访问完成，该访问 D 的右子树了。所以

走到 Tree(G) -> 依旧先左；走到 Tree(NULL, G 的左孩子)；返回到 G；

访问 G；此时 G 及 G 的左子树均访问完成，该访问 G 的右子树了。

走到 Tree(NULL, G 的右孩子)；返回到 G；

返回到 D；此时 D 及 D 的左右子树均已经访问完成。

返回到 B；此时 B 的左子树已访问完成，根据中序遍历规则，该访问根节点了，于是访问 B；此时 B 的左子树及 B 都访问完成，该访问 B 的右子树了。

走到 Tree(E)；走到 Tree(H) -> 访问 H；我们不再赘述空子树的访问过程。

回到 E；访问 E；

回到 B -> A；此时 A 的左子树已经全部访问完成，根据中序规定的访问顺序：①A 的左子树 ②A ③A 的右子树，现在 A 的左子树已经访问完成，所以接下来访问 A，然后访问 A 的右子树。

走到 Tree(C) -> 左子树为空；访问 C；

走到 Tree(F) ->

走到 Tree(I) -> 访问 I；

回到 F；访问 F；

走到 J；

访问 J；

一路回退到 A；代码结束。中序遍历得到的序列为 D, G, B, H, E, A, C, I, F, J

请注意：对于二叉树的中序遍历，根节点的访问总是在左子树和右子树的中间。因此，若已知某棵树的根节点是哪一个，就可以根据根节点在中序序列中的位置，将该中序序列从根节点处砍成两半，得到左子树和右子树的中序遍历序列。如上面的例子，若已知根节点为 A，则立即可以得到：

① 根节点左子树的中序遍历序列是

$$D, G, B, H, E$$

② 根节点右子树的中序遍历序列是

$$C, I, F, J$$

二叉树的后序遍历

有了前面的内容，后序遍历就变得很好理解。后序遍历是“左-右-根”，即根节点的访问在最后：

1. 递归遍历左子树；
2. 递归遍历右子树；
3. 访问根节点（后根）。

代码如下。

```
void PostOrderTravel(BTreeNode* root) {
    if(root == NULL) return ;   // 递归边界
    PostOrderTravel(root->lchild);   // 先递归遍历左子树
    PostOrderTravel(root->rchild);   // 再递归遍历右子树
    visit(root);         // 左子树、右子树均访问完成后，最后访问根节点。
}
```

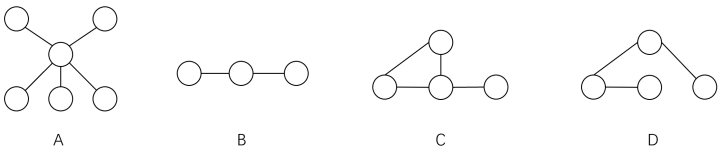
先序遍历是最先访问根节点，而后序遍历是最后访问根节点。因此对后序遍历序列来说，序列中的最后一个节点一定是根节点。对于图 4.8 所示的二叉树，其后序遍历得到的序列为 G,D,H,E,B,I,J,F,C,A。

— BONFIRE ENCOUNTERED —

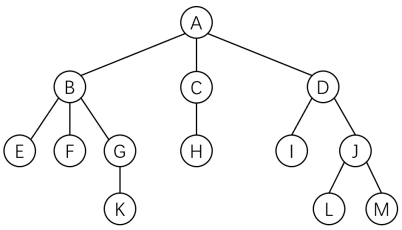
此处提示大家，二叉树这里的题目复杂多变，思维难度较大。

习题 3 （参考答案见习题 3 参考答案，第 210 页）

1. 下图中，不属于**树**的结构是 _____。



2. 对于下图所示的树，下列说法错误的是 _____。



A. 节点 B 的度为 2　　B. 节点 A 的度为 3　　C. 这是一棵二叉树　　D. 这棵树的度为 3

3. 关于上题图所示的树，下列说法错误的是

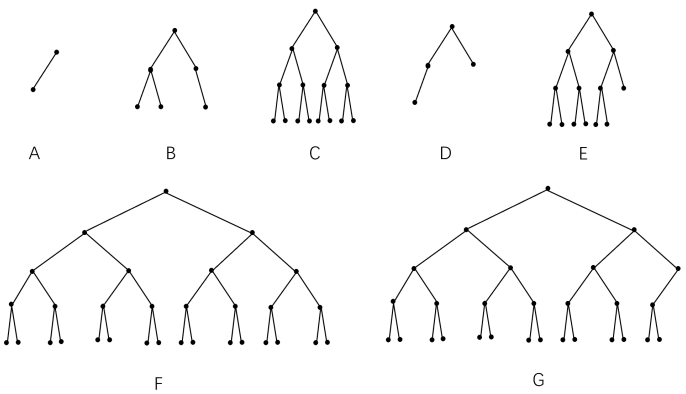
A. 若根节点高度定义为 1，则该树的高度为 4

B. 若根节点高度定义为 1，则该树的深度为 3

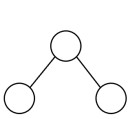
C. 节点 F 的度为 0，是叶子节点

D. 以 B、C、D 为根节点的子树的集合构成了一个森林

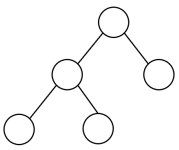
4. 下图中，是满二叉树的有 _____；不是满二叉树，但是是完全二叉树的有_____。



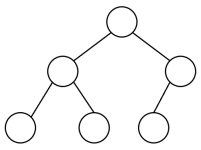
5. 请观察下列四棵二叉树，分别统计每棵树中度为 0 的节点（叶节点）和度为 2 的节点的数量，并将结果填入表格。



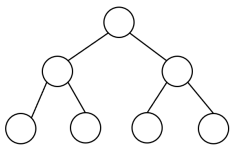
图一



图二



图三



图四

	度为 0 的节点数 (记为 n_0)	度为 2 的节点数 (记为 n_2)
图一		
图二		
图三		
图四		

请你思考：

1）对于这四棵树，度为 0 的节点数与度为 2 的节点数之间有什么关系？

2）这种关系是否可能对任何非空二叉树都成立？请尝试画出其他形状的二叉树，检验你的猜想。

3）不必从数学角度严谨证明，你能否尝试解释，为什么这两个数量之间会存在这样的关系？

6. 一棵非空二叉树中，有 8 个度为 2 的节点，3 个度为 1 的节点。请问：

1）能否确定这棵二叉树有多少个叶节点？

2）能否确定这棵二叉树一共有多少个节点？

7. 一棵非空二叉树共有 20 个节点，其中有 7 个度为 1 的节点。请问这棵二叉树的叶节点数可能是多少？

8. [多选] 若完全二叉树有 n（n > 0）个节点，从上到下，从左到右分别编号 1 至 n，则下列说法中正确的有_____

A. 若某节点不是根节点，其编号为 i，则其父节点的编号为 $\lfloor \frac{i}{2} \rfloor$

B. 若某节点编号为 i，且 2 * i > n，则节点 i 一定是叶子节点

C. 树中度为 1 的节点数可能大于 1
D. $\lfloor \frac{n}{2} \rfloor$ 一定是所有非叶子节点中，编号最大的那个

9. 思考：二叉树的第 i 层上（设根为第 1 层）至多有多少个节点？

10. 思考：二叉树的叶子节点可能出现在哪些层？完全二叉树呢？满二叉树呢？

11. （某年 408 真题）已知一棵完全二叉树的第 6 层（设根为第 1 层）有 8 个叶节点，则该完全二叉树的节点个数最多是_____

A. 39 B. 52 C. 111 D. 119

12. （某年 408 真题）若一棵完全二叉树有 768 个节点，则该二叉树中叶节点的个数是_____

A. 257 B. 258 C. 384 D. 385

13. （某年 408 真题）对于任意一棵高度为 5 且有 10 个节点的二叉树，若采用顺序存储结构保存，每个节点占 1 个存储单元（仅存放节点的数据信息），则存放该二叉树需要的存储单元数量至少是_____

A. 31 B. 16 C. 15 D. 10

14. （某年 408 真题）在一棵度为 4 的树 T 中，若有 20 个度为 4 的节点，10 个度为 3 的节点，1 个度为 2 的节点，10 个度为 1 的节点，则树 T 的叶节点个数是_____

A. 41 B. 82 C. 113 D. 122

15. 由于二叉树的链式存储本质上是二叉链表，所以某同学想到了用静态链表存储二叉树。该同学设计如下：

代码 4.2

```
struct node {
    ElemType data;
    int lchild;
    int rchild;
};
struct node nodes[MaxSize];
```

其中，nodes 是一个用于存储二叉树节点的数组，lchild 和 rchild 分别用于存储左、右孩子节点在数组中的下标；若相应的孩子节点不存在，则使用 -1 表示。根节点存放

于数组下标 1 处。下列说法中正确的是_____

A. 该存储方式下，若某节点存放在下标 i 处，则其左孩子（若存在）一定存储在下标 2i 处。
B. 该存储方式下，同一个节点的所有孩子节点在物理存储上一定是紧挨着的
C. 该存储方式通常不适用于保存最大节点数量不确定的二叉树
D. 假设节点数为 n，则所有节点从上到下、从左到右地依次存放于数组下标 1-n 处。

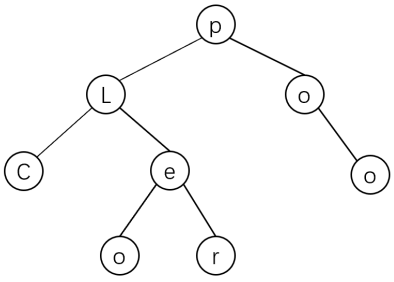
16. 设某二叉树 T 保存 int 类型的数据，采用二叉链表存储：

```
struct node {
    int data;
    node* lchild;
    node* rchild;
};
```

1）请你编写一个算法，将 T 中所有值为 5 的元素改为 10，算法返回 void 即可。

2）若算法要求：当修改了至少一个元素时返回 1，否则返回 0，请你思考该如何实现，有条件的话可以写出代码。

17. 某二叉树 T 保存 char 类型的数据，如下图所示。

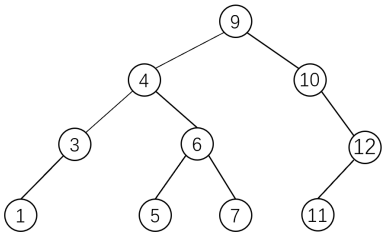


该树采用二叉链表存储：

```
struct node {
    char data;
    node* lchild;
    node* rchild;
};
```

1）请你写出后序遍历该树 T 的代码，打印每个节点的值。
2）打印出的字符序列是什么？

18. 设某二叉树 T 保存 int 类型的数据，如下图所示。



该树采用二叉链表存储：

```
struct node {
    int data;
    node* lchild;
    node* rchild;
};
```

1）观察这棵树，该二叉树中，任意一个节点的左子树与该节点存在什么关系？右子树与该节点存在什么关系？

2）如果对这棵树进行中序遍历，得到的序列是什么？该序列有何特点？

3）你能否写出一个函数，功能是在这棵树中插入一个新的值为 e 的数据，并且加入新节点后，这棵树依然具有第 1）问及第 2）问中的特性？

```
void InsertTree(node* &root, int e) {

}
```

有了前面的铺垫，相信你对树与二叉树的基本理论内容已经有了比较深刻的认识，并且对树中的递归代码建立起了一定的感觉。接下来就让我们进入本章中真正有趣（但也有一定难度）的部分，来看一些有实际应用价值的特殊的树。

4.3 哈夫曼树与哈夫曼编码

请思渡者考虑以下最为经典的“合并柴薪”问题：

在初始火焰的熔炉周围，散落着 n 堆柴薪，重量分别为 $w_1, w_2, ..., w_n$，你每次可以搬运任意两堆柴薪 i 和 j，将它们合成一堆重量为 $w_i + w_j$ 的新柴薪，这个过程需要消耗的体力为 $w_i + w_j$。显然，$n - 1$ 次搬运之后，就只剩下一堆柴薪了。请问你能否设计最优的合并次序方案，使得耗费的体力最少？并给出消耗的体力值。

例如有 3 堆柴薪，重量分别为：

$$[3, 6, 11]$$

第一次，你将 6 和 11 合并到一起，消耗体力 17，还剩两堆柴薪：

$$[3, 17]$$

第二次，你将 3 和 17 合并到一起，消耗体力 20，只剩一堆柴薪：

$$[20]$$

合并过程一共消耗了 17 + 20 = 37 点体力。显然，若你先合并 3 和 6，则可以达成更少的体力消耗：(3+6)+(9+11)=29 点体力。

我们看一个稍微复杂一点的例子，五堆柴薪重量分别是：3, 6, 2, 3, 4。我们先随意将它们合并，看看过程中会发生什么。合并过程可以如图 4.10 所示：

可以看出，一堆柴薪参与合并后，并不会从后续的合并过程中消失，而是会作为新柴薪的一部分，继续参与后续合并。例如，初始重量为 3 的柴薪与重量为 6 的柴薪合并后，形成一堆重量为 $9 = 3 + 6$ 的柴薪。虽然新柴薪的重量变成了 9，但其中仍然包含原来重量为3 的柴薪（9=3+6 嘛）。因此，当这堆重量为 9 的柴薪再次参与合并时，产生的消耗中仍然包含原来那 3 的重量。也就是说：**每堆原始柴薪在每次参与合并时，都会为总体力消耗贡献一次自己的初始重量**。于是，若初始重量为 w_i 的柴薪参与了 k_i 次合并，那么它对总消耗的贡献就是 $w_i k_i$。因此，整个合并过程的总消耗可以表示为

$$W = \sum_{i=1}^{n} w_i k_i$$

另一方面，由图 4.10 可以看出，原始柴薪越靠后参与合并，其总共参与的合并次数就越小，如 4 只参与了 1 次合并，而 3 和 6 却参与了 4 次合并。因此，自然要将原

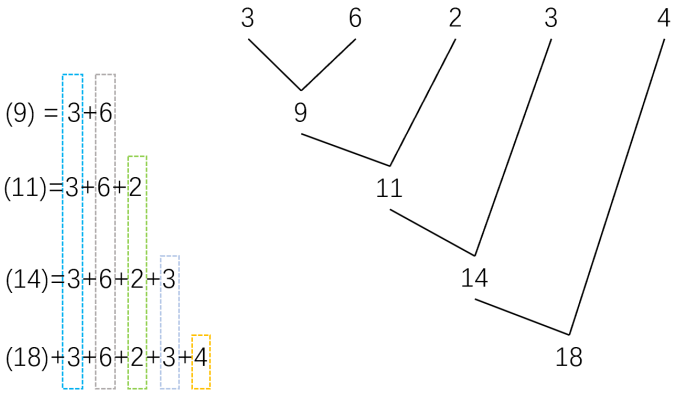


图 4.10 柴薪的合并过程

始重量更大的柴薪放在更后面合并，这样其参与的合并次数就小，因而总体造成的体力消耗就更小。

基于以上讨论，很容易设计出最优的合并方案：每次从当前所有柴薪中，选出两堆重量最小的进行合并，直至最后合并完成。对于 3, 6, 2, 3, 4 这五堆柴薪，一种最优的合并方式如图 4.11 所示。

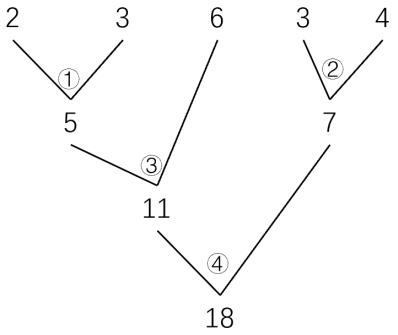


图 4.11 最优合并过程

第①步，将最小的 2 和 3 合并，消耗体力 5，合并后新的柴薪：

$$[5, 6, 3, 4]$$

第②步，将这四堆中最小的 3 和 4 合并，消耗体力 7，合并后新的柴薪：

$$[5, 6, 7]$$

第③步，将最小的 5 和 6 合并，消耗体力 11，合并后新的柴薪：

$$[11, 7]$$

第④步，将最小的 (也是仅剩的)11 和 7 合并，消耗体力 18，合并成最终的一堆柴薪：18。可以看出，消耗的总体力 $= 5 + 7 + 11 + 18 = 41$。换个角度，2、3 贡献了 3 次自己的初始重量、6 贡献了 2 次，3 和 4 贡献了 2 次，因此 $W = \sum_{i=1}^{n} w_i k_i = (2+3)*3+6*2+(3+4)*2 = 41$。

相信你早已看出来了，合并的过程其实就构成了一棵二叉树，原始的五堆柴薪就是五个叶子节点，而每堆初始柴薪参与的合并次数 k_i 就是该叶子节点到根节点的**路径长度**。在以上二叉树中，每个节点的数值代表了要搬动这堆柴薪需要消耗的体力。事实上在许多应用中都要类似现象，我们在许多场景下会将树中的节点赋予一个表示某种意义的数值，称为节点的**权**。从树根到一个节点的路径长度与该节点上权值的乘积，就叫做该节点的**带权路径长度 (WPL)**（可以理解为这堆柴薪在后续合并过程中还会贡献的总体力消耗），而树中所有叶子节点的带权路径长度之和就称为这棵树的**带权路径长度 (WPL)**，记为。

$$\mathrm{WPL} = \sum_{i=1}^{n} w_i k_i$$

其中 w_i 是第 i 个叶节点的权值，k_i 是该叶节点到根节点的路径长度。在含有 n 个带权叶节点的二叉树中，带权路径长度最小的二叉树就称为**哈夫曼树**。可以看到在该定义下，“合并柴薪”的问题其实就是求以这 n 堆柴薪为叶节点的哈夫曼树，其带权路径长度是多少。

通过这个例子，我们也理解了“带权路径长度”以及哈夫曼树的应用价值。图 4.11 所示的过程其实也就是由给定的若干个节点，构造一棵哈夫曼树的过程。这里将该过程完整地叙述如下：

1. 初始化。根据给定的 n 个权值，构造 n 棵仅包含一个节点的二叉树，每个节点的权值分别为 $w_1, w_2, \ldots, w_n$。此时，每棵树都只有一个根节点，且该节点同时也是叶节点。

2. 选择与合并。从当前所有二叉树中，选择根节点权值最小的两棵二叉树，将它们合并为一棵新的二叉树。具体而言，以一个新节点作为根节点，将选出的两棵二叉树分别作为新根节点的左、右子树，并将新根节点的权值设置为这两棵树的根节点权值之和。

3. 更新集合。将刚刚合并得到的新二叉树加入当前的二叉树集合，同时从集合中移除原先被合并的两棵二叉树。这样，集合中的二叉树数量便减少一棵。

4. 重复执行步骤 2 和步骤 3，直到当前集合中只剩下一棵二叉树。

5. 得到哈夫曼树。最终剩下的二叉树就是所构造的哈夫曼树。其所有叶节点的带权路径长度之和最小。

需要注意的是，选择权值最小的两棵树进行合并时，若存在权值相同的节点，可以任选符合条件的两棵树。不同的选择可能构造出形态不同的哈夫曼树，但所得的最小带权路径长度相同。

哈夫曼树的性质

从以上哈夫曼树的构造过程可以看出哈夫曼树具有如下特点：

1. 最初有 n 个叶节点，合并需要 $n-1$ 步，因此新增共 $n-1$ 个节点。所以：**具有 n 个叶节点的哈夫曼树，其总结点数一定是 $n+n-1=2n-1$**。
2. 所有有孩子的节点一定都是由 2 棵子树合并而来，因此**哈夫曼树中不存在度为 1 的节点**。

这两条性质在某种角度其实是相通的。

4.3.1 哈夫曼编码

我们知道数字信号是一种二值信号，通过有不同特征的电信号来表示 0 或 1。将字符编码成二进制信号，就可以实现数字信号的传输。如最经典的 ASCII 编码，'A'的编码为 65，传输过程中的数字信号为 01000001，而'a'对应的数字信号为 01100001。这种编码方式是**等长编码**，对所有字符的编码长度相等，都是 8 位，都需要 8 个 bit。但是，实际应用中，各个字符的使用率却大有差异。例如一篇新闻报道中，'e'和' '（空格）等字符的数量一定远大于'$'和'x'等字符。

显然，如果我们能够将出现频率更高的字符编码得更短，而频率低的字符编码较长，就可以减少所需的传输空间。

然而这种**不定长编码**也带来了一个新的问题：对方收到之后，如何解码？比如你将 A、B、C 分别编码为 0、1、01，那么收到 0101，应解码为 ABAB 还是 ABC 或 CC？

因此，我们还需要为编码方案增加一个重要的约束：**任何一个字符的编码，都不能是另一个字符编码的前缀**。满足这一条件的编码称为**前缀编码（Prefix Code）**。

> *** 思维拓展**
>
> 在诸多英文文献中，**前缀编码（Prefix Code）**也被称为“Prefix-free Code”，即“无前缀编码”。笔者认为后者在语义上更加直观。

于是问题就变成了：给定 n 个字符 $c_1, c_2, ..., c_n$ 及它们的权重（即字符出现的次数）$w_1, w_2, ..., w_n$，求一种前缀编码方案让总编码长度达到最小。

我们可以借助二叉树来解决这样的编码问题。我们规定从根节点向左走代表数字 0，向右走代表数字 1。如图 4.12 所示，这样，从根节点到任意节点的路径，就可以用一串二进制数字表示。如果我们将每个字符放在一片叶节点上，那么从根节点到该叶节点的路径，就可以作为这个字符的编码，路径长度也即是编码长度。该例中，A 的编

码为 000，到根节点的路径长度为 3，即是编码长度；B 的编码为 01，路径长度、编码长度为 2；C 的编码为 1。

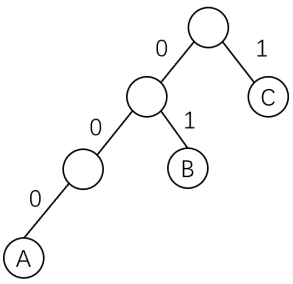


图 4.12 用二叉树进行不定长编码

假设字符 c_i 出现的次数为 w_i，其在二叉树中对应叶节点的深度为 k_i，则传输所有字符所需的总比特数为

$$L = \sum_{i=1}^{n} w_i k_i$$

而这恰好就是该树的带权路径长度（WPL）。因此，在字符及其出现次数固定的情况下，使总编码长度最小，等价于使二叉树的带权路径长度最小。因此求解该问题，实际上就是构造哈夫曼树的过程。例如，A, B, C, D, E 五个字符出现次数依次是 3, 6, 2, 3, 4，则可以构造如图 4.13 所示的哈夫曼树。

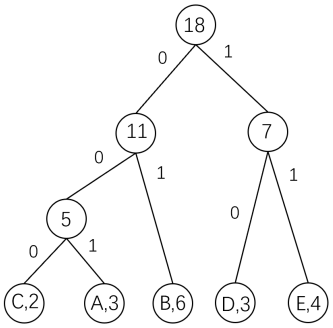


图 4.13 哈夫曼编码

A、B、C、D、E 的编码与长度如下：

	编码值	编码长度/路径长度
A	001	3
B	01	2
C	000	3
D	10	2
E	11	2

同时，由于叶节点不可能是其他节点的祖先，因此，任何一个字符的编码都不会成为另一个字符编码的前缀。这恰好满足了前缀编码的要求。这种由哈夫曼树产生的编码称为**哈夫曼编码**。显然对于相同字符集存在多种哈夫曼编码，但它们的编码长度是相同的。

提示

当然，你也可以将向左定义为 1，向右定义为 0。类似的思想已经提示过多次，此处是最后一次了，我保证。

在“合并柴薪”问题中，一堆柴薪参与合并的次数越多，它的重量就会在总代价中被累加越多次；在“字符编码”问题中，一个字符的编码越长，它消耗的传输空间就越多。二者共同的底层逻辑是：**每个对象都有自己的权值，而我们可以通过不同的安排，决定这个权值会被计入最终结果多少次。**

思考

对于“合并柴薪”问题，你能否写出一个函数，输入一组重量，输出最优的体力消耗值（只要输出这个数值就行）？

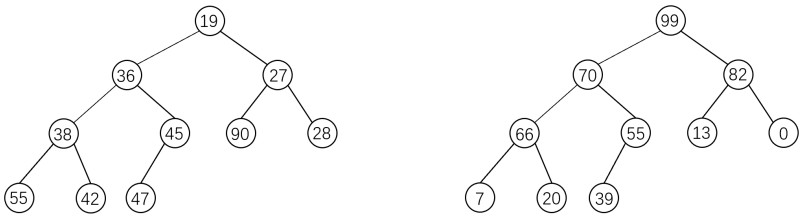
通过上面的思考你也许会发现，解决此类问题并不需要真的构造出一棵哈夫曼树。编码问题亦是如此，同样不需要在计算机中存储哈夫曼树。事实上，哈夫曼树的意义在于，对于这类问题，它为我们提供了统一的数学模型和统一的求解方法。

还有一个问题，以“合并柴薪”问题为例，我们需要每次从当前所有柴薪堆中选出两个重量最小的，合并后再放回去。使用什么数据结构保存柴薪堆，能使这个操作达到最快？显然若使用普通线性表，对于频繁插入删除的场景，链表一定更好。选择两个最小、合并后再插入的时间可以达到 O(n)。

我们在接下来的介绍中将看到，如果使用**堆**保存所有柴薪的重量，这个选择加合并后插入的过程可以达到 O(logn)。如果你熟悉 C++，应该已经想到了使用一个 std::priority_queue 来保存所有柴薪堆，实际上，C++ 标准库中的 priority_queue 正是通过**堆**来实现的。

4.4 堆

堆是一棵完全二叉树，满足其中任何一个节点的关键字的值都不大于（或不小于）其左右子树中任何一个节点的关键字的值。完全二叉树的根节点称为**堆顶**。若任何一个节点的值都不大于其左右子树，这样的堆叫做**最小堆**，也叫**小顶堆**或**小根堆**，堆顶元素一定是堆中最小的；若任何一个节点的值都不小于其左右子树，这样的堆叫做**最大堆**，也叫**大顶堆**或**大根堆**，堆顶元素一定是堆中最大的。图 4.14 （左）和（右）分别展示了一个最小堆和一个最大堆。



（左）最小堆　　（右）最大堆

图 4.14

请注意：以最大堆为例，最大堆能保证：①根节点一定是最大的； ②任何一个节点，都比其左右子树中的节点大。但是最大堆无法保证上层节点一定大于下层节点，因为它们不一定处于同一棵子树中。如图 4.14（右）中，节点 39 虽处于第 4 层，但它比第 3 层的 13、0 还要大。

*** 思维拓展**

关键字（Key）可以理解为用于标识数据或确定数据比较关系的一个或多个数据项。在排序等问题中，通常根据关键字的值来确定数据的大小关系。例如，对于字符数据，可以将其 ASCII 码值作为关键字；对于游戏中的武器数据，可以将武器的 id 或攻击力作为关键字。

4.4.1 堆及其常见操作的实现

由于堆是一棵完全二叉树，因此用一个数组顺序存储就非常合适（参见 4.2.1，第 80 页）。使用数组存储一个堆也是最常见、最方便的实现方式。通常情况下通过一个数组和一个整数，就可以表示一个堆：

```
1 int heap[MaxSize+1];
2 int size = 0;  // 堆中元素的数量
```

注意我们定义根节点编号为 1，从数组下标 1 开始存储。显然这种存储方式下，heap[1] 就能获取堆顶元素的值，时间复杂度为 O(1)。

为了方便大家建立对堆的感觉，我们先来看如何将一个已有若干元素的完全二叉树（数组存储）调整成一个堆。本节所有对堆操作的介绍均以最大堆为例。假设数组 arr[] 中已经存储了完全二叉树的 n 个节点，可以通过如下操作将这个完全二叉树调整成一个堆：

将完全二叉树调整为堆

从编号为 $\lfloor \frac{n}{2} \rfloor$ 的节点，即最后一个非叶子节点开始，逐一向前检查每一个节点，对每个节点进行**向下调整**操作。将节点**向下调整**是指，将该节点与其两个孩子（若存在）中较大的那个比较，若该节点小于较大的那个孩子，则将该节点“下沉”，与较大的那个孩子交换位置。交换后，该节点到了新的位置，继续检查能否继续“下沉”，直到该节点沉入某个无法继续与孩子交换的位置，或沉到叶节点。我们以图 4.15 所示的完全二叉树为例，介绍详细的流程。

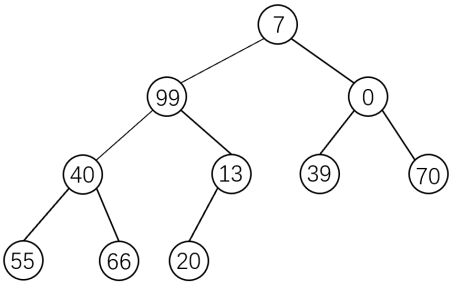


图 4.15

该树共有 10 个节点，第一个非叶子节点编号为 $\lfloor \frac{10}{2} \rfloor = 5$，因此我们逐个检查编号为 5、4、3、2、1 的节点，尝试将它们下沉。

① 如图 4.16，检查节点 13，它只有一个左孩子，因此与其左孩子比较，发现左孩子更大，所以交换。交换后，13 已经沉到底。继续检查下一个节点；

② 如图 4.17，检查节点 40，它有两个孩子节点，其中右孩子 66 较大，因此将该节点

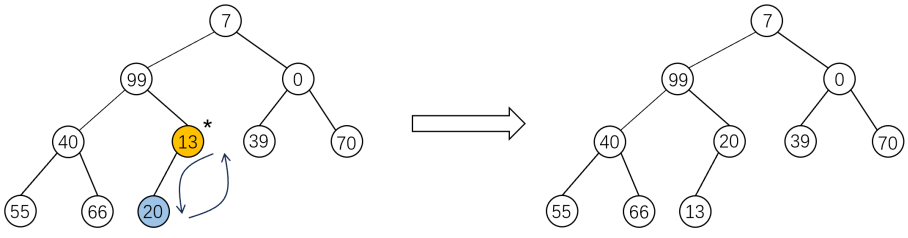


图 4.16

40 与右孩子 66 进行比较，该节点小于右孩子，于是下沉；

③ 如图 4.18，检查节点 0，它有两个孩子，其中右孩子 70 较大，因此将该节点 0 与

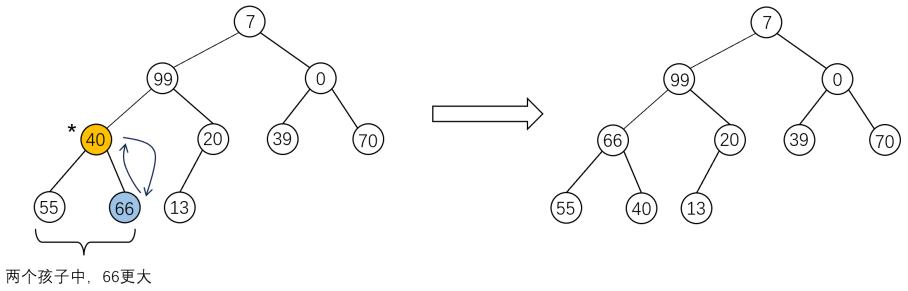


图 4.17

右孩子 70 进行比较，该节点小于右孩子，于是下沉。已沉底，继续检查下一个节点。

④ 如图 4.19，检查节点 99，它有两个孩子，其中左孩子 66 较大，因此将该节点 99

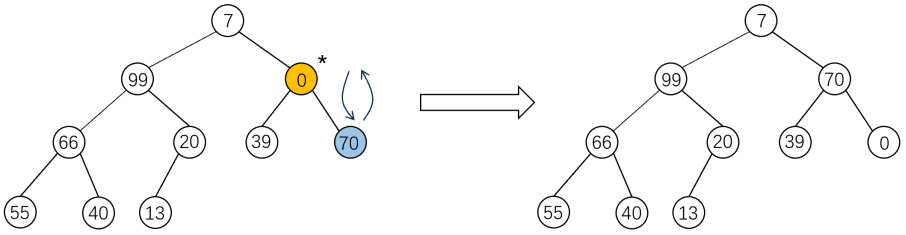


图 4.18

与左孩子 66 进行比较，该节点不小于 66，无需调整，继续检查下一个节点。

⑤ 如图 4.20，检查节点 7，它有两个孩子：99 和 70，因此 7 与 99 交换；交换后继续

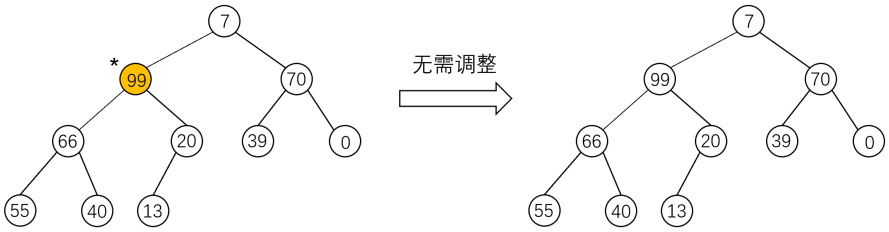


图 4.19

判断 7 是否应该继续下沉：此时 7 的两个孩子分别是 66 和 20，与 66 比较，7 比 66 小，继续和 66 交换位置；之后继续和两个孩子中较大的 55 比较，小于 55，和 55 交换，到达叶子节点，节点 7 的向下调整完成。

经过了上述过程，堆就已经建好了。从一个已有的完全二叉树建堆的过程如代码 4.3 所示。注意在向下调整的函数 DownAdjust() 中，while 循环终止有两种可能，一是

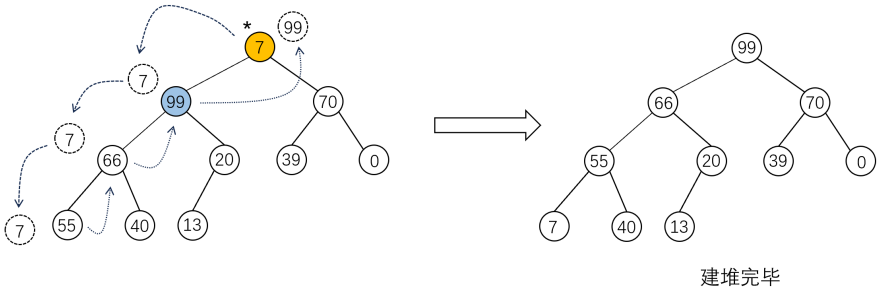


图 4.20

2*i 已经大于 n 了，即该节点已经调整到了叶子节点，没有孩子了；二是该节点已经调整到了某位置，其值不小于两个孩子中更大的那个了，无需继续下沉。

代码 4.3 堆的向下调整及建堆代码

```
// 元素数量为 n 的完全二叉树中，将编号为 i 的元素向下调整
    void DownAdjust(int arr[], int n, int i) {
        // i 表示下沉过程中，该节点已经“沉入”到了编号为多少的地方。
        // 最后一个节点编号是 n，因此 2*i<=n 表示该位置还有子节点
        while(2*i <= n) {
            int max_child = arr[2*i];   // 先假设左孩子是更大的
            int max_child_idx = 2*i;
            // 事实上右孩子存在，且右孩子更大
            if(2*i+1 <= n && arr[2*i+1] > arr[2*i]) {
                max_child = arr[2*i+1];
                max_child_idx = 2*i+1;
            }
            // 该元素较小，应该继续下沉
            if(arr[i] <= max_child) {
                int t = arr[i];
                arr[i] = arr[max_child_idx];
                arr[max_child_idx] = t;
                i = max_child_idx;
            }
            // 已经不满足继续下沉的条件，向下调整完成。
            else {
                break;
            }
        }
    }
// 将具有 n 个元素，存储在 arr[] 数组中的完全二叉树调整成一个堆。
    void CreateHeap(int arr[], int n){
        for(int i=n/2; i>=1; i--) {
            DownAdjust(arr, n, i);
        }
```

```
31      }
```

向下调整，至多需要“下沉”约 logn 次，时间复杂度为 O(logn)。而对于建堆操作，需要将约 n/2 个元素向下调整，每个元素向下调整的时间复杂度为 O(logn)，那整个建堆过程就是 O(nlogn)。

思考

可是，建堆过程是 O(nlogn) 这个说法真的是正确的吗？

这里我只给出正确的说法：建堆操作的时间复杂度应该是 O(n)，而非 O(nlogn)。这是一个很有意思的问题。大家可以从直观上想一想为什么。严谨的证明过程较复杂，感兴趣的同学可以参考《算法导论》。

向上调整、向堆中插入元素

要往堆中加入一个元素，可以先将该元素放在数组尾部（即完全二叉树的最后一个节点后面），再将该元素**向上调整**。与向下调整类似，**向上调整**就是将该节点不断地与其父节点比较，若大于父节点，则将该节点“上浮”，与父节点交换位置，之后不断重复该操作，直至不满足上浮条件，或已经上浮到根节点，如此，该节点的向上调整就完成了。向上调整与插入操作的代码实现如 4.4 所示。

代码 4.4 向上调整及插入操作

```
// 元素数量为 n 的完全二叉树中，将编号为 i 的元素向上调整
    void UpAdjust(int arr[], int n, int i) {
        // i 表示上浮过程中，该节点已经上浮到了编号为多少的地方。
        while(i/2 > 0) {    // 此位置还有父节点
            int father_idx = i/2;
            if(arr[i] > arr[father_idx]) {
                int t = arr[i];
                arr[i] = arr[father_idx];
                arr[father_idx] = t;
                i = i/2;
            }
            else {
                break;
            }
        }
    }
    // 向当前有 n 个元素的堆 arr 中，插入新元素 e
    void InsertHeap(int arr[], int& n, int e) {
        n += 1;
```

```
        arr[n] = e;
        UpAdjust(arr, n, n);
    }
```

与向下调整类似，向上调整的时间复杂度同样为 O(logn)。显然插入操作的时间复杂度也是 O(logn)。

提示

如果为了更加能体现出数组表示的数据结构是一个堆，可以将一个数组以及堆中当前元素数量进行一个封装：

```
#define MaxSize 100
    struct Heap {
        int arr[MaxSize+1];
        int n;  // 堆中当前元素数量
    };
```

这样，相关操作的接口语义会更加明确一些：

```
void UpAdjust(Heap& h, int i);  // 将堆 h 中编号为 i 的元素向上调整
void InsertHeap(Heap& h, int e); // 向堆 h 中插入一个新元素 e
```

等。此外，注意插入操作需考虑堆满的情况，我们在前面介绍过，为将关注重点放在算法的核心思想上，在许多合适的地方我们将省去这些代码，以减轻理解负担。后面将不再提及此事。

删除堆顶元素

要删除堆顶元素，只要用**堆底元素**，即完全二叉树的最后一个节点，覆盖掉堆顶元素，然后将新的堆顶向下调整即可。如代码 4.5 所示。

代码 4.5 删除堆顶元素

```
void DeleteTop(int arr[], int& n) {
    if(n == 0) return ;
    arr[1] = arr[n];
    n -= 1;
    DownAdjust(arr, n, 1);
}
```

显然删除堆顶元素的操作时间复杂度为 O(logn)。

注意以上所有操作都是以最大堆为例来介绍的。最小堆的操作类似。

4.4.2 堆的经典应用

下面介绍一些堆的经典的、最常见的应用场景。

1. 动态维护最大值或最小值

堆的一个重要价值在于，能够在数据不断变化时，以 O(logn) 的时间复杂度插入和删除元素，并以 O(1) 的时间复杂度获取当前的最大值或最小值，也就是能高效地动态维护优先级最高的元素。例如在 4.3（第 97 页）中介绍的“合并柴薪”以及“字符编码”问题，需要不断地从容器中删除两个最小的，合并之后再插入容器，这种问题非常适合使用堆来实现。

一个比较有趣的事情是，在后面的介绍中我们将看到，求解最短路径的大名鼎鼎的迪杰斯特拉算法，使用堆可以进一步提升性能。

2. 找出大量数据中最大或最小的 k 个

例如，如果需要从 10 万件商品中，筛选价格最低的 k 件商品，最朴素的做法是对这 10 万件商品遍历 k 次，第 i 次遍历找出第 i 小的价格。但是这需要某种手段标记 i 之前已经被找到了的元素。算法时间复杂度为 O(nk), 并且需要额外空间。

如果使用排序算法，我们将在后面的内容中介绍到，最好可以达到 O(nlogn) 的时间将这 10 万件商品排成一个有序的列表，再取前 k 件即可。时间复杂度为 O(nlogn)，空间复杂度最好（堆排序）为 O(1)。

一种更巧妙的做法是，我们建立一个具有 k 个元素的**最大堆**，保存 10 万个元素中最小的 k 个，让这 100000 个元素“争抢进入这个堆的权利”，谁更小，谁就能进入堆中。显然，只要某个元素至少小于堆中的某一个元素，该元素即可进入堆中。而事实上对最大堆来说：

至少小于一个元素 => 小于堆顶元素一定成立

小于堆顶元素 => 至少小于一个元素一定成立

因此至少小于一个元素 <=> 小于堆顶元素

因此我们逐一检查这 100000 个元素，将它们依次与堆顶元素进行比较，若小于堆顶元素，说明该元素是当前检查过的所有元素里前 k 小的，此时若堆已满，令其覆盖堆顶元素并调整堆，保证堆顶依然是堆中最大的；若堆未满，直接将其加入堆中。直至检查完所有元素，堆中保存的，即是这所有 10 万个元素中最小的 k 个元素。

该方法需要对所有 n 个元素进行一遍访问，每次访问可能需要对堆进行调整，对于有 k 个元素的堆，调整堆的复杂度为 O(logk)，因此整体时间复杂度为 O(nlogk)。与

排序算法可以达到的 O(nlogn) 相比，有所提升。同时该方法需要一个额外的 k 个空间的辅助堆，空间复杂度为 O(k)。

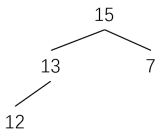
习题 4 （参考答案见习题 4 参考答案，第 216 页）

1. 若某哈夫曼树度为 2 的节点有 7 个，能否确定度为 0 的节点有几个？

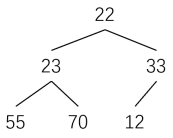
2. 若某哈夫曼树度为 2 的节点有 7 个，能否确定这棵哈夫曼树共有几个节点？

3. 练习：有 5 个元素 A、B、C、D、E 的权值分别为 1,2,2,4,5，将这五个元素作为叶节点构造一棵哈夫曼树。

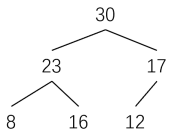
4. 下列完全二叉树中，是最小堆的有_____，是最大堆的有_____。



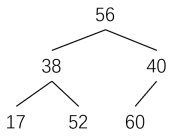
(A)



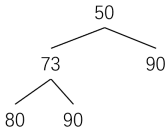
(B)



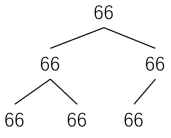
(C)



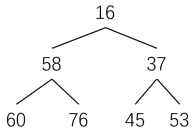
(D)



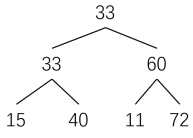
(E)



(F)

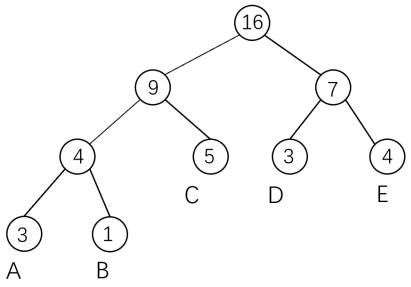


(G)



(H)

5. 对于下图所示的二叉树，各节点中的数字表示该节点的权值。下列说法错误的是_____



A. 节点 B 的带权路径长度为 3

B. 这棵树的带权路径长度为 60

C. 以 A、B、C、D、E 为叶节点构造一棵二叉树，图示的树能使得带权路径长度达到最小

D. 以 A、B、C、D、E 为叶节点构造一棵二叉树，图示的树并不是唯一的带权路径长

度最小的树

6. C++ 标准库已经封装好了一个数据结构：priority_queue，这是一个最小堆，可以通过：

```
priority_queue<int, vector<int>, greater<int> > min_heap;
```

定义并初始化一个初始为空的，名为 min_heap 的最小堆，可容纳 int 类型元素。其中

```
priority_queue<int, vector<int>, greater<int> >
```

是一种类型，只不过看起来比较长而已。读者大可不必关注其细节，只知道上述代码用该类型定义的 min_heap 是一个最小堆，并直接使用相关操作即可。min_heap 支持以下操作：

1） min_heap.push(e); // 向堆中插入一个值为 e 的元素；

2） int a = min_heap.top(); // 获取堆顶元素，赋值给 a(但不删除堆顶元素)；

3） min_heap.pop(); // 删除堆顶元素；

4） min_heap.size(); // 返回堆中元素数量；

5） min_heap.empty(); // 若堆为空，返回 true，否则返回 false。

对初始为空的 minHeap 依次执行以下操作，请指出每一次使用 min_heap.top() 获取堆顶元素时，获取到的元素值是多少。

1） min_heap.push(5);

2） min_heap.top(); // 1

3） min_heap.push(3);

4） min_heap.push(4);

5） min_heap.top(); // 2

6） min_heap.pop();

7） min_heap.top(); // 3

7. 请你借助上题给出的 priority_queue 及其相关操作来解决“合并柴薪”问题。输入一个长度为 n 的数组 w[] 表示各堆柴薪的重量（均为正整数），返回一个正整数，表示最优情况下消耗的体力值。

```
int MinCost(int w[], int n) {
    priority_queue<int, vector<int>, greater<int>> min_heap;
    // 在下方完成你的代码
}
```

8. （某年 408 真题）对 n 个互不相同的符号进行哈夫曼编码，若生成的哈夫曼树共有 115 个节点，则 n 的值是

A. 56　B. 57　C. 58　D. 60

9. （某年 408 真题）若某二叉树有 5 个叶节点，其权值分别为 10，12，16，21，30，则其最小的带权路径长度（WPL）是_____

A. 89　B. 200　C. 208　D. 289

10. 请你实现一个算法，求使用二叉链表存储的二叉树的带权路径长度。空树直接返回 0。

```
struct node {
    // int data;  本题仅节点权重有意义，数据可忽略
    int weight;
    node* lchild;
    node* rchild;
};
```

注意：二叉树的带权路径长度（WPL）定义为所有叶节点的带权路径长度之和，非叶节点不参与计算。

11. 某大陆散布着多个地下遗迹，这些地下遗迹都可以被建模成一棵二叉树，根节点为遗迹的入口，向下探索，每个节点代表一个房间，所有处于叶节点的房间都能挖出宝藏，weight(weight>0) 表示宝藏的价值。由于遗迹深处的宝藏获取难度更大，某位名为 CoreLoop 的探险者决定权衡风险与收益，$\frac{1}{\text{节点深度}} * \text{宝藏价值}$ 定义为房间的“诱人值”。请你实现一个算法，帮 CoreLoop 算出二叉树 T 中诱人值最大的房间诱人值是多少。（根节点深度定义为 1；空树诱人值定义为 0）。

```
double CalcAppeal(node* T) {
    double ans = 0;
    // your codes...

    return ans;
}
```

二叉树的节点定义同上一题所示。你完全可以实现其他函数，并在 CalcAppeal 中调用。

12. 某位名为 CoreLoop 的探险者误入了一扇魔法门从而被传送到了一片地底世界，他只能通过前方的地下城回到大陆。该地下城可以被建模为一棵二叉树，每个节点代表一个房间，根节点表示入口，也是必须经过的房间。节点的 weight(weight>0) 表示进入该房间所需消耗的能量，叶节点代表所有可以回到大陆的出口。CoreLoop 可以沿任何一条路径经出口走出地下城。请你实现一个算法帮助 CoreLoop 求出最优路径（消耗能量最小）下，逃离地下城需要消耗的能量值。root 保证不是空树。

```
struct node {
    int weight;     // 进入房间需要消耗的能量
    node* lchild;
    node* rchild;
};
int MinEscapeCost(node* root) {

}
```

4.5 二叉查找树

在一组混乱、无序的线性表中查找关键字为某值的数据，平均时间复杂度为 O(n)。若数据存放在有序的顺序表中，使用二分查找平均时间复杂度可以达到 O(logn)。但是顺序表难以维护动态数据：当表中元素增加或删除时，需要移动大量元素，移动操作的平均时间复杂度为 O(n)。

二叉查找树的思想类似二分查找，将元素通过某种有序的方式组织起来，以达到较为高效的元素查找效率。经过本节的研究我们将看到，在二叉查找树中查找元素的平均时间复杂度为 O(logn)，插入、删除元素的平均时间复杂度同样是 O(logn)。

定义

二叉查找树要么是一棵空树，要么是：

一棵由根节点、左子树和右子树构成的二叉树，其中左子树和右子树均是二叉查找树，并且左子树上所有节点的关键字均小于根节点的关键字，右子树上所有节点的关键字均大于根节点的关键字。

图 4.21 展示了几棵不同的二叉查找树。可以看到，二叉查找树其实是一种有序的

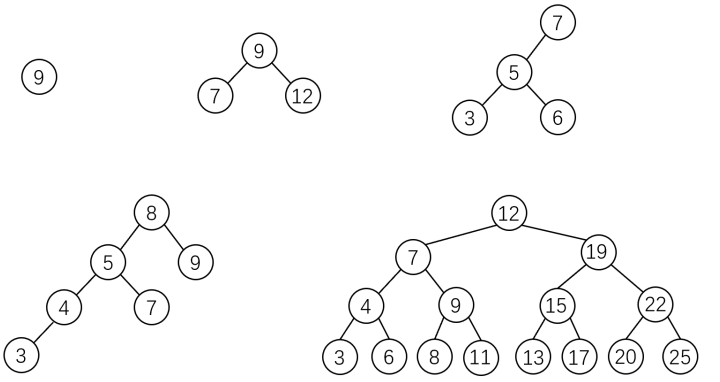


图 4.21 几棵不同的二叉查找树

树形结构。二叉查找树具有“左小、根中、右大”的特征，而按照中序遍历“左-根-右”的方式遍历二叉查找树，得到的序列就是一个有序序列。

二叉查找树最重要的意义就在于在动态变化的数据中较为高效地进行查找，如前文所述，其查找、插入和删除均能达到 O(logn) 的平均时间复杂度。接下来我们就来看看如何实现这些操作。

4.5.1 二叉查找树的实现

通常情况下，二叉查找树同样采用二叉链表实现：

```
struct BSTreeNode { // Binary Search Tree
    ElemType data;
    BSTreeNode* lchild;
    BSTreeNode* rchild;
};
```

为不引入额外的理解负担，我们假设 data 已经是某种可以比较大小的数据结构，data 本身即作为关键字。

1. 查找操作

如何在二叉查找树 T 中，查找关键字值为 e 的元素？

由于二叉查找树的特点，根节点其实作为分界线，将左右两棵子树划分成了“两个界域”，若 e 的值大于根节点，则它不可能出现在左子树中，如图 4.22，直接递归地在右子树中查找即可。

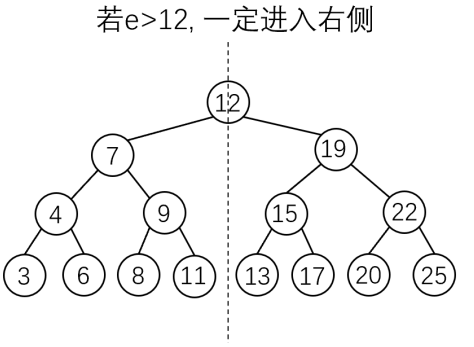


图 4.22

类似地，若 e 更小，则它不可能出现在左子树中。因此在二叉查找树中查找元素，不必像无序线性表那样将所有元素遍历一遍，而是可以舍弃掉大量元素的比较，因此效率更高。如果 e 存在，一定会在查找过程中找到。如果 e 不存在，则递归一定会进行到某个空节点。

具体来说，查找过程如下：从根节点开始，

1. 若当前节点为空，说明查找失败，返回；否则进入第 2 步。
2. 若 e 的值等于当前节点，说明查找成功，访问它；
3. 若 e 的值小于当前节点，说明应该往左，因此递归地在当前节点的左子树中查找；
4. 若 e 的值大于当前节点，说明应该往右，因此递归地在当前节点的右子树中查找。

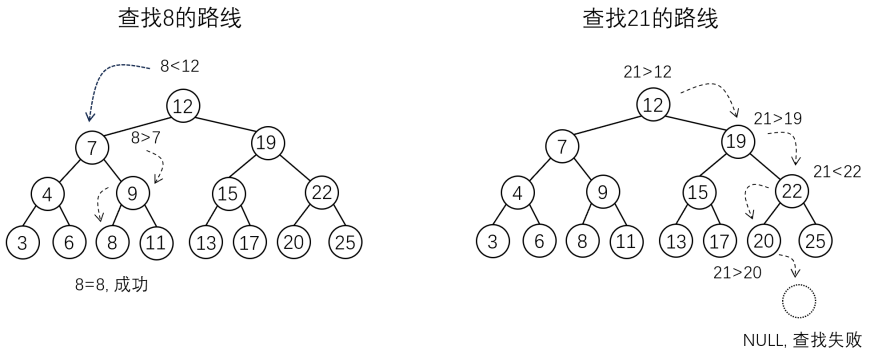


图 4.23

代码如下所示：

```
int BSTSearch(BSTreeNode* root, ElemType e) {
    if(root == NULL) {
        return 0;   // 查找失败，e 不存在
    }
    if(e == root->data) {
        visit(root);    // 找到了，做你想对它做的事吧！
        return 1;   // 查找成功
    }
    if(e > root->data) {
        return BSTSearch(root->rchild, e);
    }
    if(e < root->data) {
        return BSTSearch(root->lchild, e);
    }

}
```

在图 4.23 所示的二叉树中查找 8 和查找 21 的过程如图所示。

显然平均情况下，若二叉树中有 n 个节点，比较次数约为 log(n) / 2，因此查找的平均时间复杂度为 O(logn)。

2. 插入操作

插入节点时，可以从根节点开始，将根节点元素与待插元素比较。若待插元素较小，则说明该节点需要在左子树中插入，此时，若：

① 左子树为空，则插入位置已经找到，直接插入；

② 左子树非空，则递归地进入左子树，进行同样的操作。

反之若待插元素大于根节点，则递归地进入右子树。如此递归进行下去，直至找到正确的位置。具体实现代码如下：

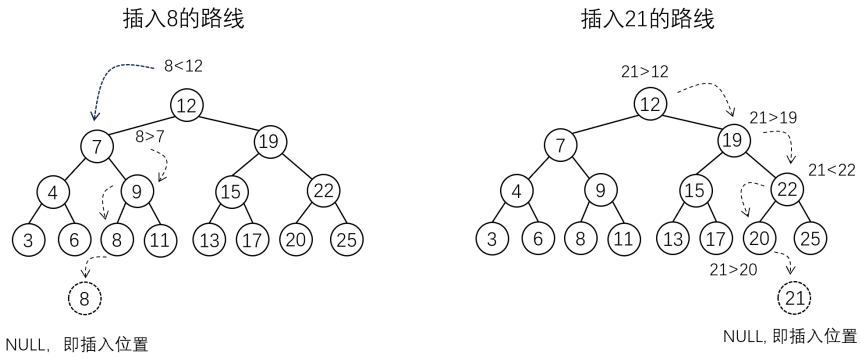


图 4.24

```
void InsertBSTree(node* &root, int e) {
    if(root == NULL) {
        // 空树，直接插入就行
        node* new_node = (node*)malloc(sizeof(node));
        new_node->data = e;
        new_node->lchild = NULL;
        new_node->rchild = NULL;
        root = new_node;
        return ;
    }
    // e 已存在，无需插入
    if(e == root->data) {
        return ;
    }
    if(e < root->data) {   // 待插元素较小
        // 递归地进入左子树
        InsertBSTree(root->lchild, e);
    }
    else if(e > root->data) {    // 待插元素较大
        // 递归地进入右子树
        InsertBSTree(root->rchild, e);
    }
}
```

在图 4.24 所示的二叉树中分别插入 8 和插入 21 的过程如图所示。注意上面的代码中，root 变量作为根节点的地址，该地址本身是有可能被修改的，因此一定要以引用作为参数，而不能是普通的拷贝传递。

3. 由已有数据建立二叉查找树

假设数组 arr[] 中存放了 n 个数据，用这些数据建立一棵二叉查找树的过程，其实就是将它们一一插入二叉查找树的过程。代码如下所示：

```
BSTreeNode* CreateBSTree(ElemType arr[], int n) {
    BSTreeNode* root = NULL;
    for(int i=0; i<n; i++) {
        InsertBSTree(root, arr[i]);
    }
    return root;
}
```

需要注意的是，即使是同样的一组数据，如果插入它们的顺序不同，也可能生成不同的二叉查找树。例如插入顺序为 1,2,3,4,5,6,7,8,9，会得到一棵“斜向右下的一条直线”的树，深度为 9；而插入顺序为 9,8,7,6,5,4,3,2,1，树形则会是“斜向左下的一条直线”。

4. 删除操作

在二叉查找树 T 中删除关键字为 e 的节点，如何做？

与插入操作不同，插入操作一定能“陷入”一棵空树作为插入位置，而对于删除操作，被删节点很可能还有复杂的子树。显然：

① 对于待删节点是叶节点，直接释放掉该节点即可。

② 若待删节点是非叶节点，该怎么办呢？常见的做法是找到待删节点的**中序前驱**或**中序后继**，用中序前驱或中序后继覆盖掉待删节点，然后问题便转化成了递归地在左子树中删除中序前驱，或在右子树中删除中序后继，如此向子树递归、向子树的子树递归、向子树的子树的子树递归..... 总会将问题化归为①，即待删节点是叶节点。

在二叉查找树中，比某节点小的最大节点一定是中序遍历序列中，该节点的前驱，称为该节点的**中序前驱**，相应地，比某节点大的最小节点称为该节点的**中序后继**。显然，节点的中序前驱一定是该节点左子树中最右边的节点（也就是从左子树根节点开始不断沿着 rchild 往右下走，直到 rchild 为 NULL 时的节点），而节点的中序后继一定是该节点右子树中最左边的节点（也就是从右子树的根节点开始不断沿着 lchild 往左下走，直到 lchild 为 NULL 时的节点）。如图 4.25 所示，节点 12 的中序前驱是 11，中序后继是 13。

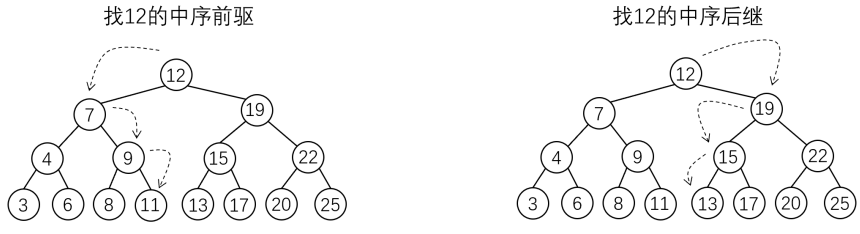


图 4.25

我们先给出两个辅助函数，用于寻找节点的中序前驱和中序后继：

```
// 寻找节点 node 的中序前驱
    BSTreeNode* FindInOrderPre(BSTreeNode* node) {
        // 该节点没有中序前驱，或中序前驱不存在于子树中
        if(node->lchild == NULL) {
            return NULL;
        }
        BSTreeNode* ret = node->lchild;
        while(ret->rchild != NULL) {
            ret = ret->rchild;
        }
        return ret;

    }
// 寻找节点 node 的中序后继
    BSTreeNode* FindInOrderPost(BSTreeNode* node) {
        // 该节点没有中序后继，或中序后继不存在于子树中
        if(node->rchild == NULL) {
            return NULL;
        }
        BSTreeNode* ret = node->rchild;
        while(ret->lchild != NULL) {
            ret = ret->lchild;
        }
        return ret;

    }
```

根据上面的讨论，我们给出在二叉查找树 T 中删除关键字为 e 的节点的详细思路：

① 如果 e 大于当前节点，说明待删元素在右子树中，递归地在右子树中删除 e；

② 如果 e 小于当前节点，类似地，递归地在左子树中删除 e；

③ 如果当前节点为空（NULL），说明树中不存在关键字为 e 的元素，直接返回即可；

④ 如果 e 等于当前节点，说明找到了要删除的节点，进入删除操作：

a）若当前节点是叶节点，直接删除即可；

b）若当前节点有左孩子，那么可以在左子树中寻找节点的中序前驱 pre，用 pre 的数据覆盖掉当前节点的数据，之后在左子树中删除节点 pre；

c）若当前节点有右孩子，那么可以在右子树中寻找节点的中序后继 post，用 post 的数据覆盖掉当前节点的数据，之后在右子树中删除节点 post。

完整的删除代码如下所示：

```
// 在树 root 中，删除值为 e 的节点
    void DeleteBSTree(BSTreeNode* &root, ElemType e) {
```

```
        if(root == NULL) {
            return ;
        }
        if(e < root->data) {
            DeleteBSTree(root->lchild, e);
        }
        if(e > root->data) {
            DeleteBSTree(root->rchild, e);
        }
        if(e == root->data) {
            // root 是叶节点
            if(root->lchild == NULL && root->rchild == NULL) {
                BSTreeNode* to_free = root;
                // root = NULL 是为了改变链条，让其父节点的孩子指针指向 NULL
                root = NULL;
                free(to_free);
            }
            // root 不是叶节点
            else {
                // 有左子树
                if(root->lchild != NULL) {
                    // 若有左子树，则一定能找到中序前驱，所以 pre 一定不是 NULL
                    BSTreeNode* pre = FindInOrderPre(root);
                    root->data = pre->data;
                    DeleteBSTree(root->lchild, pre->data);
                }
                // 有右子树
                else if(root->rchild != NULL) {
                    BSTreeNode* post = FindInOrderPost(root);
                    root->data = post->data;
                    DeleteBSTree(root->rchild, post->data);
                }
            }
        }
    }
```

4.6 平衡二叉树

在上一节中，可能你已经想到了，当我们使用序列 1,2,3,4,5,6,7,8,9 来构造一棵二叉查找树时，形成的树形是斜向右下的一条直线，高度为 9。此时，二叉查找树实际上退化为了一个线性表。许多具有类似结构特点的二叉查找树对查询、插入等操作带来的优化非常有限。

那么我们能不能在每次插入、删除元素后对树加以调整，让结构依然保持平衡？例如当插入 1 到 9 时，对树进行如图 4.26 所示的调整。这样，二叉查找树始终能保持高效的，O(logn) 级别的操作。基于这样的思想，两位先贤，名为 G.M.**A**delse-**V**elskil 以

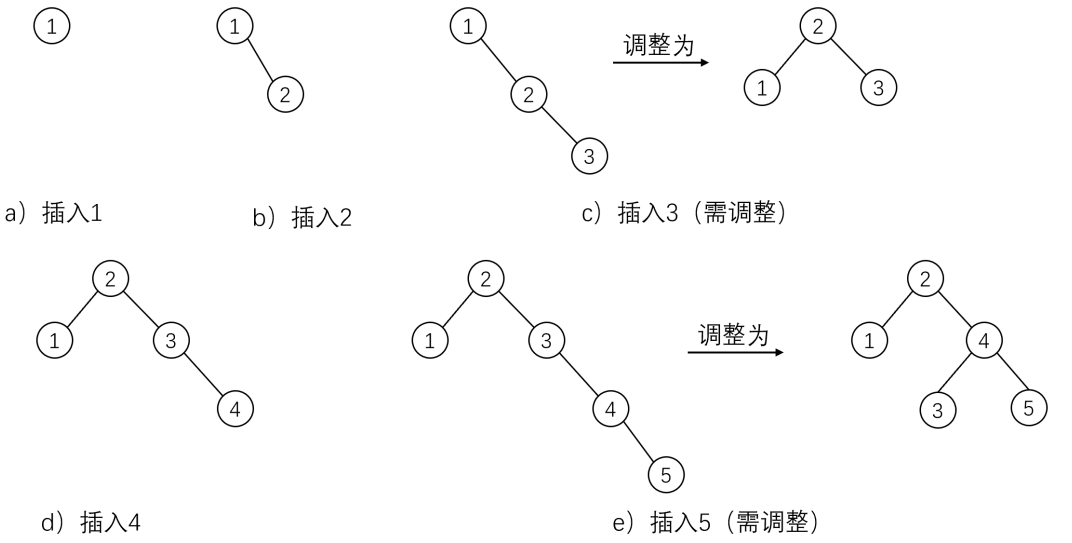


图 4.26

及 E.M.**L**andis 提出了**平衡二叉树**，也被称为 **AVL 树**。AVL 树是满足“平衡”要求的二叉查找树。所谓平衡，是指对树中任意节点来说，其左右子树的高度相差不超过 1。

对于一个节点，其左子树的高度减去右子树的高度，差值称为该节点的**平衡因子**（可以将节点理解为天平的中点，将其左右子树的高度分别理解为左右托盘的砝码数量）。于是我们知道，**平衡二叉树要求树中任何一个节点的平衡因子的绝对值不超过 1**。

图 4.26 中，c）调整前，节点 1 的左子树为空树，高度为 0，右子树高度为 2，因此节点 1 的平衡因子为 0-2=-2，左右子树高度相差 2，不是平衡二叉树，而调整后，节点 2、1、3 的平衡因子都是 0，因此是一棵平衡二叉树。显然树中只要存在一个节点平衡因子的绝对值超过 1，该树就不是平衡二叉树，因此我们需要一种手段记录每个节点的平衡因子。通常的做法并不是直接在节点的结构定义中记录平衡因子，而是记录以当前节点为根节点的子树的高度：

```
struct AVLTreeNode {
    ElemType data;
    AVLTreeNode* lchild;
    AVLTreeNode* rchild;
    int height;     // 以该节点为根的子树的高度。
}
```

提示

height 一定要理解为以该节点为根的子树的高度，为简洁不啰嗦，本节中我们表达为“节点高度”，但一定要将 height 理解为树的高度。

于是某节点的平衡因子就可以通过其左右子树高度之差计算得来。

```
// 返回节点 root 的平衡因子
    int GetBalance(AVLTreeNode* root) {
        int lheight;    // 左子树高度
        int rheight;    // 右子树高度

        // 注意不能直接通过 root->lchild->height 获取子树高度，若 root 没有左子
        ↪ 树，应认为其左子树高度为 0.
        if(root->lchild == NULL) {
            lheight = 0;
        }
        else {
            lheight = root->lchild->height;
        }
        if(root->rchild == NULL) {
            rheight = 0;
        }
        else {
            rheight = 0;
        }
        return lheight - rheight;
    }
```

为了此处及后续诸多地方的代码书写简洁，我们封装一个函数来获取节点的高度：

```
int GetHeight(AVLTreeNode* root) {
    if(root == NULL) return 0;
    return root->height;
}
```

于是获取平衡因子的函数就可以简单写成：

```
int GetBalance(AVLTreeNode* root) {
    int lheight = GetHeight(root->lchild);
    int rheight = GetHeight(root->rchild);
    return lheight - rheight;
}
```

显然，当我们在平衡二叉树中插入一个新的节点之后，只需要更新该节点的所有**祖先节点**的 height 值，而非祖先的 height 是不受该新节点的影响的，无需更新。如图 4.26e），插入 5 后，节点 1 的 height 无需更新。由于节点 root 的 height 等于其左子树的 height 与右子树的 height 的较大值加一，因此当新节点插入后，可以自下而上依次地更新新节点的每一个祖先节点的高度，通过如下函数：

```
void UpdateHeight(AVLTreeNode* root) {
    int lheight = GetHeight(root->lchild);
    int rheight = GetHeight(root->rchild);
    root->height = max(lheight, rheight) + 1;
}
```

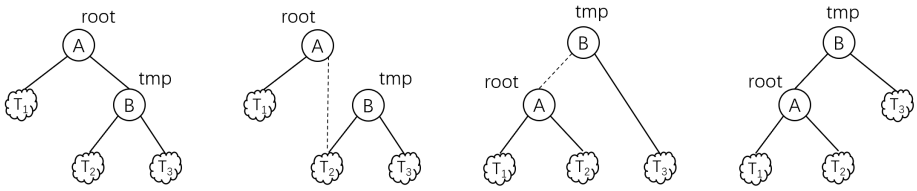
*** 思维拓展**

实际上，有了前面二叉查找树插入操作的递归思想，你可能会想得到，“当新节点插入后，自下而上依次地更新每一个新节点的祖先节点的高度”，这个更新操作的时机就是在函数递归调用，找到正确的插入位置并插入元素之后再“回溯”到访问该节点的函数栈中时。大家可以结合后面平衡二叉树的插入代码体会。

显然，平衡二叉树的查找操作与二叉查找树没有区别。而平衡二叉树的删除操作较为复杂，一般不做讨论。接下来我们就来重点研究平衡二叉树的插入操作。

4.6.1 平衡二叉树的插入操作

事实上图 4.26 c）所进行的调整，可以称为“进行了以节点 1 为根的**左旋**”。平衡二叉树的插入操作需要视情形对某些节点进行**左旋**或**右旋**。我们先来看相关概念。如



a）初始状态　b）步骤一　c）步骤二　d）步骤三

图 4.27 左旋过程示意图

图 4.27 所示的过程将以节点 A 为根的子树进行了向左的旋转，具体过程如下：

1） 让 B 的左子树 T_2 成为 A 的右子树；

2） 让 A 成为 B 的左子树；

3） 将根节点设定为节点 B。

像这样的，将原本的根节点 A 的右孩子“提升”为新的根节点，并且将 A 及其左子树“向左下沉”的操作，可以称为（以 A 为根节点的）**左旋**。在后面的描述中我们简单地称为**对节点 A 左旋**。对 root 左旋的代码如下所示：

```
// 对节点 root 左旋
    void LRotate(AVLTreeNode* &root) {
        AVLTreeNode* tmp = root->rchild;
        root->rchild = tmp->lchild; // 步骤一
        tmp->lchild = root;      // 步骤二
        UpdateHeight(root);      // 先更新下层节点，在更新上层节点
        UpdateHeight(tmp);
        root = tmp;      // 步骤三
    }
```

注意，T_1、T_2、T_3 的 height 无需更新，因为旋转过程中这三棵树并未发生改变，他们的树高不变。A、B 节点的 height 表示的是以 A、B 为根节点的子树的高度，这两棵子树发生了变化，因此需要自下而上地先更新 A 的高度，再更新 B 的高度。

相应地，**右旋**是与左旋对称的过程，图 4.28 展示了对节点 B 右旋的过程。类似地

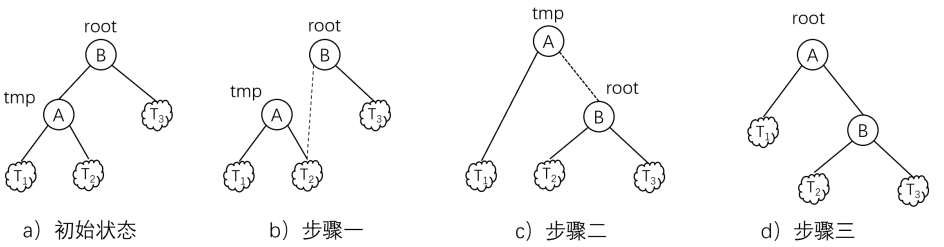


图 4.28 右旋过程示意图

可以写出代码：

```
// 对节点 root 右旋
    void RRotate(AVLTreeNode* &root) {
        AVLTreeNode* tmp = root->lchild;
        root->lchild = tmp->rchild;
        tmp->rchild = root;
        UpdateHeight(root);
        UpdateHeight(tmp);
        root = tmp;
    }
```

以上介绍的左旋和右旋操作，正是用于失衡的调整。已经被证明，若失衡，我们只需要把新插入节点的所有祖先节点中，最靠近新插入节点的那个失衡节点调整到正常，则路径上的所有节点就都会平衡。而最靠近新插入节点的失衡节点，其失衡的情形一

定是如图 4.29 所示四种情形之一。

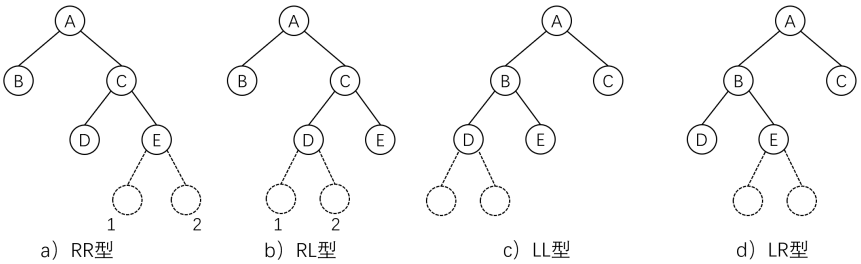


图 4.29 四种失衡情形

图中用虚线表示新插入节点，根节点 A 表示最靠近新插入节点的失衡节点。若新插入节点在根节点的右子树（R）的右子树（R）中，则称这种失衡情形为 **RR 型**，如 a）所示；若新插入节点在根节点的右子树（R）的左子树（L）中，则称这种失衡情形为 **RL 型**，如 b）所示；相应地，c）和 d）分别被称为 **LL 型**和 **LR 型**。

注意，对于 a)，无论插入位置是 1 还是 2，都是在根节点的右子树的右子树（以 E 为根的子树）中，因此都叫做 RR 型；对于 b)，无论插入位置是 1 还是 2，都是在根节点的右子树的左子树（以 D 为根的子树）中，因此都叫做 RL 型。c)、d）情况类似。

一个很让人放心的事实是，这四种情形都可以通过若干次左旋或右旋的操作调整，从而重新达到平衡。先贤们将正确的做法传承了下来，见下。

1.RR 型

对于 RR 型，直接将根节点（最靠近新插入节点的失衡节点）左旋即可。如图 4.30。

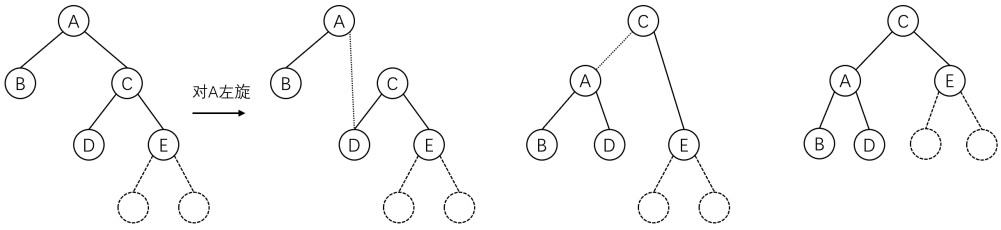


图 4.30 RR 型的调整方法

2.RL 型

如图 4.31，对于 RL 型，可以先将根节点的右子树进行右旋（对 C 右旋），这样，就化归成了 RR 型失衡。再使用 RR 型的处理方式，对根节点左旋即可。

不难发现，LL 型与 RR 型是对称的，LR 型与 RL 型是对称的。因此处理 LL 型与 LR 型使用相应的对称的旋转方法即可。

3.LL 型

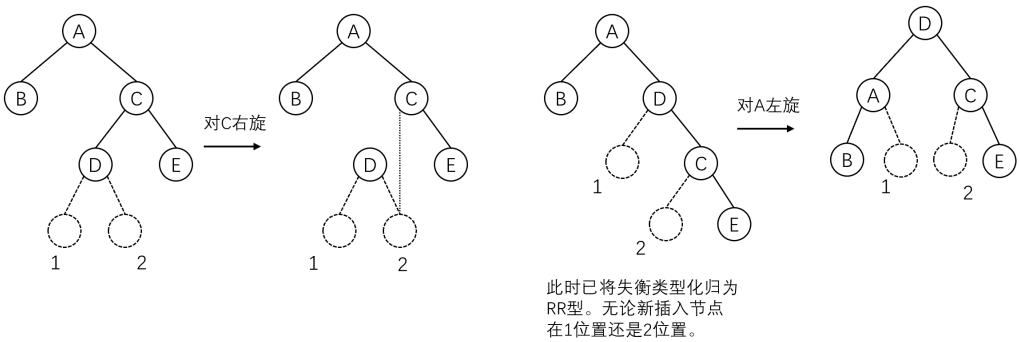


图 4.31 RL 型的调整方法

LL 型与 RR 型对称，使用与 RR 型对称的处理方法，直接将根节点右旋即可。如图 4.32。

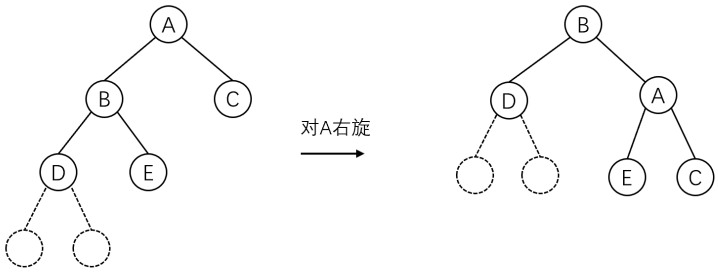


图 4.32 LL 型的调整方法

4.LR 型

LR 型与 RL 型对称。先将根节点的左子树左旋，再将根节点右旋。如图 4.33。直接将根节点右旋即可。如图 4.33。

至此，我们来总结一下平衡二叉树的插入方法。首先，像普通二叉查找树的插入一样，沿着树递归向下找到插入位置，插入节点之后，沿递归向上返回，判断每一个节点是否存在失衡，若失衡，判断失衡类型（LL、LR、RL、RR）并进行正确的调整即可。沿递归回去的路径判断失衡，第一个找到的失衡节点恰是距离新插入节点最近的祖先节点。具体实现代码如下。

```
void AVLTreeInsert(AVLTreeNode* &root, ElemType e) {
    // 找到了插入位置，插入
    if (root == NULL) {
        root = (AVLTreeNode*)malloc(sizeof(AVLTreeNode));
        root->data = e;
```

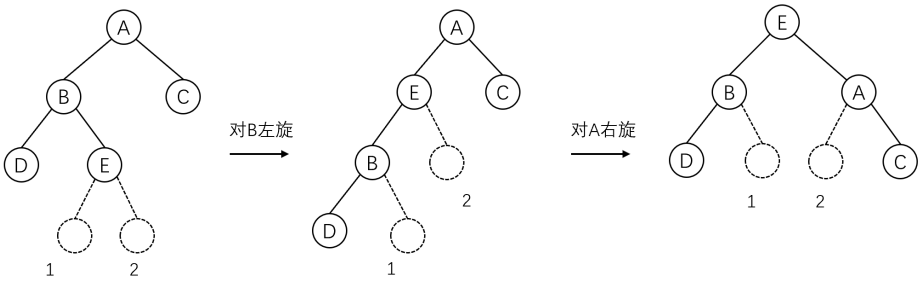


图 4.33 LR 型的调整方法

```
6           root->height = 1;
7           root->lchild = NULL;
8           root->rchild = NULL;
9           return;
10      }
11      // 关键字已存在，无需插入
12      if(e == root->data) {
13          return ;
14      }
15      // 在左子树中插入，只可能是 LL 型或 LR 型失衡
16      if(e < root->data) {
17          AVLTreeInsert(root->lchild, e);
18          UpdateHeight(root);
19          // 存在失衡
20          if(GetBalance(root) == 2) {
21              if(GetBalance(root->lchild) == 1) { // LL 型失衡
22                  RRotate(root);
23              } else if(GetBalance(root->lchild) == -1) { // LR 型失衡
24                  LRotate(root->lchild);
25                  RRotate(root);
26              }
27          }
28      }
29      // 在右子树中插入，只可能是 RR 型或 RL 型失衡
30      else if(e > root->data) {
31          AVLTreeInsert(root->rchild, e);
32          UpdateHeight(root);
33          // 存在失衡
34          if(GetBalance(root) == -2) {
35              if(GetBalance(root->rchild) == 1) { // RL 型
36                  RRotate(root->rchild);
37                  LRotate(root);
38              } else if(GetBalance(root->rchild) == -1) { // RR 型
39                  LRotate(root);
40              }
```

```
        }
    }
}
```

以上代码的骨架和二叉查找树的插入操作一致，只不过在此基础上多了失衡判断和调整处理代码：

```
void AVLTreeInsert(AVLTreeNode* &root, ElemType e) {
    // 找到了插入位置，插入
    if (root == NULL) {
        root = (AVLTreeNode*)malloc(sizeof(AVLTreeNode));
        root->data = e;
        root->height = 1;
        root->lchild = NULL;
        root->rchild = NULL;
        return;
    }
    // 关键字已存在，无需插入
    if(e == root->data) {
        return ;
    }
    // 在左子树中插入，只可能是 LL 型或 LR 型失衡
    if(e < root->data) {
        AVLTreeInsert(root->lchild, e);
        // UpdateHeight(root);
        // // 存在失衡
        // if(GetBalance(root) == 2) {
        //     if(GetBalance(root->lchild) == 1) { // LL 型失衡
        //         RRotate(root);
        //     } else if(GetBalance(root->lchild) == -1) { // LR 型失衡
        //         LRotate(root->lchild);
        //         RRotate(root);
        //     }
        // }
    }
    // 在右子树中插入，只可能是 RR 型或 RL 型失衡
    else if(e > root->data) {
        AVLTreeInsert(root->rchild, e);
        // UpdateHeight(root);
        // // 存在失衡
        // if(GetBalance(root) == -2) {
        //     if(GetBalance(root->rchild) == 1) { // RL 型
        //         RRotate(root->rchild);
        //         LRotate(root);
        //     } else if(GetBalance(root->rchild) == -1) { // RR 型
        //         LRotate(root);
```

```
40          //      }
41          // }
42      }
43  }
```

图 4.34 展示了在根节点 12（平衡因子为-1）的平衡二叉树中，插入节点 27 的插入过程：沿 12、19、22、25、26 一路递归向下，找到插入位置 pos，插入节点 27。此后递归回溯，回到 26，重新计算节点 26 的 height=max(0, 1)+1=2，更新 26 的 height 为 2。之后判断节点 26 的平衡因子，为 0-1=-1，不失衡，递归继续向上返回到 25，更新 height 为 max(1, 2)+1=3，平衡因子为 1-2=-1，不失衡，递归继续向上返回，到 22，更新 height 为 4，平衡因子为 1-3=-2，为 RR 型失衡，因此对 22 左旋，得到 b）。继续返回 19、12，函数结束。

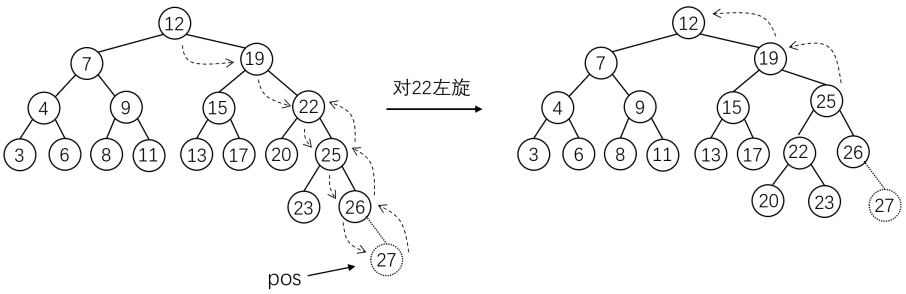


图 4.34 在平衡二叉树中插入节点 27

注意，在本节我们介绍二叉查找树以及平衡二叉树时，我们的定义是：左子树严格小于根节点、右子树严格大于根节点，即二叉树中不允许存在重复关键字。于是插入时，若关键字已存在，直接返回。

*** 思维拓展**

事实上，根据实际场景的不同需求，可能会需要允许重复关键字的树。绝大多数参考资料对此不做讨论，作为拓展，笔者给出一种较为简单的实现思路：

在每个节点中增加一个链表，将所有相同关键字的数据存储起来：

```
struct node {
    int key;    // 关键字
    LinkList<ElemType> datas;  // datas 是一个链表，保存所有关键字为 key 的数据。
    node* lchild;
    node* rchild;
    int height;
};
```

这样，插入时，和链表中第一个节点比较即可。若相等，将新元素插入链表尾部。

显然平衡二叉树的查找、插入操作的时间复杂度均为 O(logn)。有了以上插入操作的实现，通过已有的 n 个元素构建一棵平衡二叉树，只需要依次将这 n 个元素插入树中即可。代码如下：

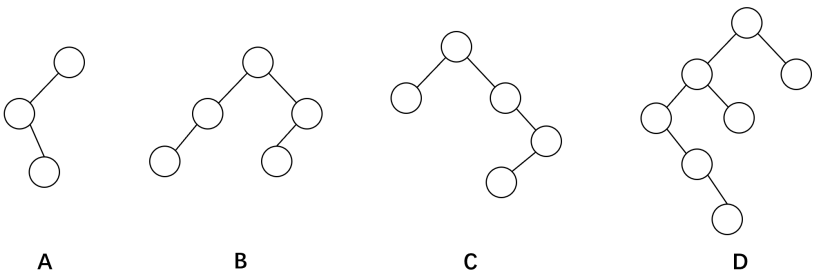
```
// 将存储在 arr 中的 n 个元素建立一棵平衡二叉树，返回树的根节点
    AVLTreeNode* CreateAVLTree(ElemType arr[], int n) {
        AVLTreeNode* root = NULL;
        for(int i=0; i<n; i++) {
            AVLTreeInsert(root, arr[i]);
        }
        return root;
    }
```

习题 5 （参考答案见习题 5 参考答案，第 220 页）

1. [判断] 二叉查找树，也称二叉排序树、二叉搜索树。

2. [判断] 平衡二叉树一定都是二叉查找树。

3. （某年 408 真题）下列二叉排序树中，满足平衡二叉树定义的是_____

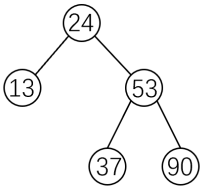


4. （某年 408 真题）对下列关键字序列，不可能构成二叉排序树中一条查找路径的是_____

A. 95, 22, 91, 24, 94, 71　B. 92, 20, 91, 34, 88, 35

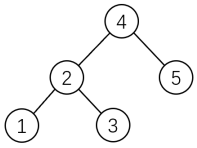
C. 21, 89, 77, 29, 36, 38　D. 12, 25, 71, 68, 33, 34

5. （某年 408 真题）在下图所示的平衡二叉树中插入关键字 48 后得到一棵新的平衡二叉树，在新平衡二叉树中，关键字 37 所在节点的左、右子节点中保存的关键字分别是_____



A. 13, 48　B. 24, 48　C. 24, 53　D. 24, 90

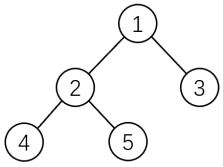
6. （某年 408 真题）下列给定的关键字输入序列中，不能生成如图所示的二叉排序树的是_____



A. 4, 5, 2, 1, 3
B. 4, 5, 1, 2, 3
C. 4, 2, 5, 3, 1
D. 4, 2, 1, 3, 5

7. (提示：本题是一个比较经典的问题，但有一定思维门槛。建议大家根据自身需求选择直接跳过，或思考 1-2 分钟，若无思路，可以直接研读参考答案，学会最近公共祖先的求法，并获得一定启发即可。)

请你设计一个算法，给定任意一棵二叉树以及两个关键字 a 和 b，返回树中 a 和 b 的最近公共祖先。假设关键字类型为 int。保证树中无重复关键字并且 a、b 一定都存在于树中。节点自身也被认为是自身的祖先。如图所示的树中，4、5 的最近公共祖先为 2；4、3 的最近公共祖先为 1；2、5 的最近公共祖先被认为是 2。




```
struct node {
    int key;
    node* lchild;
    node* rchild;
};

node* LowestCommonAncestor(node* root, int a, int b) {

}
```

8. 若上一题并非任意二叉树，而是一棵二叉搜索树，其他条件不变，则可解上题的代码一定可解本题吗？代码会有哪些优化空间吗？若你研究了上一题的思想，可以思考写出本题的代码。

9. 请你设计一个算法，查找二叉搜索树中第 k 小的元素。

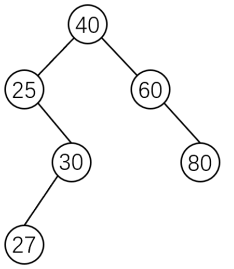
```
// 返回二叉搜索树 root 中第 k 小的元素
    int kthSmallest(node* root, int k) {

    }
```

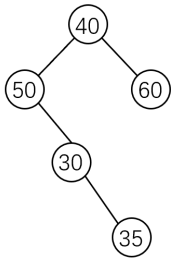
10. （某年 408 真题）已知非空二叉树 T 的节点值均为正整数，采用顺序存储方式保存，数据结构定义如下：

```
typedef struct {                // MAX_SIZE 为已定义常量
    int SqBiTNode[MAX_SIZE];    // 保存二叉树节点值得数组
    int ElemNum;                // 实际占用的数组元素个数
}SqBiTree;
```

T 中不存在的节点在数组 SqBitNode 中用-1 表示。例如，对于下图所示得两棵非空二叉树 T_1 和 T_2，



二叉树T_1



二叉树T_2

T_1 的存储结果如下：

T_1.SqBitNode	40	25	60	-1	30	-1	80	-1	-1	27		

T_1.ElemNum=10

T_2 的存储结果如下：

T_2.SqBitNode	40	50	60	-1	30	-1	-1	-1	-1	-1	35	

T_2.ElemNum=11

请设计一个尽可能高效的算法，判定一棵采用这种方式存储的二叉树是否为二叉搜索树，若是，则返回 true，否则，返回 false，要求：

1）给出算法的基本设计思想；

2）根据设计思想，采用 C 或者 C++ 语言描述算法，关键之处给出注释。

图的基本概念与存储结构。图的遍历：DFS、BFS。图的连通性与连通分量。并查集：如何高效维护动态合并的集合。最小生成树：Kruskal 算法如何利用并查集。其他图算法。



维火女

亲爱的余火大人，恭喜您成功觐见了王者—树与二叉树，并从那古老而伟大的王座之前，获得了属于王的力量。

接下来，您将踏入的区域名为万象之都。在那座古都深处，沉睡着一位孤独的王者。他无声地注视着世间万物的交织与离散，等待着命运的召唤。他的名字便是——图。相传这位孤独的王者正是那雾的彼端的初王之子。CoreLoop 已在此地刻下曾经发生在雾的彼端，那名为罗德尔的传奇王国的故事，它将伴随您走完本区域的旅程。

余火大人啊……您未曾得到火焰的指引，却仍凭着自己的意志，一步一步地穿越荆棘与黑暗，走到了这里……我曾无数次望见，您身上的伤口才刚刚愈合，转眼间却又添上了新的伤痕。可即便如此，您依旧默默地站起身来，继续走向那未知的远方。我也曾在寂静的深夜，看见您独自仰望那片遥远的星空，仿佛有千言万语萦绕心头。

您从不曾向任何人诉说自己的苦楚，也不曾让泪水从眼中落下……可是，亲爱的余火大人……我多么希望，至少在我的面前，您不必如此坚强。

倘若这漫长的旅途令您感到疲惫，便请稍稍停下脚步吧。哪怕只有片刻也好，请让我陪伴在您的身旁，让您不必独自面对这无边的黑暗。因为对我而言，您并非只是那位不断前行的思渡者。

您也是……我由衷牵挂着的人啊。

第五章　图

深夜，王都的钟声突然响起。

来自边境的信使带来了一个令人不安的消息：早已叛国的碎片君王们正带着大批军队向王国腹地推进，预计将于数月后抵达。国王立即召集所有军事指挥官，要求在天亮前制定出一套防御方案。

王国境内分布着许多城镇、要塞、仓库等据点，它们通过道路相互连接。然而，道路的长短各不相同，并且有些需要穿梭险峻的山脉，有些需要横跨湍急的河流，还有一些因年久失修，早已无法通行。



如何快速确定哪些据点依然互相连通？若某个军事要点来犯，从首都派出军队，走哪条路能以最快速度到达？如何最优地铺设通信电线，以连接所有城镇？

……

时间正在飞速流逝，而摆在指挥官们面前的，是城镇、道路与错综复杂的连接关系。

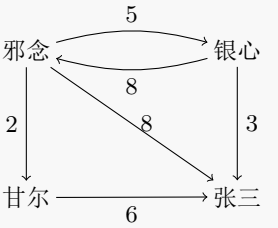
经过不懈努力，指挥官们制定出了一套行之有效的防御模型。最终，入侵者被尽数驱逐，王国重归安宁。而这套曾经帮助王国渡过危机的模型，并未随着战争的结束而被遗忘。相反，王国的后来者在此基础上不断探索、推演与完善，逐渐建立起一套严谨而完整的理论体系。虽然以上内容都是古老的传闻，但思渡者啊，在这片大陆上你看到的都并非虚妄。在你觐见王者的途中，这段古老的传说将伴随你一同前行。

经过讨论，指挥官们决定将王国建模为一个**图**。他们将王国中的城镇、要塞等据点作为图的**顶点**，将据点之间的道路作为**边**。

定义

图（Graph）是一种由**顶点**集合和顶点之间的**边**集合组成的数据结构，通常记为 $G = (V, E)$。其中，V 是有穷非空的顶点集合，E 是表示顶点之间关系的边集合。根据边是否具有方向，图可分为**无向图**和**有向图**：无向图中的边表示两个顶点之间不区分方向的关系，有向图中的边则表示从一个顶点指向另一个顶点的关系。

在第一章中（—waypoint-5—（第 2 页））我们曾经提到小队队员之间好感度的关系。更现实的情况是 A 对 B 的好感度与 B 对 A 的好感度并不相同，这时就要使用**有向图**，来描述这种从一个顶点指向另一个顶点这种有方向的关系。比如邪念对银心好感度是 5，而银心对邪念的好感度是 8：



王国的指挥官们认为，据点之间的道路若存在，则一定是可以双向通行的，因此他们使用**无向图**来描述王国。为方便交流，他们对无向图做了如下约定：

1. 将某顶点 v 的**度**定义为依附于顶点 v 的边的数量；
2. 在无向图 G 中，若顶点 v 和顶点 w 之间有路径存在，则称这两个顶点是**连通的**。若图 G 中任意两个顶点都是连通的，则称图 G 为**连通图**，否则称为**非连通图**。无向图中的一个极大连通子图称为一个**连通分量**，孤立顶点被认为是一个连通分量。
3. 由于无向图中边是双向的，因此一条边既可以看作出边，也可以看作入边。

王国的后来者对类似的概念，在有向图上做了如下约定：

1. 有向图中，某顶点 v 的度分为**入度**和**出度**。**入度**是“进来到该顶点的边数”，即以顶点 v 为终点的有向边的数量，而**出度**是“从该顶点出去的边数”，即以顶点 v 为起点的有向边的数量。顶点的出度和入度之和称为顶点的**度**。

2. 在有向图 G 中，若从顶点 v 到 w，以及从 w 到 v 之间都有路径，则称这两个顶点是**强连通的**。若图中任意一对顶点都是强连通的，则称该图为**强连通图**。类似地，有向图中的一个极大强连通子图称为一个**强连通分量**，孤立顶点被认为是一个强连通分量。

连通分量和强连通分量通常也被称为**连通块**。

为加深记忆，指挥官们以如图 5.1 的例子进行了一番交流：G_1 是一个有 5 个顶点、4 条边的图，由于它存在两个顶点不连通（如 D 与 A），因此它是一个非连通图；由 ABC 及连通它们的边 e_1、e_2、e_3 所构成的**子图**，是图 G_1 的一个连通分量，同理 D、E 及 e_4 也构成了一个连通分量。

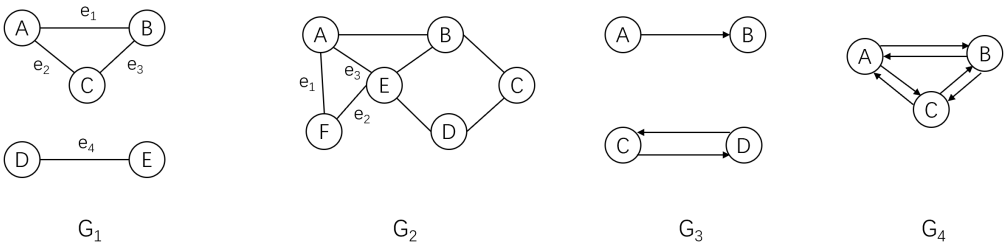


图 5.1

图 G_2 中，A、E、F 及 e_1、e_2、e_3 构成的子图不能称为一个连通分量，因为它不是**极大**的。

> **极大连通子图**是指：在一个无向图中，不能再加入原图中的其他顶点及相关边而保持连通性的连通子图。有向图中**极大强连通子图**的概念类似。

图 G_2 是一个连通图，因为任何两个顶点之间都有路径。图 G_2 中，顶点 A、B 的度为 3，顶点 E 的度为 4。

图 G_3 是一个有四个顶点、三条有向边的有向图。A 的出度为 1，入度为 0；C、D 的出度为 1，入度为 1。显然图 G_3 不是强连通图。注意：由于 B 到 A 不存在路径，因此顶点 A 与 B 并非强连通。但该图并非只有一个强连通分量，而是有如下的三个：



图 G_4 是一个强连通图，顶点 A、B、C 的出度和入度均为 2；该图只有一个强连通分量，就是它自身。

根据实际应用的不同，顶点和边可能会携带一些信息，通常将边上的某种量化属性称为**边权**。如，指挥官们可能将顶点信息和边的信息分别用如下结构记录：

```
struct VertexInfo {     // 顶点携带的信息
    char* name; // 据点的名字
    int weight; // 该据点的战略价值（重要性）
    //...
};

struct EdgeInfo {    // 边携带的信息
    int weight;     // 边权，可能是两据点间行军需要的时间，也可能是两据点间铺设通信电线需要的花费等等
}
```

这种边上带权的图称为**带权图**。绝大多数场景下，边只需表示顶点之间是否存在连接，或者额外记录一个权值，就足以解决实际问题。因此我们主要讨论这两种情况。对于确实需要利用边的其他属性或更复杂信息的应用场景，我们再根据具体问题进行说明。

王国的城镇、要塞等据点成百上千，道路错综复杂。因此，指挥官们决定将建模输入他们的信息系统来解决问题。

5.1 图的存储

要存储一个由 N 个顶点，M 条边构成的图，就需要存储所有顶点的信息、明确“哪些顶点之间存在边”以及存储这些边的权值。要存储这 N 个顶点的信息非常简单，只需要一个数组即可：

```
VertexInfo v[N];
```

那么如何存储边与顶点的关系，以及边上携带的信息？经过研究，两位聪明的指挥官格佛莱和麦丽凯分别提出了两种策略。

5.1.1 邻接矩阵

格佛莱指出，若将 N 个顶点编号为 0 到 N-1，则通过一个矩阵（二维数组）：

```
int AdjM[N][N]; // Adjacency Matrix
```

就可以明确“哪些顶点之间存在边”，以及这些边的权值是多少。例如若 AdjM[0][2] 的值是 7，说明存在一条 0 号顶点到 2 号顶点的边，并且权值是 7。显然对于无向图，

AdjM[i][j] 和 AdjM[j][i] 的值一定是相等的，因此无向图的邻接矩阵一定是一个对称矩阵。而对于有向图，AdjM[i][j] 和 AdjM[j][i] 可以分别表示 i 到 j 以及 j 到 i 是否存在边、权值是多少。若边不存在，可以用一个在此场景无意义的数表示权值，如 0、-1 或正无穷等。

对于无权图，则无需记录边的权值，因此简单地用 0、1 表示边是否存在即可。

*** 思维拓展**

CoreLoop 提示：若在你的应用场景下，-1，0，负无穷、正无穷等都是可能存在的有意义边权，则可以在边的信息中加入一个 bool 值，标识这条边是否存在。

EdgeInfo AdjM[N][N]; // EdgeInfo 中包含一个 bool 变量。

以上这种图的存储方式被称为**邻接矩阵**法。使用邻接矩阵法可以将一个图描述如下：

```
// MAX_VNUM 为该结构可存储的图的最大顶点数
    struct MGraph { // M: Matrix 版
        VertexInfo v[MAX_VNUM] ;
        int AdjM[MAX_VNUM][MAX_VNUM];   // 邻接矩阵
        int current_vnum;   // 当前顶点数
        int current_enum;   // 当前边数
    };
```

拟用 0 表示边不存在，格佛莱通过图 5.2 展示了两个图（Graph）以及其对应的邻接矩阵 AdjM。

可以看出，a）是对称矩阵。

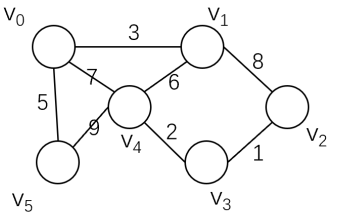
另一个指挥官麦丽凯研究了格佛莱的提议后，对此法大为认同，但也指出了该方法的一个缺点：该方法存储图需要一个二维矩阵，存储具有 N 个顶点的图，空间复杂度就是 $O(N^2)$。若图存在大量顶点，这种方法在空间上难以接受。经过思考，麦丽凯提出了一种新的方法，后来被称为**邻接表**法。

5.1.2 邻接表

邻接矩阵的方法着眼于全局，直接从全局视角记录边与点的关系。而邻接表法则是将每一个顶点作为主角，记录每个顶点的“局部信息”。

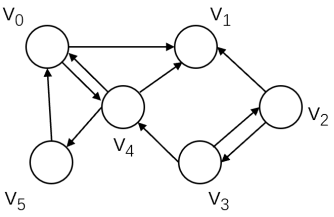
具体来说，对于有 N 个顶点（编号为 0, 1, ..., N-1）的图 G，邻接表法会存储所有顶点的所有**出边**的信息。如图 5.3 中，对于编号为 2 的顶点 v_2，我们需要记录它的三条出边，就需要分别记录这三条边的终点是哪个顶点，以及边权是多少。

于是我们定义一个结构 AdjEdge，来表示这种“出边”的信息：



	0	1	2	3	4	5
0	0	3	0	0	7	5
1	3	0	8	0	6	0
2	0	8	0	1	0	0
3	0	0	1	0	2	0
4	7	6	0	2	0	9
5	5	0	0	0	9	0

a）无向图及其临接矩阵AdjM(边带权)



	0	1	2	3	4	5
0	0	1	0	0	1	0
1	0	0	0	0	0	0
2	0	1	0	1	0	0
3	0	0	1	0	1	0
4	1	1	0	0	0	1
5	1	0	0	0	0	0

b）有向图及其临接矩阵AdjM(边不带权)

图 5.2

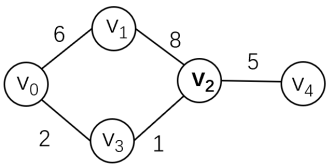


图 5.3

```cpp
struct AdjEdge {
    int u;  // 邻接顶点的编号
    int weight; // 当前顶点到 u 的边的权值。若需要更多边的信息，可自定义 EdgeInfo 类型
};
```

由于无法确定每个顶点有多少条出边，因此，—waypoint-6—（第 30 页）处介绍的不定长数组 vector 用在此处就非常合适。对于 v_2，我们就可以通过一个元素类型为 AdjEdge 的 vector，将其所有出边的信息记录下来：

```cpp
vector<AdjEdge> AdjL_v2;  // 顶点 v2 的所有出边信息
```

这个表就称为v_2 的邻接表。v_2 的邻接表如图 5.4 所示。

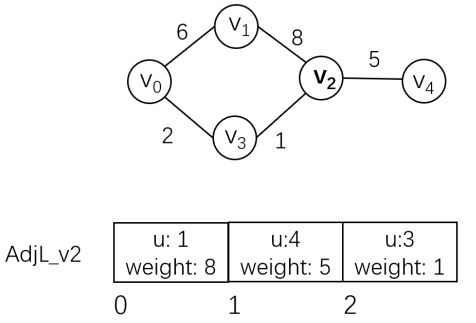


图 5.4

提示

事实上很多时候，更好的做法是通过链表来实现邻接表（可以想想为什么），但二者思想是类似的。链表实现复杂，况且大多数情况下 vector 性能已经足够好。为了给大家带来更强的实用性，本章中邻接表拟采用 vector 实现。

某个顶点的邻接表记录了与该顶点有关的所有出边信息。那么我们将图中所有顶点的出边信息都存储下来，也就是为每个顶点记录一个邻接表，一共记录 N 个邻接表，这也就完整地记录了图中所有边的关系：

```cpp
vector<AdjEdge> AdjList[N];
```

这 N 个顶点的邻接表共同构成了整个图 G 的邻接表。注意：以上代码中 AdjList 是一个 C 原生数组，而 AdjList[i] 才是一个 vector，该 vector 是顶点 i 的邻接表，保

存的是顶点 i 的所有出边的信息。

于是，使用邻接表存储图中边的信息，我们就可以将一个图表示为如下的结构：

```
struct AdjEdge {
    int u;  // 邻接顶点的编号
    int weight; // 当前顶点到 u 的边的权值。若需要更多边的信息，可自定义 EdgeInfo 类型
};
struct LGraph {
    vector<AdjEdge> AdjList[MAX_VNUM];  // 邻接表
    VertexInfo v[MAX_VNUM];      // 存放顶点信息
    int current_vnum;            // 当前顶点数
    int current_enum;            // 当前边数
};
```

由于顶点 v 的邻接表记录的是顶点 v 的**出边**的信息，因此对于无向图，若存在边 (u, v)，那么同时需要在 AdjList[u] 中记录 v，并在 AdjList[v] 中记录 u。因此，无向图中的一条实际边通常对应邻接表中的两个表项。

显然对于无权图，AdjEdge 类型只存储邻接顶点的编号 u 即可。

前面的例子中的图及其完整的邻接表如图 5.5 所示。

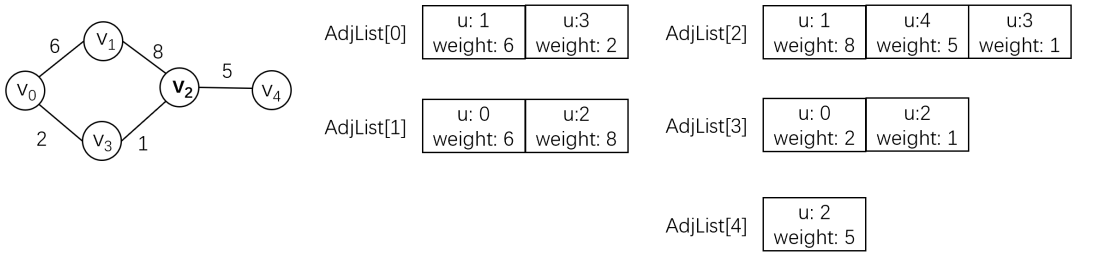


图 5.5

容易知道邻接矩阵表示法下，存储一个图需要的存储空间与图的边数 M 无关。而对于邻接表法，要存储有 N 个顶点、M 条边的图，则需要维护 N 个邻接表，对于无向图，总共需要 $2M$ 个表项，对于有向图总共需要 M 个表项，因此，邻接表法存储一个图需要的存储空间为 $O(N+M)$。

5.1.3 二者获取邻接信息的对比

可以看到，无论是邻接表存储法，还是邻接矩阵存储法，对于 N 个顶点的信息存储都是一致的，使用一个额外的数组来存储。二者区别就在于对于每个顶点的邻接信息，即顶点与顶点、顶点与边的关系如何存储。二者存储方式的不同，也带来了一些获

取邻接信息上的差异。

① 邻接矩阵

对于有向图，要知道 v 的所有出边信息，遍历一遍图的邻接矩阵的下标为 v 的那一行即可：

```
for(int i=0; i<G.current_vnum; i++) {
    if(G.AdjM[v][i] != 0) {
        // v 有一条到 i 的出边，G.AdjM[v][i] 的值即是该边的权值
    } else {
        // v 到 i 不存在出边
    }
}
```

类似地要知道 v 的所有入边信息，遍历下标为 v 的那一列即可。显然遍历的过程中若记录有效边的数量，就得到了顶点出度、入度的信息。

对于无向图，只遍历下标为 v 的行或下标为 v 的列即可得到该顶点的所有邻接信息。

```
for(int i=0; i<G.current_vnum, i++) {
    if(G.AdjM[v][i] != 0) {
        // v 到 i 存在无向边，G.AdjM[v][i] 的值即是该边的权值
    } else {
        // v 到 i 不存在边
    }
}
```

同样地，若遍历过程中记录有效边的数量，就得到了无向图某顶点度的信息。

② 邻接表

对于邻接表存储的有向图，v 的邻接表存储的就是 v 的所有出边信息。要知道顶点 v 的所有出边信息很简单，遍历 v 的邻接表即可：

```
for(int i=0; i<G.AdjList[v].size(); i++) {
    // G.AdjList[v][i] 就是 v 的邻接表的第 i 个表项，即与 v 邻接的第 i 个顶点和
    ↪ 边权的信息。
}
```

同时，表长 G.AdjList[v].size() 就是顶点 v 的出度。

但是，对于邻接表存储的有向图，要知道顶点 v 的所有入边信息，就需要将所有顶点的邻接表均查找一遍：

```cpp
// 遍历一遍所有顶点的邻接表
for(int i=0; i<G.current_vnum; i++) {
    // 遍历顶点 i 的邻接表的所有表项
    for(int j=0; j<G.AdjList[i].size(); j++) {
        // 顶点 i 的第 j 个邻接点为 v
        if(G.AdjList[i][j].u == v) {
            // 找到了 v 的一条入边，这是一条从 i 指向 v 的边，权值为 G.AdjList[i][j].weight
            // 若需要计算 v 的入度，在此处累加。
        }
    }
}
```

二者对比如表 5.1 所示：这里 N 表示顶点数，M 表示边数，$\deg^+(v)$ 表示顶点 v 的出度。

表 5.1: 邻接矩阵与邻接表的操作效率对比

操作	邻接矩阵	邻接表
查找顶点 v 的所有出边	$O(N)$	$O(\deg^+(v))$
查找顶点 v 的所有入边	$O(N)$	$O(N+M)$
获取有向图顶点 v 的出度	$O(N)$	$O(1)$（直接读取表项数量）
获取有向图顶点 v 的入度	$O(N)$	$O(N+M)$（未额外维护入边信息）

有了以上扎实的模型基础，指挥官们计划了第一步行动。他们决定先从首都派出一名信使，将大敌来犯的消息通知到每一个据点的士兵以及居民们。然而，各据点之间的道路错综复杂，信使不可能毫无章法地四处奔走。怎样规划通知顺序，才能确保每个据点都不会遗漏？

5.2 图的遍历

指挥官们给出了两种方式，也是王国的后来者总结出的两种图的遍历方式，总结如下。

5.2.1 深度优先搜索（DFS）

深度优先搜索（Depth First Search, DFS）的过程就像是“不分黑白地一股脑儿往前冲，直到撞了墙，才按原路返回，继续探索未探索过的其他路（当然也是继续冲、撞墙、返回，冲、撞墙、返回.....）。”

具体来说，信使从首都（起点，记为 v）出发，访问与 v 邻接且未被访问过的任意一个顶点 u，之后从 u 出发（而非回到 v），再访问与 u 邻接且未被访问过的任意一个顶点 u'，之后从 u'出发，继续访问与 u'邻接且未被访问过的任意一个顶点 u''，……，如此重复，直至不能继续访问下去之后，依次退回到最近被访问的顶点，若其还有未被访问的邻接顶点，则访问之…… 如此重复，直至图中所有顶点均被访问一遍。

我们以图 5.6 直观地感受一下深度优先搜索的过程。

假设起点为 v_0：

a）访问 v_0，然后看到与 v_0 邻接的顶点有 v_1 和 v_3，他们都未被访问，任意选一个访问即可。假设选 v_1。

b）访问 v_1，然后看到邻接顶点仅有 v_2 未被访问，于是

c）访问 v_2，看到 v_4、v_5 均未被访问，假设选 v_4，于是

d）访问 v_4，此时已深入最深处，与 v_4 邻接的所有顶点均已访问完成，于是退回到上一个顶点 v_2，发现它还有 v_3、v_5 两个邻接顶点未访问，任选一个，假设选 v_3。

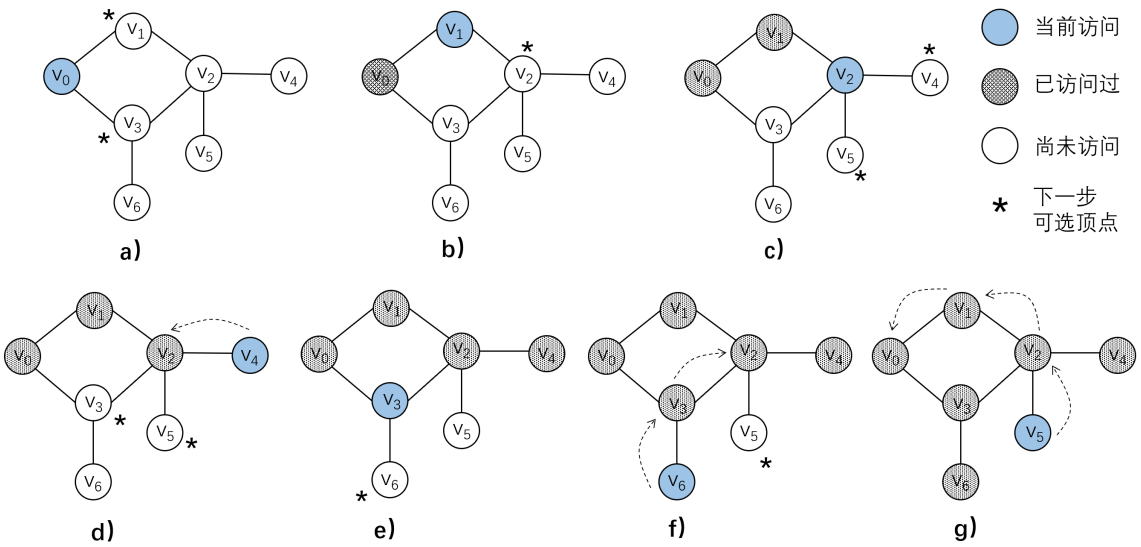


图 5.6 深度优先搜索

e）访问 v_3，看到邻接的 v_6 还没被访问。于是

f）访问 v_6，此时无法继续访问下去，于是退回到 v_3，发现 v_3 的所有邻接顶点也都已经访问完成了，于是退回 v_2，发现与 v_2 邻接的 v_5 还未访问。于是

g）访问 v_5。之后退回到 v_2（无顶点未访问），v_1（无顶点未访问）、v_0（无顶点未访问），深度优先搜索过程结束。

以上过程得到的深度优先访问序列为：$\{v_0, v_1, v_2, v_4, v_3, v_6, v_5\}$。这就是深度优

先搜索，总是尽可能“深”地探索，当无法再深入之后才原路返回，探索那些未走过的路。

5.2.2 广度优先搜索（BFS）

如果说深度优先搜索（DFS）是“一路走到底，不撞南墙不回头”，那么广度优先搜索（Breadth First Search, BFS）就是“浅尝辄止，先慢慢探清周围环境，再继续往深处走”。

实际上广度优先搜索和树的层序遍历非常相似。可以想象平静的水面落下一个小铁球而泛起层层涟漪，广度优先搜索即是先访问中心点，之后访问距中心点最近的“第一层涟漪”，之后访问“第二层涟漪”，第三层.....，直到全部访问完成。

具体来说，假设从 v 出发，首先访问起始顶点 v，并“发现”v 的所有未被访问的邻接顶点 v_1, v_2, ..., v_n，这些顶点即是“第二层涟漪”，依次访问它们，访问时，又会发现第三层涟漪 v_{11}, v_{12}, ..., v_{nm}..... 如此往复，直至图中所有顶点均被访问一遍。

我们以图 5.7 来直观地感受一下广度优先搜索的过程。拟以 v_0 作为起点：

a）访问 v_0，看到 v_0 的邻接顶点有 v_1 和 v_3，于是将这两个顶点作为“第二层”的顶点，此时“第一层涟漪”已访问完毕，“第二层涟漪”有两个，任选一个，假设选择 v_1；

b）访问 v_1，此时发现了“第三层涟漪”的一个顶点 v_2，先将它记下来。准备继续访问第二层未访问完的顶点 v_3。

c）访问 v_3，发现 v_3 的邻接顶点 v_6 尚未被访问，因此也将它记下来，作为“第三层涟漪”的一部分。至此，第二层顶点均已访问完毕，准备访问第三层，假设选择 v_2；

d）访问 v_2，发现与 v_2 邻接的有 v_4 和 v_5 还未访问，因此将它们记下来，作为“第四层涟漪”的一部分；然后准备访问第三层剩下的 v_6；

e）访问 v_6，v_6 没有尚未访问的邻接顶点，因此从 v_6 没有发现新的“第四层涟漪”。至此第三层均已访问完毕。准备开始访问第四层

f）访问 v_4，没发现新的顶点；

g）访问 v_5，没发现新的顶点。此时所有顶点均已访问过一遍，没有新的层了，结束。

以上过程得到的广度优先搜索访问序列是 $\{v_0, v_1, v_3, v_2, v_6, v_4, v_5\}$。这就是广度优先搜索，总是先“摸清楚周围的环境，再继续向更深处进发”。BFS 算法可以求解无权图或等权图的最短路径问题，我们将在习题 6 第 16、17 题中详细介绍。

至此，“深”和“广”的语义已经很明确，“深度优先”总是冲到最“深”处，而“广度优先”总是先向“广”阔的四周探索。

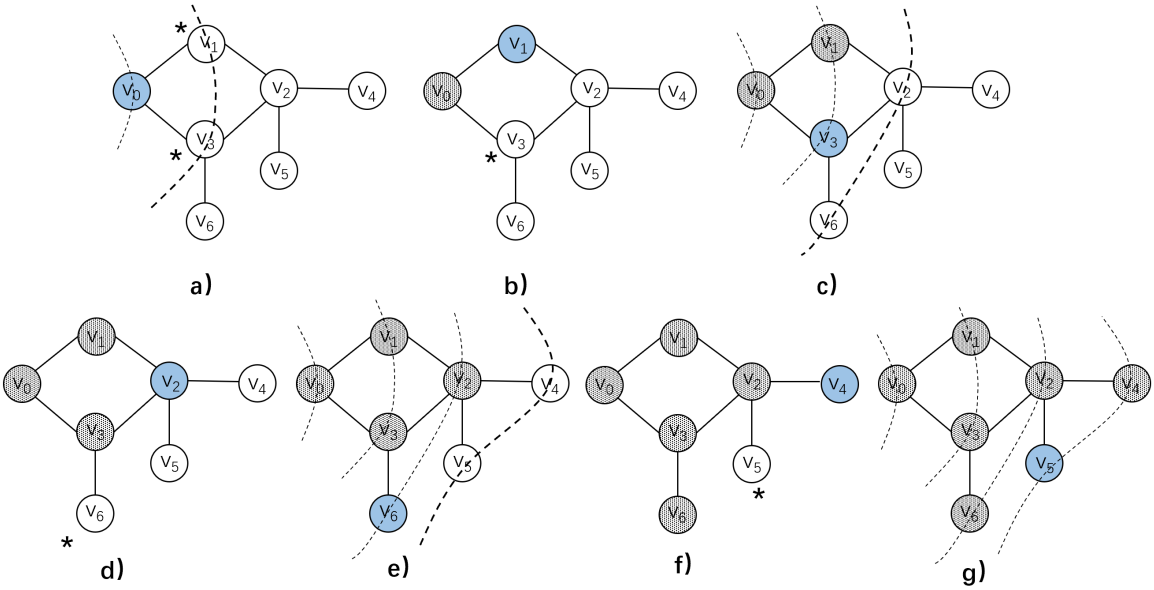


图 5.7 广度优先搜索

5.2.3 DFS 和 BFS 的实现

可以看到，广度优先搜索（BFS）总是“越早看见的顶点越先被访问”（因为层数小），而深度优先搜索（DFS）总是“越晚看见的顶点越先被访问”（因为一路往前冲而不是回头）。这正与队列和栈的“先进先出”与“后进先出”的特点相符合。

如果你研究过—waypoint-3—（第 53 页）处的问题，可以进一步理解：实际上广度优先（BFS）的过程中，“顶点们”成为了“仍然等待某事发生，并且优先处理先来者的数据”，而深度优先（DFS）的过程中，“顶点们”则是“仍然等待某事发生，并且优先处理后来者的数据”：

1. 在广度优先（BFS）中，顶点们等待被访问，并且先“被看见”的先访问；
2. 而在深度优先（DFS）中，顶点们等待被访问，并且后“被看见”的先访问。

有了以上思维过程，我们就可以非常清晰地给出二者的代码实现过程。

深度优先搜索（DFS）的实现

创建一个栈 s，访问起点，并将起点的所有邻接且尚未被访问过的顶点入栈。之后：

1. 若栈空，说明没有待访问的顶点了，算法结束，否则，进入第二步；
2. 从栈顶取出一个顶点，访问之，并将该顶点的所有邻接且未被访问过的顶点入栈；回到第一步。

如此循环直至栈空，则所有顶点均被访问一遍。

如何确定某顶点是否已经访问过？最常见的办法是维护一个一维数组：

```
bool visited[MAX_VNUM]; // 所有元素初始为 false，表示还未访问
```

visited[i] 为 true 则表示已经访问过了。

以邻接矩阵存储的图为例，深度优先搜索的代码如下所示：

```
/************** 邻接矩阵结构定义 ***************/
// // MAX_VNUM 为该结构可存储的图的最大顶点数
//     struct MGraph {
//         VertexInfo v[MAX_VNUM] ;
//         int AdjM[MAX_VNUM][MAX_VNUM];
//         int current_vnum;   // 当前顶点数
//         int current_enum;   // 当前边数
//     };

// 输入图 G 以及起点编号 start_idx，以 DFS 方式对每一个顶点执行某函数 visit()
void DFS(MGraph G, int start_idx) {
    bool visited[G.current_vnum];
    // 初始化 visited[] 数组
    for(int i=0; i<G.current_vnum; i++) {
        visited[i] = 0;
    }

    stack<int> s;   // 栈中存储顶点编号即可
    s.push(start_idx);
    visited[start_idx] = true;
    while(!s.empty()) {
        // 访问栈顶顶点 v
        int v = s.top();
        s.pop();
        visit(G.v[v]);   // 访问 v

        // 将与 v 邻接的所有尚未访问的顶点入栈
        for(int i=0; i<G.current_vnum; i++) {
            if(G.AdjM[v][i] != 0 && visited[i] == false) {
                s.push(i);
                visited[i] = true;
            }
        }
    } // end while
}
```

若图是通过邻接表存储的，与邻接矩阵存储的代码有什么区别呢？其实仅在寻找 v 的所有邻接点时，代码有区别。邻接表版本代码如下：

```
/************ 邻接表结构定义 *************/
// struct AdjEdge {
//     int u;  // 邻接顶点的编号
//     int weight; // 当前顶点到 u 的边的权值。
// };
// struct LGraph {
//     vector<AdjEdge> AdjList[MAX_VNUM];  // 邻接表
//     VertexInfo v[MAX_VNUM];     // 存放顶点信息
//     int current_vnum;           // 当前顶点数
//     int current_enum;           // 当前边数
// };

// 输入图 G 以及起点编号 start_idx，以 DFS 方式对每一个顶点执行某函数 visit()
    void DFS(LGraph G, int start_idx) {
        bool visited[G.current_vnum];
        // 初始化 visited[] 数组
        for(int i=0; i<G.current_vnum; i++) {
            visited[i] = 0;
        }

        stack<int> s;   // 栈中存储顶点编号即可
        s.push(start_idx);
        visited[start_idx] = true;
        while(!s.empty()) {
            // 访问栈顶顶点 v
            int v = s.top();
            s.pop();
            visit(G.v[v]);   // 访问 v

            // 将与 v 邻接的所有尚未访问的顶点入栈
            for(int i=0; i<G.AdjList[v].size(); i++) {
                // G.AdjList[v] 是 v 的邻接表，G.AdjList[v][i] 是该表中第 i 个
                ↪ 元素
                int u = G.AdjList[v][i].u;
                if(visited[u] == false) {
                    s.push(u);
                    visited[u] = true;
                }
            }
        } // end while

    }
```

广度优先搜索（BFS）的实现

根据前面的思维过程很容易知道，将深度优先搜索代码中的栈换成队列即是广度优先搜索。邻接矩阵版：

```
// 输入图 G 以及起点编号 start_idx，以 BFS 方式对每一个顶点执行某函数 visit()
    void BFS(MGraph G, int start_idx) {
        bool visited[G.current_vnum];
        // 初始化 visited[] 数组
        for(int i=0; i<G.current_vnum; i++) {
            visited[i] = 0;
        }

        queue<int> q;   // 队列中存储顶点编号即可
        q.push(start_idx);
        visited[start_idx] = true;
        while(!q.empty()) {
            // 访问队首顶点 v
            int v = q.front();
            q.pop();
            visit(G.v[v]);   // 访问 v

            // 将与 v 邻接的所有尚未访问的顶点入队
            for(int i=0; i<G.current_vnum; i++) {
                if(G.AdjM[v][i] != 0 && visited[i] == false) {
                    q.push(i);
                    visited[i] = true;
                }
            }
        } // end while
    }
```

邻接表版：

```
// 输入图 G 以及起点编号 start_idx，以 BFS 方式对每一个顶点执行某函数 visit()
    void BFS(LGraph G, int start_idx) {
        bool visited[G.current_vnum];
        // 初始化 visited[] 数组
        for(int i=0; i<G.current_vnum; i++) {
            visited[i] = 0;
        }

        queue<int> q;   // 队列中存储顶点编号即可
        q.push(start_idx);
        visited[start_idx] = true;
        while(!q.empty()) {
            // 访问队首顶点 v
            int v = q.front();
            q.pop();
            visit(G.v[v]);   // 访问 v

```

```
18              // 将与 v 邻接的所有尚未访问的顶点入队
19              for(int i=0; i<G.AdjList[v].size(); i++) {
20                  // G.AdjList[v] 是 v 的邻接表，G.AdjList[v][i] 是该表中第 i 个
                    ↪ 元素
21                  int u = G.AdjList[v][i].u;
22                  if(visited[u] == false) {
23                      q.push(u);
24                      visited[u] = true;
25                  }
26              }
27          } // end while
28      }
```

上面代码中使用到的 stack 和 queue 是 C++ 标准库为我们提供的栈和队列的数据结构。我们曾在—waypoint-7—（第 66 页）处介绍过。

思考

你也许会有疑问，将 visited[i] 标记为 true 的时机，难道不应该是访问 i 之后吗？代码中怎么将 i 入队（还未访问）时就将 visited[i] 标记为 true 了？

访问后标记当然是正确的做法。但事实上，入队时即标记同样是正确的，并且是更优的做法。你可以想一想，为什么正确？为什么更优？（可以结合图 5.7 的搜索过程思考）。

以上便是图最常见的两种遍历方式：广度优先搜索和深度优先搜索。信使任选一种方式，即可确保每个据点都不会遗漏。但是，信使很快发现了这样一个问题，如图 5.8 所示，信使走到据点 A 后，发现通往 D、E 的桥梁已经倒塌，而他无论如何也找不到其他路通往 D、E 这两个城镇。

信使回去汇报后，指挥官们发现类似的**非连通**区域还有很多，于是立即快马加鞭地组织人手修路搭桥，将王国地图修复成了一个**连通图**。

思渡者啊，你也许会想，若实际中某图存在从起点出发不可达的顶点，即图是一个非连通图，该如何遍历它呢？其实只需要对图中每一个顶点均执行一次上面的 DFS 或 BFS 即可，以图 5.8 为例，伪代码如下：

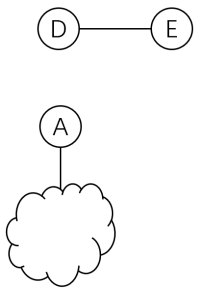


图 5.8 无法抵达对岸的城镇

```
Graph G;    // 假设 G 已经存储了图5.8所示的图
for(int i=0; i<N; i++) {
    if(!visited[i]) {
        DFS(G, i);
    }
}
```

以上代码会对 G 中所有顶点（包括顶点 D、E、A 等）依次判断，若未访问过，则对其执行 DFS。显然，当首次对 A 所在的连通块执行 DFS 时，会将其中所有顶点标记为已访问，之后循环中判断时，则不会对连通块中的其他顶点执行 DFS 了。D 和 E 所在的连通块有类似的情况。

以上两种遍历的实现都是非递归实现。事实上，由于深度优先搜索的特性，通过递归的方式实现深度优先搜索是非常简单的：

```
bool visited[N];    // 初始化为 false
void DFS(int v) {
    visit(v);   // 访问 v
    visited[v] = true;
    // 递归地访问 v 的邻接点，适配邻接矩阵或邻接表就行了。
    for(v 的每一个邻接点 u) {
        if(!visited[u]) {
            DFS(u);
        }
    }
}
```

要遍历非连通图，分别以每一个顶点为起点调用以上函数即可：

```
void Travel() {
    for(int i=0; i<N; i++) {
        DFS(i);
    }
}
```

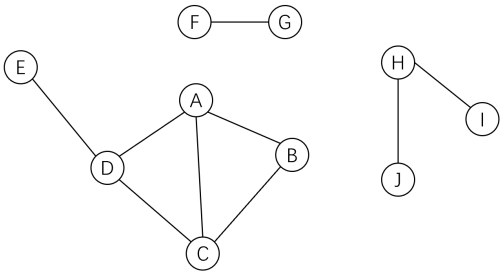
由于递归调用过程中，操作系统要开出很多个函数栈，因此递归的方式往往空间复杂度更大，但递归算法在很多情况下也是有用的。

根据指挥官给出的方法，信使无一疏漏地将大敌来犯的消息传遍了王国的每一座城池、每一座要塞。士兵和居民们纷纷武装起来，布起防御系统，准备迎接那场即将到来的大战。

— BONFIRE ENCOUNTERED —

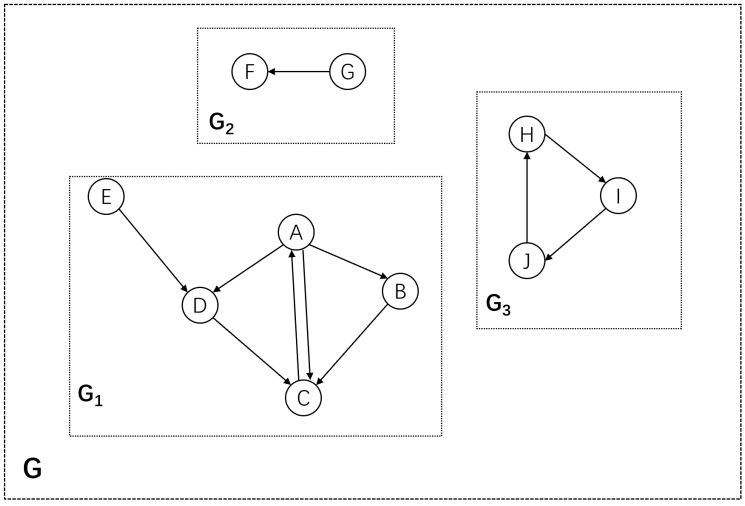
习题 6 （参考答案见习题 6 参考答案，第 227 页）

1. 关于下图所示的无向图，下列说法错误的是_____



A. 顶点 A 的度为 3
B. 顶点 A 与顶点 E 是不连通的
C. 该图有 3 个连通分量
D. 对于顶点 H 所在的连通分量，若以 H 为起点，其深度优先搜索序列可以是 {H, I, J}

2. 关于下图所示的有向图 G 以及其三个子图 G_1, G_2, G_3，下列说法错误的是_____



A. 三个子图 G_1, G_2, G_3 中，仅 G_3 是强连通的
B. 图 G 的强连通分量数大于 3
C. 顶点 A 的出度为 3
D. 顶点 F 的出度为 1

3. （某年 408 真题）下列关于无向连通图特性的叙述中，正确的是_____。
I. 所有顶点的度之和为偶数。
II. 边数大于顶点个数减 1。
III. 至少有一个顶点的度为 1。
A. 只有 I
B. 只有 II
C. I 和 II
D. I 和 III

4. 思考：有 N 个顶点的无向图，至多有多少条边？（不允许存在自环和重复边）。

5. 思考：有 N 个顶点的无向图，要保证它是连通图，至少要有多少条边？

6. （某年 408 真题）若无向图 $G=(V,E)$ 有 7 个顶点，无边。现在要向图 G 中添加若干条边。请问，要保证无论怎样添加，添加 N 条边后图一定是连通的，则 N 最小是_____

A. 6 B. 15 C. 16 D. 21

7. 设某无向图的邻接矩阵 A 如下所示，若 0 表示边不存在，则各顶点的度依次是_____

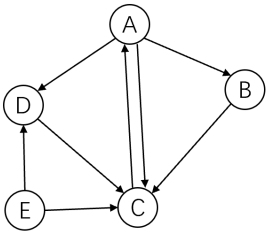
$$A=\begin{bmatrix}0&1&0&1\\1&0&1&1\\0&1&0&0\\1&1&0&0\end{bmatrix}$$

A. 2,3,1,2 B. 4,6,2,4 C. 0,1,2,3 D. 999,1000,1001,7820

8. 若上题的矩阵 A 表示的是某有向图的邻接矩阵，则各顶点的度依次是_____
提示：有向图中顶点的度定义为该顶点的入度和出度之和。

A. 2,3,1,2 B. 4,6,2,4 C. 0,1,2,3 D. 999,1000,1001,7820

9. 如下图所示的图 G 若使用邻接表存储，顶点 A 和 C 的邻接表表长分别是多少？_____



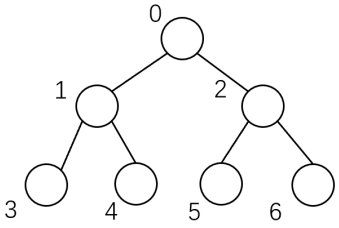
A. 3,3 B. 1,4 C. 3,1 D. 4,4

10. 对于上题图以顶点 E 为起点进行深度优先搜索，不可能得到的搜索序列是_____

A. {E,D,C,A,B} B. {E,C,A,D,B}

C. {E,C,A,B,D} D. {E,C,D,A,B}

11. 对于下图所示的图 G，以 0 号顶点为起点进行广度优先搜索，不可能得到的序列是_____



A. {0,1,2,3,4,5,6} B. {0,2,1,6,5,4,3}

C. {0,3,4,5,6,1,2} D. {0,1,2,3,4,6,5}

12. 关于图的两种遍历算法：深度优先搜索（DFS）和广度优先搜索（BFS），下列说法错误的是_____

A. 深度优先搜索通常通过队列实现，而广度优先搜索通常通过栈实现
B. 对于同一个图，深度优先搜索、广度优先搜索序列可能不唯一，取决于具体实现
C. 深度优先搜索能够访问到起点所在连通块的每一个顶点，不会遗漏
D. 广度优先搜索能够访问到起点所在连通块的每一个顶点，不会遗漏

13. 对于有 N 个顶点，未知边数的无向无权图 G, 分别编写以下两个函数：

1）若图 G 采用邻接矩阵存储，求其所有顶点的度数之和：

```
bool AdjM[N][N];    // 假设该数组已经正确存储了 G 的邻接矩阵
// 返回所有顶点的度数之和
int DegreeSum(int N) {
    // 在此处完成你的代码
}
```

2）若图 G 采用邻接表存储，求其所有顶点的度数之和；

```
vector<int> AdjList[N]; // 该数组已经正确存储了 G 的邻接表
// 返回所有顶点的度数之和
int DegreeSum(int N) {
    // 在此处完成你的代码
}
```

随意画出一个无向图，脑中模拟你代码的运行过程并计算出结果，你发现了什么规律吗？对于有向图有类似的规律吗？

【14、15、16、17、18 题基于此背景】传说在《前有数据结构》大陆某处神秘的角落有一片森林，其中有一棵樱花树无时不刻地施展着她的魔法，为世人祈福，带来幸运。可是该魔法也无时不刻地侵蚀着她自身，这些痛苦会凝聚成泪滴积于花瓣之上，将其饮下，便能抚平樱树的悲怆，化作自身的庇佑。

火光摇曳中，你在帐篷中沉睡过去。维火女凝视着你满身的疲惫与疮痍，眼底盛满了心疼与依恋。她不再忍心见你独自背负这苦难，便悄然别过营火，孤身涉足那片未知的森林，去寻找并带回那代表着祝福与庇佑的樱花之泪。

天亮了。维火女已到达了那片森林。你通过火焰之环看到了那片森林的地图。森林中有 N(N<=100) 棵树木，编号为 0 到 N-1，一些树木之间通过双向的灵魂通路相连，而有些树木则无法抵达。

14. 维火女从编号为 0 的树木处出发，试图探索可达的所有树木，以找到那具有魔法的樱花树。假设地图采用邻接表存储，请你编写一个递归版本的 DFS 函数，计算维火女从起点 0 出发，总共可以访问到的可达树木有多少棵？（包含起点本身）。

```
const int N/* = 100*/;
bool visited[N];    // 已初始化为 false

```

```
// AdjList[i] 已存放编号为 i 的顶点的邻接表，邻接表元素类型 int，含义为与 i 邻接的顶点编号
int CountAccessibleTrees(vector<int> AdjList[N]) {

}
```

15. 假设地图采用邻接表存储，请你用非递归的 DFS 或 BFS 计算，这片森林有多少个连通块？

```
const int N/* = 100*/;
bool visited[N];    // 已初始化为 false

// AdjList[i] 已存放编号为 i 的顶点的邻接表，邻接表元素类型 int，含义为与 i 邻接的顶点编号
int CountConnectedComponents(vector<int> AdjList[N]) {

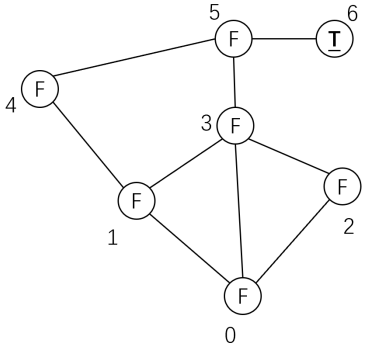
}
```

*16. N 棵树木的信息已存储于一个 char 类型的数组中：

```
char trees[N];
```

若某棵树是普通树木，其值为'F', 若它是那棵魔法樱树，则其值为'T'。假设地图采用**邻接矩阵**存储，维火女从起点 u 出发，能否抵达樱花树？若能，输出一条可行的路径，否则输出“.....”。

以下图地图为例，若起点 u 为 0，你可以输出“01356”、“0356”或“0231456”等，但不能输出“0456”、“06”等等不合法的路径；若起点 u 不在此连通块，输出“.....”。



```
const int N/* = 7 */;

char trees[N] /*= {'F', 'F', 'F', 'F', 'F', 'F', 'T'}*/;    // 保存顶点数据
int Matrix[N][N] /*= {    // 邻接矩阵
    { 0, 1, 1, 1, 0, 0, 0 },
    { 1, 0, 0, 1, 1, 0, 0 },
    { 1, 0, 0, 1, 0, 0, 0 },
    { 1, 1, 1, 0, 0, 1, 0 },
    { 0, 1, 0, 0, 0, 1, 0 },
    { 0, 0, 0, 1, 1, 0, 1 },
    { 0, 0, 0, 0, 0, 1, 0 },
}*/;

void FindPath(int u) {

}
```

*17. 若上题条件下，假设所有灵魂通路的通行时间**相等**，当樱花树可以抵达时，要求输出所有可行路径中距离最短的一条（经过的边数最少的），不可抵达时同样输出“.....”，请写出代码。

若有多条路径最短，任意输出一条即可。

*18. 若 16 题条件下，假设所有灵魂通路的通行时间**不等**，当樱花树可以抵达时，要求输出所有可行路径中距离最短的一条（花费时间最少的），不可抵达时同样输出“.....”，请写出代码。

若有多条路径最短，任意输出一条即可。邻接矩阵中已存储了各灵魂通路的通行时间。

```
const int N/* = 7 */;

char trees[N] /*= {'F', 'F', 'F', 'F', 'F', 'F', 'T'}*/;    // 保存顶点数据
int Matrix[N][N] /*= {    // 邻接矩阵，0 表示无路，非 0 表示通行时间
    { 0, 3, 2, 2, 0, 0, 0 },
    { 3, 0, 0, 2, 5, 0, 0 },
    { 2, 0, 0, 1, 0, 0, 0 },
    { 2, 2, 5, 0, 0, 6, 0 },
    { 0, 1, 0, 0, 0, 2, 0 },
    { 0, 0, 0, 6, 2, 0, 1 },
    { 0, 0, 0, 0, 0, 1, 0 },
}*/;

void FindPath(int u) {
```

```
15
16 }
```

19.（某年 408 真题）已知无向连通图存在 EL 路径的充要条件是度为奇数的顶点数为 0 或 2。设无向连通图 G 采用邻接矩阵存储，类型定义如下：

```
typedef struct {
    int numVertices, numEdges;  // 图中实际的顶点数和边数
    char VerticesList[MAXV];    // 顶点表。MAXV 为已定义常量
    int Matrix[MAXV][MAXV];       // 邻接矩阵
}MGraph;
```

若顶点 i 到 j 存在边，则 Edge[i][j] 为 1，否则为 0。请设计算法 int IsExistEL(MGraph G)，判断 G 是否存在 EL 路径，若存在，返回 1，否则返回 0。要求：
1）给出算法的基本设计思想；
2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释；
3）说明你所设计算法的时间和空间复杂度。

20.（某年 408 真题）已知通常将有向图中出度大于入度的顶点称为 K 顶点。若图采用邻接矩阵存储，类型定义如下：

```
typedef struct {
    int numVertices, numEdges;  // 图的顶点数和有向边数
    char VerticesList[MAXV];    // 顶点表，MAXV 为已定义常量
    int Matrix[MAXV][MAXV];       // 邻接矩阵
}MGraph;
```

若顶点 i 到 j 存在边，则 Edge[i][j] 为 1，否则为 0。请你设计一个算法 int PrintKVertices(MGraph G)，输出 G 中的所有 K 顶点，并返回 K 顶点的个数。要求：
1）给出算法的基本设计思想；
2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

5.3 最短路径

王国的故事还在继续。根据指挥官给出的遍历方法，信使无一疏漏地将大敌来犯的消息传遍了王国的每一座城池、每一座要塞。士兵和居民们纷纷武装起来，布起防御系统，准备迎接那场即将到来的大战。

5.3.1 迪杰斯特拉算法

接下来，一位指挥官提出了这样一个问题：若某个据点有重兵来犯，从首都派出军队去该据点支援，走哪条路，能够最快到达？这种求解从某顶点到图中其他所有顶点的最短路径的问题，后来被称为**单源最短路径**问题。

为了解决这个问题，指挥官们将王国建模成了一个无向带权图，边的权表示走此路径需要消耗的时间。如图 5.9 所示。

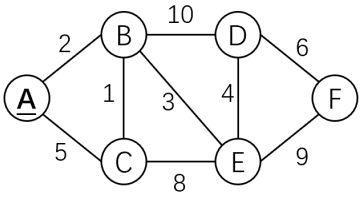


图 5.9

假设 A 为首都，如何求 A 到 F 的最短路径？一位指挥官指出，最简单的做法就是把所有可能的路径都找出来，然后比较它们的长度。这种做法通过 DFS 进行路径搜索即可实现，如**习题 6 第 18 题**（第 160 页）所做的那样。但是，这种做法存在一个明显的问题：即使其中许多路径已经不可能成为最短路径，我们仍然要花费时间去搜索它们。对图 5.9 进行 DFS 路径搜索的搜索树如图 5.10 所示。可以看到每个叶节点即代表了一条路径的终点，要么是找到了目标顶点，要么是非目标顶点的死路。

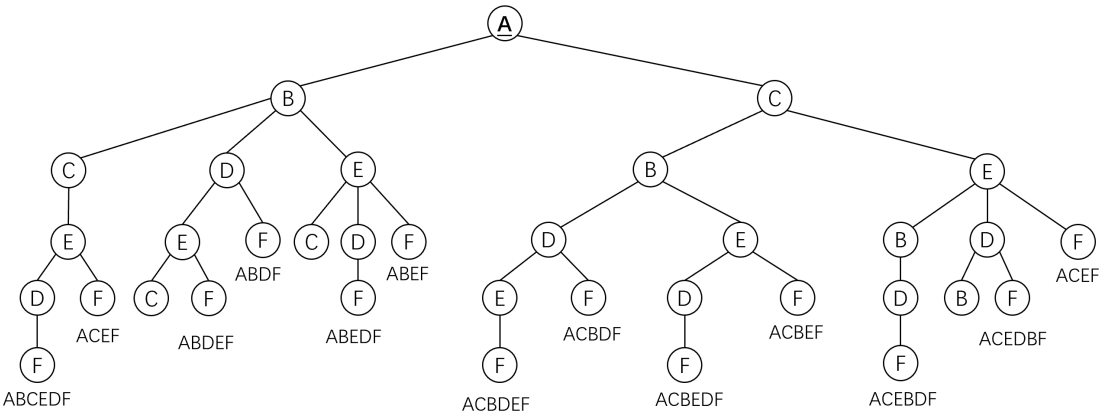


图 5.10 搜索树

当我们探索到一条可行的路径 ABDF 时，就已经知道最短路径不可能大于 4，于

是 ABEDF、ACBDEF 等路径其实完全没有必要探索，即使使用**剪枝策略**（见习题 6 第 18 题参考答案），也要至少探索 4 个顶点再回头。DFS 的穷举过程最差时间复杂度是 $O(N!)$ 量级，对于稍微复杂的图，该算法绝对无法接受。况且如果需要求出 A 到所有顶点的最短路径，就需要再重复如图 5.10 的搜索过程若干次。

CoreLoop

我们暂时把思路拉回现实世界。

执笔至此，笔者以为，接下来要介绍的迪杰斯特拉算法，大抵是本书中最令人惊叹、最具创造性的算法。我不想直接介绍迪杰斯特拉算法的过程与实现。我要尝试做一件事情：与你一起思考并探索迪杰斯特拉算法“诞生”的过程：当 DFS 绝对无法接受时，最短路径问题就摆在我们面前，我们将会有哪些思考和结论？以此过程，一方面加深大家对于算法的理解和记忆（而不是枯燥地记住这复杂的过程），另一方面，通过探索过程中的艰难到最后的柳岸花明，来体会算法开创者非凡的创造力，以此向艾兹赫尔·迪杰斯特拉（Edsger W. Dijkstra）—这位计算机领域的传奇人物致敬。

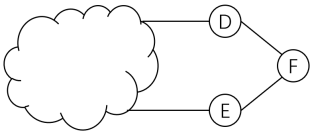
如你对此过程兴趣无多，请直接跳至—waypoint-8—（第 168 页）处，这不会影响后面的旅途。

首先，我们很容易想到这样一个想法：

假设顶点 i 正确的事实最短路径为 $mind[i]$，那么，一定有：

$$mind[i] = min(mind[j_1] + cost(j_1, i), mind[j_2] + cost(j_2, i), ..., mind[j_n] + cost(j_n, i))$$

其中，$j_1, j_2, ..., j_n$ 为 i 的所有邻接点，$cost(j, i)$ 表示顶点 j 到 i 的花费。例如如图 5.9 中，要到达 F，必然经过 D 或 E，因此对于顶点 F，一定有：



$$mind[F] = min(mind[D]+cost(D, F), mind[E]+cost(E, F))$$

也就是说，F 的最短路径就是① A 到 D 的最短路径 + D 到 F 的花费与② A 到 E 的最短路径 + E 到 F 的花费这二者之间更小的。以上是**第一个事实**。那么只要确定了 D、E 的最短路径，F 的最短路径也就确定了。

那么我们能不能先确定某些顶点的最短路径呢？

显然，顶点 A 到自身的花费（即 $mind[A]$）一定是能确定的：就是 0。这是因为路程花费时间不可能为负，所以 A -> xxx -> A 的花费一定更大。我们将这种思想记为**“不绕弯思想”**。

同时，知道了 mind[A] 为 0 之后，对于顶点 C，他有 A、B、E 三个邻接点，因此

有：

$$mind[C] = min(mind[A] + cost(A,C), mind[B] + cost(B,C), mind[E] + cost(E,C))$$

其中的 $mind[A] + cost(A,C)$ 这一项就能计算得出：$0 + 5 = 5$。于是我们可以记录下来**当前已知的（并非全局的）C 的最短路径 $cur_mind[C] = 5$**，并在未来某时刻确定了 B、E 的 $mind[B]$ 和 $mind[E]$ 时，若 $mind[B] + cost(B,C) < cur_mind[C]$ 或 $mind[E] + cost(E,C) < cur_mind[C]$，则可以用 B 或 E 来更新 $cur_mind[C]$，这样最终得到的 $cur_mind[C]$ 一定是全局最短路径。于是我们顺着这个思路会自然地有这样的想法：**在确定了一个顶点的全局最短路径 mind[] 时，要尝试更新其所有邻接点的最短路径，否则可能错过正确解。**

于是在知道了 $mind[A]$ 之后我们更新所有 A 能到达的邻接点的“当前最短路径 $cur_mind[]$”，此时：

$$cur_mind[B] = 2$$
$$cur_mind[C] = 5$$

而其他顶点暂未找到一条到达 A 的路径。如图 5.11 a）所示，带下划线的数字表示该顶点当前已知的最短路径 $cur_mind[]$，顶点有填充则表示该顶点的全局事实最短路径已经确定。

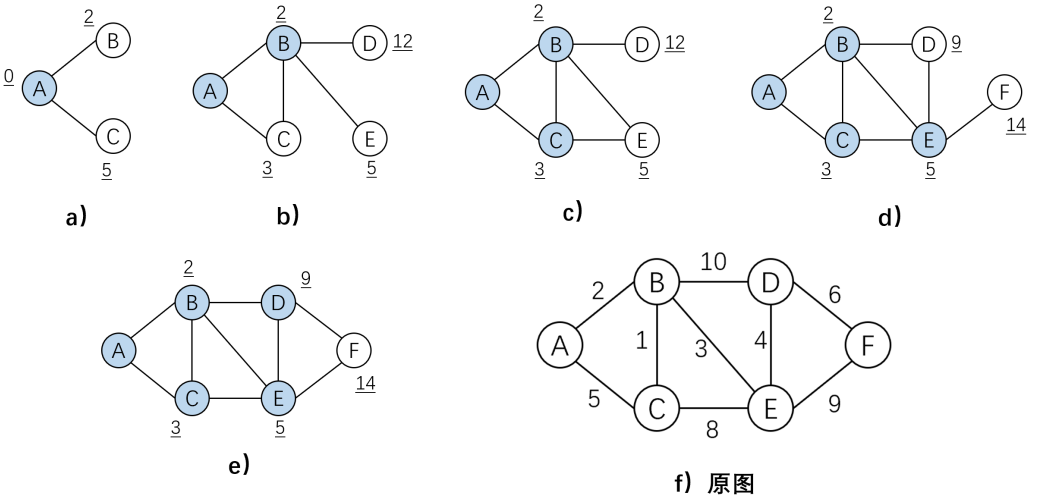


图 5.11

我们会发现，此时 $cur_mind[B] = 2$ 一定是 B 的全局最短路径（因为 B 是所有 A 的邻接点中花费最小的那个，因此 A -> B 一定比 A -> 非 B 更短，结合“不绕弯思想”，B 此时一定全局最短）。我们用集合 S 记录已经找到全局最短路径的顶点，此时

$$S = \{A, B\}$$

如前所述，确定了 $mind[B]$ 后，就可以更新与 B 邻接的 C、D、E 的当前已知最短路径 $cur_mind[]$ 信息：

$$cur_mind[C] = min(5, mind[B] + cost(B, C)) = min(5, 2 + 1) = 3$$
$$cur_mind[D] = mind[B] + cost(B, D) = 2 + 10 = 12$$
$$cur_mind[E] = 2 + 3 = 5$$

如图 5.11 b）所示。此时所有 $cur_mind[]$ 中，最小的就是 $cur_mind[C]$。根据“不绕弯思想”，显然此时 C 已知的当前最短一定也是全局最短。可是，如何证明？接下来这部分便是迪杰斯特拉算法最为精彩也是最难理解的部分，还请听我慢慢道来。

如何证明当前已知的所有最短路径 $cur_mind[]$ 中最小的那个一定就是该顶点的全局最短路径？

如图 5.11 b）所示，由于 A、B 已确定全局最短路径，C 还未确定，且所有可以到达 C 的路径中一定至少存在一个还未确定的顶点（至少有一个 C 还未确定），我们假设存在某条路径 PATH 到 C 的距离比 $cur_mind[C]$ 更短：

```
PATH: A -> xxx -> P -> X -> xxx -> C
```

我们用 X 表示路径 PATH 中的第一个尚未确定全局最短路径的顶点，则它的前一个顶点 P 的全局最短路径一定已经确定。记这条路径 PATH 的长度为 L_PATH，则我们实际上假设了

$$L_PATH < cur_mind[C]$$

接下来，我们来一点一点证出该假设与事实相悖，从而推翻该假设。

由于 P 的全局最短路径已经确定，因此 $cur_mind[X]$ 一定已经由 P 点更新过，即此时一定有：

$$cur_mind[X] <= mind[P] + cost(P, X) \quad \textbf{// 式（1）}$$

上不等式右边即是假设的那条路径 PATH 到 X 时的长度（A -> xxx -> P -> X 的长度）。而由于图没有负权边，因此

```
A -> xxx -> P -> X
```

的长度一定小于等于

```
A -> xxx -> P -> X -> xxx -> C
```

的长度，即下式一定成立：

$$mind[P] + cost(P, X) <= L_PATH \quad \text{// 式（2）}$$

于是整理式 1 式 2，有：

$$cur_mind[X] <= mind[P] + cost(P, X) <= L_PATH \quad \text{// 式（3）}$$

而事实上 C、X 均是此时未确定最短路径的顶点，且 C 是所有 $cur_mind[]$ 中最小的那个，因此一定有：

$$cur_mind[C] <= cur_mind[X] \quad \text{// 式（4）}$$

由式 3、式 4，得：

$$cur_mind[C] <= cur_mind[X] <= mind[P] + cost(P, X) <= L_PATH$$

这与我们假设的 $L_PATH < cur_mind[C]$ 相悖，因此假设不成立，即不存在一条路径，其距离比当前已知的 C 的最小距离 $cur_mind[C]$ 更小。

由以上的过程，我们就证明了：当前已知的所有最短路径 $cur_mind[]$ 中最小的那个一定就是该顶点的全局最短路径。

于是，接下来的思路就豁然开朗了：不断地选择当前已知的所有最短路径中最小的那个，作为该顶点的全局最短路径（已由上面的过程证明这是绝对正确的），接下来，以该顶点为新的“中介”，去尝试更新其所有邻接点的 $cur_mind[]$，不断重复这个过程，直至所有顶点的全局最短路径都被确定。我们从 C 开始，看一遍接下来的过程。如

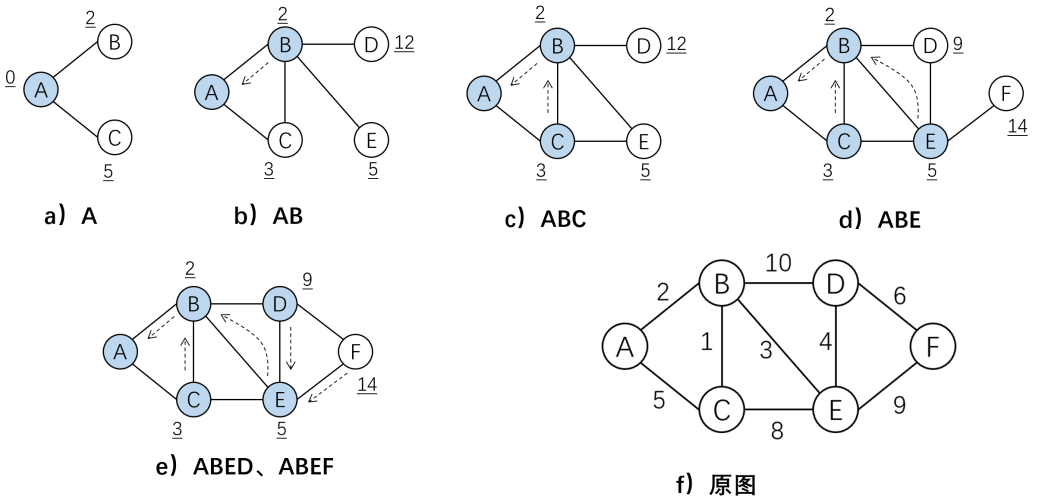


图 5.12

前所述，接下来将 C 的 $cur_mind[C]$ 作为 C 的全局最短，C 也就成了“已求解”的顶点。所有已求解的顶点集合 S 就变成了：

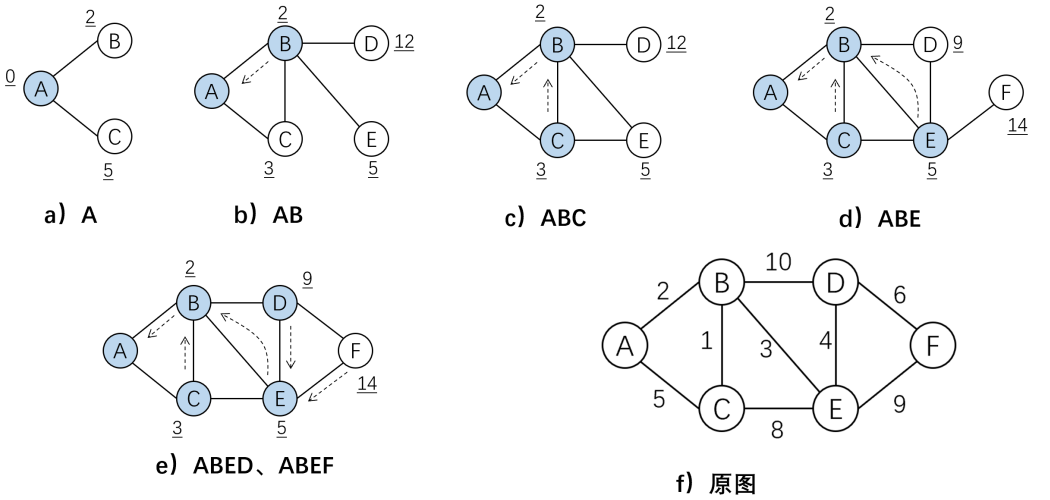
$$S = \{A, B, C\}$$

并且尝试更新 $cur_mind[E]$，但发现通过 C 到达 E 的距离：$3 + 8 = 11$ 不会比当前已知的：A -> B -> E（5）更短，因此 $cur_mind[E]$ 依然是 5。如图 5.12 c）所示。这一次我们在图中加入了路径回溯，记录每一个顶点最终的最短路径是由哪个顶点作为“中介”更新来的，如 B 是由 A 更新来的，因此指向 A，C 是由 B 最终更新来的，因此指向 B，以此方法记录最短路径本身（B 是 A -> B, C 是 A -> B -> C）。

继续从图中的 c）开始，此时 $cur_mind[D] = 12$，$cur_mind[E] = 5$，最小的是 $cur_mind[E]$，因此 E“已成功求解”，记 $mind[E] = cur_mind[E] = 5$，并以 E 作为新的中介，计算：

$mind[E] + cost(E, D) = 5 + 4 = 9$，小于当前的 $cur_mind[D] = 12$，于是更新 D
$mind[E] + cost(E, F) = 5 + 9 = 14$，发现了一条新的到达 F 的路，所以更新 F

更新后如图 d）所示。最后一步，D 作为“已成功求解”的顶点，并作为新的中介尝试更新 F，但通过 D 到 F 不会更小，因此 F 的路径前驱依然是 E，最终 F 也已成功求解，无其他顶点了，如 e）所示。



至此，所有顶点到达 A 的最短路径均已求解，$cur_mind[i]$ 中最终保存的就是顶点 i 到 A 的全局最短路径（事实最短路径），因为它一定已经经过了其所有邻接点的更新操作。

最后，需要注意我们以上讨论，都是基于一个假设：图中不存在负权边。通过上述过程也可以看得出来，若存在负边，“不绕弯思想”将不复存在，因此整个推理过程也将崩塌。正是由于这样的道理，我们马上会看到，**迪杰斯特拉算法要求图中不存在从源**

点可达的负权边。并且由以上思维过程我们知道，若图非连通，迪杰斯特拉算法也可以得出哪些顶点从源点出发是不可达的。

waypoint-8

接下来我们正式介绍赫赫有名的迪杰斯特拉算法（Dijkstra 算法）。我们先给出算法正确、完整的步骤，再由一个具体例子直观地感受一下。

迪杰斯特拉算法可用于求解单源最短路径问题。设图 $G=(V,E)$ 有 N 个顶点，$cost(u,v)$ 为 u 到 v 的权值（路径的长度），我们记 $cur_mind[i]$ 为顶点 i 当前已知的最短路径长度，则求解起点 s 到其他任意顶点的最短路径的过程如下：

1. 除起点 s 距离 s 为 0（$cur_mind[s]=0$）外，当前不知道任何其他顶点到 s 的最短路径，暂时认为所有顶点的最短路径都是正无穷 +inf，将其他所有 $cur_mind[]$ 初始化为 +inf。此时所有顶点均**未求解**。

2. 从所有**未求解**顶点中，选出已知的最短路径长度 $cur_mind[]$ 最小的那个顶点 v（第一次选的一定是 s），该长度一定就是该顶点 v 到 s 的全局最短路径长度，于是 v **“已求解”**。

3. 以 v 为中介点，尝试更新所有 v 的邻接点的 $cur_mind[]$。具体来说，对于 v 的一条指向 u 的边，若 $cur_mind[v]+cost(v,u)<cur_mind[u]$，说明经过 v 到 u 比之前已知的 u 到 s 的最短路径更短，所以将 $cur_mind[u]$ 更新为 $cur_mind[v]+cost(v,u)$。这一操作通常被称为**“松弛操作”**。

4. 重复步骤 2 和步骤 3，直到所有可从起点 s 到达的顶点均已确定最短距离。

事实上迪杰斯特拉算法就是一个不断地“选点”、“更新”的过程。我们设置一个数组 Solved[] 来标记哪些顶点已求解，哪些还未求解；并设立一个数组 Pre[] 标识每个顶点的路径前驱，如 Pre[1] 值为 0 就表示顶点 1 的最短路径的上一步是从 0 走过来的。

我们以图 5.13 f）所示的图为例来直观地感受一下迪杰斯特拉的过程。

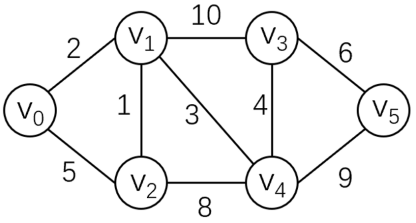
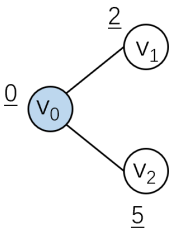


图 5.13 迪杰斯特拉算法过程演示图

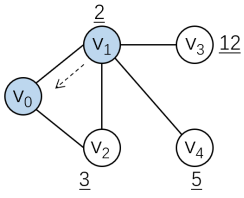
我们将 v_0 作为源点 s，初始时，所有顶点均未求解，Solved[] 为 false，除源点 s 的 $cur_mind[0]=0$ 外，所有其他顶点 $cur_mind[]$ 都是 +inf，并且所有顶点的 Pre[] 均初始化为-1，表示尚未找到一条路径，暂无前驱，如表 5.2“初始”所示。

1. 从未求解顶点集合 {0,1,2,3,4,5} 中选择当前已知最短路径 $cur_mind[]$ 最小的顶点（0 号顶点）。此时 0 号顶点的当前已知最短路径 $cur_mind[0]$ 一定也是全局最短路径，于是将 v_0 作为“已求解”顶点，Solved[0] 标记为 true。并且 v_0 就是起点，无前驱，$Pre[]$ 保持 -1 即可。然后以 0 号顶点为中介，尝试更新其邻接点 v_1 和 v_2 的 $cur_mind[]$: 由于 $cur_mind[v_0] + cost(v_0, v_1) = 0 + 2 = 2 < cur_mind[v_1] = +inf$，因此将 $cur_mind[v_1]$ 更新为 $cur_mind[v_0] + cost(v_0, v_1) = 2$，同理将 $cur_mind[v_2]$ 更新为 0+5=5，由于 v_1 和 v_2 当前是由 v_0 更新来的，因此将 Pre[1] 和 Pre[2] 均设为 0。
如图 a）及表 5.2 步骤 1 所示。



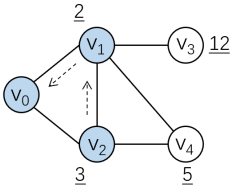
a) v_0

2. 从未求解顶点集合 {1,2,3,4,5} 中选择 $cur_mind[]$ 最小的顶点 v_1，将该顶点的全局最短路径确定为 2，并标记 v_1 的 Solved[1] 为 true。然后以 v_1 为中介，v_1 的邻接点有 v_2、v_3、v_4，于是分别计算：$cur_mind[v_1] + cost(v_1, v_2) = 3 < cur_mind[v_2] = 5$，于是将 $cur_mind[v_2]$ 更新为 3；$cur_mind[v_1] + cost(v_1, v_3) = 12 < cur_mind[v_3] = +inf$，于是将 $cur_mind[v_3]$ 更新为 12，类似地，将 $cur_mind[v_4]$ 更新为 5。并且 v_2、v_3、v_4 此时都是由 v_1 更新来的，因此将 Pre[2]、Pre[3]、Pre[4] 记为 1。
如图 b）及表 5.2 步骤 2 所示。



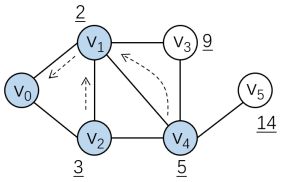
b) $v_0 v_1$

3. 从未求解顶点集合 {2,3,4,5} 中选择 $cur_mind[]$ 最小的顶点 v_2，将 Solved[2] 标记为 true，然后尝试更新 v_4 的 $cur_mind[4]$，但由于 $cur_mind[2] + cost(v_2, v_4) = 3 + 8 = 11 > cur_mind[4] = 5$，因此无需更新。
这一步其实只将顶点 v_2 标记为了“已求解”，如图 c）及表 5.2 步骤 3 所示。



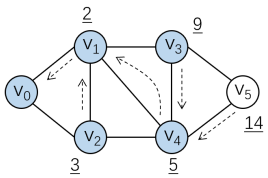
c) $v_0 v_1 v_2$

4. 从 {3,4,5} 中，选择顶点 v_4，标记 Solved[4] 为 true。并以 v_4 为中介，将 $cur_mind[3]$ 更新为 9、Pre[3] 更新为 4；将 $cur_mind[5]$ 更新为 5+9=14，将 Pre[5] 更新为 4。
如图 d）及表 5.2 步骤 4 所示。



d) $v_0v_1v_4$

5. 从 {3,5} 中，选择顶点 v_3，标记 Solved[3] 为 true。以 v_3 为中介，发现 $cur_mind[3] + cost(3,5) = 9 + 6 = 15 > cur_mind[5] = 14$，因此无需更新。此时最后一个顶点 v_5 的最短路径显然也已经确定了，为 14。
如图 e）及表 5.2 步骤 5 所示。最后，我们将 v_5 标记为已求解顶点即可。



e) $v_0v_1v_4v_3$、 $v_0v_1v_4v_5$

6. 将最后的顶点 v_5 标记为已求解。如表 5.2 步骤 6 所示。

表 5.2: 迪杰斯特拉算法过程演示

步骤 \ 顶点		0	1	2	3	4	5	剩余未求解顶点集合
初始	cur_mind[] Pre[] Solved[]	0 -1 false	+inf -1 false	+inf -1 false	+inf -1 false	+inf -1 false	+inf -1 false	{0,1,2,3,4,5}
1	cur_mind[] Pre[] Solved[]	0 -1 **true**	**2** **0** false	**5** **0** false	+inf -1 false	+inf -1 false	+inf -1 false	{1,2,3,4,5}
2	cur_mind[] Pre[] Solved[]	0 -1 true	2 0 **true**	**3** **1** false	**12** **1** false	**5** **1** false	+inf -1 false	{2,3,4,5}
3	cur_mind[] Pre[] Solved[]	0 -1 true	2 0 true	3 1 **true**	12 1 false	5 1 false	+inf -1 false	{3,4,5}
4	cur_mind[] Pre[] Solved[]	0 -1 true	2 0 true	3 1 true	**9** **4** false	5 1 **true**	**14** **4** false	{3,5}
5	cur_mind[] Pre[] Solved[]	0 -1 true	2 0 true	3 1 true	9 4 **true**	5 1 true	14 4 false	{5}
6	cur_mind[] Pre[] Solved[]	0 -1 true	2 0 true	3 1 true	9 4 true	5 1 true	14 4 **true**	{ }

可以看到，“选点”就是每次从尚未求解的集合中，“选”出一个当前已知最短路径 $cur_mind[]$ 最小的顶点 v，作为“已求解顶点”，并用它尝试“更新”其所有邻接点的最短路径 $cur_mind[]$。每次能确定一个顶点的全局最短路径，重复 N 次操作（其实 $N-1$ 次即可），即可求解所有顶点的全局最短路径，最终 $cur_mind[]$ 中保存的就是各个顶点的全局最短路径的长度，而各顶点的最短路径是什么，顺着 Pre[] 数组一路向前即可。如 v_5 到 v_0 的最短路径就是

5 -> Pre[5] = 4 -> Pre[4] = 1 -> Pre[1] = 0，即 5 -> 4 -> 1 -> 0

这就是赫赫有名的迪杰斯特拉算法。迪杰斯特拉算法可以求解出从源点 s 到各个与之连通的顶点之间的最短路径，同时也可以确定哪些顶点与 s 是不连通的。但迪杰斯特拉算法要求图中不存在从源点出发可达的负权边，**若图不满足这一条件，迪杰斯特拉算法将失效**。

迪杰斯特拉算法需要 N 次“选点”、“更新”操作，更新时需要找到该顶点的所有邻接点，而对于邻接矩阵和邻接表存储的图，找邻接点的时间复杂度有所区别，因此，迪杰斯特拉算法的时间复杂度与图的存储方式有关，下面我们就看看两种存储方式下，迪杰斯特拉算法如何实现。

1. 迪杰斯特拉算法实现（邻接矩阵版）

有了前面的思路，我们可以很轻松地写出迪杰斯特拉算法的代码。唯一需要注意的就是需要正确处理图中存在顶点不可达的情况，详请参见如下代码及注释。

```
int Matrix[N][N];   // 邻接矩阵，存放边权，若边不存在用 -1 表示
bool Solved[N];     // 已初始化为 false
int Pre[N];         // 已初始化为 -1
int cur_mind[N];    // 已初始化为 +inf。
const int inf = 0x3fffffff; // +inf
// 求解以 s 为源点的单源最短路径
void Dijkstra(int s) {
    cur_mind[s] = 0;
    // 该层循环用于执行 N 次“选点”、“更新”的操作
    for(int i=0; i<N; i++) {
        // 选出未求解的顶点中，cur_mind[] 最小的顶点 v
        int min_cur_mind = inf;
        int v = -1;
        for(int j=0; j<N; j++) {
            if(Solved[j] == false && cur_mind[j] < min_cur_mind) {
                min_cur_mind = cur_mind[j];
                v = j;
            }
        }

```

```
        /* 这里是为了处理图中存在顶点不可达的情况。
         若存在顶点不可达，则将可达顶点全选完之后，
         接下来一定会“选不到顶点”，所以若没选到顶点，
         则 v 的值是无意义的，不应执行后面的代码。*/
        if(v == -1) {
            break;
        }

        Solved[v] = true;
        // 以 v 为中介，尝试更新其所有邻接点
        for(int u=0; u<N; u++) {
            // v 到 u 有路径并且 u 的全局最短还未确定 不判断 Solved[u] == false
            ↪ 性能稍差一点，但影响不大。
            if(Matrix[v][u] != -1 && Solved[u] == false) {
                if(cur_mind[v] + Matrix[v][u] < cur_mind[u]) {
                    cur_mind[u] = cur_mind[v] + Matrix[v][u];
                    Pre[u] = v;
                }
            }
        }
    }
}
```

对于以上邻接矩阵版的代码，需要 N 次“选点”、“更新”操作，外层循环时间为 $O(N)$，每次选点时遍历一遍 cur_mind[] 数组的时间是 $O(N)$，更新时需要找到该顶点的所有邻接点，时间是 $O(N)$，整体的时间复杂度为 $O(N^2)$。空间方面，需要额外引入 Pre[]、Solved[]、cur_mind[] 三个数组，空间复杂度为 $O(N)$。

注意：邻接矩阵是存储图本身所需要的空间，不属于迪杰斯特拉算法带来的空间开销，不能算在算法的空间复杂度中。

2. 迪杰斯特拉算法实现（邻接表版）

邻接表版仅在获取顶点的邻接点信息时代码有所不同。邻接表版代码如下所示。

```
struct EdgeInfo {
    int u;      // 邻接点的编号
    int weight; // 边权
};
vector<EdgeInfo> AdjList[N];    // 邻接表
bool Solved[N];     // 已初始化为 false
int Pre[N];         // 已初始化为 -1
int cur_mind[N];    // 已初始化为 +inf。
const int inf = 0x3fffffff; // +inf
// 求解以 s 为源点的单源最短路径
void Dijkstra(int s) {
```

```
    cur_mind[s] = 0;
    for(int i=0; i<N; i++) {
        // 选出未求解的顶点中，cur_mind[] 最小的顶点 v
        int min_cur_mind = inf;
        int v = -1;
        for(int j=0; j<N; j++) {
            if(Solved[j] == false && cur_mind[j] < min_cur_mind) {
                min_cur_mind = cur_mind[j];
                v = j;
            }
        }

        if(v == -1) {
            break;
        }

        Solved[v] = true;
        // 以 v 为中介，尝试更新其所有邻接点
        for(int j=0; j<AdjList[v].size(); j++) {
            int u = AdjList[v][j].u;  // 与 v 邻接的顶点 u
            if(cur_mind[v] + AdjList[v][j].weight < cur_mind[u]) {
                cur_mind[u] = cur_mind[v] + AdjList[v][j].weight;
                Pre[u] = v;
            }
        }
    }
}
```

对于以上邻接表版的代码，外层循环时间同样为 $O(N)$，每次选点时遍历一遍 cur_mind[] 数组的时间同样是 $O(N)$，而更新时找到该顶点的所有邻接点的操作实际上与外层循环无关，是对整个图来说需要进行约 M 次，M 为图的边数。因此邻接表版时间复杂度为 $O(N^2+M)$。

空间方面，与邻接矩阵版本类似，同样是需要额外引入 Pre[]、Solved[]、cur_mind[] 三个数组，空间复杂度为 $O(N)$。

请注意，上面的代码中，每次寻找当前 cur_mind[] 最小的顶点时，我们是遍历一遍数组，为 O(N) 的时间。事实上，可以使用前面介绍过的数据结构—**堆**（C++ 中的 priority_queue）来完成，每次以 O(1) 时间即可返回最小的元素。若图采用邻接表实现，这种堆优化的迪杰斯特拉算法的**最坏**时间复杂度可以达到 $O((N+M)logN)$。由于 priority_queue 在此处的使用涉及稍复杂的 C++ 原理，因此不在此处给出代码，感兴趣的同学可以自己尝试写出。

5.3.2 Bellman-Ford 算法

几乎在迪杰斯特拉算法诞生的同期，也诞生了另外一个求解单源最短路径的算法：Bellman-Ford 算法。它与 Dijkstra 算法有一个重要区别：只要从源点出发能够到达的范围内不存在负权环，Bellman-Ford 算法就能够正确求得单源最短路径，即使图中存在负权边也没有问题。

Bellman-Ford 算法的核心思想是：反复对图中的所有边进行松弛操作。具体来说，假设图中有 N 个顶点 M 条边，源点为 s，与迪杰斯特拉算法类似，定义：cur_mind[v] 为当前已知的从 s 到 v 的最短路径长度，初始化 cur_mind[s] 为 0，其他顶点的 cur_mind[] 均为正无穷。

然后，对图中的每一条边 (u, v) 进行松弛操作：

$$dist[v] = \min(dist[v], dist[u] + cost(u, v))$$

其中，$cost(u, v)$ 表示边 (u, v) 的权值（路径长度）。对所有边完成一轮松弛后，再进行下一轮，则最多进行 $N-1$ 轮后，所有顶点的 cur_mind[] 就已经是全局最短路径。

为什么这个算法是正确的？

如图 5.14 所示，以 v_2 为起点，v_1 到 v_2 的最短路径是 1 条边的路径：$v1->v2$，因此第一轮操作之后就能考虑到，并将其长度 1 赋值给 cur_mind[1]，此后无论如何 cur_mind[1] 也不会再被更新。

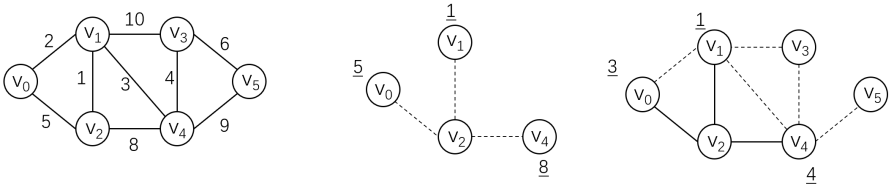


图 5.14

v_0 到 v_2 的最短路径是 $v_0->v_1->v_2$、v_4 到 v_2 的最短路径是 $v_4->v_1->v_2$，都是 2 条边的最短路径，因此第二轮操作之后就能考虑到这些路径，并且赋值给 cur_mind[0]、cur_mind[4]，之后绝对不会再改变。

Bellman-Ford 算法每完成一轮对所有边的松弛，就能够保证考虑到最多多一条边的路径，因此 $N-1$ 轮之后，就能考虑到最多 $N-1$ 条边的路径。而如果源点可达范围不存在负权环，那么任意两个顶点之间的最短路径一定不包含重复顶点，路径最多 $N-1$ 条边。

此外，如果完成 $N-1$ 轮后，再对所有边进行一轮松弛，仍然能够更新某个顶点的距离，就说明存在从源点可达的负权环。若图存在从源点可达的负环，则显然在负环中不

断转圈可以让最短路径无限地小下去，直至负无穷。因此，通过以上手段 Bellman-Ford 算法可以在图不存在负环时给出正确解，并在图存在负环时给出“问题不可解”的结论。

Bellman-Ford 算法需要做约 N 轮操作，每轮操作需要遍历所有边，对于邻接矩阵存储的图，遍历所有边需要遍历邻接矩阵，时间为 $O(N^2)$，而对于邻接表，遍历所有边需要遍历邻接表，时间为 $O(M)$。因此对于邻接矩阵存储的图，Bellman-Ford 时间复杂度为 $O(N^3)$，对于邻接表存储的图，Bellman-Ford 的时间复杂度为 $O(NM)$。

对于邻接表存储的图，Bellman-Ford 算法的实现如下所示：

```
struct EdgeInfo {
    int u;
    int weight;
};
vector<EdgeInfo> AdjList[N]; // 已存储图的邻接表
const int inf = 0x3fffffff;
int cur_mind[N];    // 已初始化为 inf
int Pre[N];     // 已初始化为 -1
bool BellmanFord(int s) {
    cur_mind[s] = 0;
    // 做 N-1 次
    for(int i=0; i<N-1; i++) {
        // 遍历所有边
        for(int v=0; v<N; v++) {
            // 与 v 邻接的所有边
            for(int k=0; k<AdjList[v].size(); k++) {
                int u = AdjList[v][k].u;
                int weight = AdjList[v][k].weight;
                if(cur_mind[v] + weight < cur_mind[u]) {
                    cur_mind[u] = cur_mind[v] + weight;
                    Pre[u] = v;
                }

            }
        }
    }

    // 再做一次，判断是否存在负环
    for(int v=0; v<N; v++) {
        for(int k=0; k<AdjList[v].size(); k++) {
            int u = AdjList[v][k].u;
            int weight = AdjList[v][k].weight;
            if(cur_mind[v] + weight < cur_mind[u]) {
                return false;
            }

```

```
        }
    }
    return true;
}
```

当图存在负环时，Bellman-Ford 算法返回 false，表示存在负环，最短路径问题不可解。否则返回 true，cur_mind[] 数组中存放的就是各顶点的最短路径。沿着 Pre[] 数组往回追溯即可得到轨迹。对于邻接矩阵版本仅在于遍历边的方式不同，这里不再给出代码。

可以看到，与 Dijkstra 算法从源点出发，逐渐向外“扩散”地求解各个顶点的最短路径的策略不同，Bellman-Ford 算法则是“放眼全局”，一步一步地让全局下的各个 cur_mind[] 慢慢向正确解“收拢”。

对于 Bellman-Ford 算法，我们自然会想到这样一个问题：我们每轮处理的时候，都需要遍历所有边，判断能否更新某个顶点的 cur_mind[]。但事实上，假设第 $i-1$ 轮仅有顶点 v 的 cur_mind[] 发生了变化，那么第 i 轮其实我们没有必要遍历所有边，而是只遍历 v 的出边即可，因为只有距离发生了变化的顶点才可能继续影响其他顶点。这样，仅处理距离发生了变化的边，就大大提高了算法的效率。因此对于上面的代码，在以下条件成立时：

```
if(cur_mind[v] + weight < cur_mind[u]) {
    cur_mind[u] = cur_mind[v] + weight;
    Pre[u] = v;
}
```

说明 u 是一个距离发生了变化的顶点，我们将其放入队列中。

于是思路就是：最初将起点 s 放入队列，并在每次处理时，将那些距离发生了改变的顶点入队。当队列为空时，说明没有新的顶点距离发生改变，也就处理完了。但是这样做有两个新的问题：

（1）反复入队问题。

如图 5.15 所示，假设 0 为起点，向上松弛的若干顶点，将这些顶点入队后，取出时，对每一个顶点的处理都可能将顶点 1 松弛一遍，如果每次都将 1 入队就造成了重复入队，会出现很多无意义的重复处理，继续向外传导，会造成指数级的性能灾难。因此要解决这个问题，就需要引入一个数组 in_queue[N]，标识每一个顶点是否已经在队列中了，顶点此时不在队列中时才将它入队。

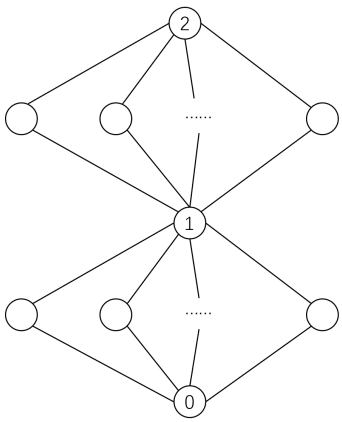


图 5.15 反复入队问题

（2）负环判断问题。这种做法一直做到队列为空，若存在

负环，则永远会有新的顶点距离变小，队列永远不会空。那怎么办？常见的做法是再记录一个数组 inq_cnt[N]，标记每个顶点入队的次数。若某个顶点已经入队了超过 $N-1$ 次（它的距离已经被更新过超过 $N-1$ 次），说明存在负环，问题不可解。

于是我们可以给出这种优化的 Bellman-Ford 算法代码（邻接表版）如下所示：

```
struct EdgeInfo {
    int u;
    int weight;
};
vector<EdgeInfo> AdjList[N]; // 已存储图的邻接表
const int inf = 0x3fffffff;
int cur_mind[N];    // 已初始化为 inf
int Pre[N];     // 已初始化为 -1

bool in_queue[N];   // 标记顶点是否当前在队列中，初始化为 false
int inq_cnt[N];         // 记录顶点入队次数，用于检测负权环，初始化为 0
bool SPFA(int s) {
    cur_mind[s] = 0;
    queue<int> q;
    q.push(s);
    in_queue[s] = true;

    while(!q.empty()) {
        int v = q.front();
        q.pop();

        // 弹出后，别忘了将标记改为 false，允许它以后再次入队
        in_queue[v] = false;

        // 遍历当前顶点的所有出边
        for(int j = 0; j < AdjList[v].size(); j++) {
            int u = AdjList[v][j].u;
            int weight = AdjList[v][j].weight;

            // 松弛操作
            if(cur_mind[v] + weight < cur_mind[u]) {
                cur_mind[u] = cur_mind[v] + weight;
                Pre[u] = v;

                // 如果 u 不在队列中，将其压入队列
                if(!in_queue[u]) {
                    q.push(u);
                    in_queue[u] = true;
                    inq_cnt[u]++; // 记录入队次数
```

```
40
41                      // 如果某个顶点入队次数超过 N-1 次，说明存在负权环
42                      if(inq_cnt[u] >= N) {
43                          return false;
44                      }
45                  }
46              }
47          }
48      }
49
50      return true;
51  }
```

这种优化后的 Bellman-Ford 算法被称为 **SPFA（Shortest Path Faster Algorithm）算法**。它的平均时间复杂度为 O(kM)，k 是每个顶点的平均入队次数，而最坏时间复杂度为 O(NM)。在大多数场景下 SPFA 算法性能要由于堆优化后的迪杰斯特拉算法，但对于极端情况，迪杰斯特拉算法最差也能保证 O((N+M)logN) 的复杂度，而 SPFA 算法可能达到 O(NM)。

5.3.3 Floyd 算法

Floyd 算法（弗洛伊德算法），是一种用于求解图中所有顶点对之间最短路径的算法，即**全源最短路径**问题。。Floyd 算法的核心思想在于：对于任意三个顶点 i、j、k，我们都可以尝试：

$$d_{ij} = \min(d_{ij}, d_{ik} + d_{kj})$$

也就是说，我们原来知道的 i 到 j 的最短距离是 d_{ij}，现在尝试让路径经过顶点 k，如果经过 k 的距离更短，就更新答案。

顶点到自身的距离初始化为 0，存在直接边的顶点之间距离初始化为边权，不存在直接边的顶点之间距离初始化为正无穷。算法实现如下：

```
const int inf = 0x3fffffff;
int d[N][N];

// 初始化距离矩阵
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        d[i][j] = (i == j ? 0 : inf);
    }
}

// 根据图中的边更新距离矩阵
// 假设存在一条从 u 到 v、权重为 w 的边
```

```
degree[u][v] = w;
// ...

// Floyd 算法
for (int k = 0; k < N; k++) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (d[i][k] != inf &&
                d[k][j] != inf) {
                d[i][j] = min(
                    d[i][j],
                    d[i][k] + d[k][j]
                );
            }
        }
    }
}
```

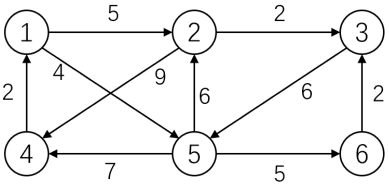
Floyd 算法支持存在负权边的图，但要求不存在从源点可达的负权环。其时间复杂度为 $O(N^3)$，空间复杂度为 $O(N^2)$。

— BONFIRE ENCOUNTERED —

火光闪烁中，你独自坐在营火旁。

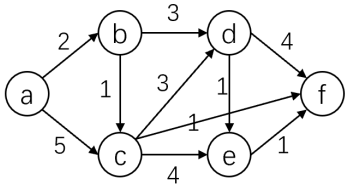
习题 7 （参考答案见习题 7 参考答案，第 240 页）

1. （某年 408 真题）使用迪杰斯特拉算法求下图中顶点 1 到其他各顶点之间的最短路径，则依次确定最短路径的目标定点是_____



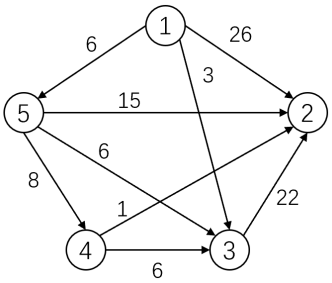
A. 5,2,3,4,6　B. 5,2,3,6,4　C. 5,2,4,3,6　D. 5,2,6,3,4

2. （某年 408 真题）对下图所示的有向带权图，若采用迪杰斯特拉算法求源点 a 到其他各顶点的最短路径，则第一个确定全局最短路径的顶点是 b，第二个确定全局最短路径的顶点是 c，后续的顶点依次是_____



A. d,e,f　B. e,d,f　C. f,d,e　D. f,e,d

3. （某年 408 真题）使用迪杰斯特拉算法求下图中从顶点 1 到其余各顶点的最短路径，将当前找到的从顶点 1 到顶点 2，3，4，5 的最短路径保存在数组 dist[] 中，则依次求出两个顶点（不包括源点 1）的最短路径后，dist 中的内容更新为_____



A. 26,3,14,6 B. 25,3,14,6 C. 21,3,14,6 D. 15,3,14,6

4. 请你说明：迪杰斯特拉算法、Bellman-Ford 算法以及 SPFA 算法应对以下情况的图时会怎样？

a）存在从源点可达的负环；
b）不存在从源点可达的负环，但存在从源点可达的负权边；

维火女被邪恶的恶魔、大陆守护神的孪生兄弟 LoopCore 抓了起来，并关在了监牢的某个牢房里。监牢可以被建模为一个图 $G = (V, E)$，顶点表示牢房。

已知该监牢有 N 个牢房，编号为 0 到 $N-1$，牢房之间均存在双向通路，用邻接表存储图模型。你毫不犹豫地动身前往牢房营救维火女，但你破门而入后意外地被魔法传送进了牢房 s。

5. 已知每个牢房（顶点）的数据类型为 char：维火女所在的牢房为'T'，其余牢房均为'F'。给出在每两个牢房之间移动所需要花费的时间 time_cost（非负），请你计算从 s 出发，到达维火女的房间所需要的最短时间是多少，若无法到达，返回-1。

```
// N 已定义
struct EdgeInfo {
    int u;          // 顶点编号
    int time_cost;  // 花费时间（边权）
};
vector<EdgeInfo> AdjList[N];    // 已存储邻接表信息
char ver_info[N];   // 已存储顶点信息
// 返回以 s 为起点到达维火女的房间需要花费的最短时间
int MinTime(int s) {

}
```

6. 在上题条件下，假设你在任何两个房间移动时需要和敌人战斗，此时边（Edge）使用如下结构定义：

```cpp
struct EdgeInfo {
    int u;
    int time_cost;      // 花费时间
    int energy_cost;    // 和敌人战斗需要消耗的能量值
};
```

保证所有边的 time_cost 和 energy_cost 均非负。你决定选择所有路线中，消耗时间最少的路线去营救维火女。若有多条路线消耗时间相等，你将选择其中消耗能量最少的一条路线。

编写一个函数，返回你所选择的路线所消耗的能量值。若无法到达，返回-1。

你成功营救出了维火女，你们一同回到大陆中，继续冒险。她手中正是那樱花之泪。

5.4 最小生成树

王国的指挥官们解决了最短路径问题之后，又提出了一个新的问题：目前据点之间通信只能通过信使传递，他们希望在据点之间铺设通信线缆，使得信息可以顺着线缆迅速传递。指挥官们认为，没有必要在任意两个据点之间都铺设线缆，而是只要能够保证任意两个据点之间都可以通过线缆连通即可。这样一来，即使信息需要经过其他据点中转，也能实现整个王国的线缆通信。

然而，铺设线缆需要耗费大量资源。不同据点之间铺设线缆所需的成本各不相同。指挥官们希望找到一种方案：既能保证所有据点都连通，又能使铺设线缆的总成本尽可能低。应该如何设计这样的方案呢？这其实就是**最小生成树**的问题：

给定一个连通的无向带权图 $G = (V, E)$，如果从图中选择若干条边，使得所有顶点连通且不存在环，那么图的所有顶点及这些边构成的子图就是原图的一棵**生成树**。在所有生成树中，边权总和最小的生成树称为**最小生成树（Minimum Spanning Tree, MST）**。

对此问题，格佛莱指出：初始时随意选择一个顶点 s，然后选择依附于 s 的边权最小的一条边，将该边与相连顶点“收进来”，此时，当前的生成树中就已经有了 2 个顶点。然后，继续选择能够连通这 2 个顶点与其他顶点的一条边权最小的边，并将该边与相连的顶点“收进来”，生成树中就有了 3 个顶点。重复这个操作，直至所有顶点都被“收进来”。这样做，得到的生成树中的所有边，一定是使得顶点连通的所有边的选集中权值之和最小的 $N-1$ 条边。如图 5.16 所示，使得三个顶点连通，可选的边集有 $\{e_0, e_1\}$、$\{e_0, e_2\}$ 以及 $\{e_1, e_2\}$，权值之和最小的一定是 $\{e_0, e_1\}$。

因此得到的树一定就是最小生成树。这就是 **Prim（普里姆）算法**的核心思想。

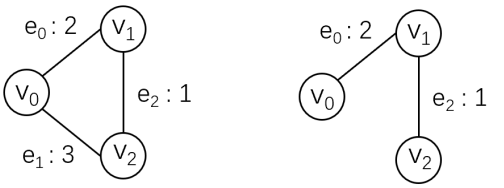


图 5.16

可以看出，对于 Prim 算法，我们需要动态维护当前生成树以及尚未收入的所有顶点到当前生成树的最小代价。详细实现如下。

5.4.1 Prim（普里姆）算法

设无向连通图 $G=(V,E)$，Prim 算法的过程如下：设置集合 S 保存已经收入当前的生成树中的顶点，cur_mind[] 保存各未收入顶点到集合 S 的最小代价。初始时随意收入一个顶点 s（即 S={s}），令与 s 邻接的所有顶点的 cur_mind[] 为相应边权，令其他所有顶点的 cur_mind[] 为正无穷。然后执行 $N-1$ 次以下两个步骤：

1. 从集合 V-S（还未收入当前生成树的顶点集）中选择距离集合 S 代价最小的一个顶点（即 cur_mind[] 最小）u，将 u 加入 S，并将 u 以及这条边加入当前生成树中。

2. 对于 u 的每个未被收入 S 的邻接点 v，执行

$$cur_mind[v]=min(cur_mind[v],cost(u,v))$$

即如果通过 u 到达 v 的代价比原来记录的更小，则更新 cur_mind[v]。

可以看到这个过程与迪杰斯特拉算法非常像，迪杰斯特拉算法是每次选择与源点 s最近的顶点，而 Prim 算法是选择与已有生成树 S最近的顶点。注意：Prim 算法中，cur_mind[v] 记录的并不是顶点 v 到源点 s 的最短距离，而是顶点 v 到集合 S 的最短距离，即 v 到 S 中的所有顶点的距离中最小的那个。

我们以图 5.17 a）所示的图，演示一遍 Prim 算法的过程：

1）任选一个点，假设选 A，收入最小生成树，更新 A 的邻接点 B 和 C 的 cur_mind[]，如 b）所示；此时集合 S 为 {A}。图中带下划线的数字表示还未收入生成树的各个顶点距集合 S 的最小代价（最短距离）；图中虚线表示该顶点最近是被哪条边所更新的，收入该顶点时沿着虚线就能找到边。

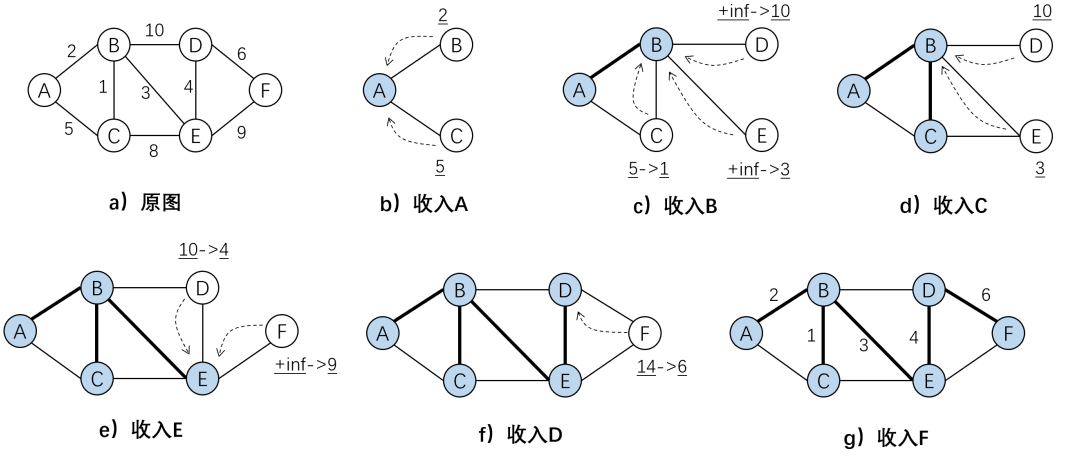


图 5.17 Prim 算法过程演示

2）未收入的顶点中，到集合 S 代价最小（最近）的顶点是 B，于是收入 B，如 c）所示，收入 B 后，由于 $cur_mind[C]=5<cost(B,C)=1$，也就是说对于新的集合

S={A, B}，C 到 B 的距离更小了，于是更新 C 到集合 S 的距离为 1。类似地，更新 cur_mind[E] 为 3、cur_mind[D] 为 10。

3）未收入的顶点中，到集合 S 代价最小（最近）的顶点是 C，于是收入 C，如 d）所示，并且对于 C 的邻接点 E，由于 $cur_mind[E] = 3 < cost(C, E) = 8$，因此无需更新。

4）收入 E，然后更新 D、F，如 e）所示；

5）收入 D，然后更新 F，如 f）所示；

6）收入 F，所有顶点均已被收入生成树，此时得到的即是最小生成树，各个边的权值之和即是最小代价：$2 + 1 + 3 + 4 + 6 = 16$。

5.4.2 Kruskal（克鲁斯卡尔）算法

指挥官麦丽凯给出了一种完全不同的思路：她不喜欢一步一步细致、缜密地规划，而是将注意力放眼全局。麦丽凯指出：初始时，没有任何顶点和边被收入生成树，将图看成是 N 个割裂的顶点构成了 N 个连通块。首先挑选整个图中边权最小的边，将它以及它连通的两个顶点加入生成树，连通块的数量便还剩 $N - 1$ 个。此后，每次选择剩余边中权值最小的，若它所连接的两个顶点处在不同连通块，则将他们连通并收入生成树，直至所有顶点均已连通。

该策略显然是正确的。因为如果两个连通块之间存在一条权值最小的边，那么最优解是一定要包含这条边的。这种放眼全局，通过不断地选择边权最小的边将不同连通块合并的策略，这就是 **Kruskal（克鲁斯卡尔）算法**的核心思想。

具体来说，设无向连通图 $G = (V, E)$ 有 N 个顶点，Kruskal 算法的过程如下：将图看成是 N 个割裂的顶点构成了 N 个连通块。然后反复按边权从小到大检查所有还未被收入生成树的边，若被检查的边所连接的两个顶点不在同一个连通块中，则将他们连通，并收入生成树，否则跳过，继续检查下一条边。如此重复，直至所有顶点均已连通；

我们以 5.18 a）所示的图演示一遍 Kruskal 算法的过程。

1）所有边中权值最小的是连接 BC 的边（为 1），将其收入，如 b）所示，此时 B、C 已被连通；

2）此时剩余所有边中，边权最小的是连接 A、B 的边（为 2），由于 A、B 此时不在相同的连通块，因此将他们收入生成树，此时 A、B、C 已被连通，如 c）所示；

3）此时剩余所有边中，边权最小的是连接 B、E 的边（为 3），由于 B、E 此时不在相同的连通块，因此将他们收入生成树，此时 A、B、C、E 已被连通，如 d）所示；

4）将 D 收入生成树，此时 A、B、C、E、D 已被连通，如 e）所示；

5）剩余所有边中，边权最小的是连接 A、C 的边（为 5），但 A、C 已经处于同一个连通块，因此跳过，继续检查下一条边；

6）剩余所有边中，边权最小的是连接 D、F 的边（为 6），并且 D、F 处在不同

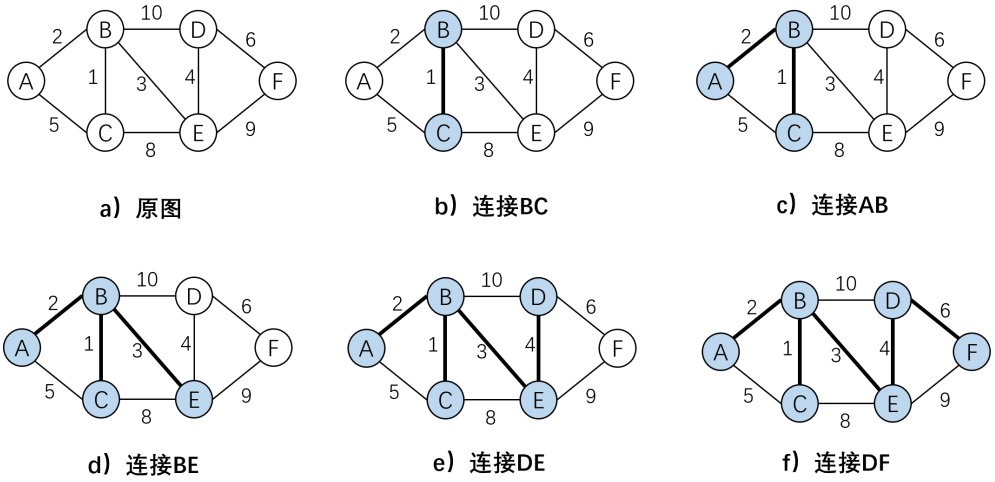


图 5.18 Kruskal 算法过程演示

的连通块，于是将它们收入生成树，如 f）所示，此时所有顶点均已被连通，算法结束。可以看到，在这个图的例子中，Kruskal 算法得到了与 Prim 算法相同的最小生成树。

C++ 重点内容整理

本书中出现了许多 C++ 中的最为重点的内容，为方便查阅，将其整理于此。由于本书重点是《数据结构》，因此此处出现的 C++ 内容只是入门级别，不会为大家带来理解上的难度。

vector

vector 是 C++ 标准库为我们提供的类模板。可以将 vector 理解成保存同类型元素的一个不定长数组。使用 vector 需要包含相应的头文件及引入命名空间：

```
#include <vector>
using namespace std;    // vector、stack 等定义在 std 命名空间（参考195页相关内容）
```

我们可以通过在尖括号中指定元素的类型，来定义不同类型的数组，如：

```
vector<int> int_arr;         // 一个空的，容纳 int 型元素的不定长数组
vector<char> char_arr;       // 一个空的，容纳 char 型元素的不定长数组
vector<double> double_arr;   // 一个空的，容纳 double 型元素的不定长数组
vector<vector<int> > vectorint_arr; // 容纳元素类型为 vector<int> 的不定长数组
```

返回 vector 中元素的数量

可以通过 **v.size() 返回 v 中元素的数量**。

访问 vector 中的元素

vector 中的元素下标从 0 开始，支持下标访问运算符：“[]”。可以通过 **v[idx] 返回 v 中第 idx 个位置上元素的引用**。注意：

1. 由于返回的是引用，因此可以修改原数组中的元素，这一点和 C 语言原生数组类似；

2. 下标访问只能访问 v 中已有元素。任何非法的下标访问（idx < 0 或 idx >= v.size() ）都是不安全的，将导致**未定义行为（Undefined Behavior，UB）**。

向 vector 对象中添加元素

我们可以使用 vector 的成员函数 **push_back** 向其中添加元素，如：

```
vector<int> v;  // 空的 vector 对象，v
for(int i=0; i<100; i++) {
    v.push_back(i); // 依次把整数值放到 v 的尾部
}
// 循环结束后 v 有 100 个元素，值分别为 0 到 99
```

注意：不能用下标形式添加元素。如：

```
vector<int> v;
for(int i=0; i<100; i++) {
    v[i] = i;
}
```

这段代码是绝对错误的。因为 v 是一个空的 vector，根本不包含任何元素，所以不能通过下标去访问任何元素。正确的做法是使用 push_back。

引用

类似指针，**引用**也是一种复合类型，引用为对象起了另外一个名字，是某对象的一个别名。就像通过“ ”符号定义指针那样，可以通过“&”符号来定义引用，如：

```
int a = 5;
int* pa = &a;   // pa 是指向 a 的指针
int& ra = a;    // ra 是 a 的一个别名
cout << a << endl;  // 打印 a，输出：5
```

注意“&”在 C++ 中不仅有取地址的含义，在声明引用时，“&”组成了声明符的一部分。就像“ ”既可以表示取指针变量指向的值，又可以作为声明符声明指针一样。

由于引用类型的对象只是另外某对象的一个别名，因此引用在定义时必须初始化。如：

```
int& rb;    // 编译报错，引用必须被初始化
```

定义了一个引用之后，对其进行的所有操作都是在与之绑定的对象上进行的：

```
ra = 10;    // 把 10 赋值给 ra 所引用的对象，即赋值给 a
cout << a << endl;  // 输出：10
cout << ra << endl; // 输出：10
```

为引用赋值，实际上是把值赋给了与引用绑定的对象。获取引用的值，实际上是获取了与引用绑定的对象的值。甚至取引用对象的地址，取的都是引用对象所引用的那个对象的地址：

```
cout << (&ra == &a) << endl;    // 输出：1
```

这也就是为什么**函数传参时，作为引用传参能够修改原变量的值**，类似指针一样：

```
void ref(int& a) {
    a = 10;
}
void pointer(int* a) {
    *a = 20;
}
void value(int a) {
    a = 30;
}
int main() {
    int t = 5;

    ref(t);
    cout << t << endl;  // 输出：10

    pointer(&t);
    cout << t << endl;  // 输出：20

    value(t);
    cout << t << endl;  // 输出：20

    return 0;
}
```

stack

类似 vector，stack 同样是 C++ 标准库为我们提供的类模板，模板参数就是栈中存放的元素类型。使用 stack 需要 include 相应头文件及引入命名空间：

```cpp
#include <stack>
using namespace std;
```

通过指定模板参数，我们就可以定义存放不同类型元素的栈。如：

```cpp
stack<int> is;  // 存放 int 的栈
stack<int> ds;  // 存放 double 的栈
```

甚至：

```cpp
stack<vector<int> > vis;    // 存放不定长数组的栈
```

栈 s 的一些常用操作如下：

1） s.push(e); // 将值为 e 的元素入栈

2） int t = s.top(); // 返回 s 的栈顶元素的引用（但不弹出该元素），将其值拷贝给 t

3） s.pop(); // 弹出栈顶元素

4） s.empty(); // 若栈 s 为空，此处返回值为 true，否则返回 false。

5） s.size(); // 返回栈 s 中元素的数量

注意，使用 top() 操作和 pop() 操作时，一定要保证栈是非空的，否则将导致**未定义行为（Undefined Behavior，UB）**。

queue

类似 vector 和 stack，queue 同样是 C++ 标准库为我们提供的类模板，模板参数就是队列中存放的元素类型。使用 queue 需要 include 相应头文件及引入命名空间：

```cpp
#include <queue>
using namespace std;
```

通过指定模板参数，我们就可以定义存放不同类型元素的队列。如：

```cpp
queue<int> iq;  // 存放 int 的队列
queue<int> dq;  // 存放 double 的队列
```

甚至：

```
queue<vector<int> > viq;    // 存放不定长数组的队列
```

队列 q 的一些常用操作如下：

1） q.push(e); // 将值为 e 的元素入队

2） int t = q.front(); // 返回 q 的队首元素的引用（但不将该元素出队），将其值拷贝给 t

3） q.pop(); // 将队首元素出队

4） q.empty(); // 若队列 q 为空，此处返回值为 true，否则返回 false。

5） q.size(); // 返回队列 q 中元素的数量

注意，使用 front() 操作和 pop() 操作时，一定要保证队列是非空的，否则将导致**未定义行为（Undefined Behavior，UB）**。

C++ 的 Hello, World

一个入门 C 语言的源文件，假设叫他“helloworld.c”，大概长这样：

```c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

而 C++ 源文件的名字通常以 cpp 为后缀，和 C 语言很相似，一个“helloworld.cpp”大概长这样：

```cpp
#include <iostream>

using namespace std;
int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

我们逐一将上面的代码拆解。

iostream 与 cin、cout

与 C 类似，

```
#include <iostream>
```

是引入 iostream 这个头文件。iostream 是 C++ 标准库提供的标准输入输出流，其中有两个最为常用的功能：cin 和 cout。

正如它们的名字 in 和 out 那样，cin 用于从标准输入流读（in）数据，而 cout 用于将数据输出（out）到标准输出流。来看一个简单的例子：

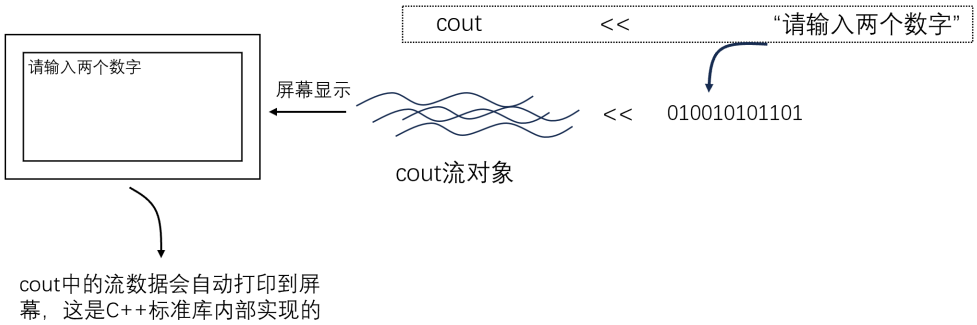
```
int a, b;
cout << " 请输入两个数字：";
cin >> a >> b;
cout << " 它们的和是：" << a + b << endl;
```

这个程序运行效果如下图所示：

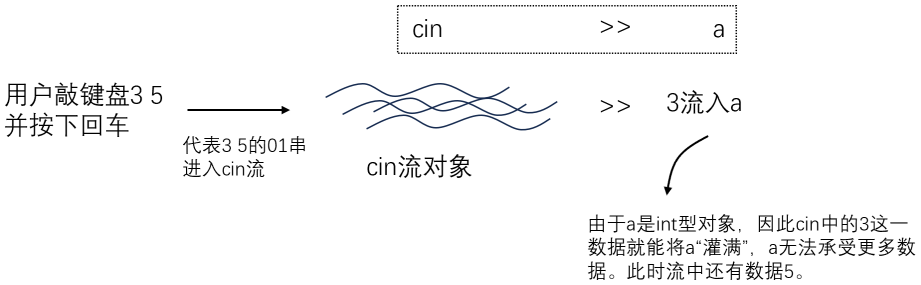


程序运行后，输出图 1，然后等待用户输入，用户输入两数并按下回车后，输出图 3 所示的结果。可以将“endl”简单理解为一个换行符'\n'。

在 C++ 中，cin 和 cout 是“流对象”，符号“«”和“»”生动地表达了数据的流向。cout 的原理（简单理解）如下图所示：



类似地，cin 的原理（简单理解）如下：



所以使用 cin 可以连续接收多个数据，cout 类似。

进一步理解，事实上“«”与“»”都是二元运算符，其左侧操作对象是 cout 或 cin，右侧对象是要读入或输出的数据，而该运算的返回值，事实上就是操作后的相应流对象（cin 或 cout）。比如 cin » a » b; cin » a 执行完成后，返回值即是新的 cin 对象（此时流中还剩” 5” 的数据），于是 cin » a » b 相当于先将 3 读入 a，之后再执行 cin » b，只不过此时 cin 流中的数据只有” 5” 了，所以将 5 读入 b。cout 也有类似的原理。

同时，printf 需要使用%c, %d 等控制输出类型，与之不同，cout 可以自动识别不同类型的数据。如：

```
int a = 5;
double b = 3.1415;
char c = 'a';
string d = "hello, world";

cout << a << " " << b << " " << c << endl;   // 输出: "5 3.1415 a\n"
cout << d << endl;  // 输出: "hello, world\n"
```

namespace—命名空间

接下来来看这段代码：

```
using namespace std;
```

这行代码的作用是：使用标准库命名空间 std。这样我们在使用 C++ 标准库中的名称时，就可以省略前面的 std::。例如，在前面的代码中：

```
cout << " 请输入两个数字: ";
cin >> a >> b;
```

实际上 cin 和 cout 都属于 std 这一**命名空间**。如果没有写 using namespace std;，我们就需要这样写：

```
std::cout << " 请输入两个数字：";
std::cin >> a >> b;
```

using namespace std; 并不是 C++ 程序必须写的一行代码。即使不写它，只要使用 std::cout、std::endl 这样的完整名称，程序依然可以正常运行。

事实上，vector、stack、queue 以及绝大多数标准库的重要工具都是定义在 std 命名空间的。如：std::stack<int>, std::queue<int> 等。从工程实践来看，using namespace std; 可能引入名称冲突，因此通常不建议在大型项目的头文件中使用。但对于以学习为目的的编程者，使用它可以减少代码中的额外符号，让读者更容易把注意力放在程序本身。

顺带说一个很有意思的事：事实上任何合法的 C 代码，一定是合法的 C++ 代码，并且 C 风格的字符串 char* 是可以直接打印到 cout 的。比如：

```
#include <iostream>
#include <stdio.h>
using namespace std;

int main() {
    printf("hello, ");
    char tmp[10] = "world!";
    cout << tmp << endl;
}
```

输出：

```
hello, world!
```

习题参考答案

习题 1 参考答案

1. D。线性表要求元素存在位置上的逻辑关系，A 错误；线性表是一种逻辑结构，通常可以用数组或链表实现，B 错误；线性表要求同一个表中的数据类型相同，C 错误；实际需求中我们要处理 int 型的数据，就会使用一个 int 型的线性表，要处理 Role 类型的数据，就会使用 Role 类型的线性表，D 正确。

2. C。A、B 选项都需要移动若干元素，时间复杂度为 O(n)；D 项需要从表头开始，将每一个元素的值与 x 对比直至找到，时间复杂度为 O(n)。C 项，由于每个元素的地址可以由其位序直接计算得来，因此时间复杂度为 O(1)。

3. C。

4. B。由于链表结构中保存有表首元素指针，因此在表头插入元素时间复杂度一定是 O(1)。对于 C 选项，要在单链表的表尾插入元素，需要先遍历链表，拿到尾节点的指针，再操作链条，时间复杂度为 O(n)。我们将在后面看到，为了解决单链表无法高效在尾部插入元素的问题，可以通过单链表的多种变态解决，如：记录尾指针的循环单链表、循环双链表等。

5. 事实上，这种存储方式下，还需要一个字段 capacity 标记当前已动态分配的空间总量：

```
struct SeqList {
    ElemType* data;     // 数组起始地址
    int current_size;  // 当前表中的元素数量
    int capacity;      // 当前已分配的数组容量
};
```

例如：capacity = 100，current_size = 80 表示分配了 100 个位置的内存空间，目前使用了 80 个，还剩 20 个。操作方面，显然，初始化动态分配版本的顺序表需要为 data 分配一个初始空间，初始分配多大你可以自己定义。

而对于插入操作，回忆静态存储的顺序表，若表满，只能拒绝插入（当然你可以规定淘汰算法，接受新值淘汰旧值，这取决于你，但静态存储已经无法存储更多了）。而动态存储版本，当已分配空间满了的时候，我们可以申请一块更大的内存空间，将已有表中数据整体拷贝到新空间，并接受新元素。C++ 中的 std::vector 就采用了类似机制实现。

6. 拟采用带头节点的实现。代码如下：

1）要构建 $a_1 \to a_2 \to \ldots \to a_{10}$，不难想出，只需要按顺序将每一个元素依次链接

到链表尾部就可以，代码如下：

```
LinkList f1(int a[], int n = 10) {
    // 创建头结点
    LinkList head = (int*)malloc(sizeof(ListNode));
    head->next_node = NULL;
    // 动态维护 tail，让其始终指向当前链表最后一个节点
    LinkList tail = head;
    for (int i = 0; i < n; i++) {
        LinkList node = (int*)malloc(sizeof(ListNode));   // 为新数据分配一个
        ↪  节点
        node->data = a[i];
        node->next_node = NULL;
        // 新节点插入在最后一个节点之后（尾插）
        tail->next_node = node;
        tail = node;    // 动态维护 tail 指针，让 tail 始终指向最后一个节点
    }
    return head;
}
```

2）要构建 $a_{10} \to a_9 \to \ldots \to a_1$，实际也很简单，只需要按顺序将每一个元素依次插入链表头部即可，代码如下：

```
LinkList f2(int a[], int n = 10) {
    // 创建头结点
    LinkList head = new ListNode;
    head->next_node = NULL;
    for (int i = 0; i < n; i++) {
        LinkList node = new ListNode;
        node->data = a[i];
        // 新节点插入在头结点之后（头插）
        node->next_node = head->next_node;
        head->next_node = node;
    }
    return head;
}
```

事实上，函数 f1 就是许多经典教材中**“尾插法”**的实现，函数 f2 就是**“头插法”**的实现。

7. 思路：遍历一遍链表，“检查”每一个节点的值，若等于 e，将其删除，并重构该节点附近链条。代码如下。

```c
void DeleteValue(LinkList L, ElemType e) {
    ListNode* pre = L;          // 当前“检查”的节点的前驱节点
    ListNode* cur = L->next_node; // 当前“检查”的节点
    while (cur != NULL) {
        if (cur->data == e) {
            ListNode* temp = cur;
            // 重构链条
            pre->next_node = cur->next_node;
            // 推移 cur 指针，准备检查下一个节点
            cur = cur->next_node;
            // 释放删除节点
            free(temp);
        }
        else {
            // 当前节点不是目标节点
            // 前驱和当前节点同时后移，继续检查下一个节点
            pre = cur;
            cur = cur->next_node;
        }
    }
}
```

8. 相信大家很自然地就能想到的一种做法是：先遍历一遍链表，计算出表长 n，再从头开始查找 $\frac{n+1}{2}$ 位置处的节点将其返回。代码如下：

```c
NODE* FindMiddle1(NODE *L) {
    int n = 0;
    // 第一次遍历，统计节点数量
    NODE *p = L->next;  // 带头结点，L->next 是第一个数据节点
    while(p != NULL) {
        n++;
        p = p->next;
    }
    // 空表情况
    if(n == 0)
        return NULL;
    // 第二次遍历，寻找第 (n+1)/2 个节点
    int pos = (n + 1) / 2;
    p = L->next;    // p 指针重置为第一个数据节点，从这里开始找
    for(int i = 1; i < pos; i++) {
        p = p->next;
    }
    return p;
}
```

事实上，还有一种更高效的算法，就是所谓的经典的**“双指针算法”**。我们设置两

个指针，slow 指针每次走一步，fast 指针每次走两步，这样，slow“走过的里程”永远是 fast 的一半，所以，当 fast 走到表尾时，slow 就处在中间。需要注意的就是边界判断及奇偶的处理。代码如下：

```
NODE* FindMiddle2(NODE *L) {
    NODE *slow = L->next;
    NODE *fast = L->next;
    if(slow == NULL)
        return NULL;
    while(fast->next != NULL && fast->next->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

9. 本题思路很简单。对于带头节点的单链表，我们依次将（本来就在头节点后的 a_1、）a_2、a_3、...、a_n 插入到第一个数据节点之前（即头节点之后）就行。我们动态维护指针 p，让其指向当前“待插”节点。代码如下。

```
NODE* Reverse(NODE* L) {
    NODE* p = L->next;  // L 是头节点，L->next 是第一个数据节点
    L->next = NULL;     // 插入过程中，让链表的“已定形部分”的尾节点指向 NULL
    while(p) {
        // next 为下一个待插节点
        NODE* next = p->next;
        // 插入当前节点
        p->next = L->next;
        L->next = p;
        // 动态维护 p，使其指向下一个待插节点
        p = next;
    }
    return L;
}
```

这个过程其实也可以理解为按顺序将当前链表节点用头插法（见习题 1 第 6 小题）“重新插入该链表”。

对于不带头结点的单链表同样是将每一个节点依次插入到第一个数据节点之前：

```
NODE* Reverse(NODE* L) {
    NODE *new_head = NULL;
    NODE *p = L;
    while(p != NULL) {
```

```
        NODE *next = p->next;
        p->next = new_head;
        new_head = p;
        p = next;
    }
    return new_head;
}
```

10. 经过观察分析不难看出，题目要我们将“倒数第 i 个元素”插入到“第 i 个元素”后面，换句话说，如果我们将链表从正中间“砍成两半”，题目要求的正是我们将链表的“后半部分”从后往前依次插入“前半部分”中。大家可能会想，后半部分如果有反向链条就好了对吧？就可以依次一个一个插入了。那没有，怎么办呢？相信聪明的你已经想到了，在第上一个习题中，我们介绍了链表的倒置算法。我们将后半部分链表倒置一下不就可以了？没错！接下来我们就给出本题的完整思路：

我们先用双指针算法（见习题 1 第 8 小题）找到链表的中点，将链表砍成两半。然后将后半部分链表逆置，再依次插入前半部分的对应位置中。

若元素个数为奇数个，那“中点”应该分给第一部分还是第二部分呢？其实经过简单的思考不难发现：其实给哪部分都行，我们是可以通过代码控制的。

完整的代码实现如下：

```
void Rearrange(NODE *head) {
    if(head == NULL || head->next == NULL)
        return;

    // 双指针寻找链表中点，slow 最终指向第一部分的最后一个节点
    NODE *slow = head;
    NODE *fast = head;
    while(fast->next != NULL &&
          fast->next->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    /* 将链表“砍成两半”
     * 第一部分：
     * head->next ... slow
     * 第二部分：
     * second-> ...
     */
    NODE *second = slow->next;  // second 即第二部分的“第一个节点”。注意：第二部分事实上是一个“不带头结点”的单链表，其第一个节点就是 second 指向的数据节点。
    slow->next = NULL;
    /*
```

```
     * 第二步:
     * 逆置第二部分链表
     */
    NODE *prev = NULL;
    NODE *cur = second;
    while(cur != NULL) {
        NODE *next = cur->next;
        cur->next = prev;
        prev = cur;
        cur = next;
    }
    second = prev;
    /*
     * 第三步:
     * 交替合并两个链表
     */
    NODE *first = head->next;
    NODE *p = first;    // 动态维护 p，让其始终指向第一部分的插入位置
    NODE *q = second;   // 动态维护 q，让其始终指向第二部分的待插节点
    while(p != NULL && q != NULL) {
        NODE *pNext = p->next;
        NODE *qNext = q->next;
        // 插入 q 节点
        p->next = q;
        q->next = pNext;
        // 推移 p 和 q 两个指针，准备插入下一个节点
        p = pNext;
        q = qNext;
    }
}
```

寻找中点、逆置第二部分链表和交替合并的过程时间复杂度均为 O(n)，因此整个算法时间复杂度为 O(n)。

11. 本题意在帮助大家学会 C++ 标准库中最重要的容器（几乎可以说没有之一）—vector 的基本使用方法。

我们定义一个 vector<int> ret; 遍历一遍原数组 arr，若 arr[i] > k，则将其 push_back 到 ret 中，最终返回 ret 即可。

```
vector<int> FilterGreaterThanK(vector<int> arr, int k) {
    vector<int> ret;
    for(int i=0; i<arr.size(); i++) {
        if(arr[i] > k) {
            ret.push_back(arr[i]);
```

```
        }
    }
    return ret;
}
```

注意，向 vector 中加入元素 e，一定要用 arr.push_back(e)，而不能是 arr[arr.size()] = e。arr[arr.size()] 这种代码实际上属于越界访问，会导致**未定义行为**。关于 vector 的更多内容感兴趣请参考本书末，C++ 重点内容整理（第 189 页）。

习题 2 参考答案

1. 选 ABD。

2. 选 C。栈的插入删除只能在栈顶进行，C 错误。

3. 选 D。对于 C 项，队列仅要求队首元素可以出队，但没要求仅队首元素可以被读取。你当然可以为队列实现获取第 i 个元素的函数接口，但通常来说我们使用队列的场景往往不会有这个需求。额外提示：你当然可以为你的“队列”实现第 i 个元素“出队”的函数接口，但这样做的话，你所实现的数据结构就不能叫做**队列**了。

4. 选 D。2134 可由入入出出入出入出达成；2341 可由入入出入出入出出达成；3241 可由入入入出出入出出达成；栈满足后进先出，如论如何 4 一定比 3 先出，2 一定比 1 先出，D 无法达成。

5. 选 B。对于 A，顺序栈的 Push 和 Pop 操作均在顺序表的表尾实现，时间均可达到 O(1)；对于 C，仅有 head 指针的单链表实现“头删”，即出队，可达到 O(1)，但要实现“尾删”，即出队，则需要先找到最后一个节点，时间为 O(n)。单链表中额外记录一个尾指针 rear，则入队和出队均可达成 O(1) 时间。对于 D，循环队列的入队和出队无需移动元素。

6. 选 D。通过本题给了我们一个提示，对于循环队列来说，初始 front 和 rear 不一定非要指向 0，因为从环的几何结构来讲，“队首元素”保存在 0 位置和保存在 i 位置是没有区别的。仅有初始状态（队空）会满足 front == rear。

7. 选 A、C、D、G。对于 B，无论栈中有多少个元素，一次出栈操作只需要访问当前栈顶元素，然后修改栈顶指针即可，时间复杂度为 O(1)。
对于 C，可由两栈的栈顶下标直接分别计算得到两栈的元素数量，时间均为 O(1)，再相加即可得到总数量，总体时间为 O(1)，C 正确；
对于 D，要注意：当栈 S2 中没有元素，而栈 S1 将空间全部占满时，同样可以用 S1.top+1==S2.top 判断空间满，当 S2 将空间占满时同样如此，D 正确；
E、F 显然错误；
对于 G，当栈满时，S1.top 指向 S1 栈顶元素的下一个位置，即 S2 的栈顶元素位置；对应地，S2.top 指向 S1 的栈顶元素位置，S1.top>S2.top 成立，G 正确。

8. 选 C。队列的特性决定了第 i 个入队的元素一定是第 i 个出队的，而栈则不同，栈的出栈顺序是会受到操作过程影响的，参考本习题第 4 题。

9. 选 AC。

10. 选 BCD。

11. 选 A。对于循环双链表，可以在 O(1) 时间立即拿到尾节点指针，因此在头、尾操作均为 O(1)。

12. 选 C。A 项可以通过①①②②①①③③得到；B 项可通过②①①①①①③得到；D 项输入与输出刚好相反，显然通过栈就能逆序：②②②②②②③③③③③③。对于 C，3 在 1、2 前输出，因此 1、2 必按顺序先入栈，所以 1 必晚于 2 输出，C 错误。

13. 选 C。这是一个非常有意思的题。不妨假设仅有左边能出队。无论如何 a 一定先入队，摆在最中央的位置。接下来，每一个元素入队，你都可以选择将其“摆在队中已有元素的左边或右边”。所以你需要以 a 为中心努力地去构造四个选项所给出的序列。如 A 选项要求的序列是 b,a,c,d,e，于是你将 b 摆在 a 左边，将 c 摆在 ba 的右边，将 d 摆在 bac 的右边，再将 e 摆在 bacd 的右边，便得到了 A 项所要求的序列，从左边出队，便得到了 b,a,c,d,e。B、D 项也都可以通过同样的方法得到。

14. 选 D。

15. 选 B。详细过程已经在 3.3.1 中介绍过，不再赘述。

16. 选 B。本题比较简单，注意出栈顺序别搞错，同时注意二元运算符 op 的操作数顺序是 b op a 而不是 a op b，不要搞错就行。

17. 本题意在帮助大家学会 C++ 标准库中提供的栈—stack 的基本使用方法。注意 top() 仅返回栈顶元素而不将其弹出即可。

1） s.push(3);

2） s.size(); // 1；此时栈中元素：3

3） s.push(5);

4） s.top(); // 5；栈中元素：3，5

5） s.top(); // 5；栈中：3，5

6） s.size(); // 2；栈中：3，5

7） s.push(6);// 栈中：3，5，6

8） s.pop(); // 栈中：3，5

9） s.pop(); // 栈中：3

10） s.size(); // 1；栈中：3

11） s.top(); // 3；栈中：3

18. 本题意在帮助大家练习 C++ 标准库中提供的栈—stack 以及队列—queue 的基本使用方法。使用它们需要 include 相应头文件。iostream 是 C++ 的标准输入输出流，可以简单理解为 stdio.h。cout 是 iostream 中提供的向标准输出打印的函数，可以简单理解为 printf。

```cpp
#include <iostream>
#include <stack>
#include <queue>
using namespace std;

int arr[5] = {1,2,3,4,5};

int main() {
    stack<int> s;
    queue<int> q;
    for(int i=0; i<5; i++) {
        s.push(arr[i]);
    }
    while(!s.empty()) {
        cout << s.top();    // 打印栈顶元素
        q.push(s.top());    // 栈顶元素入队
        s.pop();    // 弹出栈顶元素
    }
    // 打印一个换行符，相当于'\n'
    cout << endl;
    while(!q.empty()) {
        cout << q.front();  // 打印队首元素
        q.pop();    // 队首元素出队
    }

    return 0;
}
```

关于 iostream, using namespace std; 这些，大概是入门 C++ 的“Hello, World”内容。请大家参考 C++ 的 Hello, World(第 193 页) 中的相关内容。

19. 这个问题如果你在思考的过程中自然地想到了使用某种“一端进，两端出”的

数据结构，那么你可以开一瓶香槟庆祝一下。如果你只想到了时间复杂度为 $O(n \times k)$ 的算法，你应该惩罚自己周末不许出去玩。

本题最容易想到的方法是：从前向后遍历一遍 delay[] 数组，检查每一个延迟数值 delay[i] 时，从 i 向前找，直到遇见下标 0 或 i-k+1 的元素，在这个过程中记录下一个最大值，作为 i 时刻的延迟最大值，即 ans[i]。该方法的时间复杂度为 $O(n \times k)$。当 k 较大时，复杂度可能接近 $O(n^2)$。

但是，以上方法与“天气预报问题”（见—waypoint-3—（第 53 页）处）的朴素解法有着类似的问题：当我们检查后面的元素时，前面的元素其实已经被检查过若干遍了。我们能不能在每个元素第一次被检查的时候，就想办法将它保存到某个结构 some_structure 中，直至该元素“不可能成为后面任何一个 i 的解”，再将它丢弃呢？

我们从前向后遍历，当检查下标为 i 的元素时：
1）显然，下标 <=i-k 的元素已经不再可能成为后面任何一个元素（包括 i）的解。所以需要将所有下标 <=i-k 的元素移出 some_structure。这正是“先进先出”。
2）当我们检查 i 时，由于 i 可以作为自己的解，所以 some_structure 中所有值小于 i 的元素均已不再可能成为后面任何一个元素的解，移出它们。经过分析不难发现，该特性决定了 some_structure 中元素单调不增。所以移出时应该将 delay[i] 与“后进来的”元素先比较。

所以这个结构应该是一个“尾端能进能出、首端能出”的一个结构。并且当检查元素 i 时，移出满足上述 1）、2）条件的元素后，队首元素就是该元素 i 的解。

如正文中介绍过，通常称这种结构为**双端队列**。此处我们是在解决问题（而不是封装某种数据结构给别人或自己复用），因此我们不兴师动众地封装一个“双端队列”的数据结构，并为他写出“首端出队”、“尾端入队、尾端出队”等操作，而是直接将操作展开，作为逻辑的一部分实现。完整代码如下所示。

```
void MaxDelay(int delay[], int n, int k, int ans[]) {
    // deque[] 保存候选元素的下标。
    // deque[front] 是队首，deque[rear - 1] 是队尾。
    int deque[n];
    int front = 0;
    int rear = 0;

    for (int i = 0; i < n; i++) {
        while (front < rear && deque[front] <= i - k) {
            // 动态维护队首元素（i 的解）
            front++;     // 注意入队元素一定不会超过 n，因此不用考虑循环
        }

```

```
        // 移除所有不可能再成为最大值的元素。
        while (front < rear &&
            delay[deque[rear - 1]] <= delay[i]) {
            rear--;
        }

        // 当前元素加入队尾。
        deque[rear] = i;
        rear++;

        // 队首就是当前窗口中的最大值。
        ans[i] = delay[deque[front]];
    }
}
```

习题 3 参考答案

1. 选 C。一个最快的方法就是判断节点是否存在环。根据树的定义，若存在环则节点之间的前驱后继关系一定不符合树的定义。笔者不建议读者在此处逻辑上浪费过多时间。

2. 选 C。根据度的定义知道 A、B、D 正确。二叉树中任何一个节点至多有两棵子树，C 错误。

3. 选 B。树的高度和树的深度一定相等，并且等于树的层次的数量。该图中的树有四层，因此树的高度与深度都是 4，A 正确；D 显然正确。

4. 是满二叉树的有C、F；不是满二叉树，但是是完全二叉树的有A、D、E。完全二叉树只能是在最底层、从右开始有连续的残缺。F 项倒数第二层存在残缺；B 项最底层的残缺并非从右开始并且连续，因此 F、B 都不是完全二叉树。

5. 如下表。

	度为 0 的节点数	度为 2 的节点数
图一	2	1
图二	3	2
图三	3	2
图四	4	3

事实上，**记二叉树中度为 2 的节点的数量为 n_2，度为 0 的节点（即叶节点）数为 n_0，则 $n_0 = n_2+1$ 对任何非空二叉树均成立**。对于数学上严谨的证明，感兴趣的同学可以参考相关资料，我们这里只给出直观上的理解。

相信你也已经看出来了，最初对于只有一个节点的二叉树，$n_0 = n_2+1$ 显然成立。此后，在二叉树中增加节点，只可能从度为 0 的节点“引出边”，或者从度为 1 的节点“引出边”。

1）从一个度为 0 的节点“引出一条边”，不会增加 n_2 的数量，并且 n_0 会被吃掉一个，也会新增一个，相当于没变。所以该操作不影响 n_0 和 n_2 的数量关系；

2）从一个度为 1 的节点“引出一条边”，则会将 n_0 和 n_2 的数量都增加一个。该操作同样不影响 n_0 和 n_2 的数量关系；

所以，只要二叉树中至少有一个节点，此后无论加入多少个节点，n_0 和 n_2 的关系都不会变化。$n_0 = n_2 + 1$ **对任何非空二叉树都成立。事实上这是二叉树中一个非常重要的性质**，很多教材喜欢将此直接作为“二叉树的性质”给出，但笔者认为，探索、发现规律的过程可以锻炼能力、培养创造力和解决问题的能力。并且这对知识的记忆会

是一个很强的冲击。如果直接记住公式，非常容易忘，并且很可能出现“n_0 是啥意思来着？是谁等于谁加一来着？”这样的现象。而有了这个过程，当你忘记的时候，拿出笔画几个圈，立即能回忆起来。

6. 由上一题的结论可知，非空二叉树中度为 2 的节点 $n_2 = 8$，则度为 0 的节点数一定是 $8 + 1 = 9$，即叶节点一定是 9 个。题目又说了度为 1 的节点有 3 个，所以二叉树中节点总数为 $n = n_0 + n_1 + n_2 = 9 + 3 + 8 = 20$。

7. 由已知信息可知，有 $n_0 + n_2 = 13$ 成立。又根据前面的结论有 $n_0 = n_2 + 1$ 成立，解二元一次方程，得 $n_0 = 7$。即叶节点有 7 个。

8. 选 A、B、D。对于此类问题，动手画一画图就非常容易得到答案。事实上本题中的几个选项同样可以作为二叉树的性质来记忆：
若完全二叉树有 n（n > 0）个节点，从上到下，从左到右分别编号 1 至 n，则：
a）若某节点编号为 i，且 2 * i > n，则节点 i 一定是叶子节点；
b）对完全二叉树来说，树中度为 1 的节点数要么是 0，要么是 1，绝不可能大于 1。
c）$\lfloor \frac{n}{2} \rfloor$ 一定是所有非叶子节点中，编号最大的那个

此处笔者提醒大家对性质 c）稍作留意。该选项是后面我们介绍堆时会用到的一个完全二叉树的性质。也可以利用该性质快速解决一些问题。

9. 各层及其最大节点数有如下规律：

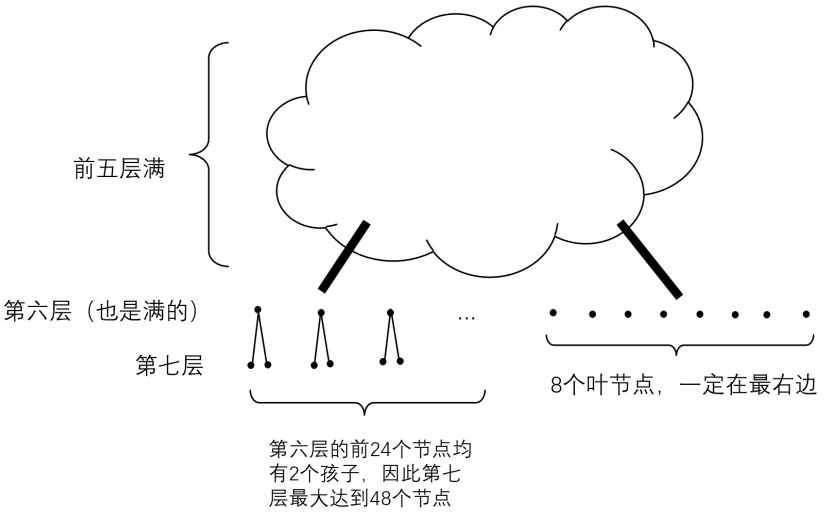
层数	最大节点数
1	1
2	2
3	4
4	8
5	16
⋮	⋮

由数学归纳法可以证明，第 i 层的最大节点数为 2^{i-1}。该结论可以作为二叉树的性质来记忆：**二叉树中第 i 层的最大节点数为 2^{i-1}。**

10. 随手画画图，就很容易发现结论：对于二叉树，其叶子节点可能出现在任何一层；**对于完全二叉树，叶子节点仅可能出现在最底层或者自下而上的第二层**；对于满二

叉树，叶子节点仅可能出现在最底层。完全二叉树叶子节点仅可能出现在最下两层这一性质会带来一些有趣的问题，我们已在后面的题目中给出。

11. 选 C。对于完全二叉树，第 6 层有叶节点，说明树要么共 6 层，要么共 7 层。显然树为 7 层时节点更多。因此要达到节点最多，树形一定如下图：



所以最大节点数就是，前六层是满的：$1+2+4+8+16+32=63$，再加上第七层的节点数 48，共 111。

我们求前六层的节点数时，也给了我们一个提示：**高为 h 的满二叉树，节点数是** $1+2+...+2^{h-1}=2^h-1$。换句话说，**高为 h 的二叉树，至多有** $1+2+...+2^{h-1}=2^h-1$ **个节点。**该性质同样可以作为二叉树的性质记忆，也可以根据每层最大节点数用等比数列求和公式现推。

12. 选 C。最直观（但复杂）的想法应该就是先算出二叉树有多少层、最后一层有多少节点，这样就能求出倒数第二层的叶节点的数量，再加上最后一层的节点数就是总共的叶节点数。

更简单的做法是，利用**本习题第 8 小题给出的性质 c）**：最后一个非叶节点的编号是 $\lfloor \frac{768}{2} \rfloor = 384$，即非叶节点有 384 个。所以叶节点数量为 768 - 384 = 384 个。

还有一种不太直观的解法，利用**本习题第 5 小题给出的性质）**：$n_0 = n_2 + 1$，有 $2n_0 + n_1 - 1 = 768$，又因为完全二叉树中 n_1 只能取 1 或 0，当 n_1 取 0 时，n_0 为小数，显然不行，所以 n_1 只能取 1，得到 $n_0 = 384$。

13. 选 A。要满足任意一棵高度为 5 且有 10 个节点的二叉树都能存得下，需要考

虑最坏情况，即存储单元消耗最大的情况。顺序存储通过从上到下，从左到右依次保存每一个位置的节点，高度为 5，则最坏情况是第五层最右边的节点被占用，需要 31 个存储空间。而 10 个节点能否达到此情况？容易得知是可以的。所以选 A，31。

14. 选 B。树的度为 4，说明树中只有度为 0、1、2、3、4 的节点，又已知了度为 1、2、3、4 的节点数量分别为 10、1、10、20，根据“树的所有节点度数之和等于树的边数”的性质，树的边数可以直接求得：

$$e = 10 * 1 + 1 * 2 + 10 * 3 + 20 * 4 = 122$$

于是树的节点数 $n = 122 + 1 = 123$。于是叶节点，即度为 0 的节点数为 $123 - 10 - 1 - 10 - 20 = 82$。

15. 选 C。虽然本题目提出的存储方式与 4.2.1 小节中提出的二叉树的顺序存储物理上都是使用一个数组存储，但逻辑上对于二叉树中节点关系的描述方式不同。A、B、D 显然不合逻辑。对于 C 项，数组的长度是确定的，若某场景下无法确定需要使用的二叉树是否有节点数上限，则不适合使用固定大小的数组存储，C 正确。

16. 第 1）小问很简单，将树遍历一遍就行。这里拟用先序遍历，代码如下：

```
void travel(node* root) {
    if(root == NULL) return ;
    if(root->data == 5) root->data = 10;
    travel(root->lchild);
    travel(root->rchild);
}
void five2ten(node* T) {
    travel(T);
}
```

事实上，第 2）问也不难。定义一个 int 变量，这里取名为 ret，标识是否有元素的值被修改过。递归时将 ret 的地址传入递归函数，在某节点处修改了值时，将该 ret 的值置为 1 就好。代码如下：

```
void travel(node* root, int* value_changed) {
    if(root == NULL) return ;
    if(root->data == 5) {
        root->data = 10;
        *value_changed = 1;
    }
    travel(root->lchild, value_changed);
```

```
    travel(root->rchild, value_changed);
}
int five2ten(node* T) {
    int ret = 0;    // 此时未修改任何值
    travel(T, &ret);
    return ret;
}
```

还是提示一下，value_changed 只是形参，它实际传入的实参是 five2ten() 中 ret 这个变量的地址。

17. 1）代码如下：

```
void PostOrderTravel(node* root) {
    if(root == NULL) return ;
    PostOrderTravel(root->lchild);
    PostOrderTravel(root->rchild);
    printf("%c", root->data);
}
void f(node* T) {
    PostOrderTravel(T);
    printf("\n");
}
```

2）后序遍历序列为 CoreLoop。

18.

1）通过观察不难发现，对于树中的任意一个节点，其左子树中所有节点的值均小于该节点的值，右子树中所有节点的值均大于该节点的值。

2）中序遍历，得到的序列是 1, 3, 4, 5, 6, 7, 9, 10, 11, 12；该序列是大小有序的。

3）插入节点时，可以从根节点开始，将根节点元素与待插元素比较。若待插元素较小，则说明该节点需要在左子树中插入，此时，若：

① 左子树为空，则插入位置已经找到，直接插入；

② 左子树非空，则递归地进入左子树，进行同样的操作。

反之若待插元素大于根节点，则递归地判断右子树。如此递归进行下去，直至找到正确的位置。具体实现代码如下：

```
void InsertTree(node* &root, int e) {
    if(root == NULL) {
        // 空树，直接插入就行
```

```
        node* new_node = (node*)malloc(sizeof(node));
        new_node->data = e;
        new_node->lchild = NULL;
        new_node->rchild = NULL;
        root = new_node;
        return ;
    }
    // e 已存在，无需插入
    if(e == root->data) {
        return ;
    }
    if(e < root->data) {   // 待插元素较小
        // 递归地进入左子树
        InsertTree(root->lchild, e);
    }
    else if(e > root->data) {    // 待插元素较大
        // 递归地进入右子树
        InsertTree(root->rchild, e);
    }
}
```

事实上，我们将在后面的内容中介绍到，有着这样特点的树叫做**二叉查找树**，或者叫**二叉搜索树**、**二叉排序树**。正如其名，这种数据结构可以利用节点值之间的大小关系进行较为高效的查找操作，我们将在后面的内容中详细介绍。

习题 4 参考答案

1. 答：能确定度为 0 的节点有 8 个。哈夫曼树一定是二叉树，因此根据 $n_0 = n_2+1$，度为 0 的节点有 $7 + 1 = 8$ 个。

2. 答：能确定哈夫曼树共有 15 个节点。由前面的讨论（参见哈夫曼树的性质，第 100 页）可知，哈夫曼树中不存在度为 1 的节点，因此总节点数 $n = n_0 + n_1 + n_2 = n_0 + n_2 = 7 + 8 = 15$。

3. 略。

4. 最小堆有：E、F、G；最大堆有：A、C、F。

5. 选 B。节点 B 的带权路径长度为 3*1 = 3，A 正确；树的带权路径长度是所有叶节点的带权路径长度之和：3*(3+1) + 2*(5+3+4) = 12 + 24 = 36，B 错误；C、D 显然正确。

6. 显然，本题我们只要关注堆顶元素（即最小的那一个）是什么即可：

1） min_heap.push(5); // 堆中元素：5；堆顶为 5

2） min_heap.top(); // 获取到的为 5

3） min_heap.push(3); // 堆中元素：5、3；堆顶为 3

4） min_heap.push(4); // 堆中元素：5、3、4，堆顶为 3

5） min_heap.top(); // 获取到的为 3

6） min_heap.pop(); // 删除堆顶元素 3，4 成为新的堆顶

7） min_heap.top(); // 获取到的为 4

7. 每次在堆中选出两个最小的来合并，计算相应体力消耗，并将合并后的新柴薪加入堆中。做 n-1 次即可。

```
int MinCost(int w[], int n) {
    priority_queue<int, vector<int>, greater<int>> min_heap;
    // 在下方完成你的代码
    int total_cost = 0;

    for(int i=0; i<n; i++) {
        min_heap.push(w[i]);
    }
```

```
    // 做 n-1 次
    for(int i=1; i<=n-1; i++) {
        // 取出两个重量最小的：w1 和 w2
        int w1 = min_heap.top();
        min_heap.pop();
        int w2 = min_heap.top();
        min_heap.pop();

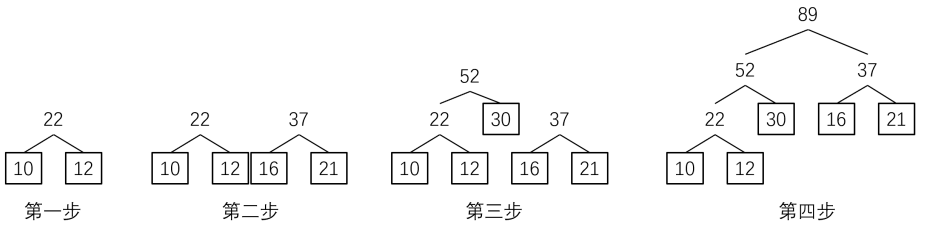
        // 体力消耗为 w1+w2
        int union_cost = w1 + w2;
        total_cost += union_cost;

        // 合成一堆后放回堆中
        min_heap.push(union_cost);
    }
    return total_cost;
}
```

8. 选 C。哈夫曼编码，被编码的符号一定都是叶节点，又：哈夫曼树中没有只有叶节点和度为 2 的节点，因此有: $n_0 + n_2 = n_2 + 1 + n_2 = 115$，得 $n_2 = 57, n_0 = 58$。

另外一种理解，生成哈夫曼树，需要合并 n-1 次，因此生成的度为 2 的节点为 n-1 个，代入计算也可。

9. 选 B。可以构造如下图所示的哈夫曼树。求出带权路径长度 WPL=3*(10+12) + 2*(30+16+12) = 200。



10. 本题可以记录一个变量 total，递归地访问每一个节点，将所有访问到的叶节点的带权路径长度累加记录，最终 total 的值就是树的带权路径长度 WPL。这里我们给出另一种思路的实现代码：

不难发现，每棵子树 T 的 WPL 等于其左子树与右子树的 WPL 之和。因此求 T 的 WPL，可以递归地转化为求 T->lchild 以及 T->rchild 的 WPL。递归边界就是左子树或右子树为空树。代码如下所示。

```
int WPL(node* root, int depth) {
    // 空树，直接返回 0
    if(root == NULL) return 0;
    // 当前递归到的，以 root 为根节点的子树，其左右子树都是空树，因此 root 的 WPL 可以直接计算。
    if(root->lchild == NULL && root->rchild == NULL) {
        return root->weight * (depth-1);    // 深度为 depth 的节点，根节点到其路径长度为 depth-1
    }
    // root 还有子树，需要递归地计算子树的 WPL 并加到一起，作为 root 的 WPL 返回
    int total = 0;
    if(root->lchild != NULL) {
        total += WPL(root->lchild, depth+1);
    }
    if(root->rchild != NULL) {
        total += WPL(root->rchild, depth+1);
    }

    return total;
}
```

调用 WPL(T, 1) 即可求出 T 的带权路径长度。注意：我们这里按照惯例，将根节点深度定义成了 1，这恰好能正确处理“只有一个根节点的树”的情况。

11. 实际上本题依然是通过叶节点的深度和权值计算出叶结点的“诱人值”，可以看成某种“特殊的带权路径长度”，只不过求 WPL 是累加，而本题是“记录最大”。我们可以通过一个引用变量（或指针）记录递归过程中的最大值，代码如下：

```
void visit(node* root, int depth, double& max_appeal) {
    if(root == NULL) return ;

    // 叶子节点，有宝藏的房间
    if(root->lchild == NULL && root->rchild == NULL) {
        double appeal = (1.0/dpeth) * root->weight;
        // 本房间的“诱人值”比之前遇到的所有房间都大，更新 max_appeal
        if(appeal > max_appeal) {
            max_appeal = appeal;
        }
    }
    if(root->lchild != NULL) {
        visit(root->lchild, depth+1, max_appeal);
    }
    if(root->rchild != NULL) {
        visit(root->rchild, depth+1, max_appeal);
```

```
    }

}

double CalcAppeal(node* T) {
    double ans = 0;
    // your codes...
    visit(T, 1, ans);
    return ans;
}
```

12. 本题的核心思想在于，对于以节点 u 为起点的逃生路线有：

$$MinCost(u) = weight(u) + \min(MinCost(left), MinCost(right))$$

left、right 分别表示 u 的左右子树。想通了这一点，题目就变得非常简单。请参见代码与注释。

```
int MinEscapeCost(node* root) {
    if(root == NULL) return 0;

    // 叶子节点，出口找到了
    if(root->lchild == NULL && root->rchild == NULL) {
        return root->weight;
    }
    // 左边是死路，只能从右边走
    if(root->lchild == NULL) {
        return root->weight + MinEscapeCost(root->rchild);
    }
    // 右边是死路
    if(root->rchild == NULL) {
        return root->weight + MinEscapeCost(root->lchild);
    }

    // 两边都能走，递归计算两边代价后，选代价小的那一条路
    int ltree_cost = MinEscapeCost(root->lchild);
    int rtree_cost = MinEscapeCost(root->rchild);

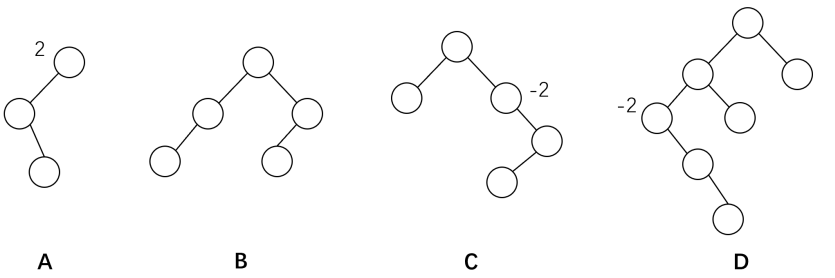
    return root->weight + min(ltree_cost, rtree_cost);
}
```

习题 5 参考答案

1. 正确。

2. 正确。

3. 选 B。A、C、D 均存在平衡因子的绝对值大于 1 的节点，如图所示。



4. 选 A。各选项查找过程如下图所示：

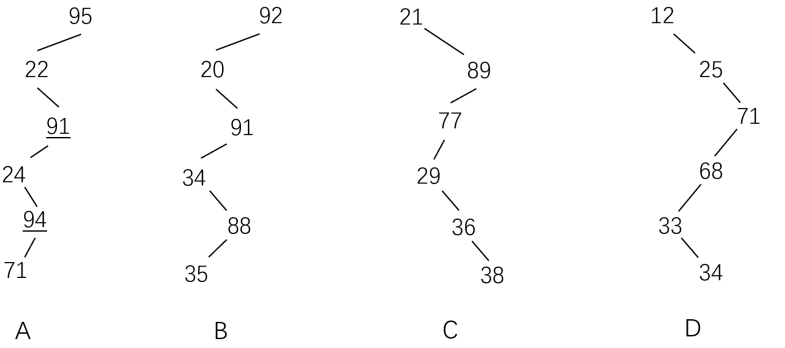
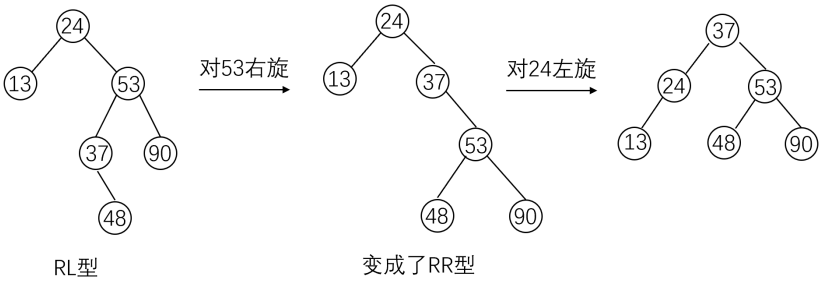
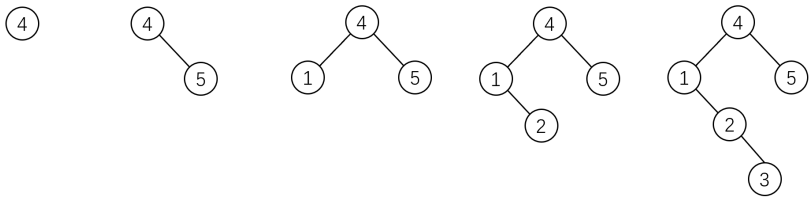


图 A，91 的左子树出现了 94，大于 91，因此不可能是二叉排序树的查找路径。

5. 选 C。插入 48 后，据新插入节点 48 最近的失衡节点为 24（根节点），平衡因子为 1-3=-2，为 RL 型失衡。先对 53 右旋，再对 24 左旋，调整后 37 的左、右孩子分别是 24、53，如下图所示。



6. 选 B。B 项插入序列生成的二叉排序树如图所示：



7. 我们从根节点开始，向左右子树寻找两个目标节点。如果发现左边找到了一个、右边也找到了一个，那么当前节点一定是它们的最近公共祖先。完整代码如下：

```
node* LowestCommonAncestor(node* root, int a, int b) {
    if( root == NULL ) return NULL;

    if( root->key == a || root->key == b ) return root;

    node* left = lowestCommonAncestor(root->left, a, b);
    node* right = lowestCommonAncestor(root->right, a, b);
    if( left != NULL && right != NULL ) return root;
    if( left ) return left;
    if( right ) return right;

    return NULL;

}
```

寻找公共祖先的代码是极难理解的。别急，我和大家一起慢慢将其拆解。建议大家结合下面的代码注释以及图中所示例子的查找过程共同来理解。首先我们从根节点开始，向左右子树寻找两个目标节点。如果发现左边找到了一个、右边也找到了一个，那么当前节点一定是它们的最近公共祖先：

```
node* left = LowestCommonAncestor(root->lchild, a, b);
node* right = LowestCommonAncestor(root->rchild, a, b);

if (left != NULL && right != NULL)
    return root;
```

如果沿途找到了某 NULL 节点，说明此条路径上没有有意义的信息，直接返回 NULL；

若找到了某待查关键字，或两个关键字的公共祖先之后，则将相应指针沿路径向上返回：

```
// 沿路径查找到了 NULL，说明该路径上不存在有用的信息。
    if( root == NULL ) return NULL;

    node* left = LowestCommonAncestor(root->lchild, a, b);
    node* right = LowestCommonAncestor(root->rchild, a, b);

// 若找到了某待查关键字，将该节点指针向上返回
    if( root->key == a || root->key == b ) return root;

// 这种情况成立，则一定是 left 和 right 分别是待查找的两个关键字的指针。此时 root 就是公共祖先，向上返回，一直传递上去。
    if (left != NULL && right != NULL)
        return root;

    // 此前（更下层的递归中）存在有用的信息（左子树中有待查关键字，或已经在左子树中找到了最近公共祖先），将该有用的信息向上返回，传递上去
    if(left) return left;
    // 此前（更下层的递归中）存在有用的信息（右子树中有待查关键字，或已经在右子树中找到了最近公共祖先），将该有用的信息向上返回，传递上去
    if(right) return right;

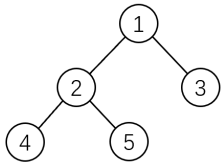
    // 若左右子树都不存在有用的信息，返回 NULL，表示这条路径上没有有用的信息。
    return NULL;
```

left、right 实际上有两层含义：

第一层：未找到公共祖先时，left 或 right 表示该节点左（右）子树中待查找的关键字的指针。（第一种有用的信息）

第二层：找到公共祖先后，left 及 right 的含义就是该节点的左（右）子树中是否存在要找的公共祖先节点，若存在，left 或 right 表示的就是公共祖先节点的指针，若不存在，left 或 right 为 NULL。（第二种有用的信息）

我们给出在下图所示的二叉树中，寻找 4 和 5 的公共祖先的过程。



第一步，当前节点 root 是 1，它既不是 4，也不是 5。因此，算法继续向下查找：
- 左子树：从节点 2 开始。
- 右子树：从节点 3 开始。

第二步：进入节点 2，节点 2 又会向左右子树查找：

- 左边进入节点 4。
- 右边进入节点 5。

当递归来到节点 4 时，发现它就是目标节点，于是直接返回节点 4。同理，来到节点 5 时，直接返回节点 5。于是，节点 2 这一层得到：

node* left = LowestCommonAncestor(root->lchild, 4, 5); // left 是节点 4 的指针
node* right = LowestCommonAncestor(root->rchild, 4, 5); // right 是节点 5 的指针

两边都不为空！说明两个目标节点分别出现在节点 2 的左右子树中。因此，返回节点 2。

第三步：返回节点 1，节点 2 的递归函数中返回了节点 2，意味着节点 1 的左侧查找结果是节点 2。而节点 3 的子树里没有 4 或 5，所以右侧返回 nullptr。因此，节点 1 得到：

node* left = LowestCommonAncestor(root->lchild, 4, 5); // left 是节点 2 的指针
node* right = LowestCommonAncestor(root->rchild, 4, 5); // right 是 NULL

因为只有左边有结果，节点 1 的递归函数栈中就把左边的结果（节点 2 的指针）继续返回。得到最终答案是节点 2。

8. 事实上有两点可以优化：
1）若 a 和 b 与某节点相比一大一小，则该节点一定就是最近公共祖先，而无需继续向下找 a 和 b；
2）a 和 b 都更大或都更小时，无需两棵子树都进行递归，拿到 left 和 right 的信息，而是直接进入 lchild 或 rchild 即可。

此外，由于查找结点一定存在，而在二叉查找树下由上面的 2），我们能确定每次递归进入的路径下方一定存在公共祖先，因此一定不会进入 NULL 节点。

```
node* LowestCommonAncestor(node* root, int a, int b) {
    // a 和 b 分别在 root 的左右子树中，root 就是最近公共祖先
    if(a < root->key && b > root->key) return root;
    if(a > root->key && b < root->key) return root;

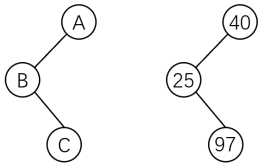
    // root 就是 a 或 b，root 一定是最近公共祖先
    if(root->key == a || root->key == b) return root;
    // a 和 b 都在左子树或都在右子树中，递归地进入左子树或右子树。
    if(a > root->key && b > root->key) return
    ↪  LowestCommonAncestor(root->right, a, b);
    return LowestCommonAncestor(root->left, a, b);
}
```

9. 中序遍历中，第 k 个访问的节点就是树中第 k 小的元素。因此我们可以在中序遍历过程中，用一个变量 visited_num 记录当前已经有多少个节点被访问过了。当访问到某节点使得 visited_num == k 时，返回该节点即可，代码如下。

```
void InOrderTravel(node* root, int k, int& visited_num, int& result) {
    if(root == NULL) return ;
    InOrderTravel(root->lchild, k, visited_num, result);

    // 代码执行到此处时，才是中序遍历中“访问”一个节点的时候
    visited_num += 1;
    if(visited_num == k) {
        result = root->val;
        return ;
    }
    // 若 visited_num 已经 >=k，说明找到了，无需继续递归遍历了。
    if(visited_num < k) {
        InOrderTravel(root->rchild, k, visited_num, result);
    }
}
int kthSmallest(node* root, int k) {
    int result = 0;
    int visited_num = 0;
    InOrderTravel(root, k, visited_num, result);
    return result;
}
```

10. 相信一定有人的思路是这样：遍历一遍所有节点 i，判断：其左孩子（即 2i）小于节点 i 的值，并且其右孩子（即 2i+1）大于节点 i 的值，若存在一个节点该条件不成立，则返回 false。否则返回 true。事实上，这种思路是错误的。如图，



以上思路能保证 B<A、B<C 成立，但不能保证 C<A 成立。正确的做法是：

【解法一】遍历一遍所有节点，不能只判断其左右孩子，而是要判断其左子树中的最大值是否满足小于该节点的值、右子树中的最小值是否满足大于该节点的值。只有所有节点都满足以上条件，才是一棵二叉搜索树。

我们维护两个数组：local_min[] 和 local_max[]，local_min[i] 和 local_max[i] 的值分别表示以节点 i 为根的子树中的最小值和最大值。从后向前遍历一遍所有节点，判断是否满足二叉搜索树条件的同时，更新这两个数组。具体过程请参见代码及注释。

```
bool IsBST(SqBiTree tree) {
    // local_max[i]、local_min[i] 分别表示以 i 为根的子树中元素的最大、最小值
    int local_max[MAX_SIZE];
    int local_min[MAX_SIZE];
    // 每个节点为根的子树，其中最大或最小值至少是它们自己
    for(int i=0; i<MAX_SIZE; ++i) {
        local_min[i] = local_max[i] = tree.SqBiTNode[i];
    }

    for(int i=tree.ElemNum; i>=1; i--) {
        int lchild_idx = 2*i;
        int rchild_idx = 2*i+1;

        // 节点 i 有左孩子
        if(lchild_idx <= tree.ElemNum && tree.SqBiTNode[lchild_idx] != -1) {
            if(tree.SqBiTNode[i] <= local_max[lchild_idx]) {    // 当前节点的值小于等于左子树中的最大值，不符合条件，返回 false
                return false;
            }
            // 以该节点 i 为根的子树中的最小值更新为 i 的左子树中的最小值。
            local_min[i] = local_min[lchild_idx];
        }

        // 节点 i 有右孩子
        if(rchild_idx <= tree.ElemNum && tree.SqBiTNode[rchild_idx] != -1) {
            if(tree.SqBiTNode[i] >= local_min[rchild_idx]) {    // 当前节点的值大于等于右子树中的最小值，不符合条件，返回 false
                return false;
            }
            // 以该节点 i 为根的子树中的最大值更新为 i 的右子树中的最大值。
            local_max[i] = local_max[rchild_idx];
        }
    } // end for()

    // 以上 for 循环结束，说明所有节点均满足二叉搜索树的条件，返回 true。
    return true;
}
```

【解法二】本题还有一种更简单的解法：根据二叉搜索树中序遍历一定是有序的，我们在中序遍历的过程中，记录当前访问的节点 i 的中序前驱的值 pre，若存在某个节点，其值小于等于 pre，则返回 false。

```cpp
void InOrderTravel(SqBiTree tree, int i, int& pre, bool& ans);

bool IsBST(SqBiTree tree) {
    // 先假设是二叉搜索树，若遍历过程发现不满足，将 ans 置为 false。
    bool ans = true;
    // 树中节点都是正整数，因此将第一个节点的前驱初始化为 0 或任意负数即可。
    int pre = -10;
    InOrderTravel(tree, 1, pre, ans);
    return ans;
}

void InOrderTravel(SqBiTree tree, int i, int& pre, bool& ans) {
    // 节点不存在，直接返回
    if(i > tree.ElemNum || tree.SqBiTNode[i] == -1) return;
    // 已经可以做否定断定，无需继续遍历
    if(ans == false) return ;

    // 先访问左子树
    InOrderTravel(tree, 2*i, pre, ans);

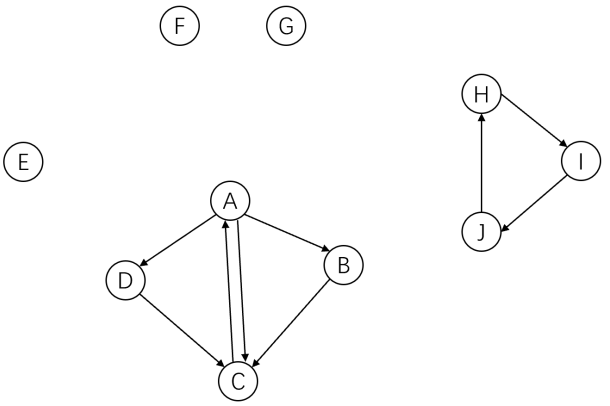
    // 若当前访问节点小于等于其前驱，说明不是二叉搜索树
    if(tree.SqBiTNode[i] <= pre) {
        ans = false;
        return;
    }
    // 将下一个待访问节点的中序前驱更新为当前节点
    pre = tree.SqBiTNode[i];

    // 访问右子树
    InOrderTravel(tree, 2*i+1, pre, ans);
    return;
}
```

习题 6 参考答案

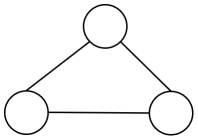
1. 选 B。顶点 A 与顶点 E 存在相互连通的路径: A -> D -> E 以及 E -> D -> A，选项 B 错误。

2. 选 D。A 项，存在两顶点 A、E，A 到 E 无路径，所以 G_1 不是强连通的；对于 G_2，F 到 G 无路径，因此 G_2 不是强连通的；G_3 任意两顶点之间都有路径，是强连通的。B 项，图 G 存在以下强连通分量：



显然数量大于 3。C 项，顶点 A 出度为 3，入度为 1，C 正确。D 项，顶点 F 出度为 0，入度为 1，D 错误。

3. 选 A。下图所示的图是一个无向连通图，但不具有 II 与 III 的特性。只有 I 正确。



4. 要使得边数达到最大，一定是任意两个顶点之间都有一条边。因此最大边数是从 N 个顶点选出 2 个的组合数：

$$C_N^2 = \frac{N(N-1)}{2}$$

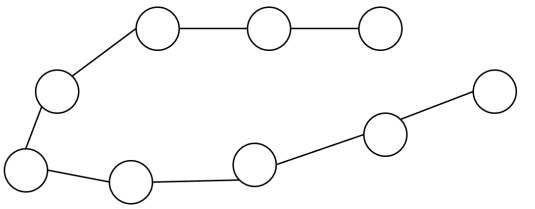
另一种理解方式：第 1 个顶点可以与另外 N-1 个顶点各形成一条边，共 N-1 条；第 2 个顶点可以和剩下的 N-2 个顶点各形成一条边，共 N-2 条..... 第 N-1 个顶点可以和剩下的 1 个顶点组成 1 条边，此时，任意两个顶点都有边存在了，达到最大。因此最大边数为

$$(N-1)+(N-2)+...+2+1 = \frac{(N-1)(1+N-1)}{2} = \frac{N(N-1)}{2}$$

事实上，对于不存在重复边、不存在自环（即指向自身的边）的图，通常称其为**简单图**。若无特殊说明，本书中提到的所有图均为简单图。

对于有 N 个顶点的无向图，其边数最大为 $\frac{N(N-1)}{2}$，有 $\frac{N(N-1)}{2}$ 条边的无向图就被称为**完全图**，也被称为**简单完全图**，完全图中的任意两个顶点之间均存在边。而对于有向图，显然边数最大为 $2 \cdot \frac{N(N-1)}{2} = N(N-1)$，因此，将有 $N(N-1)$ 条边的有向图称为**有向完全图**，有向完全图中的任意两个顶点之间都存在方向相反的两条边。

5. 要保证连通性，边数最少的情况就是“用一根线将所有顶点串起来”，如图所示：



在此情况下，再少一条边，则图将不连通。显然此情况边数等于顶点数减一。即：**有 N 个顶点的无向图，要保证它是连通图，至少要有 N-1 条边。换句话说，有 N 个顶点的无向图，若边数小于 N-1，则它一定是不连通的。**

也可以这样理解：对于 N 个顶点、无边的无向图，显然有 N 个连通块。之后每添加 1 条边，一定能让两个本不连通的块连通起来。因此，添加 N-1 条边后，就只剩一个连通块了。

6. 选 C：16。若 N 为 15，则存在一种方式：将其中 6 个顶点通过这 $\frac{6\cdot(6-1)}{2} = 15$ 条边连接为一个完全图，将第七个顶点孤立在外，使得图 G 不连通。而若 N 为 16，多出来的一条边一定可以让第七个顶点与另外 6 个连通。

7. 选 A。图的邻接矩阵 AdjM[i][j] 存储的是编号为 i 的顶点到编号为 j 的顶点的边的信息。0 表示无边，那么编号为 i 的顶点的度就等于邻接矩阵第 i 行（或第 i 列）中非零元素的数量。

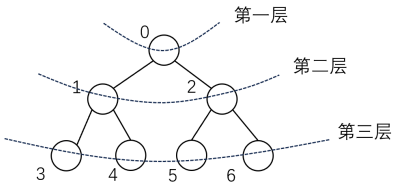
8. 选 B。如果是有向图，那么 AdjM[i][*]，即第 i 行的非 0 元素数量，是顶点 i 的出度，而 AdjM[*][i]，即第 i 列的非 0 元素数量，是顶点 i 的入度，二者加起来才是顶点 i 的度。

9. 选 C。邻接表存储方式，为每个顶点维护一个线性表，存放其所有**出边**的信息，顶点 A 有 3 条出边，顶点 C 有 1 条出边，选 C。

10. 选 D。D 项序列 {E,C,D,A,B}，探索完前两个顶点 E、C，与 C 邻接的顶点尚有 A 未被访问，仍可以向更深处探索，而序列中下一个探索了 D，说明不是深度优先

的策略。

11. 选 C。本题意在更直观地揭示树的层序遍历与图的广度优先遍历之间的关联。它们都是像涟漪一样从中心向外“扩散”：



12. 选 A。广度优先是越早看见的顶点越先被访问，是“先进先出”的，因此通常用队列实现，而不是栈；深度优先是“后进先出”的，通常用栈实现，A 错误。

13. 1）邻接矩阵版：

```
bool AdjM[N][N];    // 假设该数组已经正确存储了 G 的邻接矩阵
// 返回所有顶点的度数之和
int DegreeSum(int N) {
    // 在此处完成你的代码
    int ret = 0;
    for(int i=0; i<N; i++) {
        for(int j=0; j<N; j++) {
            if(AdjM[i][j] == 1) {
                ret += 1;
            }
        }
    }
    return ret;
}
```

2）邻接表版：

```
vector<int> AdjList[N]; // 该数组已经正确存储了 G 的邻接表
// 返回所有顶点的度数之和
int DegreeSum(int N) {
    // 在此处完成你的代码
    int ret = 0;
    for(int i=0; i<N; i++) {
        ret += AdjList[i].size();
    }
    return ret;
}
```

事实上，对于无向图，一条边必定连接两个顶点，为这两个顶点各贡献一个度。因

此，无向图中所有顶点的度数之和一定是图的边数的 2 倍。这个事实被称为**“牵手定理”**。

类似地，对于有向图，一条有向边始于一个顶点（贡献 1 个出度）、终于一个顶点（贡献 1 个入度），因此有向图中所有顶点的出度之和（或入度之和）等于图的边数 M，而有向图的总度数之和（入度 + 出度）同样是边数的 2 倍（$2M$）。

14. 显然，本题从起点出发，DFS 遍历一遍，每访问到一个新的顶点，就将计数器加一即可。“访问”一个节点的过程就是将全局计数器加一的过程，而不需要对顶点自身做任何其他操作。代码如下：

```
const int N/* = 100*/;
bool visited[N];    // 已初始化为 false

void DFS(int v, int& count, vector<int> AdjList[N]) {
    count += 1;
    visited[v] = true;
    for(int i=0; i<AdjList[v].size(); i++) {
        int adj_vertex = AdjList[v][i];
        if(!visited[adj_vertex]) {
            DFS(adj_vertex, count, AdjList);
        }
    }
}

int CountAccessibleTrees(vector<int> AdjList[N]) {
    int count = 0;
    // 从起点 0 出发，全局计数器 count 以引用传递，每访问一个顶点时计数。
    DFS(0, count, AdjList);
    return count;
}
```

15. 这是非常经典的求图的连通块的问题。我们检查一遍图的所有顶点（代码中的循环①），若其没被访问过，说明这是一个新的需要计数的连通块，于是计数加一，并以该顶点为起点执行 BFS 或 DFS，将该连通块的所有顶点标记为已访问。反之若顶点已被访问，说明它属于之前遍历过的某个连通块，已经被计数过了，跳过即可。

```
const int N/* = 100*/;
bool visited[N];    // 已初始化为 false

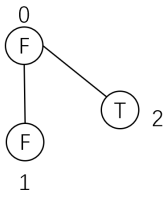
// 以 v 为起点，遍历 v 所在的连通块
void BFS(int v, vector<int> AdjList[N]) {
```

```
    queue<int> q;
    q.push(v);
    visited[v] = true;
    while(!q.empty()) {
        int cur_vertex = q.front();
        q.pop();
        for(int i=0; i<AdjList[v].size(); i++) {
            int adj_vertex = AdjList[cur_vertex][i];
            if(!visited[adj_vertex]) {
                q.push(adj_vertex);
                visited[adj_vertex] = true;
            }
        }
    }
}

int CountConnectedComponents(vector<int> AdjList[N]) {
    int count = 0;
    for(int i=0; i<N; i++) {    // 循环①
        if(!visited[i]) {   // 该顶点所在的连通块没被计数过
            count += 1;
            BFS(i, AdjList);
        }
    }
    return count;
}
```

*16. 本题思路比较简单，通过 BFS 或 DFS 从起点开始搜索，当搜索到'T' 时即找到了一条路径。问题在于搜索过程中如何将涉足过的轨迹记录下来？

①对于递归的 DFS 代码，算法一定是一步一个脚印地，沿着合法路径在前进或回溯。当走进一条错误的路径之后，如何正确地进行**回溯**？如图所示，



若从 0 出发进入了 1，此时记录的路径是“01”，回到 0 后，要将路径回溯到“0”。显然，由于回退时总是将“后路过的顶点先从路径中拿掉”，因此可以使用一个栈保存所经过的路径。回溯时，将顶点从栈顶弹出即可。

②而对于非递归的 BFS 或 DFS 代码，不同的是，非递归算法的下一个“出发点”是由队列（或栈）中元素决定的，顶点的访问顺序未必是一条合法路径。所以对于非递

归代码，要记录路径，可以设一个 Pre[] 数组，如 Pre[3] 值为 0 就表示顶点 3 的上一步是顶点 0。

我们给出非递归的 BFS 与递归的 DFS 代码如下：

代码 5.1 非递归的 BFS 解法

```
const int N/* = 7 */;
char trees[N] /*= {'F', 'F', 'F', 'F', 'F', 'F', 'T'}*/;    // 保存顶点数据
int Matrix[N][N] /*= {    // 邻接矩阵
    { 0, 1, 1, 1, 0, 0, 0 },
    { 1, 0, 0, 1, 1, 0, 0 },
    { 1, 0, 0, 1, 0, 0, 0 },
    { 1, 1, 1, 0, 0, 1, 0 },
    { 0, 1, 0, 0, 0, 1, 0 },
    { 0, 0, 0, 1, 1, 0, 1 },
    { 0, 0, 0, 0, 0, 1, 0 },
}*/;
int Pre[N]; // 记录路径
bool visited[N];

// 以 v 为起点，BFS 搜索一条找到'T'的路径。若找到了，返回'T'所在顶点的编号。
↪ 若'T'不可达，返回-1。
int BFS(int v) {
    queue<int> q;
    int ret = -1;
    if(trees[v] == 'T') {
        return v;
    }
    q.push(v);
    visited[v] = true;
    while(!q.empty()) {
        int step = q.front();
        q.pop();
        for(int i=0; i<N; i++) {
            if(Matrix[step][i] != 0 && !visited[i]) {
                // 记录轨迹的上一个顶点编号
                Pre[i] = step;
                if(trees[i] == 'T') {
                    ret = i;
                    return ret;
                }
                q.push(i);
                visited[i] = true;
            }
        }
    }
    return ret;
```

```
}

void FindPath(int u) {
    for(int i=0; i<N; i++) {
        Pre[i] = -1;
        visited[i] = false;
    }
    int flower_idx = BFS(u);
    if(flower_idx == -1) {
        cout << "......" << endl;
        return;
    }
    // 从 flower_idx 向前找到的路径顶点顺序是反的，用一个栈正向打印
    stack<int> to_print;
    for(int i=flower_idx; i!=-1; i=Pre[i]) {
        to_print.push(i);
    }
    while(!to_print.empty()) {
        cout << to_print.top();
        to_print.pop();
    }
    cout << endl;
}
```

以上 BFS 算法要访问约 N 个顶点，访问每个顶点时要遍历该顶点的邻接矩阵以找出邻接点，时间为 $O(N)$，因此总的时间复杂度为 $O(N^2)$。

递归的 DFS 代码：

代码 5.2　递归的 DFS 解法

```
const int N/* = 7 */;
char trees[N] /*= {'F', 'F', 'F', 'F', 'F', 'F', 'T'}*/;    // 保存顶点数据
int Matrix[N][N] /*= {    // 邻接矩阵
    { 0, 1, 1, 1, 0, 0, 0 },
    { 1, 0, 0, 1, 1, 0, 0 },
    { 1, 0, 0, 1, 0, 0, 0 },
    { 1, 1, 1, 0, 0, 1, 0 },
    { 0, 1, 0, 0, 0, 1, 0 },
    { 0, 0, 0, 1, 1, 0, 1 },
    { 0, 0, 0, 0, 0, 1, 0 },
}*/;
// 存在解返回 true，否则返回 false
bool DFS(int v, stack<int>& track, bool visited[]) {
    visited[v] = true;
```

```
    track.push(v);  /* 先假设经由该顶点 v 能到达樱树，在轨迹中记录此顶点，若后面发现无法到达，回溯。 */
    bool solved = false;    // 先假设不存在解，若递归过程中找到了樱树，将结果一层一层地向上带。
    if(trees[v] == 'T') {
        solved = true;
        return solved;
    }

    // 从此顶点 v 开始，向邻接点递归地探索
    for(int i=0; i<N; i++) {
        if(!visited[i] && Matrix[v][i] == 1) {
            // 以此顶点 v 为（半路）起点，可以找到樱树，返回 true
            if(DFS(i, track, visited) == true) {
                solved = true;
                return solved;
            }
        }
    }
    /* 若代码执行至此，说明事实上由此顶点 v 作为（半路）起点的所有路径均无法找到樱树，于是从轨迹中弹出此顶点，回溯到上一个顶点，继续探索其他路径。 */
    track.pop();
    return solved;
}

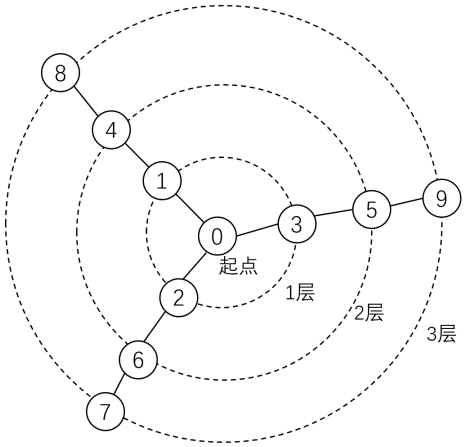
void FindPath(int u) {
    // 动态记录轨迹
    stack<int> track;
    bool visited[N];
    for(int i=0; i<N; i++) {
        visited[i] = false;
    }
    // 存在解
    if(DFS(u, track, visited) == true) {
        // 栈中保存的轨迹是相反的，保存到一个 vector 中之后从后向前打印
        vector<int> to_print;
        while(!track.empty()) {
            to_print.push(track.top());
            track.pop();
        }
        for(int i=to_print.size()-1; i>=0; i--) {
            cout << to_print[i];
        }
        cout << endl;
    } else {    // 不存在解
        cout << "......" << endl;
    }
```

```
59 }
```

该算法的性能瓶颈在于 DFS，最坏情况下，假设每个顶点都有接近 N 个邻接点，则 DFS 被调用接近 N 次，并且每次调用执行接近 N 次循环，因此算法最坏时间复杂度为 $O(N^2)$。

*17. 事实上，上题的非递归 BFS 解法，求出来的那条路径一定是最短路径。为什么？

如图所示，由于 BFS 总是先搜索完当前层的所有顶点，才会进入下一层，因此它一定会按照从近到远的顺序发现顶点。所以当 BFS 第一次发现某个顶点时，就已经找到了从起点到达该顶点、经过边数最少的路径。



当图中所有边的权重都相同时，路径的总权重只取决于它经过了多少条边。而 BFS 恰好能够保证：第一次到达某个顶点时，经过的边数最少。因此对于无权图或等权图，BFS 可以用于求解最短路径问题。

而对于带权图，经过的边数最少的路径不一定是总长度最短的路径。因此 BFS 算法无法求解带权图的最短路径。而 DFS 可以将所有的路径都探索一遍，通过穷举路径的方式计算出路径最短的是哪一条。因此通过 DFS 方式穷举路径的方法，是一种求带权图最短路径的算法，我们将在下一题中体会。

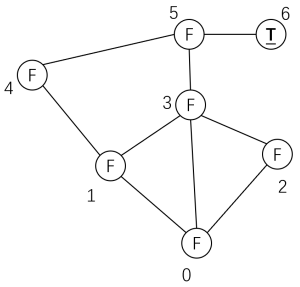
*18. 如上题所述，对于带权图无法使用 BFS 求解最短路径，但是可以 DFS 递归地将所有路径穷举一遍。在搜索到'T'后回溯，继续寻找其他路径。一种简单的想法是，在 DFS 过程中，将当前已经找到的每一条路径轨迹及其长度都记录下来，最后再选择一条长度最短的输出其轨迹。我们做如下定义：

```
// 定义一条路径
struct track {
    stack<int> nodes;   // 轨迹序列，如 0356
    int length;         // 该路径的长度
};
track t;    // 当前走过的路径
vector<track> tracks;   // 用于保存所有路径
```

于是我们可以在 DFS 递归过程中动态维护当前走过的路径 t（包括轨迹顶点集 nodes 以及轨迹的长度 length），当找到'T'时，就将当前记录的路径 t 放入 tracks 尾部：

```
tracks.push_back(track);
```

需要注意的是，在寻找路径的过程中，每个顶点有可能被多次访问，因此回溯时不仅要维护 t.nodes 与 t.length，还要将当前节点重新标记为未访问。比如在下图中，假设已经找到了一条路径 01356，回溯时若不把这些顶点的 visited[] 标记为 false，会导致后面重新从 0 出发时，无法达到 1 和 3，从而错过最短路径：“0356”。



完整的代码如下所示。

```
const int N/* = 7 */;
char trees[N] /*= {'F', 'F', 'F', 'F', 'F', 'F', 'T'}*/;    // 保存顶点数据
int Matrix[N][N] /*= {    // 邻接矩阵，0 表示无路，非 0 表示通行时间
    { 0, 3, 2, 2, 0, 0, 0 },
    { 3, 0, 0, 2, 5, 0, 0 },
    { 2, 0, 0, 1, 0, 0, 0 },
    { 2, 2, 5, 0, 0, 6, 0 },
    { 0, 1, 0, 0, 0, 2, 0 },
    { 0, 0, 0, 6, 2, 0, 1 },
    { 0, 0, 0, 0, 0, 1, 0 },
}*/;
struct track {
    stack<int> nodes;
```

```
    int length;
};

void DFS(int v, int pre_v, track& t, bool visited[], vector<track>& tracks) {
    visited[v] = true;
    t.nodes.push(v);
    t.length += Matrix[pre_v][v];
    if(trees[v] == 'T') {
        tracks.push_back(t);

        // 回溯
        visited[v] = false;
        t.nodes.pop();
        t.length -= Matrix[pre_v][v];
        return ;
    }

    for(int i=0; i<N; i++) {
        if(Matrix[v][i] != 0 && !visited[i]) {
            DFS(i, v, t, visited, tracks);
        }
    }

    // 回溯
    visited[v] = false;
    t.nodes.pop();
}

void FindPath(int u) {
    vector<track> tracks;
    track t;
    bool visited[N];
    for(int i=0; i<N; i++) {
        visited[i] = false;
    }

    DFS(u, u, t, visited, tracks);
    // tracks 为空说明从未找到过任何一条路径，不可达。
    if(tracks.empty()) {
        cout << "......" << endl;
    } else {
        // 判断一下哪条路径长度最短
        int min_length = 0x3fffffff;
        int ans_idx = -1;   // 最短路径的下标
        vector<int> min_path;
        for(int i=0; i<tracks.size(); i++) {
            if(tracks[i].length < min_length) {
```

```
                min_length = tracks[i].length;
                ans_idx = i;
            }
        }
        vector<int> to_print;
        while(!tracks[ans_idx].nodes.empty()) {
            to_print.push_back(tracks[ans_idx].top());
            tracks[ans_idx].pop();
        }
        for(int i=to_print.size()-1; i>=0; i--) {
            cout << to_print[i] ;
        }
        cout << endl;
    }
}
```

最坏情况下，假设 N 个顶点任意两顶点之间都存在边，边的数量达到了 $\frac{N(N-1)}{2}$（见本习题第 4 题），从起点出发，下一步有 (N-1) 条路，这 (N-1) 个新的起点，每个起点又有 (N-2) 条路可以选择，.....，于是需要辗转的顶点数量为

$$(N-1)\cdot(N-2)...\cdot 2\cdot 1$$

时间复杂度为 $O(N!)$ 量级。对于稍微复杂一点的图，这是根本不可能接受的！

事实上，在 DFS 过程中，可以再维护一个变量，记录当前已经找到的所有可行路径的最短长度，在递归到某顶点时，若此时当前路径长度已经大于最短长度，则无需继续递归下去，而是直接回溯即可。这种优化策略通常被称为**剪枝**，意为在递归树中剪掉一些无意义的路线。

此外，完全可以在 DFS 搜索过程中只维护和更新“当前已找到的所有路径中最短的那一条路径”，而不必将所有路径都存储起来，这对于存储空间是一个极大的优化。感兴趣的同学可以自己尝试写出代码。

但是，以上两种优化策略也仅仅是杯水车薪，带来的优化比较有限。在 5.3 节中，我们将介绍一种极具创造性的算法—迪杰斯特拉算法，来解决**单源最短路径问题**。

19. 计算所有顶点的度即可。代码如下：

```
int IsExistEL(MGraph G) {
    int odd_deg_num = 0;    // 计算度为奇数的顶点个数
    for(int i=0; i<G.numVertices; i++) {
        int i_degree = 0;   // 计算顶点 i 的度
        for(int j=0; j<G.numVertices; j++) {
            if(G.Matrix[i][j] == 1) {
```

```
                i_degree += 1;
            }
        }
        if(i_degree%2 == 1) {   // 顶点 i 是度为奇数的顶点
            odd_deg_num += 1;
        }
    }
    return (odd_deg_num == 0) || (odd_deg_num == 2);
}
```

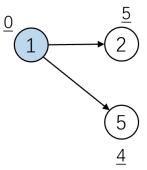
显然时间复杂度为 $O(n^2)$，空间复杂度为 $O(1)$。

20. 遍历一遍邻接矩阵，计算各顶点的入度和出度即可。

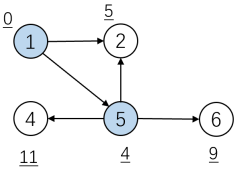
```
int PrintKVertices(MGraph G) {
    int ret = 0;    // 计算 K 顶点的个数
    for(int i=0; i<G.numVertices; i++) {
        int out_deg = 0;    // 顶点 i 的出度
        int in_deg = 0;     // 入度
        for(int j=0; j<G.numVertices; j++) {
            // i 到 j 有路径，i 的出度 +1
            if(G.Matrix[i][j] != 0) out_deg += 1;
            // j 到 i 有路径，i 的入度 +1
            if(G.Matrix[j][i] != 0) in_deg += 1;
        }
        // 出度大于入度，是一个 K 顶点，输出
        if(out_deg > in_deg) {
            cout << G.VerticesList[i];
            ret += 1;
        }
    }
    return ret;
}
```

习题 7 参考答案

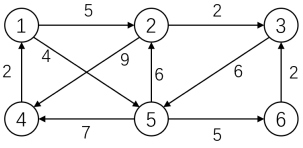
1. 选 B。每一步所选的点，及选完该点后所有点的当前已知最短路径 cur_mind[] 如下图所示（带下划线的数字表示该顶点的 cur_mind[]）。



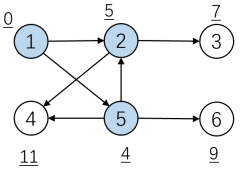
第一步，选1



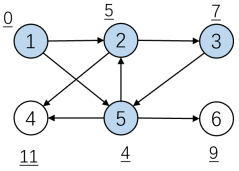
第二步，选5



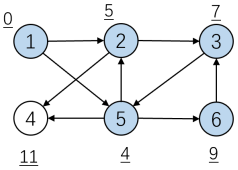
原图



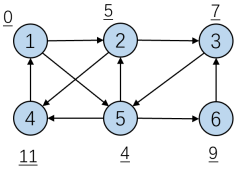
第三步，选2



第四步，选3



第五步，选6

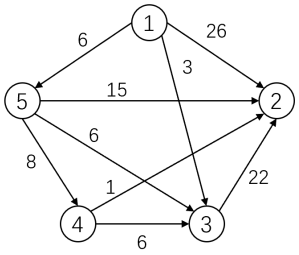


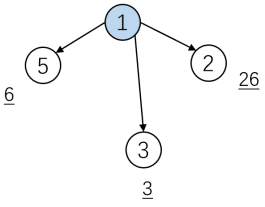
第六步，选4

于是选点顺序是 1,5,2,3,6,4，选 B。

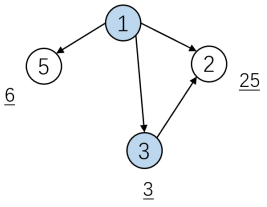
2. 选 C。需要注意 c 到 f 是有一条边的，别看漏了就行。

3. 选 C。求解前两个顶点（不包括源点）的最短路径过程如下所示。

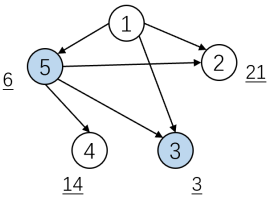




第一步



第二步



第三步

4. a）若存在从源点可达的负环，则源点 s 与任何一个顶点 v 的最短路径，都可以通过“先走到负环，然后一直绕圈，再走到 v”的方式，将最短路径“刷到负无穷”，因此若图中存在源点可达的负环，通常认为最短路径问题不可解，或者为负无穷；而

Bellman-Ford 算法可以在图存在负环时给出“不可解”的结论，Dijkstra 算法则难以做到这一点。

b）若存在源点可达的负权边，那么迪杰斯特拉算法将失效，求解出可能错误的最短路径；而 Bellman-Ford 及 SPFA 算法可以应对存在负权边的图，给出正确的最短路径。

三种算法总结如下：

图的特点	Dijkstra	Bellman-Ford	SPFA
所有边权非负	可以	可以	可以
存在负权边，无可达负环	不保证正确	可以	可以
存在可达负权环	不保证正确，无法可靠检测	可以检测	可以检测

5. 本题意在给大家练习求解最短路径的迪杰斯特拉算法、SPFA 算法的代码。显然，我们使用算法求出各顶点的最短路径，返回维火女所在顶点的最短路径即可。使用迪杰斯特拉算法求解的代码如下：

```
// N 已定义
struct EdgeInfo {
    int u;          // 顶点编号
    int time_cost;  // 花费时间（边权）
};
vector<EdgeInfo> AdjList[N];    // 已存储邻接表信息
char ver_info[N];   // 已存储顶点信息
// 返回以 s 为起点到达维火女的房间需要花费的最短时间
int MinTime(int s) {
    const int inf = 0x3fffffff;
    int cur_mind[N];
    bool solved[N];
    // 记录维火女所在的牢房编号 firekeeper_idx，函数最后返回
    ↪ cur_mind[firekeeper_idx]。
    int firekeeper_idx = -1;

    // 初始化的过程顺便找一下维火女在哪
    for(int i=0; i<N; i++) {
        cur_mind[i] = inf;
        solved[i] = false;
        if(ver_info[i] == 'T') {
            firekeeper_idx = i;
        }
    }
    cur_mind[s] = 0;

```

```
    for(int i=0; i<N-1; i++) {
        // 从尚未求解的顶点中选出一个 cur_mind[] 最小的顶点 v
        int min_curmind = inf;
        int v = -1;
        for(int j=0; j<N; j++) {
            if(cur_mind[j] < min_curmind && !solved[j]) {
                v = j;
                min_curmind = cur_mind[j];
            }
        }

        if(v == -1) {
            break;
        }
        solved[v] = true;
        // 已经计算出维火女了，就别再计算其他的了，赶紧去救她吧！
        if(v == firekeeper_idx) {
            return cur_mind[v];
        }

        for(int j=0; j<AdjList[v].size(); j++) {
            int u = AdjList[v][j].u;
            int time_cost = AdjList[v][j].time_cost;
            if(cur_mind[v] + time_cost < cur_mind[u]) {
                cur_mind[u] = cur_mind[v] + time_cost;
            }
        }

    }
    return cur_mind[firekeeper_idx] == inf? -1 : cur_mind[firekeeper_idx];
}
```

使用 SPFA 算法求解的代码如下：

```
// N 已定义
struct EdgeInfo {
    int u;          // 顶点编号
    int time_cost;  // 花费时间（边权）
};
vector<EdgeInfo> AdjList[N];    // 已存储邻接表信息
char ver_info[N];   // 已存储顶点信息
// 返回以 s 为起点到达维火女的房间需要花费的最短时间
int MinTime(int s) {
    const int inf = 0x3fffffff;
    int cur_mind[N];
    bool in_queue[N];
```

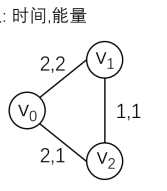
```
    /* 该场景明确图中无负边，因此无需进行负环
    检测，也就不需要 inq_cnt[] 数组，但这里还是
    以注释给出，帮助大家再次复习若需要检测负环
    时，代码该怎么写。 */
    // int inq_cnt[N];
    int firekeeper_idx = -1;
    for(int i=0; i<N; i++) {
        cur_mind[i] = inf;
        in_queue[i] = false;
        // inq_cnt[i] = 0;
        if(ver_info[i] == 'T') {
            firekeeper_idx = i;
        }
    }
    cur_mind[s] = 0;
    queue<int> q;
    q.push(s);
    in_queue[s] = true;
    while(!q.empty()) {
        int v = q.front();
        q.pop();
        in_queue[v] = false;
        for(int i=0; i<AdjList[v].size(); i++) {
            int u = AdjList[v][i].u;
            int time_cost = AdjList[v][i].time_cost;
            if(cur_mind[v] + time_cost < cur_mind[u]) {
                cur_mind[u] = cur_mind[v] + time_cost;
                if(!in_queue[u]) {
                    in_queue[u] = true;
                    q.push(u);
                    // inq_cnt[u] += 1;
                }

                // 若可能存在负环，在此处进行负环判断

            }
        }
    }
    return cur_mind[firekeeper_idx] == inf ? -1 : cur_mind[firekeeper_idx];
}
```

6.

若使用迪杰斯特拉算法，就要同时记录每个点的当前已知最短时间及对应最短时间路径的最小能量消耗：cur_mint[i] 为 j 表示顶点 i 到 s 的最短时间为 j，并且 cur_mine[i] 为 k 表示顶点 i



到 s 的最短时间路径下，最小消耗为 k。

而且每次“选点”时，就不能只选择时间花费最小的，而是要选的时间花费最小的条件下，能量消耗最小的那个点。如图，以 v_0 为起点，若直接选择了 v_1，则会得到错误答案。在更新时类似，当时间更短，或时间相等但能量消耗更少时更新即可。代码如下：

```
// N 已定义
struct EdgeInfo {
    int u;
    int time_cost;      // 花费时间
    int energy_cost;    // 和敌人战斗需要消耗的能量值
};
vector<EdgeInfo> AdjList[N];    // 已存储邻接表信息
char ver_info[N];   // 已存储顶点信息
// 返回以 s 为起点到达维火女的房间花费时间最短的路径中，消耗能量最少的那条路径所消耗的能量值
int MinEnergyCost(int s) {
    int cur_mint[N];    // 当前已知的各顶点最短时间
    int cur_mine[N];    // 对应最短时间路径中的最小能量消耗
    bool solved[N];
    const int inf = 0x3fffffff;
    int firekeeper_idx = -1;
    for(int i=0; i<N; i++) {
        cur_mint[i] = inf;
        cur_mine[i] = inf;
        solved[i] = false;
        if(ver_info[i] == 'T') {
            firekeeper_idx = i;
        }
    }

    cur_mint[s] = 0;
    cur_mine[s] = 0;

    for(int i=0; i<N-1; i++) {
        int min_timecost = inf;
        int min_energycost = inf;
        int v = -1;
        for(int j=0; j<N; j++) {
            if(!solved[j] && (cur_mint[j] < min_timecost ||
                cur_mint[j] == min_timecost && cur_mine[j] < min_energycost)
            ) {
                min_timecost = cur_mint[j];
                min_energycost = cur_mine[j];
                v = j;
            }
```

```
        }

        if(v == -1) {
            break;
        }
        solved[v] = true;
        if(v == firekeeper_idx) {
            break;
        }
        for(int j=0; j<AdjList[v].size(); j++) {
            int u = AdjList[v][j].u;
            int time_cost = AdjList[v][j].time_cost;
            int energy_cost = AdjList[v][j].energy_cost;
            if(cur_mint[v] + time_cost < cur_mint[u]) {
                cur_mint[u] = cur_mint[v] + time_cost;
                cur_mine[u] = cur_mine[v] + energy_cost;
            }
            else if(cur_mint[v] + time_cost == cur_mint[u] && cur_mine[v] +
            ↪  energy_cost < cur_mine[u]) {
                cur_mine[u] = cur_mine[v] + energy_cost;
            }
        }
    }
    return cur_mine[firekeeper_idx] == inf ? -1 : cur_mine[firekeeper_idx];
}
```

事实上本题可以从另一个角度去理解：有些时候我们可能在求时间最短的最短路径，有些时候求路程最短，有些时候求花费最少……，时间、路程、花费的大小比较规则就是比数值。

而本题“时间”和“能量”共同组成了某个东西，拟叫它“代价”好了，那么本题其实就是在求“代价”最短的最短路径。只不过“代价”有着不太一样的大小比较规则：对于两个表示“代价”的变量 a 和 b，

1）a“小于”b 定义为：a 的时间比 b 小，或 a 与 b 时间相等并且 a 的能量比 b 小；

2）a“等于”b 定义为：a 与 b 的时间、能量完全相等；

3）a“大于”b 定义为：a 的时间比 b 大，或 a 与 b 时间相等并且 a 的能量比 b 大；

因此，我们在每次“选点”时，就像要以时间、路程、花费等为标尺选出最小的一样，本题我们就是在选出“代价”最小的；在更新时，是在比较“代价”的大小，若以 v 为中介，能够使得 u 的“代价”更小，则更新 u。这也从另一个角度理解了以上代码实现。

同时，通过 C++ 的运算符重载功能，我们可以对这种结构体的大小比较进行定

义：

```
struct cost {
    int time ;
    int energy;
    // 定义 cost 类型对象的“<”比较规则
    bool operator < (const cost& other) const {
        return this->time < other.time ||
            (this->time == other.time && this->energy < other.energy);
    }

    // 定义两个代价相加规则
    cost operator + (const cost& other) const {
        cost ret;
        ret.time = this->time + other.time;
        ret.energy = this->energy + other.energy;
        return ret;
    }
};
cost cur_minc[N];   // 这一个数组就包含了 cur_mint 和 cur_mine 的功能
```

然后每次“选点”时：

```
cost min_cost;
min_cost.time = inf;
min_cost.energy = inf;
int v = -1;
for(int j=0; j<N; j++) {
    if(!solved[j] && cur_minc[j] < min_cost ) {
        min_cost = cur_minc[j];
        v = j;
    }
}
...
```

更新时：

```
for(int j=0; j<AdjList[v].size(); j++) {
    int u = AdjList[v][j].u;
    cost c;
    c.time = AdjList[v][j].time_cost;
    c.energy = AdjList[v][j].energy_cost;
    if(cur_minc[v] + c < cur_minc[u]) {
        cur_minc[u] = cur_minc[v] + c;
    }
```

```
9 }
```

这种运算符重载规则，可以在我们使用 C++ 标准库提供的排序函数 std::sort() 时来自定义一些排序规则，我们将在后面的内容中看到。

BONFIRE HIT

谢谢你带我来到这里，伴火同进者，终有一天会遇见命定之死，永别了。